A TEN-WEEK TEXT FOR CMSC 270 AT LAWRENCE UNIVERSITY

Where Things Live

Data Structures in Modern C++

Where does it live?
layout and ownership in memory

What does it cost?
on paper and on real hardware

What does it promise?
interface and invariants

How do you know?
tests, tools, and measurement

First draft edition, Fall 2026

Where Things Live: Data Structures in Modern C++
First draft edition, Fall 2026. Prepared for CMSC 270 at Lawrence University, Appleton, Wisconsin.

This text was drafted by Claude, an AI model made by Anthropic, at the request of the course instructor, in November 2026. Every C++ listing was compiled with warnings enabled and run under AddressSanitizer and UndefinedBehaviorSanitizer before it was typeset; the figures are generated by code; the timing tables were measured rather than estimated. None of that makes the book free of errors. The colophon at the back says how the book was made and how far to trust it. Report mistakes to your instructor.

There is a web edition of this book with interactive versions of many of the figures. Where a figure caption says *Interactive in the web edition*, the printed figure is a single frame of something you can operate there.

Typeset with LaTeX in XCharter, Fira Sans, and Inconsolata.

Contents

Read This First **vi**
The four questions . . . vi
The ten weeks . . . vii
How the book works . . . vii
Learning with a language model, and without one . . . ix
What you need . . . x
A note for instructors . . . xi
Answers to check-yourself questions . . . xi

1 Memory Is the Material **1**
1.1 A first program, and the machine that runs it . . . 2
1.2 Objects are bytes . . . 3
1.3 Variables are places . . . 5
1.4 Pointers: addresses as values . . . 7
1.5 Lifetimes: the stack and the heap . . . 10
1.6 The everyday kit: `vector` and `string` . . . 13
1.7 Tools that catch you . . . 15
Exercises . . . 18
Chapter notes . . . 19
Answers to check-yourself questions . . . 20

2 What Does It Cost? **21**
2.1 Counting steps . . . 21
2.2 Growth rates . . . 22
2.3 Analyzing code . . . 25
2.4 Binary search . . . 28
2.5 Measuring . . . 31
2.6 When the model lies . . . 33
Exercises . . . 36
Chapter notes . . . 38
Answers to check-yourself questions . . . 38

3 Building a Vector **39**
3.1 Three numbers . . . 40
3.2 Growing, and what it costs . . . 42
3.3 Copying . . . 46
3.4 Moving . . . 48
3.5 Templates and iterators . . . 50

3.6 How do you know? . . . 54
Exercises . . . 57
Chapter notes . . . 59
Answers to check-yourself questions . . . 59

4 Links, Stacks, and Queues 60
4.1 Nodes and links . . . 61
4.2 Rewiring . . . 63
4.3 Doubly linked lists and sentinels . . . 67
4.4 Stacks and queues . . . 69
4.5 Contiguous versus linked, on real hardware . . . 73
Exercises . . . 77
Chapter notes . . . 79
Answers to check-yourself questions . . . 79

5 Hashing 80
5.1 From keys to indices . . . 80
5.2 Separate chaining . . . 83
5.3 Open addressing . . . 86
5.4 Using hash tables well . . . 91
5.5 Bloom filters . . . 94
Exercises . . . 97
Chapter notes . . . 98
Answers to check-yourself questions . . . 99

6 Recursion and Trees 100
6.1 Recursion . . . 100
6.2 Trees . . . 103
6.3 Binary search trees . . . 107
6.4 Removal . . . 110
6.5 The shape problem . . . 111
Exercises . . . 114
Chapter notes . . . 116
Answers to check-yourself questions . . . 116

7 Balance 117
7.1 Rotations . . . 117
7.2 AVL trees . . . 119
7.3 The family of balanced trees . . . 125
7.4 B-trees . . . 126
7.5 Ordered maps in practice . . . 130
Exercises . . . 132
Chapter notes . . . 133
Answers to check-yourself questions . . . 134

8 Heaps and Sorting **135**
8.1 Priority queues and the binary heap . . . 135
8.2 Sorting with a heap, and by merging . . . 140
8.3 Quicksort . . . 143
8.4 How fast can sorting be? . . . 145
8.5 Sorting in practice . . . 147
Exercises . . . 150
Chapter notes . . . 152
Answers to check-yourself questions . . . 152

9 Graphs **153**
9.1 Graphs and how to store them . . . 154
9.2 Breadth-first search . . . 156
9.3 Depth-first search . . . 159
9.4 Shortest paths with weights . . . 162
9.5 Union-find . . . 165
Exercises . . . 170
Chapter notes . . . 171
Answers to check-yourself questions . . . 171

10 Data Structures in the Wild **173**
10.1 Choosing . . . 173
10.2 Combining structures: an LRU cache . . . 175
10.3 Tries . . . 177
10.4 Four case studies . . . 180
10.5 What we left out, and what to keep . . . 185
Exercises . . . 186
Chapter notes . . . 187
Answers to check-yourself questions . . . 188

A The Toolchain **189**
A.1 Getting a compiler . . . 189
A.2 Building programs . . . 189
A.3 Reading compiler errors . . . 190
A.4 Reading sanitizer reports . . . 191
A.5 When the program is simply wrong . . . 193

B A Little Mathematics **194**
B.1 Exponents and logarithms . . . 194
B.2 Floors and ceilings . . . 195
B.3 Sums . . . 195
B.4 Induction . . . 196
B.5 Factorials . . . 196
B.6 A little probability . . . 196

C A Field Guide to the Standard Containers **198**
C.1 The containers . . . 198
C.2 What each operation costs . . . 199
C.3 When iterators and references stop being valid . . . 199
C.4 Idioms . . . 200

Bibliography **203**

How This Book Was Made **207**
Who wrote it . . . 207
What was checked, and how . . . 207
What was not checked . . . 208
How to report a problem . . . 209
Tools . . . 209

Read This First

Every program you have ever written kept its data somewhere. A list of names, a table of scores, the squares of a game board: each one sat in the computer's memory in some particular arrangement, and that arrangement decided what your program could do quickly, what it could do only slowly, and what it could not do at all. Most of the time, in a first programming course, somebody else chose the arrangement for you. You wrote `list` or `ArrayList` or `dict` and got on with your day.

This book is about making that choice yourself. A **data structure** is a decision about where things live in memory, together with a set of promises about how you can get at them. Ten weeks from now you should be able to look at a problem, say what operations it needs, and pick or build an arrangement of memory that makes those operations cheap. You should also be able to look at somebody else's choice, whether that somebody is a library author, a teammate, or a language model, and say whether it was a good one.

The four questions

There are a great many data structures. Rather than present them as a catalog, this book asks the same four questions of each one.

Where does it live? What does the structure look like in memory? Is it one contiguous block or many small pieces connected by pointers? Who is responsible for creating that memory and for giving it back? You will be surprised how many of a structure's virtues and vices follow from the answer to this one question.

What does it promise? What operations does it offer, and what is always true of it between operations? Those always-true facts are called *invariants*, and they are the difference between a data structure and a heap of bytes. A binary search tree is not a picture of circles and lines; it is a promise that everything to the left of a node is smaller than it.

What does it cost? How does the time for each operation grow as the structure fills up? How much memory does it use? We will answer on paper, with the mathematics of growth rates, and then we will answer again with a stopwatch, because modern hardware does not always agree with the paper.

How do you know? How do you establish that an implementation keeps its promises and meets its costs? This is the question that older textbooks spend the least time on and the one that matters most now. You are going to spend your career surrounded by code you did not write, some of it produced by people and a growing share of it produced by machines. Being able to *check* is worth more than being able to type.

Each chapter ends with a short card that answers the four questions for the structures it introduced. If you can reproduce those cards from memory at the end of term, with reasons, you have learned the course.

The ten weeks

Lawrence runs on ten-week terms, so this book has ten chapters, one per week. A semester-length text would have room for more structures. This one has room for fewer, and uses the space to go deeper on the ones that carry the most weight in real software.

Week	Chapter	You will build
1	Memory Is the Material	small programs that expose addresses, copies, and lifetimes
2	What Does It Cost?	binary search; a timing harness
3	Building a Vector	your own `std::vector`
4	Links, Stacks, and Queues	a linked list, a ring-buffer queue
5	Hashing	two hash tables and a Bloom filter
6	Recursion and Trees	a binary search tree
7	Balance	an AVL tree
8	Heaps and Sorting	a binary heap; mergesort and quicksort
9	Graphs	breadth-first search, Dijkstra's algorithm, union-find
10	Data Structures in the Wild	an LRU cache and a trie

The order is not the traditional one, and three of the departures are deliberate.

Memory comes first. Before any data structure, Chapter 1 spends a week on what C++ objects, pointers, and lifetimes actually are. Every later chapter depends on it, and students who are shaky on it stay shaky all term.

Hash tables come before trees. In working software the hash table is the most heavily used structure after the array. It also needs nothing but arrays and a little arithmetic, so there is no reason to make it wait behind three weeks of trees.

Linked lists are taught as an idea more than as a tool. You need to be fluent with pointers and nodes, because trees and graphs are made of them. But on current hardware a plain linked list loses to an array at almost everything, and Chapter 4 shows you the measurements instead of asking you to take it on faith.

How the book works

The code is real. Every listing in this book is an excerpt from a file in the `code/` directory that accompanies it; the file's path is printed above the listing. Each of those files was compiled with warnings turned all the way up and run under tools that detect memory errors before the text was typeset. You should do the same: type or copy the programs, compile them, change them, and break them. There is no code editor inside the web edition, on purpose. Running a compiler on your own machine, and reading what it tells you, is one of the skills of the course.

The figures move. The book comes in two editions built from the same source. The web edition has interactive versions of many figures, in which you can insert into a tree, fill a hash table, or step through a sort one comparison at a time. The print edition shows one frame of each and marks it "Interactive in the web edition." If you are reading on paper, it is worth opening the web edition when you reach one of those.

The boxes mean things. Seven kinds of box recur.

> **THIS WEEK**
>
> At the top of every chapter: what you should be able to do by the end of the week.

> **CHECK YOURSELF 1**
>
> A question to answer *before* reading on. In the web edition the answer is folded away beneath the question; in print the answers are collected at the end of the chapter. The questions are not decoration. Trying to answer, and sometimes failing, is what makes the next paragraph stick.

> **PITFALL**
>
> A mistake that nearly everyone makes at least once, described so that you can make it only once.

> **IN THE WILD**
>
> How the idea shows up in production systems: standard libraries, databases, operating systems, the software behind language models.

> **ASIDE**
>
> A detail or a digression that can be skipped on a first reading.

> **ASK AN LLM**
>
> A prompt worth giving to a language model, with a note on what to do with the answer. The next section explains how these are meant to be used.

> **BY HAND FIRST**
>
> A task you should do with the model closed, and the reason.

The exercises are the course. Reading about data structures is like reading about swimming. Each chapter ends with exercises in roughly increasing difficulty. Do them.

Learning with a language model, and without one

Here is the awkward fact that any honest data structures textbook now has to open with. A large language model (an LLM: Claude, ChatGPT, Gemini, and their relatives) can write every data structure in this book. Ask for a templated AVL tree in C++20 and you will have one in seconds, usually a correct one. This book itself was drafted by such a model; the colophon at the back explains how, and what was done to check it.

So why learn any of this?

Because *producing* a data structure was never the hard part of the job. The hard parts are knowing which structure a problem calls for, knowing what can go wrong with it, and knowing how to tell whether the thing in front of you is right. An LLM does not remove those problems. It moves them all onto you. When the code arrives in seconds, the only remaining human contribution is judgment, and judgment about data structures comes from having built them, broken them, and measured them yourself. People who cannot tell a subtly broken hash table from a working one are no better off for having received it instantly.

There is also the plainer matter of how learning works. A long line of research finds that people learn most durably from effortful attempts, including failed ones, made *before* they are shown the answer [8, 31]. The effort is the mechanism, not an unfortunate side effect. An LLM is extremely good at removing effort, and that makes it extremely good at removing learning unless you use it with some care. The early evidence from classrooms fits the theory. In one large field experiment, students given open access to a chat model during practice did noticeably better on the practice problems and then *worse* on a later exam taken without it than students who had never had the tool; a version of the tool designed to give hints instead of answers largely avoided the harm [4]. Studies of novice programmers describe a widening gap: students who already have a solid footing use these tools to move faster, while students who are struggling can come away with an illusion of competence, feeling that they understand code they could not have written or debugged [46].

None of this is an argument for pretending the tools do not exist. They are part of how software is made now, and they can be superb tutors. It is an argument for a rule.

Generate before you consult

Before you ask a model anything about the material, produce something of your own: a prediction, a drawing, an attempt at the code, a guess at the running time, even a wrong one. *Then* consult. The model's answer lands very differently on a mind that has already committed to a position.

It helps to think of four roles a model can play.

The examiner asks you questions and withholds the answers until you respond. This is the most valuable role and the least used. "Quiz me on this, one question at a time, and do not reveal the answer until I have tried" is the best prompt in this book.

The critic reviews something you have already written or drawn. "Here is my `insert`. Do not rewrite it. Tell me which inputs would break it."

The explainer explains something you have already struggled with, at the level you ask for, as many times as you need, without getting tired or making you feel slow. Use it freely

for compiler errors, unfamiliar library functions, and second explanations of ideas the book did not land for you.

The ghostwriter writes the thing so that you do not have to. For work you already know how to do, this is a legitimate time-saver and you will use it for the rest of your career. For the thing you are currently trying to learn, it is self-sabotage with a friendly interface.

The **Ask an LLM** boxes in this book are all written for the first three roles. The **By hand first** boxes mark the places where the fourth is most tempting and most costly.

What models are good and bad at, in this course

These are tendencies, not laws, and they shift as the models change. They held when this book was written.

Models are usually *reliable* at: explaining what a compiler error means; explaining what a standard library function does; describing a well-known data structure or algorithm in general terms; generating practice questions; suggesting test cases you had not thought of; pointing out style problems.

Models are *less reliable* at: tracing pointer-heavy code step by step without slipping; counting (comparisons, swaps, loop iterations); the exact running time of an unusual loop; anything that depends on undefined behavior, where the honest answer is "this program has no defined meaning" and the tempting answer is a confident specific number; performance claims, which need to be measured on a real machine and cannot be remembered; and citations, which they can fabricate in perfect bibliographic format.

And one tendency matters more than all the rest: **a model sounds exactly as confident when it is wrong as when it is right.** A human tutor who is unsure will hedge, slow down, or look something up. A model's fluency carries no such signal. You therefore cannot use the *feel* of an answer to judge it. You need independent checks, and this course will give you four of them: compile it with warnings on; run it under a sanitizer; test it against something known to be correct; and measure it. That is the fourth question again. It applies to your own code, to the model's code, and, for that matter, to this book.

A note on rules

Whether and how you may use an LLM on graded work in CMSC 270 is decided by your instructor and stated in the syllabus. Nothing in this book overrides that. Where the syllabus allows these tools, the advice here is about using them so that you still learn. Where it does not, the By hand first boxes will at least tell you why.

What you need

Background. One prior course in programming, in any language. You should be comfortable with variables, loops, functions, and the idea of a class or object. You do *not* need to know C++; Chapter 1 starts it from the beginning, though it moves quickly and assumes you have programmed before. A little comfort with algebra, exponents, and logarithms will help from Chapter 2 onward, and Appendix B reviews what is needed.

Software. A C++ compiler that supports the C++20 standard (recent versions of GCC or Clang), a terminal, and a text editor. Appendix A walks through setup and explains the one compile command you will use all term:

```
$ g++ -std=c++20 -Wall -Wextra -g -fsanitize=address,undefined prog.cpp -o prog
```

Why C++? Because it lets you see. Languages such as Python and Java manage memory for you, which is a kindness when you are building applications and an obstacle when you are trying to learn how data is laid out. C++ makes you say where every object lives and when it dies. That is exactly the subject of this course. It is also the language in which a great deal of the world's performance-critical infrastructure is written, including most of the software that trains and runs language models.

We use modern C++ throughout: C++20 as the baseline, with an occasional pointer toward C++23. If you have seen older C++ (or C) and remember it as a thicket of `malloc`, raw arrays, and manual bookkeeping, some of what follows will be a pleasant surprise. You will still learn the raw machinery, because you cannot understand the modern conveniences without it, but you will learn it in order to build the safe tools and then use them.

A note for instructors

Each chapter is sized for one week of three class meetings and is divided into sections that roughly match those meetings. The chapters are close to equal in length; the last, for week 10, introduces the least new machinery and can absorb review. Topics a semester course would add, and which this text deliberately leaves to a later algorithms course, include red-black tree implementation, string matching, dynamic programming, minimum spanning trees beyond a brief application of union-find, and formal amortized analysis by potential functions. The exercises marked **Lab** are sized for a week-long programming assignment. The source for the book, its figures, and its code are all in the accompanying repository and are meant to be edited.

Answers to check-yourself questions

1 Like this. If you peeked without trying, that is your business, but you have cheated only yourself.

CHAPTER 1

Memory Is the Material

WEEK 1 OF 10

THIS WEEK

By the end of this week you should be able to:

- compile and run a C++ program with warnings and sanitizers turned on;
- explain what an object's type, size, address, and lifetime are;
- predict when C++ copies an object and when it shares one, and choose between passing by value, by reference, and by `const` reference;
- read and write code that uses pointers, including pointer arithmetic on arrays;
- draw the stack and the heap for a short program, and recognize a leak, a dangling pointer, and an out-of-bounds access when you see one.

Here are two programs. The first is Python.

```
a = [1, 2, 3]
b = a
b.append(4)
print(len(a))
```

The second is C++, and does what looks like the same thing.

code/ch01/copies.cpp

```
std::vector<int> a{1, 2, 3};
std::vector<int> b = a;   // b is a new vector: a copy of a
b.push_back(4);

std::cout << a.size() << ' ' << b.size() << '\n';   // prints 3 4
```

CHECK YOURSELF 1.1

The Python program prints the length of `a`. The C++ program prints the sizes of `a` and `b`. What does each print? Commit to an answer before you read on.

Neither language is wrong. They made different decisions about what a variable *is*. In Python, a variable is a label tied to an object that lives somewhere else, and assignment ties a second label to the same object. In C++, a variable *is* the object: a region of memory with a name, and assignment copies the contents of one region into another.

That difference is the whole reason this course is taught in C++. A data structure is an arrangement of memory, and C++ is a language in which you can see, and must

decide, how memory is arranged. This chapter is a fast tour of the machinery for doing that: objects, references, pointers, the stack, and the heap. It moves quickly, because you already know how to program. What you may not know yet is where your programs have been keeping things.

1.1 A first program, and the machine that runs it

Tradition demands this one.

code/ch01/hello.cpp

```
#include <iostream>

int main() {
    std::cout << "Hello, CMSC 270!\n";
    return 0;
}
```

To run it you first **compile** it: a program called a compiler translates the text into machine instructions and writes them to a file, which you then run.

```
$ g++ -std=c++20 -Wall -Wextra -g -fsanitize=address,undefined hello.cpp -o
↪  hello
$ ./hello
Hello, CMSC 270!
```

If you have only used Python, notice what is different. There is no interpreter standing between your program and the processor while it runs. The translation happens once, ahead of time, and whole categories of mistakes (a misspelled name, a string where a number should be) are reported by the compiler before the program ever starts. The price is the extra step, and error messages that take some practice to read.

A few words about the pieces. `#include <iostream>` pulls in the declarations for input and output. Execution begins at the function named `main`, which returns an `int` to the operating system: zero for success. `std::cout` is the standard output stream; `<<` sends things to it. The `std::` prefix says that `cout` belongs to the standard library's namespace. Statements end with semicolons, and blocks are marked with braces, not indentation.

The long compile command is worth understanding once, because you will use it all term.

Flag	Meaning
`-std=c++20`	use the 2020 version of the language
`-Wall -Wextra`	turn on the compiler's warnings about suspicious code
`-g`	include debugging information, so that tools can name lines
`-fsanitize=address,undefined`	add run-time checks for memory errors and undefined behavior
`-o hello`	name the output file `hello`

The middle three are your safety equipment. Section 1.7 shows what they catch, and Appendix A covers installation and has more on reading what the compiler says.

ASK AN LLM

Compiler errors in C++ can be long and strange, and explaining them is something language models do well. When you are stuck on one, try:

> *Here is a C++ compiler error and the code that produced it. Explain in plain language what the compiler is complaining about and which line is the real cause. Do not give me corrected code; I want to fix it myself.*

The last sentence matters. An explanation teaches you to read the next error on your own. A silently corrected file teaches you nothing, and you will meet the same error again next week.

1.2 Objects are bytes

A computer's memory is, to a very good approximation, one enormous row of numbered boxes. Each box is a **byte**: eight bits, enough to hold a number from 0 to 255. The number of a box is its **address**. A laptop with 16 gigabytes of memory has about sixteen billion boxes.

That is all there is. Every integer, every string, every image, every tree and hash table in this book is, in the end, a pattern of values in some of those boxes. An **object**, in C++, is a region of memory that the program is using to hold a value. Every object has four properties, and this chapter is organized around them:

- a **type**, which says how to interpret its bytes;
- a **size**, the number of bytes it occupies;
- an **address**, the number of its first byte;
- a **lifetime**, the stretch of the program's execution during which it exists.

The `sizeof` operator reports sizes.

code/ch01/sizes.cpp

```
std::cout << "char:      " << sizeof(char) << '\n'
          << "bool:      " << sizeof(bool) << '\n'
          << "int:       " << sizeof(int) << '\n'
          << "long long: " << sizeof(long long) << '\n'
          << "double:    " << sizeof(double) << '\n'
          << "int*:      " << sizeof(int*) << '\n';
```

On the machine this book was written on, and very likely on yours, that prints:

```
char:      1
bool:      1
int:       4
long long: 8
double:    8
int*:      8
```

The standard guarantees only that a `char` is one byte and that the other integer types are at least certain minimum sizes; the numbers above are what current desktop and laptop systems use.

So an `int` is four bytes: 32 bits. Figure 1.1 shows an `int` holding 270 and a `char` holding the letter A, sitting in memory.

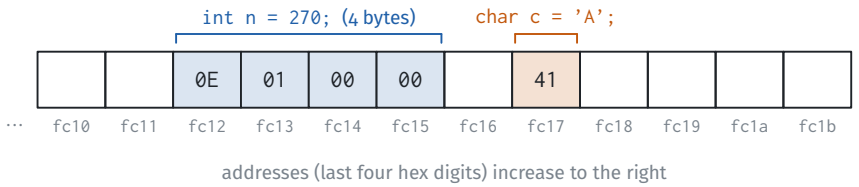


Figure 1.1 Two objects in memory. The `int` occupies four consecutive bytes; 270 is `0x0000010E` in hexadecimal, and this machine stores the low-order byte first. The `char` occupies one byte; 65 (`0x41`) is the code for `A`. Nothing in the bytes themselves says which is a number and which is a letter. The type does that.

Two consequences follow at once.

The type is not stored in memory. The four bytes of an `int` and the four bytes of a `float` look the same. It is the compiler, which knows the declared type of every variable, that decides to generate integer-addition instructions for one and floating-point instructions for the other. This is why C++ wants every variable's type declared, or deducible, at compile time.

Fixed size means limited range. Thirty-two bits can distinguish about 4.3 billion values. For a signed `int` that is the range from −2,147,483,648 to 2,147,483,647. Adding 1 to the largest `int` does not produce a bigger number, because there is no room for one. For *unsigned* types the result wraps around to zero, reliably. For signed types the language says something more alarming, which we will come back to: the behavior is *undefined*.

You can ask where an object lives with the address-of operator, `&`.

code/ch01/sizes.cpp

```
int n = 270;
char c = 'A';
double d = 2.7;
std::cout << "n lives at " << &n << '\n'
          << "c lives at " << static_cast<void*>(&c) << '\n'
          << "d lives at " << &d << '\n';
```

```
n lives at 0x7ffd5ce7b7cc
c lives at 0x7ffd5ce7b7cb
d lives at 0x7ffd5ce7b7d0
```

Addresses are conventionally printed in hexadecimal. The exact values will differ on your machine and from run to run, but look at the pattern: `c` is at an address ending `cb`, `n` occupies the four bytes from `cc` through `cf`, and `d` begins at `d0`. The three variables are packed side by side. Memory is a row of boxes, and you have just read the numbers off three of them. (This output is from a build *without* `-fsanitize`. AddressSanitizer

deliberately spaces local variables apart, with guard zones between them, so that it can catch one overrunning into the next; under it you will see larger gaps.)

> **ASIDE**
>
> One type deserves early mention. Sizes and indices in the standard library are of type `std::size_t`, an unsigned integer big enough to count every byte of memory (eight bytes on current machines). `v.size()` returns one. Because it is unsigned, it cannot be negative, and comparing it with a plain `int` earns a compiler warning, since a negative `int` would be silently reinterpreted as a huge positive number. The habit this book follows: use `std::size_t` for anything that is a size or an index into a container.

1.3 Variables are places

Return to the puzzle that opened the chapter. In C++ a variable names a region of memory, and initialization or assignment from another variable **copies**: the bytes that make up one object's value are duplicated into the other. Afterward the two objects have nothing to do with each other.

That is obviously what should happen for an `int`. C++ applies the same rule to everything, including types you define yourself.

code/ch01/copies.cpp

```
struct Point {
    double x;
    double y;
};
```

code/ch01/copies.cpp

```
Point p{1.0, 2.0};
Point q = p;      // copies both doubles
q.x = 99.0;       // p.x is still 1.0
```

A `struct` is a type made by placing other objects side by side; a `Point` is two `double`s, sixteen bytes. Copying it copies the sixteen bytes. For a `std::vector`, copying means allocating a new block of memory and copying every element into it, which is exactly why `a` and `b` in the opening example went their separate ways. This behavior is called **value semantics**: variables hold values, and copies are independent. The left side of Figure 1.2 shows the result.

Value semantics is safe and predictable: nobody can change your object behind your back. It is also sometimes exactly what you do not want. A function that is supposed to modify the caller's vector cannot do so if it only receives a copy. And copying a vector of a million elements, merely to add them up, does a million elements' worth of pointless work.

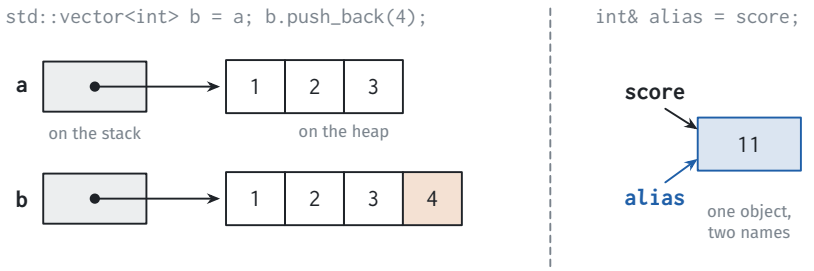


Figure 1.2 Left: in C++, `b = a` makes `b` a separate vector with its own storage, so appending to `b` leaves `a` alone. Right: a reference is a second name for the same object. The Python and Java meaning of `b = a` is the picture on the right.

References

For those situations C++ has **references**. A reference is another name for an existing object.

code/ch01/copies.cpp

```
int score = 10;
int& alias = score;   // alias is another name for score
alias = 11;           // score is now 11
```

The `&` after the type makes `alias` a reference to an `int` and not an `int` of its own. No new integer is created. From that line on, `alias` and `score` are two names for one region of memory. A reference must be bound to an object when it is created, and it stays bound to that object for its whole life.

References earn their keep in function parameters. There are three ways to pass an argument, and choosing among them is a decision you will make in every function you write for the rest of the course.

code/ch01/copies.cpp

```
// By value: the function gets its own copy. The caller's vector is safe,
// but every element is copied on every call.
int sum_by_value(std::vector<int> v) {
    int total = 0;
    for (int x : v) total += x;
    return total;
}

// By const reference: no copy, and a promise not to modify.
// This is the default way to pass anything bigger than a few bytes.
int sum(const std::vector<int>& v) {
    int total = 0;
    for (int x : v) total += x;
    return total;
}

// By (non-const) reference: no copy, and the function may modify
```

```
// the caller's object. Use it when modification is the point.
void double_all(std::vector<int>& v) {
    for (int& x : v) x *= 2;
}
```

The `const` in the middle version is a promise, enforced by the compiler, that the function will not modify the object it has been lent. This gives you the cheapness of sharing with the safety of copying, and it is the right default for anything larger than a number.

You want to…	Pass by	Spelled
read a small, cheap thing (`int`, `double`, `char`, a pointer)	value	`int x`
read anything bigger	`const` reference	`const std::vector<int>& v`
modify the caller's object	reference	`std::vector<int>& v`

CHECK YOURSELF 1.2

What is wrong with this function, which is meant to set every element of a vector to zero?

```
void clear_all(std::vector<int> v) {
    for (int& x : v) x = 0;
}
```

The range-based `for` loop follows the same rules. `for (int x : v)` copies each element into `x` in turn; `for (int& x : v)` makes `x` a reference to each element, so assigning to `x` changes the vector; `for (const std::string& s : names)` reads each string without copying it.

1.4 Pointers: addresses as values

You saw that `&n` produces the address of `n`. An address is just a number, and numbers can be stored. A **pointer** is an object whose value is the address of another object.

code/ch01/pointers.cpp

```
int n = 270;
int* p = &n;     // p holds the address of n
*p = 271;        // follow the pointer; assign to what it points at
std::cout << n << '\n';   // prints 271

int* nobody = nullptr;    // a pointer that points at nothing
```

Read `int* p` as "p is a pointer to `int`." The unary `*` operator **dereferences** a pointer: `*p` means "the object whose address is in p." So `*p = 271` stores 271 into `n` by way of its address.

A pointer can also hold the special value `nullptr`, meaning that it points at nothing. Dereferencing a null pointer is an error; on most systems the program crashes immediately, which is the *good* outcome, as we will see.

For pointers to structs there is a shorthand that you will type thousands of times this term.

code/ch01/pointers.cpp

```
Point pt{3.0, 4.0};
Point* pp = &pt;
(*pp).x = 5.0;   // follow pp, then select member x
pp->y = 12.0;    // the same thing, spelled the way everyone spells it
```

Pointers and references are close relatives: both let one piece of code reach an object that lives somewhere else. The differences are what make each useful.

	Reference	Pointer
Can be null?	no	yes (`nullptr`)
Can be re-aimed at a different object?	no	yes
Syntax to reach the object	none; use it like the object	`*p` or `p->member`
Typical use	function parameters	links inside data structures; optional or changing targets

A linked list node must be able to say "there is no next node," and must be able to change which node comes next. Those are things only a pointer can do, and it is why from Chapter 4 onward this book is full of pointers.

Arrays and pointer arithmetic

The oldest data structure is the **array**: some number of objects of the same type, placed side by side in memory with no gaps.

code/ch01/pointers.cpp

```
int a[5] = {10, 20, 30, 40, 50};   // five ints, side by side
int* first = a;                    // an array's name converts to
                                   // a pointer to its first element

assert(*first == 10);
assert(*(first + 3) == 40);        // three ints further along
assert(a[3] == *(a + 3));          // indexing IS pointer arithmetic
```

Because the elements are the same size and are contiguous, finding element `i` requires no searching. If the array starts at address A and each element is s bytes, then element i is at address $A + i \cdot s$: one multiplication and one addition, however long the array is. Figure 1.3 shows the arithmetic.

Adding an integer to a pointer moves it forward by that many *elements*, not that many bytes; the compiler scales by the element size. And the indexing notation `a[i]` is *defined* to mean `*(a + i)`. Indexing is pointer arithmetic with nicer punctuation.

Hold on to this fact. It is the first cost result of the course, and one of the most important:

Reaching an element of an array, given its index, takes the same small amount of time no matter how large the array is.

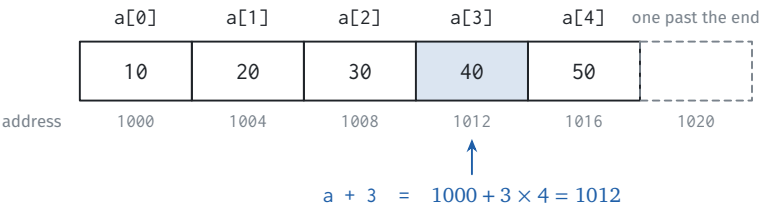


Figure 1.3 An array of five `int`s beginning at address 1000. Each `int` is four bytes, so `a[3]` is at $1000 + 3 \times 4 = 1012$. The pointer expression `a + 3` computes that address; the compiler does the multiplying by four for you.

> Everything that makes arrays, vectors, hash tables, and heaps fast comes back to it. Pointers can walk through an array, too.

code/ch01/pointers.cpp

```
int total = 0;
for (int* q = a; q != a + 5; ++q) {   // a + 5 is "one past the end"
    total += *q;
}
```

The address `a + 5` does not belong to the array; it is the position just past the last element. Forming that "one past the end" address is permitted, precisely so that loops like this can use it as a stopping point. Dereferencing it is not. This pattern, a pair of positions marking a beginning and a just-past-the-end, is how all of C++'s standard library describes ranges, and Chapter 3 returns to it.

No one checks

What happens if you write `a[7]`, when `a` has five elements?

The compiler computes $A + 7 \cdot 4$ and reads whatever is there. Perhaps it is another variable. Perhaps it is bookkeeping data that some other part of the program depends on. There is no check that the index is in range, because a check costs time on every access, and C++ is built on the principle that you do not pay for what you do not ask for.

The C++ standard calls a program that does this, or dereferences a null pointer, or overflows a signed integer, a program with **undefined behavior**. The term is exact: the language places *no requirements at all* on what such a program does. It may crash. It may print garbage. It may appear to work perfectly for months and then fail when the surrounding code changes. The compiler is allowed to assume that undefined behavior never happens and to optimize on that assumption, which can produce results that seem to defy logic.

> **PITFALL**
>
> "It ran and printed the right answer" is not evidence that a C++ program is correct. A program with undefined behavior often prints the right answer, on your machine, today. This is the single most important difference between C++ and the languages you already know, where

an out-of-range index reliably raises an error.

This sounds like a terrible deal, and without tools it would be. Section 1.7 introduces the tools.

ASK AN LLM

Undefined behavior is a good way to calibrate how far to trust a model's reading of code. Take one of the small buggy programs from this chapter's `code/ch01/bugs/` directory and ask:

What does this C++ program print? Answer precisely.

A careful answer says that the program has undefined behavior and that no particular output is guaranteed. A careless one names a specific number. Try it a few times, and with a few different programs. You are not trying to catch the model out for sport; you are learning what kind of question tempts it to answer more confidently than the facts allow.

1.5 Lifetimes: the stack and the heap

The fourth property of an object is its lifetime: when it comes into existence and when it ceases to exist. C++ gives you two main kinds of lifetime, and they correspond to two regions of memory.

Automatic storage: the stack

A variable declared inside a function is created when execution reaches its declaration and destroyed when execution leaves the enclosing block. You do nothing to make this happen, which is why such variables are said to have **automatic** storage.

code/ch01/lifetime.cpp

```
int square(int x) {
    int result = x * x;   // result lives in square's stack frame
    return result;        // its value is copied out; then the frame is gone
}

int sum_of_squares(int a, int b) {
    int s = square(a);
    int t = square(b);
    return s + t;
}
```

Automatic variables live in a region called the **stack**. When a function is called, a block of memory called a **stack frame** is set aside for its parameters and local variables, immediately beyond the frame of the function that called it. When the function returns, its frame is released, and the next call will reuse the same memory. Calls nest, so frames pile up and come off in last-in-first-out order, like a stack of plates. That is where the name comes from, and in Chapter 4 we will meet the stack again as a data structure in its own right.

When `main` calls `sum_of_squares(3, 4)`, which calls `square(3)`, there are three frames on the stack. The variable `result` exists only while `square` is running. Its *value* is copied back to the caller as the return value; then the object itself is gone.

Stack allocation is extremely cheap, since claiming a frame is a single adjustment of one processor register, and it requires no bookkeeping from you. It has two limitations. The stack is small, typically a few megabytes, so it is no place for a million-element array. And lifetimes are tied rigidly to function calls: nothing on the stack can outlive the function that created it.

> **PITFALL**
>
> That second limitation has teeth. A function must never return a reference or pointer to one of its own local variables:
>
> ```
> int& biggest(int a, int b) {
>     int result = (a > b) ? a : b;
>     return result;   // result dies here; the reference outlives it
> }
> ```
>
> The caller receives a reference to memory that is no longer an `int` and will shortly be overwritten by some other function's frame. A pointer or reference to an object whose lifetime has ended is said to **dangle**. GCC warns about this one (`reference to local variable 'result' returned`), which is one reason we compile with warnings on. Return the value instead: `int biggest(int a, int b)`.

Dynamic storage: the heap

For objects that must outlive the function that creates them, or whose size is not known until the program runs, there is a second region of memory, the **heap** (also called the free store). You request heap memory explicitly with `new`, which returns a pointer to it, and you return it explicitly with `delete`.

code/ch01/lifetime.cpp

```
// Returns a pointer to n ints on the heap, all set to `fill`.
// The caller now OWNS that memory and must delete[] it.
int* make_array(int n, int fill) {
    int* block = new int[n];
    for (int i = 0; i < n; ++i) block[i] = fill;
    return block;   // the pointer is copied out; the block lives on
}
```

code/ch01/lifetime.cpp

```
int* one = new int{42};   // one int on the heap
*one += 1;
delete one;               // give it back

int* many = make_array(1000, 7);
assert(many[999] == 7);
delete[] many;            // arrays are returned with delete[]
```

Look carefully at what lives where in `make_array`. The pointer variable `block` is a local variable on the stack, eight bytes holding an address. The thousand `int`s it points to are on the heap. When `make_array` returns, `block` is destroyed like any other local, but its value, the address, has been copied out to the caller, and the heap block is unaffected. Heap objects live until someone deletes them. Figure 1.4 lets you step through this and a few other small programs and watch both regions change.

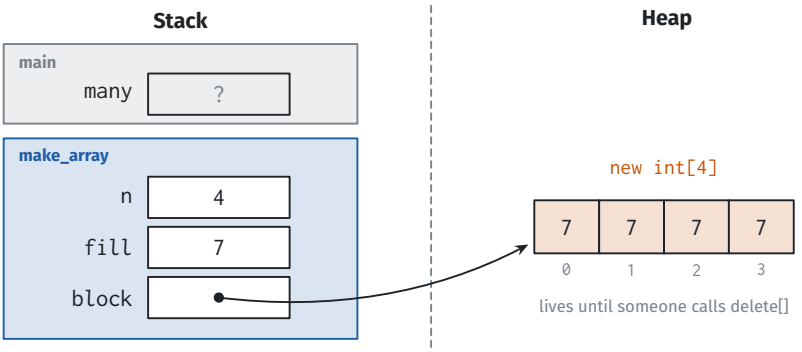


Figure 1.4 The stack and the heap during a call to `make_array`. Local variables, including pointers, live in stack frames that vanish when their function returns. Heap blocks persist until deleted. In the web edition, choose a program and step through it line by line. [Interactive in the web edition.]

With that freedom comes an obligation. Every block obtained with `new` must be released with `delete` exactly once (and with `delete[]` if it was allocated with `new[]`). There are three ways to get this wrong, and each has a name.

A **memory leak** is a block that is never deleted. If the only pointer to a heap block is overwritten or goes out of scope, the block can never be freed; it stays allocated until the program ends. A leak in a short program is harmless. A leak inside a loop in a program that runs for weeks is how servers die.

A **dangling pointer**, or **use after free**, is a pointer that is used after the block it points to has been deleted. The memory may by then belong to some unrelated object. Reading through the pointer gives nonsense; writing through it corrupts whatever now lives there.

A **double free** is deleting the same block twice, which corrupts the allocator's own records.

The second and third are undefined behavior. All three stem from one question that C++ leaves to you: for each block on the heap, **who is responsible for deleting it?** The usual word is **ownership**. In the code above, `make_array` hands ownership of the block to its caller, which is why the comment says so.

> **BY HAND FIRST**
>
> For the next few weeks, whenever a program uses pointers or the heap, draw it: a column of stack frames with a box for each variable, a separate area for heap blocks, and an arrow from each pointer to what it points at. Step through the code and update the drawing by hand, crossing out frames as functions return. Do this on paper, without a model.
>
> There are two reasons. First, this drawing *is* the mental model that the rest of the course

> runs on, and you only acquire it by producing it yourself, slowly, a number of times. Second, language models are text predictors, not memory simulators. Ask one for a step-by-step memory diagram of pointer-heavy code and it will usually be right and sometimes be subtly wrong, with an arrow aimed at the wrong box, and until you can draw the diagram yourself you cannot tell which.

What modern C++ does about it

If managing heap memory by hand is so error-prone, how does anyone write large programs in C++?

By almost never doing it by hand. The central technique of modern C++ is to wrap each heap allocation inside an object whose own lifetime is automatic. `std::vector` is the leading example. A `vector` variable is a small object on the stack holding a pointer to a heap block where the elements live. When the vector's lifetime ends, a special function called its *destructor* runs automatically and deletes the block. The heap memory gets the easy, automatic lifetime of a stack variable, and you never write `delete` at all.

So the rule for everyday code is simple: **use `std::vector` and `std::string`, and do not write `new` or `delete`.**

This course is not everyday code, though. Our business is to understand how types like `vector` are built, so in Chapter 3 you will write one, `new` and `delete` and all, and in later chapters you will build lists and trees out of individually allocated nodes. You will do it with the safety equipment on.

1.6 The everyday kit: `vector` and `string`

You will lean on two standard library types from the first day. Here is a working knowledge of each.

code/ch01/vector_tour.cpp

```
std::vector<int> v;             // empty
v.push_back(10);                // {10}
v.push_back(20);                // {10, 20}
v.push_back(30);                // {10, 20, 30}

std::cout << v.size() << '\n';  // 3
std::cout << v[0] << '\n';      // 10: no bounds check
std::cout << v.at(2) << '\n';   // 30: at() checks and throws if bad
v[1] = 25;                      // {10, 25, 30}
v.pop_back();                   // {10, 25}

std::vector<double> zeros(5, 0.0);        // five 0.0s
std::vector<std::string> names{"Ada", "Grace", "Barbara"};

for (const std::string& name : names) {   // read each, no copies
    std::cout << name << ' ';
}
std::cout << '\n';
```

```
for (double& z : zeros) {                // modify each in place
    z = 1.5;
}
```

A `std::vector<T>` is a sequence of objects of type `T`, stored contiguously, that can grow. The type in angle brackets says what it holds. `push_back` appends; `size` reports the count; square brackets index from zero. Indexing with `[]` performs no bounds check, exactly as with a raw array, while `at()` checks and reports an error if the index is out of range.

`std::string` is similar, specialized for characters.

code/ch01/vector_tour.cpp

```
std::string s = "data";
s += " structures";              // strings grow, like vectors
assert(s.size() == 15);
assert(s[0] == 'd');
assert(s.substr(5, 6) == "struct");
assert(s.find("ruct") == 7);
assert(s < "database");          // comparison is alphabetical
```

Here is a complete small program in the style you will write this week. It reads numbers from standard input, however many there are, and reports on them.

code/ch01/stats.cpp

```
int smallest(const std::vector<int>& v) {
    int best = v[0];
    for (int x : v) {
        if (x < best) best = x;
    }
    return best;
}

double mean(const std::vector<int>& v) {
    long long total = 0;
    for (int x : v) total += x;
    return static_cast<double>(total) / static_cast<double>(v.size());
}
```

code/ch01/stats.cpp

```
int main() {
    std::vector<int> data;
    int x;
    while (std::cin >> x) {   // read until input runs out
        data.push_back(x);    // the vector grows as needed
    }

    if (data.empty()) {
        std::cout << "no data\n";
        return 0;
    }

    std::cout << "count:    " << data.size() << '\n'
```

```
              << "smallest: " << smallest(data) << '\n'
              << "mean:     " << mean(data) << '\n';

    for (std::size_t i = 0; i < data.size(); ++i) {
        if (data[i] < 0) {
            std::cout << "negative value at index " << i << '\n';
        }
    }
}
```

```
$ ./stats < stats.in
count:    6
smallest: -3
mean:     13.6667
negative value at index 2
```

Things to notice: the functions take the vector by `const` reference, so nothing is copied. The loop `while (std::cin >> x)` reads one number per iteration and stops when the input is exhausted. The index variable `i` is a `std::size_t`, to match the type of `data.size()`. And `mean` adds into a `long long`, so that a long list of large values does not overflow a 32-bit total.

> **CHECK YOURSELF 1.3**
>
> The function `smallest` begins with `int best = v[0];`. Under what circumstances is that line a bug, and how does `main` protect against it?

1.7 Tools that catch you

The fourth of the book's questions is *how do you know?* This section is its first appearance, and it is about not having to rely on luck.

Here is a program with a classic mistake: the loop condition uses <= where it should use <, so the last iteration reads one element past the end of the vector.

code/ch01/bugs/oob.cpp

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v{10, 20, 30};
    int total = 0;
    for (std::size_t i = 0; i <= v.size(); ++i) {   // bug: <= should be <
        total += v[i];
    }
    std::cout << total << '\n';
}
```

Compiled plainly, with no special flags, and run three times, it prints:

```
60
60
60
```

That is the correct sum of 10, 20, and 30. The out-of-bounds read happened to find a zero. The program has undefined behavior, and it looks perfectly healthy. Submit it, and it might fail on the grader's machine, or it might not fail for a year.

Now compile it with the flags from the beginning of the chapter, and run it again.

```
$ g++ -std=c++20 -Wall -Wextra -g -fsanitize=address,undefined oob.cpp -o oob
$ ./oob
==21309==ERROR: AddressSanitizer: heap-buffer-overflow on address
↪  0x50200000001c
READ of size 4 at 0x50200000001c thread T0
    #0 0x5be00d361731 in main ch01/bugs/oob.cpp:8
    ...
0x50200000001c is located 0 bytes after 12-byte region
↪  [0x502000000010,0x50200000001c)
allocated by thread T0 here:
    #0 0x78b4b38fe548 in operator new(unsigned long)
    ...
```

The program now stops at the moment of the bad access and tells you what happened (a read of 4 bytes beyond a heap block), where (`oob.cpp`, line 8), and what it ran into (the end of a 12-byte region, which is three `int`s). This is **AddressSanitizer**, enabled by `-fsanitize=address` [51]. It works by having the compiler insert a check before every memory access and by surrounding every allocation with poisoned "red zones." It roughly doubles a program's running time, which is a bargain.

AddressSanitizer catches all three of the heap errors described in Section 1.5. A use after free:

code/ch01/bugs/dangling.cpp

```
#include <iostream>

int main() {
    int* p = new int{42};
    delete p;                 // the int is gone...
    std::cout << *p << '\n';  // ...but p still holds its old address
}
```

```
==21312==ERROR: AddressSanitizer: heap-use-after-free on address 0x502000000010
READ of size 4 at 0x502000000010 thread T0
    #0 0x5f9fdc021475 in main ch01/bugs/dangling.cpp:6
    ...
freed by thread T0 here:
    #1 0x5f9fdc021309 in main ch01/bugs/dangling.cpp:5
```

Without the sanitizer, the same program printed `-452675544` on one run. A leak:

code/ch01/bugs/leak.cpp

```cpp
#include <iostream>

int* make_array(int n, int fill) {
    int* block = new int[n];
    for (int i = 0; i < n; ++i) block[i] = fill;
    return block;
}

int main() {
    int* scores = make_array(1000, 7);
    scores = make_array(2000, 9);   // the first block is now unreachable
    std::cout << scores[0] << '\n';
    delete[] scores;
}
```

```
==21315==ERROR: LeakSanitizer: detected memory leaks
Direct leak of 4000 byte(s) in 1 object(s) allocated from:
    #1 0x5c7e03bb1385 in make_array(int, int) ch01/bugs/leak.cpp:4
    #2 0x5c7e03bb1486 in main ch01/bugs/leak.cpp:10
```

Four thousand bytes is a thousand `int`s: the first block, whose only pointer was overwritten on line 11.

The other half of the flag, `-fsanitize=undefined`, enables **UndefinedBehaviorSanitizer**, which catches a different family of errors, such as signed overflow.

```
overflow.cpp:5:9: runtime error: signed integer overflow:
2147483647 + 1 cannot be represented in type 'int'
```

Sanitizers only examine the code that actually runs. If your tests never trigger the faulty line, the sanitizer never sees it. So the tools do not relieve you of testing; they make your tests far more informative. A test that passes under the sanitizers has shown not only that the output was right but that no memory error occurred along the way.

One more tool is so simple that it is easy to overlook: `assert`. Writing `assert(condition);` (after `#include <cassert>`) makes the program stop with a message if the condition is false. Assertions are how you write down what you believe to be true at a point in the program, in a form the machine will check. The example programs in this book use them constantly, and so should yours.

> **IN THE WILD**
>
> Memory errors are not a beginner's problem that professionals grow out of. For years, large software vendors have reported that the majority of their serious security vulnerabilities trace back to memory-safety bugs in C and C++ code: out-of-bounds accesses and uses after free, the same two bugs you just watched a sanitizer catch. This is a principal reason for the industry's growing interest in memory-safe languages such as Rust, and for the steady movement of C++ itself toward safer defaults. It is also why this book has you compile with sanitizers from day one. The habit is professional practice, not training wheels.

THE FOUR QUESTIONS A C++ object

Where does it live? At an address, occupying `sizeof` bytes. Local variables live in their function's stack frame. Objects created with `new` live on the heap. A `vector` or `string` variable is a small object that owns a block on the heap.

What does it promise? Its type says how its bytes are interpreted. A variable is the object itself, so assignment copies. A reference is another name for an existing object. A pointer holds an address and may be null.

What does it cost? Reaching element `i` of an array is one multiplication and one addition, regardless of the array's length. Stack allocation is nearly free. Copying costs time proportional to the size of the thing copied, which is why large objects are passed by `const` reference.

How do you know? Compiler warnings, AddressSanitizer, UndefinedBehaviorSanitizer, and `assert`. A program that merely prints the right answer has not been shown to be correct.

Exercises

1. Compile and run `hello.cpp` with the full compile command from this chapter. Then introduce three different errors, one at a time (delete a semicolon, misspell `cout`, remove the `#include`), and read what the compiler says about each. Which messages point at the actual mistake, and which point somewhere nearby?
2. Write a program that prints `sizeof` for `short`, `long`, `float`, `long double`, `std::size_t`, `std::string`, and `std::vector<int>`. The last two results are small and do not depend on how much the string or vector holds. Explain why, in terms of where their contents live.
3. Without running it, predict the output of this program. Then run it and check.

```
void bump(int a, int& b, int* c) { a += 1; b += 1; *c += 1; }

int main() {
    int x = 10, y = 20, z = 30;
    bump(x, y, &z);
    std::cout << x << ' ' << y << ' ' << z << '\n';
}
```

4. Write a function `void swap_ints(int& a, int& b)` that exchanges the values of its two arguments. Then write `void swap_ints(int* a, int* b)` that does the same through pointers. How do the two calls look different at the call site? Which version makes it more obvious to a reader of the *calling* code that the arguments may be modified?
5. Given `int a[6] = {5, 10, 15, 20, 25, 30};` and `int* p = a + 2;`, give the value of each expression, or say that it is undefined: `*p`, `p[1]`, `*(p - 2)`, `p[-1]`, `*(a + 6)`, `p - a`, `&a[4] - p`.

6. Draw the stack and heap, by hand, at the moment just before `make_array` returns in the `lifetime.cpp` example when it is called as `make_array(3, 7)`. Then draw them again just after it returns. Which boxes disappeared?
7. Each of these functions has a lifetime or ownership bug. Name the bug (leak, dangling, double free, or out-of-bounds) and say how to fix it.

```
int* f1() { int x = 5; return &x; }

void f2() { int* p = new int[10]; p = new int[20]; delete[] p; }

void f3() { int* p = new int{1}; int* q = p; delete p; delete q; }

int f4() { int* p = new int[4]{1, 2, 3, 4}; int s = p[4]; delete[] p;
↪  return s; }
```

8. Compile each program in `code/ch01/bugs/` twice, once with only `-std=c++20` and once with the full set of flags, and run both versions. Make a table: for each program, what did the plain build do, and what did the sanitized build report? Was the plain build's behavior the same every time you ran it?
9. Write a function `std::vector<int> evens(const std::vector<int>& v)` that returns a new vector containing only the even elements of `v`, in order. Why is it safe to return a `vector` by value when it was not safe to return a reference to a local variable?
10. Write a function `bool is_palindrome(const std::string& s)` that reports whether `s` reads the same forwards and backwards. Use two indices that start at opposite ends and move toward each other. What should it return for the empty string, and why?
11. The expression `v.size() - 1` is dangerous when `v` is empty. Explain what value it has and why, and rewrite the loop `for (std::size_t i = 0; i <= v.size() - 1; ++i)` so that it is correct for every vector, including the empty one.
12. **With a model, as critic.** Write a function that reverses a `std::vector<int>` in place, and test it yourself on vectors of length 0, 1, 2, and 5. *Then* give your function to a language model with the prompt: "Do not rewrite this. List any inputs for which it misbehaves, and any C++ style problems." Evaluate each of its claims: which were correct, which were wrong, and which were matters of taste?
13. **Lab.** Write a program `histogram` that reads whole numbers between 0 and 100 from standard input and prints a text histogram with one row per range of ten (0–9, 10–19, and so on), using one `*` per value in the range. Store the counts in a `std::vector<int>`. Decide what the program should do with a number outside 0–100 and with the value 100 itself, and document your decision in a comment. The program must run cleanly under both sanitizers on an input file of your own devising that includes those edge cases.

Chapter notes

This chapter compresses a great deal of C++ into a few pages. For a fuller and gentler introduction to the language by its designer, see Stroustrup's *Programming: Principles and*

Practice Using C++ [56]; for a brisk overview aimed at people who already program, his *A Tour of C++* [55]. The reference site cppreference.com [17] is the place to look up any library function or language rule, and is more reliable than a model's recollection of the same.

The picture of memory given here, a flat row of bytes with a stack and a heap, is a simplification of what the operating system and hardware provide. Bryant and O'Hallaron's *Computer Systems: A Programmer's Perspective* [11] gives the real story, and is the natural next book if this chapter made you curious. AddressSanitizer is described by its authors in Serebryany et al. [51]. The advice to prefer `vector` and `string` and to avoid naked `new` and `delete` follows the C++ Core Guidelines [57], which are worth browsing as a statement of what experienced C++ programmers consider good practice.

Answers to check-yourself questions

1.1 Python prints `4`. C++ prints `3 4`. In Python (and Java, and JavaScript), `b = a` makes `b` a second name for the *same* list, so appending through `b` changes what `a` sees. In C++, `b = a` makes a *new* vector with its own copy of the three elements; `a` never hears about the fourth.

1.2 It takes `v` by value, so it zeroes a private copy and then throws the copy away. The caller's vector is untouched. The parameter should be `std::vector<int>& v`. The compiler will not warn you: modifying your own copy is perfectly legal, merely useless.

1.3 If `v` is empty there is no `v[0]`, and reading it is an out-of-bounds access: undefined behavior. `main` checks `data.empty()` and returns before calling `smallest`. A more defensive design would have `smallest` itself check, or state its requirement in a comment. Requirements like "must not be called on an empty vector" are called *preconditions*, and we will be much more deliberate about them starting in Chapter 3.

CHAPTER 2

What Does It Cost?

WEEK 2 OF 10

THIS WEEK

By the end of this week you should be able to:

- count the basic steps a piece of code performs as a function of its input size, in the best and worst cases;
- state the definitions of O, Ω, and Θ, and use them correctly;
- determine the growth rate of loops, including nested loops and loops that halve;
- write binary search from its invariant, and explain why it takes about $\log_2 n$ steps;
- time a program honestly, estimate its growth rate from a doubling experiment, and say why two programs with the same Big-O can differ by a factor of thirty.

I am thinking of a whole number between 1 and 1,000,000. After each guess I will tell you whether my number is higher, lower, or equal.

CHECK YOURSELF 2.1

How many guesses do you need, at most, to be certain of finding the number? Decide on a strategy and an actual figure before reading on.

A million guesses or twenty: same problem, same answer, and a difference in cost of a factor of fifty thousand. That gap had nothing to do with typing speed or processor speed. It came from the method, and from one fact about how the data was arranged: the numbers were in order, so a single comparison could discard half of them.

This chapter is about the third of the book's four questions. We need a way to talk about what an operation costs that does not depend on whose laptop it runs on, that lets us compare two approaches before writing either, and that tells us what will happen when the data gets a thousand times bigger. Then, having built that tool, we will take out a stopwatch and find out where it is wrong.

2.1 Counting steps

Here is the guess-in-order strategy as code: search a vector for a value by looking at every element in turn.

code/ch02/search.hpp

```cpp
// Returns the index of the first element equal to target,
// or v.size() if there is none.
inline std::size_t linear_search(const std::vector<int>& v, int target) {
    for (std::size_t i = 0; i < v.size(); ++i) {
        if (v[i] == target) return i;
    }
    return v.size();
}
```

The function returns the index where it found the target. If the target is absent it returns `v.size()`, which cannot be a valid index and so serves as "not found." (The standard library uses the same convention, as we will see.)

How long does `linear_search` take? In seconds, the question has no good answer: it depends on the machine, the compiler, and what else the computer is doing. So we ask a better question. How many *steps* does it take, as a function of the size of the input?

By "step" we mean any operation that takes a fixed amount of time regardless of the data: comparing two integers, adding, assigning, following a pointer, indexing an array (recall from Section 1.4 that indexing is one multiplication and one addition). The loop above does a handful of such steps per iteration: compare `i` with the size, index `v`, compare with the target, increment `i`. Whether a "handful" is four or seven turns out not to matter, so we simplify further and count just one representative operation, the comparison of an element against the target.

Let n be the number of elements. Then the number of comparisons depends on where the target is.

- **Best case.** The target is the first element: 1 comparison.
- **Worst case.** The target is last, or absent: n comparisons.
- **Average case.** If the target is present and equally likely to be anywhere, the search examines $(n + 1)/2$ elements on average.

We will usually care most about the worst case. It is a guarantee: the operation never takes longer than this. It is also frequently the common case in practice, because searching for something that is not there is hardly unusual. The average case matters too, but it requires an assumption about what inputs are typical, and such assumptions have a way of being wrong. The best case is rarely interesting. Any algorithm can get lucky.

So: in the worst case, linear search takes a number of steps proportional to n. Double the data and you double the work. That simple statement is already the kind of thing we want to be able to say about every operation on every data structure.

2.2 Growth rates

The reason to count steps as a function of n is to learn how the cost *grows*. Here is what several common growth rates mean in practice, on a machine that performs a billion steps per second.

n	$\log_2 n$	n	$n \log_2 n$	n^2	2^n
10	3 ns	10 ns	33 ns	100 ns	1 μs
100	7 ns	100 ns	660 ns	10 μs	4×10^{13} years
1,000	10 ns	1 μs	10 μs	1 ms	
1,000,000	20 ns	1 ms	20 ms	17 minutes	
1,000,000,000	30 ns	1 s	30 s	32 years	

Read down the columns. A logarithmic-time operation barely notices that the input grew from a thousand to a billion. A linear one goes from instantaneous to perceptible. A quadratic one goes from instantaneous to *thirty-two years*. And the exponential column runs out of universe almost at once. Figure 2.1 plots the same functions; in the web edition you can change the scale and see how quickly the faster-growing curves leave the picture.

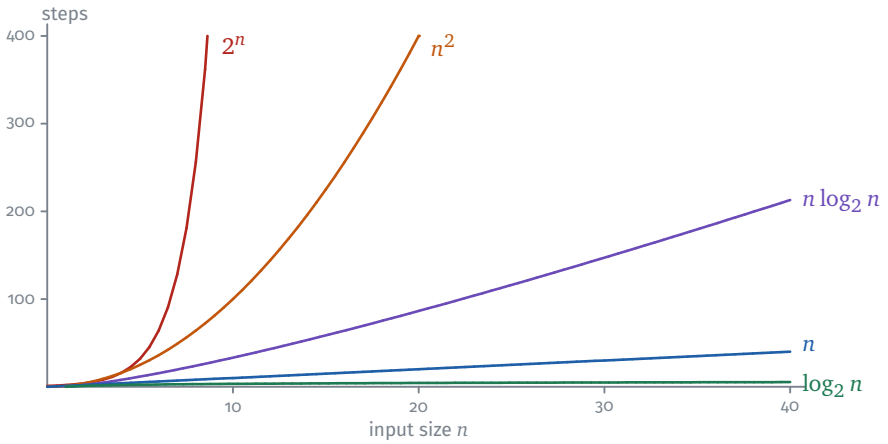


Figure 2.1 Five growth rates. On any scale large enough to matter, each curve eventually towers over the ones below it, regardless of constant factors. [Interactive in the web edition.]

Notice what the table did *not* need: the exact number of steps per loop iteration. If linear search took $7n$ steps and not n, the linear column would be seven times larger and the story would be exactly the same. The *shape* of the growth dominates everything else once n is large. That observation is the basis for the notation used throughout computer science to describe costs.

Big-O, and its relatives

We say that a function $f(n)$ is $O(g(n))$, read "big-oh of g," if, for large enough n, f is at most a constant multiple of g. Precisely:

$f(n)$ is $O(g(n))$ if there are constants $c > 0$ and n_0 such that $f(n) \le c \cdot g(n)$ for every $n \ge n_0$.

The two constants each excuse something. The multiplier c says that we do not care about constant factors. The threshold n_0 says that we do not care what happens for small inputs. Figure 2.2 shows the picture.

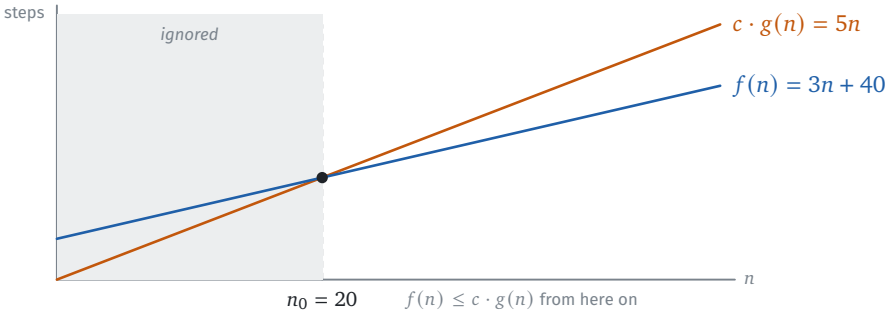


Figure 2.2 The definition of Big-O in a picture. The function $f(n) = 3n + 40$ is $O(n)$: with $c = 5$, the line $5n$ lies above f for every n beyond $n_0 = 20$. What happens to the left of n_0 is ignored.

For example, suppose a careful count shows that some function performs $f(n) = 3n^2 + 10n + 50$ steps. Then $f(n)$ is $O(n^2)$. To prove it, find the constants: for $n \ge 10$ we have $10n \le n^2$ and $50 \le n^2$, so $f(n) \le 3n^2 + n^2 + n^2 = 5n^2$. Take $c = 5$ and $n_0 = 10$.

In practice nobody hunts for constants. Two rules of thumb get you there directly: **keep only the fastest-growing term, and drop its coefficient.** So $3n^2 + 10n + 50$ becomes n^2; $n \log n + 100n$ becomes $n \log n$; 5 becomes 1, and we write $O(1)$ for anything that takes a bounded number of steps regardless of n, called **constant time**.

Big-O is an *upper bound*, and that leads to a subtlety. Linear search is $O(n)$. It is also, according to the definition, $O(n^2)$, and $O(2^n)$, in the same way that a person who is under two meters tall is also under three meters tall. True, but not informative. Two companion notations tighten things up.

- $f(n)$ is $\Omega(g(n))$ ("big-omega") if f is *at least* a constant multiple of g for large n: a lower bound.
- $f(n)$ is $\Theta(g(n))$ ("big-theta") if it is both $O(g(n))$ and $\Omega(g(n))$: f grows at the *same rate* as g.

The worst-case cost of linear search is $\Theta(n)$: proportional to n, no more and no less. When programmers say "this is $O(n)$" they almost always mean $\Theta(n)$, and this book will follow that loose convention when no confusion can result, using Θ when the distinction matters.

PITFALL

Three misreadings of Big-O are common enough to be worth naming.

"$O(n)$ means the worst case." No. Big-O describes a function; best, worst, and average case describe *which* function you are talking about. Linear search's best case is $\Theta(1)$ and its worst case is $\Theta(n)$. Both statements use the notation; they bound different things.

"An $O(n)$ algorithm is faster than an $O(n^2)$ one." Only for large enough n. The constants that Big-O discards are real. An algorithm that takes $1000n$ steps loses to one that takes n^2 steps for every n below a thousand. Several real libraries switch to a "slow" quadratic sort when the input is small, for exactly this reason (Chapter 8).

> *"Big-O tells you the running time."* It tells you how the running time *scales*. Two programs with the same Big-O can differ in speed by an order of magnitude, as the end of this chapter will demonstrate.

The growth rates that turn up constantly have names.

Notation	Name	Typical source
$\Theta(1)$	constant	indexing an array; arithmetic
$\Theta(\log n)$	logarithmic	repeatedly halving: binary search
$\Theta(n)$	linear	one pass over the data
$\Theta(n \log n)$	"n log n," linearithmic	good sorting algorithms
$\Theta(n^2)$	quadratic	all pairs; two nested loops
$\Theta(2^n)$	exponential	trying every subset

A note on logarithms, since they are about to be everywhere. $\log_2 n$ is the number of times you can halve n before reaching 1, or equivalently the power to which 2 must be raised to give n. The base does not matter inside Big-O, because logarithms to different bases differ only by a constant factor ($\log_{10} n = \log_2 n / \log_2 10$), so we write simply $O(\log n)$. Outside Big-O, in this book, log with no base means base 2. Appendix B reviews the handful of facts about logarithms and sums that the book uses.

2.3 Analyzing code

With practice you can read the growth rate of most code straight off its structure. A few patterns cover nearly everything in this course.

Statements in sequence add. A $\Theta(n)$ loop followed by another $\Theta(n)$ loop is $\Theta(n)$ in total. A $\Theta(n)$ loop followed by a $\Theta(n^2)$ loop is $\Theta(n^2)$: the larger term wins.

Nested loops multiply. If an outer loop runs n times and its body is a loop that runs n times, the innermost statement executes n^2 times.

A loop whose inner bound depends on the outer variable needs a sum. This function checks whether any value appears twice, by comparing every pair.

code/ch02/analysis.cpp

```
bool has_duplicate(const std::vector<int>& v) {
    for (std::size_t i = 0; i < v.size(); ++i) {
        for (std::size_t j = i + 1; j < v.size(); ++j) {
            if (v[i] == v[j]) return true;
        }
    }
    return false;
}
```

In the worst case, with no duplicates, the inner loop runs $n - 1$ times when $i = 0$, then $n - 2$ times, and so on down to zero. The total number of comparisons is

$$(n-1) + (n-2) + \cdots + 1 + 0 = \frac{n(n-1)}{2}.$$

That sum, of the first so-many whole numbers, appears so often in the analysis of algorithms that you should recognize it on sight. It works out to $\frac{1}{2}n^2 - \frac{1}{2}n$, which is $\Theta(n^2)$. Comparing all pairs is quadratic even though each pair is examined only once.

A loop that halves runs $\log n$ times.

code/ch02/analysis.cpp

```
int halvings(int n) {
    int steps = 0;
    while (n > 1) {
        n = n / 2;
        ++steps;
    }
    return steps;
}
```

Starting from n and halving until you reach 1 takes $\lfloor \log_2 n \rfloor$ steps: about 10 for a thousand, 20 for a million, 30 for a billion. Whenever an algorithm can throw away a constant *fraction* of its remaining work at each step, a logarithm appears.

> **CHECK YOURSELF 2.2**
>
> What is the growth rate of this fragment, in terms of n?
>
> ```
> int count = 0;
> for (int i = 1; i < n; i = i * 2) {
>     for (int j = 0; j < n; ++j) {
>         ++count;
>     }
> }
> ```

An example with two answers: insertion sort

Sorting is the subject of Chapter 8, but one simple sorting algorithm is worth meeting now, because its analysis shows why we distinguish best and worst cases. **Insertion sort** is how most people sort a hand of playing cards: take the cards one at a time, and slide each into its proper place among the cards already in order.

code/ch02/analysis.cpp

```
void insertion_sort(std::vector<int>& v) {
    for (std::size_t i = 1; i < v.size(); ++i) {
        int x = v[i];                    // next element to place
        std::size_t j = i;
        while (j > 0 && v[j - 1] > x) {  // slide larger elements right
            v[j] = v[j - 1];
            --j;
        }
        v[j] = x;                        // drop x into the gap
    }
```

```
}
```

The outer loop always runs $n - 1$ times. The inner loop is where the cost varies.

If the input is *already sorted*, then `v[j - 1] > x` is false immediately, every time. The inner loop body never runs. Total: about n comparisons, $\Theta(n)$.

If the input is in *reverse order*, each new element is smaller than everything before it and must be slid all the way to the front: 1 shift, then 2, then 3, up to $n - 1$. That is the same sum as before, $n(n-1)/2$ shifts, $\Theta(n^2)$.

So insertion sort is linear in the best case and quadratic in the worst, and on a random input it does about half the worst-case work, which is still quadratic. An algorithm like this, which does less work on inputs that are closer to sorted, is called *adaptive*, and that property will make insertion sort useful later despite its poor worst case.

ASK AN LLM

Reading growth rates off loops is a skill that comes from repetition, and a model makes a tireless drill partner. Use it as an examiner:

> *I am practicing asymptotic analysis. Show me one short C++ loop nest at a time and ask me for its running time in Theta notation, in terms of n. Do not tell me the answer until I have given mine and my reasoning. Start easy and get harder. Include some where the inner loop's bound depends on the outer loop's variable, and some where a variable is multiplied or divided each time through.*

Answer each one on paper first. When the model says you are wrong, do not simply accept it: make it show the sum.

BY HAND FIRST

When you want to know the growth rate of a loop *you* wrote, work it out yourself before asking a model. For textbook shapes the model will be right. For unusual shapes it is easy to be led astray by a confident answer. Consider:

```
for (int i = 1; i <= n; ++i)
    for (int j = i; j <= n; j += i)
        ++count;
```

Two nested loops that each "go up to n" look quadratic, and that is a tempting answer for person and model alike. But the inner loop takes steps of size `i`, so it runs about n/i times, and the total is $n/1 + n/2 + n/3 + \cdots + n/n$, which is approximately $n \ln n$. The honest way to settle any such question is the one in Section 2.5: add a counter, run it for several values of n, and look at how the count grows.

Space is a cost too

Everything above was about time, but the same notation describes memory. `linear_search` uses a couple of extra variables no matter how long the vector is: $O(1)$ extra space. A function that builds a copy of its input uses $\Theta(n)$ extra space. Time and space can often be traded against each other, and several data structures in this book (hash tables most of all) are, at bottom, schemes for spending memory to buy speed.

2.4 Binary search

Now for the twenty-guesses strategy. It rests on a precondition: **the data must be sorted.** Given that, a single comparison against the middle element tells you which half the target must be in, and the other half can be discarded.

The idea is simple. Getting the details right is notoriously not. Jon Bentley, who taught the algorithm to professional programmers for years, reported that when he asked them to write it, given ample time, about nine in ten produced a version with a bug [6]. He adds, drawing on Knuth's history of the algorithm [34], that although the first binary search was published in 1946, it was 1962 before a published version was free of bugs. The trouble is always at the boundaries: an off-by-one error that skips an element, or a range that fails to shrink and loops forever.

The way to get it right is to decide, before writing any code, exactly what the variables *mean*, and then to make every line respect that meaning. A statement about the variables that is true every time execution reaches the top of a loop is called a **loop invariant**. Here is ours:

> *If the target is anywhere in the vector, it is in the range of indices from* `lo` *up to but not including* `hi`.

The range that includes its lower end and excludes its upper end is called a **half-open range**, written $[lo, hi)$. Half-open ranges are the convention everywhere in C++ (recall "one past the end" from Section 1.4), and they make the arithmetic clean: the number of elements in the range is `hi - lo`, and the range is empty exactly when `lo == hi`.

code/ch02/search.hpp

```
// Precondition: v is sorted in increasing order.
// Returns the index of an element equal to target, or v.size() if none.
inline std::size_t binary_search(const std::vector<int>& v, int target) {
    std::size_t lo = 0;
    std::size_t hi = v.size();
    // Invariant: if target is anywhere in v, it is in v[lo..hi),
    // the half-open range that includes lo and excludes hi.
    while (lo < hi) {
        std::size_t mid = lo + (hi - lo) / 2;
        if (v[mid] == target) {
            return mid;
        } else if (v[mid] < target) {
            lo = mid + 1;   // everything up to and including mid is too small
        } else {
            hi = mid;       // mid and everything after it is too big
        }
    }
    return v.size();        // the range is empty, so target is not here
}
```

Check each line against the invariant.

At the start, `lo` is 0 and `hi` is `v.size()`, so the range is the whole vector, and the invariant holds trivially.

In the loop, `mid` is some index in the range. If `v[mid]` is the target, we are done. If `v[mid]` is too small then, because the vector is sorted, so is everything before it; the target can only be in $[mid + 1, hi)$, so we set `lo = mid + 1`. If `v[mid]` is too big, the target can

only be in $[lo, mid)$, so we set `hi = mid`. Note the asymmetry: `mid + 1` in one case and plain `mid` in the other. It comes directly from the range being half-open, and it is where most broken versions go wrong.

At the end, the loop exits when `lo == hi`: the range is empty. The invariant says the target, if present, is in an empty range. So it is not present.

One more thing has to be true: the loop must actually stop. It does, because `mid` is always a valid index within the range, and both branches produce a strictly smaller range.

Figure 2.3 steps through a search and shows the invariant as a shaded region.

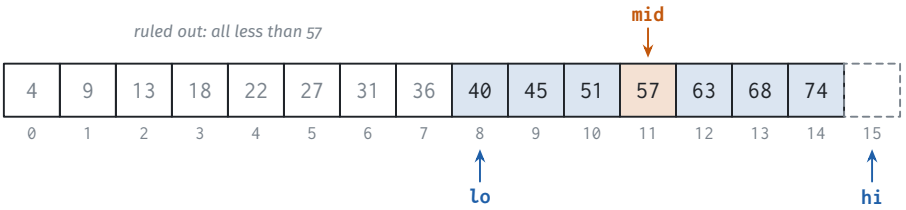


Figure 2.3 Binary search for 57 in a sorted array of fifteen elements. The shaded cells are the half-open range `[lo, hi)` that might still contain the target. In the web edition, pick a target and step through the search; switch to linear search to compare the number of comparisons. [Interactive in the web edition.]

> **PITFALL**
>
> The midpoint is computed as `lo + (hi - lo) / 2` and not as the more obvious `(lo + hi) / 2`. When indices are stored in a 32-bit `int` and the array has more than about a billion elements, `lo + hi` can exceed the largest `int` and overflow. This is not a hypothetical: the binary search in Java's standard library had exactly this bug for nine years before anyone noticed [9]. With 64-bit `std::size_t` indices the sum cannot realistically overflow, but the safe form costs nothing and is the idiom you should write by habit.

How fast?

Each pass through the loop at least halves the size of the range. A range of n elements can be halved only about $\log_2 n$ times before it is empty; the exact bound is $\lfloor \log_2 n \rfloor + 1$ passes. Binary search is $\Theta(\log n)$ in the worst case.

For a million elements that is 20 comparisons, against a million for linear search. For a billion elements, 30. The benefit of a logarithm is hard to overstate: you can double the size of the data and pay for it with *one* additional comparison.

A more useful version

The function above answers "is it there, and where?" A small change yields something more broadly useful: the position of the *first element that is not less than the target*. If the target is present, that is its first occurrence. If it is absent, that is where it would have to be inserted to keep the vector sorted.

```
// Precondition: v is sorted in increasing order.
// Returns the first index i with v[i] >= target, or v.size() if every
// element is smaller. This is where target is, or where it would go.
inline std::size_t lower_bound_index(const std::vector<int>& v, int target) {
    std::size_t lo = 0;
    std::size_t hi = v.size();
    // Invariant: everything before lo is < target;
    //            everything at or after hi is >= target.
    while (lo < hi) {
        std::size_t mid = lo + (hi - lo) / 2;
        if (v[mid] < target) {
            lo = mid + 1;
        } else {
            hi = mid;
        }
    }
    return lo;
}
```

The loop is even simpler, having two cases instead of three, and it has a new invariant, given in the comment. When the loop ends, `lo == hi`, and the invariant then says that everything before `lo` is smaller than the target and everything from `lo` onward is not. That is exactly the position we wanted.

The standard library provides this as `std::lower_bound` (and a sibling, `std::upper_bound`, that finds the first element *greater* than the target). In real programs, use those. They are correct, and they work on any sorted range.

> **BY HAND FIRST**
>
> Close the book and write binary search from scratch, starting from the invariant. Do it more than once this week, on different days. This is one of a small number of algorithms that you should be able to produce from understanding, without memory and without a model, because the reasoning pattern (state an invariant; make every line preserve it; check that the loop makes progress) is how every data structure in the rest of the book is built and debugged. If you let a model write it you will have a binary search, which you did not need, since the standard library already has one. What you need is the ability to think this way.

How do you know it is right?

Given the algorithm's history, your own confidence is not evidence. The test file for this chapter checks a few cases chosen by hand (an empty vector, a single element, targets at both ends, targets that are absent) and then does something more powerful.

code/ch02/search_test.cpp

```
// Then thousands of random cases, checked against the standard library.
std::mt19937 rng(270);
for (int trial = 0; trial < 5000; ++trial) {
    std::size_t n = rng() % 40;
    std::vector<int> a(n);
    for (int& x : a) x = static_cast<int>(rng() % 50);
```

```
    std::sort(a.begin(), a.end());
    int target = static_cast<int>(rng() % 52) - 1;

    auto it = std::lower_bound(a.begin(), a.end(), target);
    auto expected = static_cast<std::size_t>(it - a.begin());
    assert(lower_bound_index(a, target) == expected);

    bool present = std::binary_search(a.begin(), a.end(), target);
    std::size_t where = binary_search(a, target);
    assert(present == (where != a.size()));
    if (present) assert(a[where] == target);
}
```

It generates thousands of small random sorted vectors and random targets, and for each one compares our function's answer with the answer from the standard library, which we trust. Any disagreement stops the program. This technique is called **differential testing**: checking an implementation against a reference, on many generated inputs [39]. It finds boundary bugs that hand-picked examples miss, because random small inputs hit the awkward cases (empty ranges, duplicates, targets off either end) far more often than a person would think to. Small sizes are deliberate; bugs in this kind of code live at sizes zero through three.

We will use differential testing in every chapter from here on. It is the single most effective answer this book has to the question *how do you know?*, and it works just as well on code that a model wrote as on code that you wrote.

2.5 Measuring

Analysis predicts. Measurement checks the prediction, and sometimes overturns it. You should be able to do both.

The standard library's `<chrono>` header provides clocks. Here is the small timing function this book uses.

code/ch02/bench/timing.cpp

```
// Runs `work` once and returns how long it took, in seconds.
template <typename Work>
double seconds(Work work) {
    auto start = std::chrono::steady_clock::now();
    work();
    auto stop = std::chrono::steady_clock::now();
    return std::chrono::duration<double>(stop - start).count();
}
```

It takes a piece of work, packaged as a function object (in practice a *lambda*, written `[&] { ... }`), runs it, and returns the elapsed time. `steady_clock` is the right clock for measuring intervals because it never jumps backward or gets adjusted.

A single timing tells you little. What you want is the growth rate, and the cleanest way to see it is a **doubling experiment**: time the code at size n, then $2n$, then $4n$, and look at the *ratio* of each time to the one before.

code/ch02/bench/timing.cpp

```
std::printf("%10s %12s %8s\n", "n", "seconds", "ratio");
double previous = 0.0;
for (std::size_t n = 4000; n <= 128000; n *= 2) {
    std::vector<int> v(n);
    for (std::size_t i = 0; i < n; ++i) v[i] = static_cast<int>(i);

    bool answer = false;
    double t = seconds([&] { answer = has_duplicate(v); });

    std::printf("%10zu %12.4f", n, t);
    if (previous > 0.0) std::printf(" %8.2f", t / previous);
    std::printf("%s\n", answer ? "  (?!)" : "");
    previous = t;
}
```

Here is what that program printed for `has_duplicate`, on the laptop this book was written on (a 2023 Intel Core i9-13900H, compiled with GCC 13 at `-O2`).

n	seconds	ratio to previous
4,000	0.0031	
8,000	0.0077	2.5
16,000	0.0284	3.7
32,000	0.1033	3.6
64,000	0.4246	4.1
128,000	1.6728	3.9

Doubling the input multiplies the time by about four. That is the signature of quadratic growth, since $(2n)^2 = 4n^2$. In general, if the time grows like n^b, then doubling n multiplies the time by 2^b: a ratio near 2 means linear, near 4 quadratic, near 8 cubic, and a ratio that hovers just above 1 means logarithmic or constant. The first ratios in the table are off because the runs are so short that noise and processor warm-up dominate; the pattern settles as the times get longer. With one more line of arithmetic you can extrapolate: at this rate a million elements would take about $1.67 \times (1{,}000{,}000/128{,}000)^2 \approx 100$ seconds.

And here are linear search and binary search, side by side, as average nanoseconds per search. (The binary search timed is `lower_bound_index`.)

n	linear search	binary search
1,000	165 ns	38 ns
10,000	1,600 ns	51 ns
100,000	16,200 ns	67 ns
1,000,000	162,000 ns	100 ns
10,000,000	2,190,000 ns	214 ns
100,000,000	20,200,000 ns	402 ns

The linear column grows tenfold with each row, as predicted. The binary column barely moves: a hundred thousand times more data, and the search takes about ten times as long. At a hundred million elements, binary search is fifty thousand times faster.

Look more closely at that second column, though. Theory says each tenfold increase in n should add about 3.3 comparisons, a steady, gentle climb. From a thousand to a million elements that is roughly what happens. But from a million to a hundred million, $\log_2 n$ grows by only a third, while the measured time *quadruples*. Something that the step-counting model does not capture has started to matter. Hold that thought for one section.

PITFALL

Timing code is easy to do badly. Four rules will keep you out of the worst trouble.

1. **Turn on optimization and turn off sanitizers.** Compile benchmarks with `-O2` and without `-fsanitize`. Unoptimized code can be ten times slower in ways that have nothing to do with your algorithm, and sanitizers add their own overhead. (Run your *tests* with sanitizers. Run your *timings* without.)
2. **Use the result.** If a program computes something and never uses it, the optimizer may delete the computation, and you will proudly measure an empty loop. The programs here print a checksum of their results for that reason.
3. **Make the run long enough.** Timings shorter than a few milliseconds are mostly noise. Use a bigger input or repeat the work many times.
4. **Run it more than once.** Other programs, power management, and the processor's own heat all perturb timings. Look for a pattern across runs and sizes, never at one number.

ASK AN LLM

A model has never run your program on your machine, so any specific timing it offers ("binary search will be about 400 times faster") is a guess dressed as a fact. It can, however, review a benchmark you have written:

> *Here is a C++ micro-benchmark I wrote. Without rewriting it, list the ways its measurements could be misleading: dead-code elimination, too-short runs, cache warm-up, anything else you see.*

Then decide for yourself which of its objections apply. Performance questions are settled by measurement, on the machine that matters, and by nothing else.

2.6 When the model lies

The step-counting model made a quiet assumption: that every basic step costs about the same. For arithmetic that is close enough to true. For memory access it is false, and the way in which it is false will shape half the design decisions in this book.

Consider adding up every element of a square grid of numbers. We store the grid the way C++ programs usually do, as one long vector holding the first row, then the second row, and so on, so that the element in row r and column c of an $n \times n$ grid is at index $r \cdot n + c$. Here are two ways to compute the sum.

code/ch02/bench/rowcol.cpp

```
double by_rows = seconds([&] {
    for (std::size_t r = 0; r < n; ++r)
        for (std::size_t c = 0; c < n; ++c)
            sum_rows += grid[r * n + c];
});
```

code/ch02/bench/rowcol.cpp

```
double by_cols = seconds([&] {
    for (std::size_t c = 0; c < n; ++c)
        for (std::size_t r = 0; r < n; ++r)
            sum_cols += grid[r * n + c];
});
```

The two differ only in which loop is on the outside. Each visits every element exactly once and does one addition per element. By any step count they are identical: $\Theta(n^2)$ with the same constant.

> **CHECK YOURSELF 2.3**
>
> Will the two versions take the same time? If not, which will be faster, and by roughly how much: ten percent? A factor of two? More?

n	by rows	by columns	columns ÷ rows
1,024	0.0004 s	0.0027 s	7
2,048	0.0012 s	0.0095 s	8
4,096	0.0056 s	0.0755 s	13
8,192	0.0228 s	0.6936 s	30

The column-order version is between seven and thirty times slower, and the gap widens as the grid grows. Same algorithm, same Big-O, same number of additions.

The explanation is in the hardware. Main memory is slow compared with the processor: fetching a value from it takes on the order of a hundred nanoseconds, time enough for the processor to execute hundreds of instructions. To hide this, processors keep copies of recently used memory in small, fast **caches**, arranged in levels; the fastest can supply a value in about a nanosecond. Two aspects of how caches work matter to us.

First, caches do not fetch single bytes. They fetch **cache lines**, blocks of (typically) 64 consecutive bytes. When your program touches one `int`, the fifteen neighboring `int`s on the same line arrive with it and are ready, essentially free, if you ask for them next.

Second, processors watch for patterns. If a program is marching through memory in order, the hardware notices and begins fetching the lines ahead before they are requested. This is called **prefetching**.

Now look at Figure 2.4. The row-order loop walks straight through the vector from beginning to end. Every cache line it pays for delivers sixteen useful values, and the prefetcher keeps the next lines arriving on schedule. The column-order loop, by contrast,

takes a stride of n elements between consecutive accesses. For large n, every single access lands on a different cache line, far from the last one, and by the time the loop returns to the neighborhood of an earlier access, that line has long since been evicted to make room for others. Nearly every access is a **cache miss**, a full-price trip to main memory. (The compiler can also use the processor's vector instructions on the simple row-order loop, which accounts for some of the difference. That, too, is a benefit of walking memory in order.)

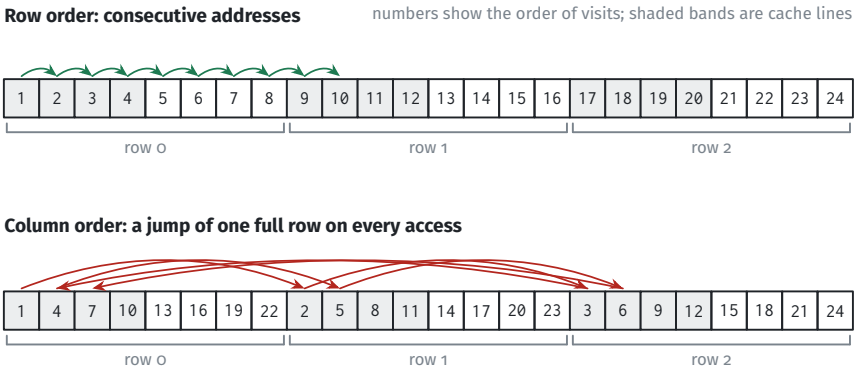


Figure 2.4 The same grid, stored row after row in one block of memory, visited in two orders. Row order (top) walks through memory consecutively, so each 64-byte cache line that is fetched serves sixteen accesses in a row. Column order (bottom) jumps a whole row's width between accesses and lands on a different cache line every time.

The property that the fast loop has and the slow loop lacks is called **locality**: its memory accesses are close together, and in a predictable order. This also explains the mystery from the last section. A binary search over a hundred million elements hops all over a 400-megabyte array; almost every probe is a cache miss. The count of comparisons grew gently, as the theory said. The cost of each one did not stay fixed.

Here, then, is the working attitude for the rest of the book.

Big-O comes first. No amount of cache-friendliness will rescue a quadratic algorithm from a linearithmic one when n is large. The table at the start of Section 2.2 is still the most important table in the chapter.

Among structures with the same Big-O, layout often decides. A structure that keeps its data contiguous and visits it in order can be many times faster than one that scatters its data across memory and reaches it through pointers, even when a step count rates them equal. This is the first of the four questions, *where does it live?*, turning up inside the third. It is the main reason that the array-based structures of the next chapter are the default choice in modern software, and it is the lens through which Chapter 4 will look at linked lists.

When it matters, measure. The theory tells you what to expect; the stopwatch tells you what happened. A professional uses both.

THE FOUR QUESTIONS **A sorted array, with binary search**

Where does it live? One contiguous block. Any element can be reached by index in constant time, which is what lets binary search jump to the middle.

What does it promise? The elements are in increasing order. That invariant is what makes a single comparison able to rule out half the data. The search itself is built around a loop invariant on the half-open range `[lo, hi)`.

What does it cost? Searching: $\Theta(\log n)$ comparisons, against $\Theta(n)$ for an unsorted array. But keeping the array sorted is not free: inserting a new element means shifting everything after it, $\Theta(n)$. Much of this book is a search for structures that make *both* cheap.

How do you know? Differential testing against `std::lower_bound` on thousands of small random inputs, plus hand-chosen boundary cases. For costs: count operations, and run a doubling experiment.

Exercises

1. For each function, give the simplest $g(n)$ such that the function is $\Theta(g(n))$: (a) $5n^3 + 2n^2 + 100$; (b) $n \log n + 50n$; (c) 2^{10}; (d) $n(n+1)/2$; (e) $\log_2(n^3)$; (f) $n + \sqrt{n}$; (g) 2^{n+1}; (h) $(n+1)(n+2)(n+3)$.
2. Prove from the definition that $4n^2 + 20n + 7$ is $O(n^2)$ by exhibiting constants c and n_0. Then prove that it is *not* $O(n)$. (For the second part, suppose constants existed, and derive a contradiction for large n.)
3. Put these in order from slowest-growing to fastest-growing: n^2, $\log n$, $n!$, 2^n, $n \log n$, $\sqrt{n}$, n, 1, n^3, $\log \log n$.
4. Give the worst-case running time of each fragment in Θ notation, in terms of `n`. Show the sum where there is one.

```
// (a)
for (int i = 0; i < n; ++i)
    for (int j = 0; j < 10; ++j) ++count;
// (b)
for (int i = 0; i < n; ++i)
    for (int j = 0; j < i; ++j) ++count;
// (c)
for (int i = n; i > 0; i /= 2) ++count;
// (d)
for (int i = 0; i < n; ++i)
    for (int j = 1; j < n; j *= 2) ++count;
// (e)
for (int i = 1; i < n; i *= 2)
    for (int j = 0; j < i; ++j) ++count;
```

   Part (e) is the subtle one. Its answer is *not* $\Theta(n \log n)$.

5. Add a counter to the "harmonic" loop in the By hand first box of Section 2.3 and run it for $n = 1{,}000$, $10{,}000$, $100{,}000$, and $1{,}000{,}000$. Compare the counts with n^2, with $n \log_2 n$, and with $n \ln n$. Which fits?

6. Trace `binary_search` by hand on the vector `{3, 8, 12, 17, 21, 30, 44}` for targets 17, 3, 44, and 20. For each, list the values of `lo`, `hi`, and `mid` at every iteration.
7. Each of these changes to `binary_search` introduces a bug. For each, find a small input on which the function gives a wrong answer or never returns. (a) Replace `hi = mid` with `hi = mid - 1`. (b) Replace `lo = mid + 1` with `lo = mid`. (c) Replace `while (lo < hi)` with `while (lo <= hi)`.
8. Write `std::size_t upper_bound_index(const std::vector<int>& v, int target)`, returning the first index whose element is strictly greater than `target`. State its loop invariant in a comment. Then use it together with `lower_bound_index` to write a function that counts how many times `target` occurs in a sorted vector, in $O(\log n)$ time. Test both against the standard library using the differential-testing pattern from this chapter.
9. A sorted vector of n distinct integers has been *rotated*: some number of elements were moved from the front to the back, as in `{30, 44, 3, 8, 12, 17, 21}`. Write a function that finds the index of the smallest element in $O(\log n)$ time. What is your loop invariant?
10. Binary search works on anything with the right "false, false, ..., false, true, true, ..., true" shape, not only on arrays. Write `int integer_sqrt(int n)` that returns the largest integer r with $r^2 \le n$, using binary search on the range of possible answers. Take care that your arithmetic cannot overflow.
11. Run the row-versus-column experiment on your own machine. Do you get similar ratios to the table in Section 2.6? Then change the grid's element type from `int` to `char`, and separately to `double`. Predict before you measure: how should the element size affect the gap?
12. Insertion sort does about $n^2/4$ shifts on a random input. Write a doubling experiment that confirms the quadratic growth. Then time it on inputs that are *already sorted*, and on inputs that are sorted except for ten randomly chosen elements. What do you conclude about when insertion sort is a reasonable choice?
13. **With a model, as critic.** Write your own binary search *without looking at this chapter's code*, and test it. Then ask a model: "Here is my binary search. Without rewriting it, state the loop invariant it appears to maintain, and tell me whether every line preserves that invariant." Compare the invariant it states with the one you had in mind. If they differ, who is right?
14. **Lab.** You are given three functions (your instructor will supply them, or use `std::sort`, `has_duplicate`, and a function of your own) and must determine each one's growth rate *empirically*. Build a timing harness using `seconds` that runs a doubling experiment on each, prints a table of sizes, times, and ratios, and reports an estimated exponent $b = \log_2(\text{ratio})$. Your write-up should state, for each function, your estimate and how confident you are, and should identify at least one thing you did to make the measurements trustworthy.

Chapter notes

Asymptotic notation was brought into the analysis of algorithms largely by Knuth, whose *The Art of Computer Programming* [33, 34] remains the deepest treatment of the mathematics. For a thorough modern textbook account, with many more techniques than a ten-week course can cover, see Cormen, Leiserson, Rivest, and Stein [16]. Sedgewick and Wayne's *Algorithms* [48] puts unusual emphasis on doubling experiments and the scientific method, and influenced this chapter's approach to measurement.

Bentley's *Programming Pearls* [6] has a wonderful chapter on writing binary search correctly, organized around the invariant; the story of the overflow bug in Java's library is told by the engineer who wrote that code [9]. Differential testing was named and described by McKeeman [39].

On the memory hierarchy, the standard reference for programmers is Drepper's long article *What Every Programmer Should Know About Memory* [21]; Bryant and O'Hallaron [11] cover the same ground at textbook pace, and Hennessy and Patterson [27] explain why the hardware is built this way.

Answers to check-yourself questions

2.1 Twenty. Guess the middle of the range that remains; each answer rules out half of it. One million possibilities become 500,000, then 250,000, and after twenty halvings there is one left, because $2^{20} = 1{,}048{,}576$ is just over a million. If your strategy was to guess 1, 2, 3, ... you might need a million guesses.

2.2 $\Theta(n \log n)$. The outer loop variable doubles each time (1, 2, 4, 8, ...), so the outer loop runs about $\log_2 n$ times. The inner loop runs n times on every one of those passes. Multiply.

2.3 See the table that follows. Very few people guess the size of the effect the first time.

CHAPTER 3

Building a Vector

WEEK 3 OF 10

THIS WEEK

By the end of this week you should be able to:

- implement a growable array as a C++ class with a pointer, a size, and a capacity, and state its representation invariant;
- explain why doubling the capacity makes `push_back` cost $O(1)$ *amortized*, and why growing by a fixed amount does not;
- write a destructor, copy constructor, copy assignment, move constructor, and move assignment for a class that owns memory, and say when a class needs none of them;
- turn a class into a template, and give it iterators so that range-`for` and the standard algorithms work on it;
- test a container by checking its invariant and by comparing it against a trusted reference on random operations.

You have been using `std::vector` since the first day. It behaves like an array that grows: you call `push_back` as many times as you like and it always has room. But Chapter 1 established that an array is a block of memory of a fixed size, with other objects living on either side of it. There is no operation for making a block bigger where it stands.

CHECK YOURSELF 3.1

A vector is holding four `int`s in a heap block with room for exactly four. The program calls `push_back`. What has to happen? And if the program then calls `push_back` a million more times, how many times in total do you think an element gets copied from one place to another?

This week we build `std::vector`. Not because the world needs another one, but because it is the perfect first data structure. It is simple enough to hold in your head and it is the most widely used container in all of C++. And building it properly requires nearly every important idea in the language's approach to memory: classes that own resources, constructors and destructors, copying and moving, templates, and iterators. We will go through all four of the book's questions with care, because the pattern set here repeats in every later chapter.

3.1 Three numbers

A growable array needs a block of memory on the heap, and it needs to know two things about that block: how many elements it can hold, and how many it currently does hold. The trick that makes growth affordable is to let those two numbers differ. We allocate *more* room than we need, and keep track of how much is in use.

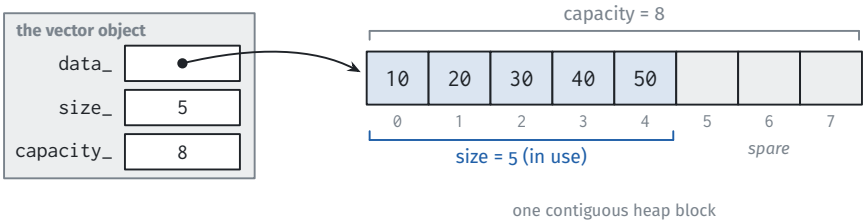


Figure 3.1 The layout of a vector. The object itself is three machine words, wherever the variable lives. The elements live in a separate heap block. `size` counts the elements in use; `capacity` is how many the block can hold before it must be replaced.

So the whole representation is three numbers, shown in Figure 3.1: a pointer to the block, the **size**, and the **capacity**. We will build a version that holds `int`s first, called `IntVec`, and generalize later.

code/ch03/intvec.hpp

```
int* data_ = nullptr;        // the heap block (null when capacity_ is 0)
std::size_t size_ = 0;       // how many elements are in use
std::size_t capacity_ = 0;   // how many the block has room for
```

If you are coming from Java or Python, the shape of a C++ class will be familiar: data members and member functions, some `public` (usable by anyone) and some `private` (usable only by the class's own functions). The trailing underscore on member names is a common convention, used throughout this book, to tell members from parameters and locals. The `= nullptr` and `= 0` are default initializers, so a freshly created `IntVec` starts out empty with no block at all.

The promise

Before writing any operations, write down what must always be true of those three members. This is the **representation invariant**: the conditions that every public member function may assume when it starts and must re-establish before it returns.

1. `size_ <= capacity_`.
2. If `capacity_` is zero, `data_` is null. Otherwise `data_` points to a heap block with room for exactly `capacity_` elements, and this object is the only one responsible for deleting it.
3. The vector's contents are `data_[0]` through `data_[size_ - 1]`. Slots from `size_` onward are spare room and mean nothing.

Everything else follows from these three sentences. If you are ever unsure whether a line of code in a member function is right, the question to ask is whether the invariant holds afterward.

Birth and death

code/ch03/intvec.hpp

```
IntVec() = default;              // an empty vector: no block yet

~IntVec() { delete[] data_; }    // runs automatically when an IntVec dies
```

A **constructor** is a function with the same name as the class; it runs when an object is created, and its job is to establish the invariant. Ours has nothing to do beyond the default initializers, so we ask the compiler to generate it with `= default`.

A **destructor** is named with a tilde. It runs *automatically* at the end of the object's lifetime: when a local variable goes out of scope, when a containing object is destroyed, or when `delete` is applied to a heap object. Ours gives the block back. (`delete[]` on a null pointer is defined to do nothing, so an empty `IntVec` is handled correctly.)

That one-line destructor is the most important line in the chapter. It is what Section 1.5 promised: heap memory tied to an object with automatic lifetime. A function that declares an `IntVec`, fills it, and returns, by any path, cannot leak the block, because the destructor runs no matter how the function exits. The general pattern, in which a constructor acquires a resource and the destructor releases it, goes by the unlovely name **RAII**, for "resource acquisition is initialization." It is the central idiom of C++, and the reason well-written modern C++ contains almost no explicit `delete`.

Access

code/ch03/intvec.hpp

```
std::size_t size() const { return size_; }
std::size_t capacity() const { return capacity_; }
bool empty() const { return size_ == 0; }

int& operator[](std::size_t i) {
    assert(i < size_);
    return data_[i];
}
const int& operator[](std::size_t i) const {
    assert(i < size_);
    return data_[i];
}
```

Three things here are new.

The `const` after a member function's parameter list is a promise that the function does not modify the object. Only such functions can be called on a `const IntVec&`, which is how you will usually receive a vector as a function parameter.

`operator[]` is how a class defines what square brackets mean. It returns a *reference* to the element, which is why `v[i] = 7` works: the assignment goes through the reference

into the block. There are two versions. The first is used on a modifiable vector and returns a reference you can assign through. The second is used on a `const` vector and returns a reference you can only read.

And each begins with an `assert` stating its **precondition**: the index must be in range. `std::vector`'s own `operator[]` does not check, for speed. Ours does, because we are learning, and an assertion that fires at the scene of the crime is worth an hour of debugging.

Growing

code/ch03/intvec.hpp

```
void push_back(int value) {
    if (size_ == capacity_) grow();
    data_[size_] = value;
    ++size_;
}

void pop_back() {
    assert(size_ > 0);
    --size_;
}
```

`push_back` is short. If there is spare room, the new element goes in the first unused slot and the size goes up by one. If the block is full, we first call a private helper to get a bigger block.

code/ch03/intvec.hpp

```
// Move to a block twice as big.
void grow() {
    std::size_t new_capacity = (capacity_ == 0) ? 1 : 2 * capacity_;
    int* bigger = new int[new_capacity];
    for (std::size_t i = 0; i < size_; ++i) bigger[i] = data_[i];
    delete[] data_;
    data_ = bigger;
    capacity_ = new_capacity;
}
```

`grow` does what the opening question said it must: allocate a larger block, copy the elements across, delete the old block, and update the pointer and the capacity. Check the invariant: afterward `data_` points to a block of `capacity_` slots that we own, the first `size_` of them hold the contents, and `size_ < capacity_`, so `push_back` can proceed.

The one real decision in `grow` is the first line. How much bigger should the new block be? The code doubles. The next section is about why.

3.2 Growing, and what it costs

Most calls to `push_back` do a constant amount of work: one comparison, one write, one increment. But a call that triggers `grow` copies every element: $\Theta(n)$ for a vector of size n. So the worst-case cost of `push_back` is $\Theta(n)$.

That sounds bad, and it is misleading. The expensive calls are rare, and they are rare in a very particular way. To see it, count the total work for a whole sequence of calls instead of the worst single call.

The program `growth.cpp` simulates n calls to `push_back` under four growth policies and counts how many element copies `grow` performs in total.

code/ch03/bench/growth.cpp

```
// Counts the element copies caused by n push_backs, where `next` maps
// the current capacity to the capacity after one growth step.
template <typename Policy>
std::uint64_t copies_for(std::uint64_t n, Policy next) {
    std::uint64_t copies = 0, size = 0, capacity = 0;
    for (std::uint64_t i = 0; i < n; ++i) {
        if (size == capacity) {
            copies += size;             // every element moves to the new block
            capacity = next(capacity);
        }
        ++size;
    }
    return copies;
}
```

n	grow by 1	grow by 1000	grow by half	double
1,000	499,500	0	2,137	1,023
10,000	49,995,000	45,000	24,284	16,383
100,000	4,999,950,000	4,950,000	276,521	131,071
1,000,000	499,999,500,000	499,500,000	2,099,753	1,048,575

Look at the last row. Growing by one slot at a time, a million appends cause half a *trillion* copies. Growing by a thousand slots at a time is a thousand times better and still terrible: half a billion. But doubling causes about one million copies for one million appends, roughly **one copy per element**. And look down the columns: when n goes up by ten, the first two columns go up by a hundred, and the last two go up by ten.

Why doubling works

Under doubling, the capacities are 1, 2, 4, 8, and so on, and `grow` is called when the size reaches each of those. The copies it makes are

$$1 + 2 + 4 + \cdots + 2^k,$$

where 2^k is the last capacity that filled up, which is less than n. A sum of doubling terms is always one less than the next term: $1 + 2 + 4 + \cdots + 2^k = 2^{k+1} - 1$. Since $2^k < n$, the total is less than $2n$.

So n calls to `push_back` do n element writes plus fewer than $2n$ copies: fewer than $3n$ steps altogether. The *average* cost per call, over the whole sequence, is less than 3. A constant.

Here is another way to see the same thing, which you may find more convincing. Imagine that every call to push_back is charged three coins. One coin pays for writing the new element. The other two are left sitting on that element, saved. Now suppose a block of capacity c has just filled. Half of its elements, $c/2$ of them, were added since the last time the vector grew, and each of those is carrying two saved coins: c coins in all, exactly enough to pay for copying all c elements into the new block. Every expensive operation has been paid for in advance by the cheap ones that led up to it.

Figure 3.2 shows the process; in the web edition you can push elements one at a time under each policy and watch the cost per operation and the running average.

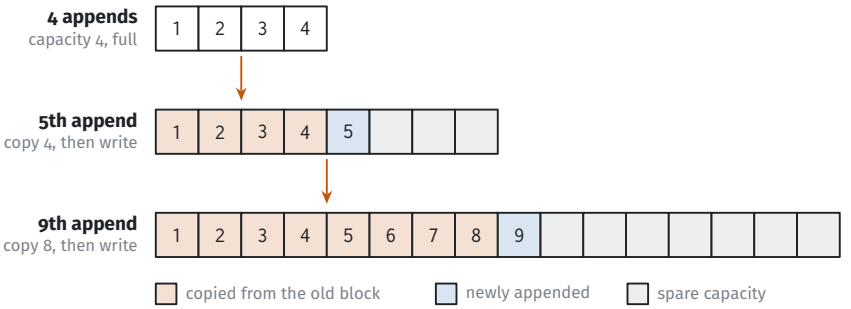


Figure 3.2 Appending to a vector that doubles. Most appends write one element. The appends that find the block full (the fifth and ninth here) also copy everything into a block twice as large. The total number of copies stays below twice the number of elements. [Interactive in the web edition.]

Amortized cost

This kind of reasoning has a name. We say that push_back has **amortized** cost $O(1)$: any sequence of n calls takes $O(n)$ time in total, even though an individual call may take much longer than the average [59]. The word comes from accounting, where a large occasional expense is spread over the period that benefits from it.

Be careful to distinguish amortized cost from average-case cost. An average-case bound depends on an assumption about the inputs, that they are random in some way, and can fail if the assumption does. An amortized bound involves no probability and no assumption. It is a *guarantee* about every possible sequence of operations: the total can never exceed the bound. The only thing given up is the guarantee about each individual operation.

Sometimes that matters. If your program must respond within a millisecond every single time (controlling a motor, say, or producing an audio sample) then a push_back that occasionally stops to copy a hundred million elements is unacceptable, however good the average. For most programs, total time is what counts, and amortized $O(1)$ is as good as $O(1)$.

Why growing by a constant fails

If the capacity instead grows by a fixed amount k, the copies are $k + 2k + 3k + \cdots$, about n/k terms in all. That is our old friend the sum of the first so-many whole numbers, and

it comes to about $n^2/2k$. The total work is quadratic. A larger k shrinks the constant but cannot change the shape, as the second column of the table shows.

The difference is easy to measure. Here are the times for n appends under the "grow by 1000" policy and the doubling policy, on the same laptop as before.

n	grow by 1000	double
250,000	0.0055 s	0.0007 s
500,000	0.0219 s	0.0011 s
1,000,000	0.0815 s	0.0020 s
2,000,000	0.3875 s	0.0039 s
4,000,000	2.4104 s	0.0090 s

Apply the doubling test from Section 2.5. The right-hand column roughly doubles from row to row: linear. The left-hand column roughly quadruples, and then some: quadratic. At four million elements the gap is a factor of 270.

The general principle is that growth must be **geometric**: each new capacity must be a constant *multiple* of the old one. Any multiplier greater than 1 gives amortized constant time. The choice of multiplier is a trade between time and space. A factor of 2 means fewer reallocations but up to half the block sitting idle. A factor of 1.5 wastes less space and copies somewhat more, as the table's third column shows.

IN THE WILD

Every mainstream language's growable array uses geometric growth; they differ only in the factor. The GNU and LLVM implementations of `std::vector` double. Microsoft's grows by a factor of 1.5, as does Java's `ArrayList`. Python's `list` over-allocates by about one-eighth. The smaller factors spend a little more time copying in exchange for less idle memory.

When you know in advance roughly how many elements are coming, you can skip the intermediate reallocations altogether by calling `v.reserve(n)` first. It is one of the cheapest optimizations in C++, and you will see it in careful code everywhere.

ASK AN LLM

Amortized analysis is an idea that often needs to be heard twice, in different words. Once you have read this section and can state the coin argument in your own words, try the *teach-back* pattern:

> *I am going to explain why push_back on a doubling dynamic array is amortized O(1). Do not correct me until I am finished. Then tell me precisely which steps of my explanation were wrong or hand-wavy, and ask me one follow-up question that tests whether I really understand it.*

Then explain it, out loud or in writing, without looking at the book. A good follow-up question is "what happens if the array also *shrinks* by half whenever it becomes half empty?" (The answer is in the exercises, and it is a trap.)

3.3 Copying

Our `IntVec` can be created, filled, and destroyed. Now consider this innocent line:

```
IntVec b = a;
```

Chapter 1 said that initialization copies. But we have not told the compiler how to copy an `IntVec`, so it does the only thing it can: it copies each member. `b.data_` gets the value of `a.data_`. `b.size_` gets `a.size_`. Member by member, bit for bit.

> **CHECK YOURSELF 3.2**
>
> After that member-by-member copy, draw the stack and heap. How many heap blocks are there? What happens when b and then a are destroyed?

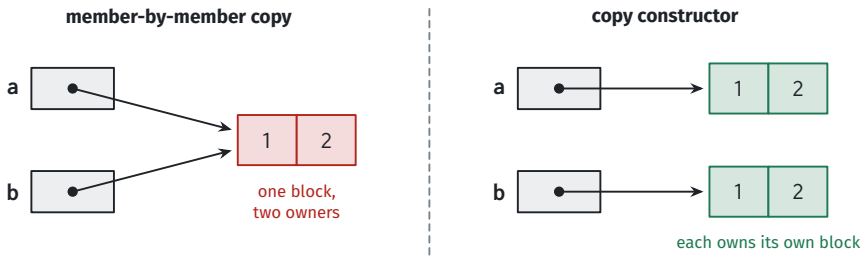


Figure 3.3 Left: a member-by-member (“shallow”) copy duplicates the pointer, so two objects believe they own one block. Right: a proper (“deep”) copy allocates a second block and copies the elements, so the invariant holds for both objects.

Here is a minimal class with exactly this flaw, and what the sanitizer says when one object is copied to another and both are destroyed.

code/ch03/bugs/shallow.cpp

```
// What goes wrong if a class that owns memory has no copy constructor.
#include <cstddef>

class Naive {
public:
    Naive() : data_(new int[4]{1, 2, 3, 4}) {}
    ~Naive() { delete[] data_; }
    // no copy constructor: the compiler supplies one that copies data_,
    // the POINTER, so two objects end up owning one block
private:
    int* data_;
};

int main() {
    Naive a;
    Naive b = a;    // b.data_ == a.data_
}                   // b's destructor frees the block; then a's frees it again
```

```
==26587==ERROR: AddressSanitizer: attempting double-free on 0x502000000010
    #1 0x6367f90ed71e in Naive::~Naive() ch03/bugs/shallow.cpp:7
    #2 0x6367f90ed3a1 in main ch03/bugs/shallow.cpp:17
```

The member-by-member copy broke clause 2 of our invariant: two objects are now responsible for the same block. To fix it we must tell the compiler what copying an `IntVec` means, by writing two special member functions.

code/ch03/intvec.hpp

```
// Copy constructor: build a new IntVec as an independent copy of other.
IntVec(const IntVec& other)
    : data_(other.size_ > 0 ? new int[other.size_] : nullptr),
      size_(other.size_),
      capacity_(other.size_) {
    for (std::size_t i = 0; i < size_; ++i) data_[i] = other.data_[i];
}

// Copy assignment: make an existing IntVec into a copy of other.
IntVec& operator=(const IntVec& other) {
    if (this != &other) {
        IntVec copy(other);   // may throw; *this is untouched so far
        swap(copy);           // take the copy's block, give it our old one
    }                         // copy's destructor frees our old block
    return *this;
}
```

The **copy constructor** builds a brand-new object from an existing one. Its parameter is a `const` reference to an object of the same class. The part between the colon and the opening brace is a *member initializer list*, which sets up the members directly: a new block just big enough for the other vector's elements, and matching size and capacity. The body then copies the elements. The result is the right side of Figure 3.3.

Copy assignment, `operator=`, handles `c = a` where `c` already exists and already owns a block. It has more to worry about: the old block must be freed, the object might be assigned to itself (`a = a`), and if allocating the new block fails, the object should be left as it was. The version here takes care of all three with a tidy idiom called *copy-and-swap*. It makes a copy of `other` using the copy constructor, then swaps the insides of `*this` with the copy. Now `*this` has the new contents, and the local variable `copy` holds our old block, which its destructor frees as the function returns. (`this` is a pointer to the object the member function was called on; `*this` is that object.)

We have now written three functions that manage the block: a destructor, a copy constructor, and a copy assignment operator. They travel together.

> ***The rule of three.*** *If a class needs a hand-written destructor, copy constructor, or copy assignment operator, it almost certainly needs all three.*

The reasoning: needing any one of them means that the class owns something that member-by-member handling gets wrong. The other two then get it wrong as well.

3.4 Moving

Copying is now correct. It is also sometimes wasteful. Consider a function that builds a vector and returns it.

code/ch03/intvec_test.cpp

```
IntVec squares(int n) {
    IntVec result;
    for (int i = 0; i < n; ++i) result.push_back(i * i);
    return result;   // moved (or constructed in place), not copied
}
```

When `squares(1000000)` returns, the local `result` is about to be destroyed. It would be absurd to copy its million elements into the caller's variable and then immediately throw the original away. The caller could simply *take over* the block.

That is a **move**. Since C++11 a class can define what it means to be moved from, with two more special member functions.

code/ch03/intvec.hpp

```
// Move constructor: take over other's block; leave other empty.
IntVec(IntVec&& other) noexcept
    : data_(other.data_), size_(other.size_), capacity_(other.capacity_) {
    other.data_ = nullptr;
    other.size_ = 0;
    other.capacity_ = 0;
}

// Move assignment: release our block, then take over other's.
IntVec& operator=(IntVec&& other) noexcept {
    if (this != &other) {
        delete[] data_;
        data_ = other.data_;
        size_ = other.size_;
        capacity_ = other.capacity_;
        other.data_ = nullptr;
        other.size_ = 0;
        other.capacity_ = 0;
    }
    return *this;
}
```

The parameter type `IntVec&&`, with two ampersands, is an *rvalue reference*. For our purposes it means: "an object that is about to be discarded, whose contents I am therefore allowed to take." The move constructor copies the three members, which costs three machine words, and then sets `other` to the empty state so that its destructor will not free the block we just took. No elements are touched. Moving a vector of a million elements costs the same as moving a vector of none.

The compiler uses these automatically wherever it can see that the source is a temporary or is about to go out of scope, as with `return result;`. When you want to move from a named object that the compiler would otherwise copy, you say so with `std::move`.

code/ch03/intvec_test.cpp

```
IntVec d = std::move(b);   // move constructor: d takes b's block
```

Despite its name, `std::move` moves nothing. It is a cast, a note to the compiler that says "I am finished with b; you may treat it as discardable." After the move, b is still a valid object (its invariant holds) but its contents are gone. The only sensible things to do with a moved-from object are to destroy it or assign it a new value.

The `noexcept` on both functions promises that they will not throw exceptions. That promise matters more than it looks. When a `std::vector<IntVec>` grows, it will *move* its elements to the new block only if their move constructor is `noexcept`; otherwise it falls back to copying, to be safe. Forgetting the keyword silently costs performance.

With five special members, the rule of three becomes the **rule of five**: destructor, copy constructor, copy assignment, move constructor, move assignment. If you write one, consider all five.

The rule of zero

Having learned to write all five, you should now learn when to write none of them. Look at this class.

```
struct Student {
    std::string name;
    std::vector<int> scores;
};
```

It has no destructor, no copy constructor, nothing. And it is entirely correct. The compiler-generated versions copy, move, and destroy member by member, and each *member* knows how to copy, move, and destroy itself properly. A `Student` can be copied (deeply), moved (cheaply), and destroyed (without leaks) for free.

The rule of zero. *If every member of a class manages itself, write none of the five special member functions.*

> **PITFALL**
>
> The rule of zero has one trap. The compiler-generated *move* moves each member, and moving a plain number just copies it. If a class keeps a count beside a container, say a `std::vector` and a separate `std::size_t` recording how many slots are in use, then after a move the source's vector is empty but its count is not. The C++ convention is that a moved-from object need only be safe to destroy or to assign a new value to, and that is all the classes in this book promise unless they say otherwise. Do not go on using an object you have moved from.

This is the rule for nearly every class you will write outside this course. Build your classes from `vector`, `string`, and other self-managing types, and all the machinery of this section happens for you. The handful of classes that must follow the rule of five are the ones that directly own a raw resource, such as the containers in this book. That is the reason for building them: so that you understand what the self-managing types are doing on your behalf.

BY HAND FIRST

Before you study the copy and move functions in `intvec.hpp` closely, close the book and write all five special members for `IntVec` yourself, with a small `main` that exercises each one, and run it under the sanitizers until it is clean. You will get something wrong: forget to null out the source of a move, mishandle self-assignment, leak the old block in assignment. Each of those mistakes, caught by AddressSanitizer and then understood, teaches a piece of C++'s object model that no amount of reading does.

This is also the first place in the course where asking a model to write the code is seriously tempting, because the functions are fiddly and a model writes them flawlessly. That is exactly why it is the wrong place. The fiddliness *is* the content.

ASK AN LLM

Once you have a version that you believe works, use a model as a critic:

> *Here is a C++ class that owns a heap array. Do not rewrite it. For each of the five special member functions, tell me whether it is present and correct. If one is wrong, describe the bug and give me a short test program that would expose it under AddressSanitizer.*

Run the test programs it gives you. If one of them does *not* fail, the model's claim was wrong, and you have learned something about both your code and the model.

3.5 Templates and iterators

`IntVec` holds `int`s. Nothing about its logic depends on that. A vector of `double`s or of `std::string`s would have the same three members and the same functions, with one type changed throughout. C++ lets you write the class once, with the element type left as a parameter.

code/ch03/vec.hpp

```
template <typename T>
class Vec {
public:
    Vec() = default;

    Vec(std::initializer_list<T> items) {   // allows  Vec<int> v{1, 2, 3};
        Vec built;                          // owns the block while we fill it
        built.reserve(items.size());
        for (const T& item : items) built.push_back(item);
        swap(built);
    }
```

`template <typename T>` makes `Vec` a **class template**: a pattern from which the compiler generates a separate class for each element type you ask for. `Vec<int>` and `Vec<std::string>` are different classes, each compiled as if you had written it by hand with `T` replaced. This is quite different from generics in Java, where one compiled class serves all element types by treating them as references to objects. A C++ `Vec<int>` stores actual `int`s, side by side, with no indirection: the template costs nothing at run time.

One practical consequence is that the compiler needs to see the whole template wherever it is used, so templates live entirely in header files. That is why the code for this chapter is in `vec.hpp` and not split into a header and a source file.

The complete `Vec<T>` is in `code/ch03/vec.hpp`. It is `IntVec` with `int` replaced by `T`, plus the additions discussed below.

> **ASIDE**
>
> Our `Vec` takes one shortcut that the real `std::vector` does not. It allocates its block with `new T[capacity]`, which creates `capacity` fully formed `T` objects at once, including the spare ones. That requires `T` to have a default constructor, and it means the spare slots hold real (default-valued) objects. `std::vector` instead allocates *raw* memory and constructs each element individually at the moment it is added, using lower-level tools that are beyond this course. The costs and the interface are the same; the bookkeeping is more delicate. Our `Vec` is also less careful than the real thing about *exceptions*: if copying an element fails partway through a reallocation (a `std::string` running out of memory, say), `std::vector` guarantees that nothing is leaked and the vector is unchanged, and achieving that takes more machinery than we show.

Passing elements efficiently

code/ch03/vec.hpp

```
void push_back(T value) {
    if (size_ == capacity_) reserve(capacity_ == 0 ? 1 : 2 * capacity_);
    data_[size_] = std::move(value);
    ++size_;
}
```

When `T` was `int`, passing by value was obviously fine. For a `T` like `std::string`, we would like to avoid copying when we can. This version takes the parameter by value and then *moves* it into the block. If the caller passes a temporary, such as `words.push_back("word" + suffix)`, the temporary is moved into `value` and then moved into the block: no copy at all. If the caller passes a named variable, it is copied once into `value` (as it must be; the caller still wants its own) and then moved. The growing logic has been factored into `reserve`, which also uses `std::move` to transfer elements to the new block.

Assignment gets the same treatment, and the result is pleasingly short.

code/ch03/vec.hpp

```
// One assignment operator for both copy and move. The parameter is
// taken by value: the caller's argument is copied (or moved) into
// `other`, we swap with it, and its destructor cleans up our old block.
Vec& operator=(Vec other) noexcept {
    swap(other);
    return *this;
}
```

Because the parameter is taken by value, the caller's argument is copied *or moved* into `other`, whichever is appropriate, before the function body runs. Then we swap. One function serves as both copy assignment and move assignment.

Insert and erase

A vector's elements are contiguous and in order. That is the source of its speed, and it has a price: putting a new element anywhere but the end means making a gap, and removing one means closing a gap.

code/ch03/vec.hpp

```
// Insert value so that it ends up at position `index`.
// Precondition: index <= size(). Everything from index on moves right.
void insert(std::size_t index, T value) {
    assert(index <= size_);
    if (size_ == capacity_) reserve(capacity_ == 0 ? 1 : 2 * capacity_);
    for (std::size_t i = size_; i > index; --i) {
        data_[i] = std::move(data_[i - 1]);
    }
    data_[index] = std::move(value);
    ++size_;
}

// Remove the element at position `index`.
// Precondition: index < size(). Everything after it moves left.
void erase(std::size_t index) {
    assert(index < size_);
    for (std::size_t i = index; i + 1 < size_; ++i) {
        data_[i] = std::move(data_[i + 1]);
    }
    --size_;
}
```

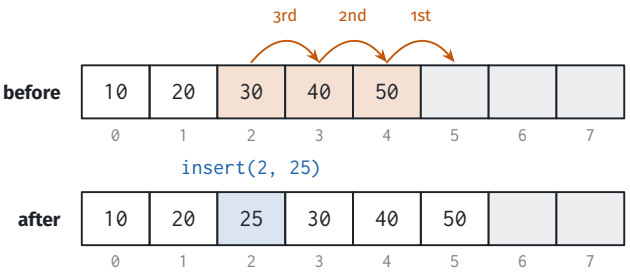


Figure 3.4 Inserting 25 at index 2. Every element from that position onward must first move one slot to the right, starting from the far end so that nothing is overwritten.

The loop in `insert` runs from the back toward `index`, as Figure 3.4 shows; run it front to back and each element would overwrite the next before it had been moved. Inserting at position i in a vector of n elements moves $n - i$ elements. At the end, that is zero. At the front, it is all of them.

Operation	Cost
`v[i]`, `at(i)`	$\Theta(1)$

Operation	Cost
`push_back`	$\Theta(1)$ amortized; $\Theta(n)$ worst case
`pop_back`	$\Theta(1)$
`insert` or `erase` at index i	$\Theta(n - i)$: constant at the back, linear at the front
searching for a value	$\Theta(n)$, or $\Theta(\log n)$ if kept sorted
copy	$\Theta(n)$
move	$\Theta(1)$

That fourth row is the vector's one real weakness, and the motivation for the linked structures of the next chapter.

Iterators

How should code outside the class visit every element? Indexing works for a vector, but not every container can be indexed efficiently. The C++ library's answer is the **iterator**: an object that marks a position in a container and can be advanced to the next position. Every container provides two.

- `begin()` returns an iterator to the first element.
- `end()` returns an iterator to the position *one past* the last element.

Together they describe a half-open range, exactly like `[lo, hi)` in binary search. An iterator supports `*it` to reach the element, `++it` to advance, and `!=` to compare. If that list of operations sounds familiar, it should. A pointer into an array supports all of them. For a vector, whose elements are contiguous, a plain pointer *is* a perfectly good iterator.

code/ch03/vec.hpp

```
T* begin() { return data_; }
T* end() { return data_ + size_; }
const T* begin() const { return data_; }
const T* end() const { return data_ + size_; }
```

Those four one-line functions buy a great deal. The range-based `for` loop is defined in terms of `begin()` and `end()`, so it now works on our class:

code/ch03/vec_test.cpp

```
int total = 0;
for (int x : v) total += x;
```

And so does the whole standard library of algorithms, which are written to operate on *any* pair of iterators, from any container.

code/ch03/vec_test.cpp

```
Vec<int> nums{31, 4, 15, 9, 26};
std::sort(nums.begin(), nums.end());              // {4, 9, 15, 26, 31}
auto where = std::find(nums.begin(), nums.end(), 15);
assert(where != nums.end() && where - nums.begin() == 2);
```

`std::sort` and `std::find` have never heard of `Vec`. They were handed two pointers and asked to work on the range between them. This separation of algorithms from containers, with iterators as the connecting interface, was the founding idea of the Standard Template Library [53], and it is why learning the iterator conventions pays off across all of C++.

PITFALL

An iterator, pointer, or reference to a vector element is an address inside the vector's current block. When the vector grows, that block is deleted. Every iterator, pointer, and reference into it is left dangling. This is called **invalidation**, and it is the classic `std::vector` bug.

code/ch03/bugs/invalidate.cpp

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v{10, 20, 30};
    int& first = v[0];          // a reference into the vector's block

    for (int i = 0; i < 100; ++i) {
        v.push_back(i);         // sooner or later: a new, bigger block
    }

    first = 99;                // writes into the OLD block, long since freed
    std::cout << v[0] << '\n';
}
```

```
==26591==ERROR: AddressSanitizer: heap-use-after-free on address
↪  0x502000000010
WRITE of size 4 at 0x502000000010 thread T0
    #0 0x5cc0fc8e69ff in main ch03/bugs/invalidate.cpp:12
```

The rule: after any operation that might change a vector's capacity (`push_back`, `insert`, `reserve`), or that shifts elements (`insert`, `erase`), do not use iterators, pointers, or references obtained before it. Indices remain meaningful; addresses do not. This also means you must never `push_back` onto a vector inside a range-`for` loop over the same vector.

3.6 How do you know?

We have a container with a dozen operations, several of which rearrange memory. How confident should we be that it is right, and where does the confidence come from?

Start with the invariant. We wrote it down in English at the beginning of the chapter. Here is part of it as code.

code/ch03/vec.hpp

```
// The representation invariant, as executable code.
bool check_invariant() const {
    if (size_ > capacity_) return false;
    if ((capacity_ == 0) != (data_ == nullptr)) return false;
    return true;
```

```
}
```

Not everything in the English version can be checked from inside the object. There is no portable way to ask whether a pointer really refers to a block of the claimed size, which is what the sanitizer is for. But what can be checked should be, after every operation in the tests.

Next, a reference. `std::vector` already exists and is trusted. If we perform the same sequence of operations on a `Vec<int>` and on a `std::vector<int>`, they should contain the same elements afterward, every time.

code/ch03/vec_test.cpp

```
// Do our Vec and the reference std::vector hold the same sequence?
template <typename T>
bool same(const Vec<T>& ours, const std::vector<T>& ref) {
    if (!ours.check_invariant()) return false;
    if (ours.size() != ref.size()) return false;
    for (std::size_t i = 0; i < ref.size(); ++i) {
        if (ours[i] != ref[i]) return false;
    }
    return true;
}
```

And finally, volume. Instead of hand-writing a few sequences of operations, generate them at random, by the thousand.

code/ch03/vec_test.cpp

```
// Apply the same random operations to both; they must never disagree.
void fuzz(unsigned seed, int steps) {
    std::mt19937 rng(seed);
    Vec<int> ours;
    std::vector<int> ref;

    for (int step = 0; step < steps; ++step) {
        int value = static_cast<int>(rng() % 1000);
        switch (rng() % 6) {
        case 0:
        case 1:   // push_back is the common case, so give it two slots
            ours.push_back(value);
            ref.push_back(value);
            break;
        case 2:
            if (!ref.empty()) {
                ours.pop_back();
                ref.pop_back();
            }
            break;
        case 3: {
            std::size_t at = rng() % (ref.size() + 1);   // may equal size()
            ours.insert(at, value);
            ref.insert(ref.begin() + static_cast<std::ptrdiff_t>(at), value);
            break;
```

```
        }
        case 4:
            if (!ref.empty()) {
                std::size_t at = rng() % ref.size();
                ours.erase(at);
                ref.erase(ref.begin() + static_cast<std::ptrdiff_t>(at));
            }
            break;
        case 5:
            if (!ref.empty()) {
                std::size_t at = rng() % ref.size();
                ours[at] = value;
                ref[at] = value;
            }
            break;
        }
        assert(same(ours, ref));
    }
}
```

This is the differential testing of Section 2.4, now applied to a whole data structure: a random sequence of operations, mirrored on a reference implementation, with a comparison after every step. The test for this chapter runs 200 such sequences of 400 operations each. All 80,000 operations run under AddressSanitizer, so a test passes only if the answers were right *and* no memory was misused on the way.

A few design points are worth copying into your own tests.

- **Small values, small sizes.** Insert positions are chosen from `0` to `size()` *inclusive*, so the edge cases (front, back, empty) come up constantly.
- **Weight the operations.** `push_back` gets two of the six slots, so the vector tends to grow and exercise reallocation, not hover near empty.
- **Use a seed.** The random generator is seeded with a known number, so a failure can be reproduced exactly by rerunning with the same seed.
- **Check after every step.** When the assertion fires, the bug is in the operation that just ran.

How good is the test? There is a direct way to find out: break the code on purpose. Change `i > index` to `i >= index` in `insert`, or delete the `++size_`, and run the test. If it still passes, the test is not looking where you thought it was. Deliberately injecting bugs to evaluate a test suite is called *mutation testing*, and it is the answer to "how do you know?" asked one level up: how do you know your tests would notice?

> **ASK AN LLM**
>
> Models are good at proposing test cases, and a useful second opinion on what you have forgotten to test.
>
> *Here is the public interface of a vector class I wrote, and the tests I have so far. List specific cases my tests do not cover. Do not write the tests; describe each case in one sentence.*

> Then write those tests yourself. And be wary of the reverse arrangement, in which a model writes both the code and its tests. Tests written by the author of the code tend to share the author's blind spots, and that is as true of a model as of a person. A differential test against an independent reference has no such blind spot, which is why this book leans on it.

THE FOUR QUESTIONS **Vec<T>, and std::vector**

Where does it live? The object is three words: a pointer, a size, a capacity. The elements are in one contiguous heap block that the object owns. The block is replaced by a larger one when it fills.

What does it promise? `size <= capacity`; elements `0` through `size - 1` are the contents, in order, contiguous. Any operation that may reallocate or shift elements invalidates iterators, pointers, and references into the vector.

What does it cost? Indexing: $\Theta(1)$. `push_back`: amortized $\Theta(1)$ with geometric growth, though a single call can be $\Theta(n)$. Insert or erase at position i: $\Theta(n - i)$. Copy: $\Theta(n)$; move: $\Theta(1)$. Space: up to twice what the elements need. Cache behavior: as good as it gets.

How do you know? An executable invariant check; differential testing against `std::vector` on random operation sequences; AddressSanitizer running under all of it; and mutation testing to check the tests.

Exercises

1. Starting from an empty `IntVec`, list the capacity after each of the first ten calls to `push_back`. At which calls does `grow` run, and how many elements does each copy?
2. Using the coin argument, explain why charging *two* coins per `push_back` is not enough to pay for doubling. Then find the smallest number of coins per `push_back` that suffices when the growth factor is 1.5, and when it is 3.
3. `Vec::pop_back` never releases memory. Suppose we "fix" this by halving the capacity whenever the vector becomes exactly half full. Describe a sequence of `push_back` and `pop_back` calls, starting from a full vector of capacity c, in which *every* call triggers a reallocation. What is the amortized cost per operation in that sequence? What shrinking rule avoids the problem? (Hint: shrink at one quarter.)
4. Add `void resize(std::size_t n)` to `IntVec`: afterward the size is `n`; if the vector grew, the new elements are zero. Add `void shrink_to_fit()`, which reallocates so that capacity equals size. State which parts of the invariant each must be careful to restore.
5. What does this print, and why? What would it print if `IntVec` had no user-written copy constructor (assuming the program did not crash)?

```
IntVec a;
a.push_back(1);
IntVec b = a;
```

```
b[0] = 2;
std::cout << a[0] << ' ' << b[0] << '\n';
```

6. In `IntVec`'s copy assignment operator, what specifically goes wrong on `a = a` if you remove the `if (this != &other)` test *and* replace the copy-and-swap body with: delete the old block, allocate a new one, copy the elements from `other`?
7. For each class, say whether it should follow the rule of zero or the rule of five, and why: (a) a `Matrix` holding `std::vector<double> cells; int rows; int cols;`; (b) a `Buffer` holding `char* bytes; std::size_t length;` where `bytes` was allocated with `new[]`; (c) a `Roster` holding `std::vector<Student> students; std::string course;`.
8. Suppose `Vec::push_back` were declared as `void push_back(const T& value)` and began by calling `reserve` when full, as ours does. Explain how `v.push_back(v[0])` could then write garbage into the vector. Why is the by-value version in this chapter immune?
9. Implement `Vec<T>::erase(std::size_t first, std::size_t last)`, which removes the elements in the half-open index range `[first, last)` in a single pass, moving each later element exactly once. What is its cost? Why is calling the one-element `erase` in a loop, to remove k elements from the front, much worse?
10. Write a function template `template <typename T> void remove_all(Vec<T>& v, const T& value)` that removes every element equal to `value` in $\Theta(n)$ total time. (Hint: keep two indices, one reading and one writing. This is the idea behind the standard library's `std::erase`.)
11. Add the differential-test operations `clear`, copy construction, and assignment to the `fuzz` function in `vec_test.cpp`. Then perform mutation testing: introduce each of these bugs into `vec.hpp`, one at a time, and record whether the tests catch it and how. (a) `reserve` copies only `size_ - 1` elements. (b) `erase` forgets `--size_`. (c) `insert`'s loop runs `i > index + 1`. (d) the move constructor does not null out `other.data_`.
12. **With a model, as examiner.** Ask a model to quiz you on iterator invalidation: "Show me short snippets that use `std::vector`, one at a time, and ask me whether each one has undefined behavior due to invalidation. Don't reveal the answer until I respond." Do ten. Verify any answer you doubt by compiling the snippet with AddressSanitizer.
13. **Lab.** Implement `Vec<T>` yourself, from an empty file, with the full rule of five, `push_back`, `pop_back`, `operator[]`, `at`, `insert`, `erase`, `reserve`, `begin`, `end`, and `check_invariant`. Write a differential fuzz test against `std::vector` for both `Vec<int>` and `Vec<std::string>`. Then add a constructor parameter or template parameter for the growth factor, measure the time for ten million `push_back`s at factors 1.25, 1.5, 2, and 4, and report both the time and the peak capacity for each. Which factor would you choose, and what does the choice depend on?

Chapter notes

Amortized analysis was given its name and its systematic form by Tarjan [59]; the coin argument here is the "banker's method" from that paper. Cormen et al. [16] devote a chapter to the technique and analyze the dynamic table, including the shrinking problem of Exercise 3, in detail.

The design of `std::vector`, and of the iterator-based library around it, is due to Alexander Stepanov and his collaborators [53]. Stroustrup's *A Tour of C++* [55] walks through the construction of a vector class much as this chapter does, at greater speed, and is a good second account. For move semantics, rvalue references, and the many subtleties passed over here, the standard guide is Meyers's *Effective Modern C++* [41]. The rules of three, five, and zero are stated in the C++ Core Guidelines [57].

Testing with randomly generated inputs checked against stated properties was popularized by the QuickCheck library for Haskell [14], and the idea has since spread to every major language under the name *property-based testing*.

Answers to check-yourself questions

3.1 The vector must allocate a new, larger block, copy the four elements into it, delete the old block, and then add the fifth. The answer to the second question depends entirely on *how much* larger each new block is, and working that out is the centerpiece of this chapter. With the policy real vectors use, the answer is: about a million copies. With the most obvious policy, it is about five hundred *billion*.
3.2 There is one block, with two `IntVec` objects pointing at it: the left side of Figure 3.3. A `push_back` through `b` would be invisible to `a`'s size, and if `b` grew, it would delete the block out from under `a`. Even if nobody touches either vector, when they go out of scope both destructors run, and both `delete[]` the same block: a double free.

CHAPTER 4

Links, Stacks, and Queues

WEEK 4 OF 10

THIS WEEK

By the end of this week you should be able to:

- implement a singly linked list, including insertion, removal, reversal, and a correct destructor, and draw what each operation does to the pointers;
- explain how a sentinel node removes the special cases from a doubly linked list;
- say what an abstract data type is, and implement a stack on a vector and a queue on a circular buffer;
- give the costs of the main operations on vectors and linked lists, and explain why measured performance often contradicts a naive reading of those costs;
- decide, with reasons, when a linked structure is the right choice.

The vector of Chapter 3 has one weakness. Its elements sit shoulder to shoulder in a single block, so putting a new element anywhere except the end means sliding everything after it over by one. Inserting at the front of a million-element vector moves a million elements.

CHECK YOURSELF 4.1

Design, in your head, a way of storing a sequence in which a new element can be inserted at the front, or between any two existing elements, *without moving any of the others*. What does each element need to carry besides its value? What ability of the vector do you have to give up?

That design is the **linked list**, the second fundamental answer to the question *where does it live?* A vector keeps its elements in one place and the order is implicit in the layout. A linked list scatters its elements across the heap and makes the order explicit with pointers. Nearly every structure in the second half of this book (trees, graphs, the chains inside a hash table) is built from the same idea of nodes joined by pointers, so fluency here is not optional.

But this chapter also has a second job, which is to be honest about linked lists. They occupy a place of honor in the traditional curriculum, and a table of Big-O costs makes them look like the natural choice whenever a program inserts and deletes a lot. On the hardware of the last twenty years that is mostly false, and the final section shows the measurements.

4.1 Nodes and links

A linked list is made of **nodes**. Each node holds one element and a pointer to the next node.

code/ch04/list.hpp

```
struct Node {
    T value;
    Node* next;   // the following node, or nullptr at the end
};
```

The list object itself holds a pointer to the first node, traditionally called the **head**, and (for convenience) a count. The last node's `next` is `nullptr`, which is how you know it is the last. An empty list is a null head. Each node is a separate heap allocation; the nodes may be anywhere in memory, in any order. Figure 4.1 shows a list of three.

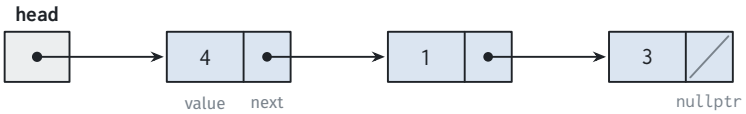


Figure 4.1 A singly linked list. The list object holds `head`; each node holds a value and the address of the next node; the last node's `next` is null. In the web edition you can insert, remove, and reverse, and watch each pointer assignment happen in order. [Interactive in the web edition.]

Notice that `Node` refers to itself. A `Node` cannot *contain* a `Node` (how big would it be?), but it can contain a pointer to one, since a pointer is eight bytes whatever it points to.

Adding at the front

code/ch04/list.hpp

```
void push_front(T value) {
    head_ = new Node{std::move(value), head_};
    ++size_;
}

void pop_front() {
    assert(head_ != nullptr);
    Node* doomed = head_;
    head_ = head_->next;
    delete doomed;
    --size_;
}

T& front() {
    assert(head_ != nullptr);
    return head_->value;
}
```

`push_front` is one line, and it repays slow reading. The expression `new Node{std::move(value), head_}` allocates a node whose `value` is the new element and whose `next` is *the current head*. Only then is the result assigned to `head_`. Two pointer changes, in a definite order: first aim the new node at the old first node, then aim the head at the new node. Do them in the other order and the address of the old first node is overwritten before it has been saved anywhere. The whole existing list becomes unreachable: a leak of every node.

That is the first lesson of linked structures. **Before you overwrite a pointer, make sure the address it held is saved somewhere, or is no longer needed.**

`pop_front` shows the same care from the other side. It saves the address of the doomed node, moves the head past it, and only then deletes it.

No element was moved in either function. The cost is $\Theta(1)$, regardless of the list's length.

Walking the list

code/ch04/list.hpp

```
bool contains(const T& value) const {
    for (const Node* p = head_; p != nullptr; p = p->next) {
        if (p->value == value) return true;
    }
    return false;
}
```

This `for` loop is the idiom for visiting every node, and you will write it hundreds of times: start `p` at the head; continue while `p` is not null; advance with `p = p->next`. Compare it with the loop over an array, `for (i = 0; i < n; ++i)`. They have the same shape. Only the meaning of "the next one" differs.

Here is the price of linking. To reach the element at position i there is no arithmetic to do; you must walk i links from the head. Access by index is $\Theta(n)$, which also rules out binary search: there is no cheap way to jump to the middle.

Destroying a list

A list owns its nodes, so its destructor must delete every one.

code/ch04/list.hpp

```
~List() { clear(); }

void clear() {
    while (head_ != nullptr) {
        Node* doomed = head_;
        head_ = head_->next;   // step forward BEFORE deleting
        delete doomed;
    }
    size_ = 0;
}
```

The loop must read `head_->next` *before* deleting the node, because afterward the node's contents no longer exist. Writing `delete head_; head_ = head_->next;` is a use-after-free, and a very common one.

The copy constructor, similarly, must make a new node for every node of the original. It keeps a pointer to the last node of the copy so that each new node can be attached in constant time. It builds the copy in a local `List` and swaps at the end, for a reason worth a moment: if an allocation fails halfway through, the half-built chain belongs to the local variable, whose destructor frees it. Built directly in the object under construction, those nodes would leak, because a destructor does not run for an object whose constructor did not finish.

code/ch04/list.hpp

```
List(const List& other) {
    List copy;              // owns the new nodes until we are finished
    Node* tail = nullptr;   // last node of the copy so far
    for (Node* p = other.head_; p != nullptr; p = p->next) {
        Node* fresh = new Node{p->value, nullptr};
        if (tail == nullptr) copy.head_ = fresh;
        else tail->next = fresh;
        tail = fresh;
        ++copy.size_;
    }
    std::swap(head_, copy.head_);   // take the finished chain
    std::swap(size_, copy.size_);
}
```

With the destructor, this copy constructor, a move constructor that steals the head pointer, and a by-value assignment operator like the one in `Vec`, the list follows the rule of five. The complete class is in `code/ch04/list.hpp`.

> **CHECK YOURSELF 4.2**
>
> The copy constructor walks the original list with `p` and builds the copy by adding each new node at the *end*. Why not use `push_front` for each element, which would be simpler?

4.2 Rewiring

The characteristic operations on a linked list are not about moving data. They are about changing which node points to which. This is delicate work, and the only reliable way to do it is with a picture.

Removing a node

To remove a node from the middle of a singly linked list, the node *before* it must be made to point to the node *after* it. So as we search for the node to remove, we must keep track of its predecessor.

code/ch04/list.hpp

```
// Removes the first node holding `value`. Returns whether one was found.
bool remove(const T& value) {
    Node* prev = nullptr;
    Node* cur = head_;
    while (cur != nullptr && !(cur->value == value)) {
        prev = cur;
        cur = cur->next;
    }
    if (cur == nullptr) return false;            // not found

    if (prev == nullptr) head_ = cur->next;      // removing the first node
    else prev->next = cur->next;                 // route around cur
    delete cur;
    --size_;
    return true;
}
```

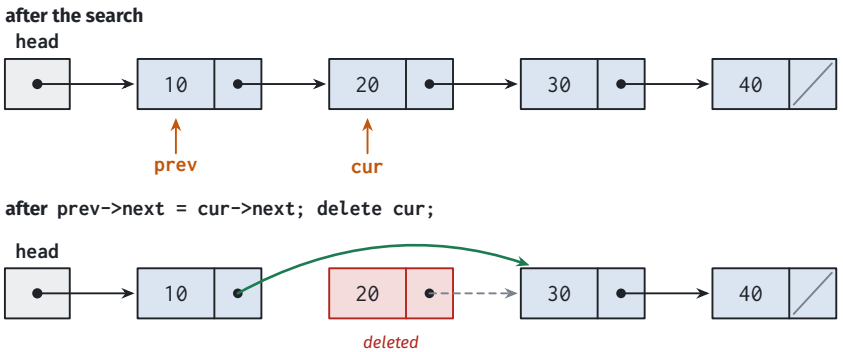


Figure 4.2 Removing the node holding 20. The search stops with `cur` at the doomed node and `prev` one behind it. A single assignment, `prev->next = cur->next`, routes the list around the node, which can then be deleted.

The loop advances `prev` and `cur` in step until `cur` reaches the node to remove or runs off the end. Then there are three cases. If `cur` is null the value was not found. If `prev` is null then `cur` is the first node, which has no predecessor; the pointer that must change is `head_` itself. Otherwise the pointer that must change is `prev->next`, as in Figure 4.2.

That middle case, the special treatment of the first node, is the nuisance of singly linked lists. The first node is reached through `head_` and every other node is reached through some node's `next`, so code that changes "the pointer that leads to this node" has to ask which kind it is dealing with.

Reversing a list

A classic exercise, worth doing because it is pure pointer manipulation: reverse a list in place, without allocating anything, by turning every arrow around.

code/ch04/list.hpp

```
// Reverses the list in place by re-aiming every next pointer.
void reverse() {
    Node* done = nullptr;      // the already-reversed part
    Node* cur = head_;         // the part still to do
    while (cur != nullptr) {
        Node* rest = cur->next;   // remember where to go next
        cur->next = done;         // flip this node's pointer
        done = cur;
        cur = rest;
    }
    head_ = done;
}
```

The loop maintains an invariant: `done` is the head of a correctly reversed list containing the nodes processed so far, and `cur` is the head of the remaining, untouched part. Each step detaches the first node of the remaining part and attaches it to the front of the reversed part. The first line of the loop body is the one people forget. It saves `cur->next` before the second line overwrites it.

> **BY HAND FIRST**
>
> Take a sheet of paper. Draw a list of four nodes as boxes with arrows, and draw `done`, `cur`, and `rest` as labeled arrows. Execute `reverse` by hand, one statement at a time, crossing out and redrawing arrows as they change. Then do the same for `remove`, once for a node in the middle, once for the first node, and once for a value that is not present.
>
> This is not optional busywork, and it cannot be delegated. Pointer rewiring is the one skill in this course that is almost entirely *visual and manual*: people who can do it see the boxes and arrows in their heads, and they got that way by drawing them on paper many times. A model can produce a correct `reverse` instantly. What it cannot do is install the picture in your head, and you will need that picture for trees in two weeks.

Pointers to pointers

There is a neat way to make the first node's special case disappear. In `remove`, the thing that needs to change is always a `Node*`: sometimes the one named `head_`, sometimes the one named `next` inside some node. A pointer is an object like any other, with an address of its own. So keep track of *the address of the pointer that leads to the current node*.

code/ch04/list.hpp

```
// Precondition: the list is in increasing order. Inserts value in order.
void insert_sorted(T value) {
    Node** link = &head_;   // the pointer we might need to change
    while (*link != nullptr && (*link)->value < value) {
        link = &(*link)->next;
    }
    *link = new Node{std::move(value), *link};
    ++size_;
}
```

Here `link` has type `Node**`, pointer to pointer to node. It begins by pointing at `head_`. Each step of the loop re-aims it at the `next` member of the following node. Whenever the loop stops, `*link` is the pointer that must be changed to splice in the new node, whether that happens to be the head or a `next` field, and the code does not need to know which. The new node's `next` is set to the old value of `*link`, and `*link` is then set to the new node.

This technique is more abstract than the `prev`/`cur` version, and you are free to prefer the latter. But it will come back in Chapter 6, where the "pointer that leads to this node" is the natural thing to pass to a recursive function.

Iterating

A vector's iterator was a plain pointer, because advancing a pointer moves to the next element of a contiguous block. In a list the next element is wherever `next` says it is, so the iterator must be a small class that knows how to follow a link.

code/ch04/list.hpp

```
class Iterator {
public:
    explicit Iterator(Node* node) : node_(node) {}
    T& operator*() const { return node_->value; }
    Iterator& operator++() {     // advance: follow the link
        node_ = node_->next;
        return *this;
    }
    bool operator==(const Iterator& other) const = default;

private:
    Node* node_;
};

Iterator begin() { return Iterator(head_); }
Iterator end() { return Iterator(nullptr); }
```

The iterator wraps a node pointer. Dereferencing it gives the node's value; incrementing it follows `next`; the end iterator wraps a null pointer. With `begin()` and `end()` in place, a range-`for` loop works on our list exactly as on a vector:

code/ch04/list_test.cpp

```
List<std::string> todo;
todo.push_front("sleep");
todo.push_front("debug");
todo.push_front("code");          // code -> debug -> sleep

std::string all;
for (const std::string& item : todo) all += item + ' ';
```

Code that uses the loop cannot tell whether it is walking a contiguous block or chasing pointers across the heap. That is the point of iterators: one interface, many arrangements of memory. The difference shows up in what the iterator *can* do. A vector's iterator can jump (`it + 500`) in constant time; a list's can only step. The standard library classifies

iterators by these abilities. An algorithm that must jump, such as `std::sort`, requires the stronger kind and will not compile for a list. Binary search will accept a list's iterators, but it has to step where it would have jumped, which throws away the point of it.

ASK AN LLM

Finding the bug in a broken linked-list function is excellent practice, and a model can supply an endless stream of broken functions.

> *Write a C++ function that operates on a singly linked list (insert, remove, reverse, or similar) and contains exactly one subtle pointer bug. Do not tell me what the bug is or hint at where it is. I will find it by drawing the list and tracing your code, and then you can tell me whether I am right.*

Find the bug with pencil and paper, not by running the code. When you think you have it, state a specific list and call that triggers it. That is what distinguishes having found the bug from having a feeling about a line.

4.3 Doubly linked lists and sentinels

A singly linked list can be walked in one direction only, and removing a node requires access to its predecessor. Give each node a second pointer, to the *previous* node, and both limitations go away: the list can be walked backward, and a node can be removed given nothing but its own address, since it knows both of its neighbors.

The cost is another eight bytes per node and twice as many pointers to keep consistent. With the obvious representation (a head pointer, a tail pointer, null at both ends), every operation sprouts special cases: is this the first node? The last? The only one?

There is a classic trick that removes all of them. Add one extra node, a **sentinel**, that holds no element and is never removed, and join the list into a ring: the sentinel's `next` is the first real node, and its `prev` is the last. An empty list is a sentinel pointing to itself.

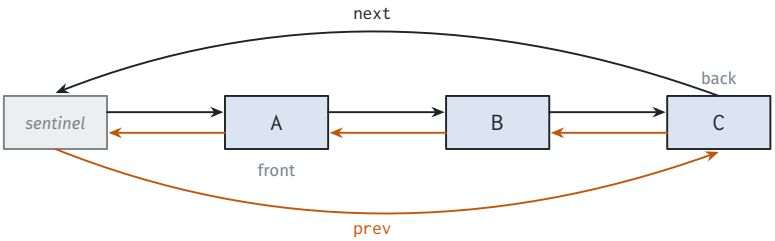


Figure 4.3 A doubly linked list of three elements with a sentinel. Every node, including the sentinel, has a valid `prev` and `next`, so there are no null pointers and no special cases at the ends. The sentinel doubles as the "one past the end" position.

Now *every* real node has a real node or the sentinel on each side. No pointer in the structure is ever null. Insertion becomes four pointer assignments and removal two, with no conditionals in either.

code/ch04/dlist.hpp

```
// Inserts value just before pos. Returns an iterator to the new node.
Iterator insert(Iterator pos, T value) {
    Node* after = pos.node_;
    Node* before = after->prev;
    Node* fresh = new Node{std::move(value), before, after};
    before->next = fresh;
    after->prev = fresh;
    ++size_;
    return Iterator(fresh);
}

// Removes the node at pos. Returns an iterator to the node after it.
Iterator erase(Iterator pos) {
    Node* doomed = pos.node_;
    assert(doomed != sentinel_);
    Node* after = doomed->next;
    doomed->prev->next = after;
    after->prev = doomed->prev;
    delete doomed;
    --size_;
    return Iterator(after);
}
```

Trace `insert` on an empty list, where `pos` is `end()`, the sentinel. Then `after` and `before` are *both* the sentinel, and the four assignments correctly produce a ring of two nodes. The empty-list case, usually the most bug-prone, simply works. Every other operation is a one-line call to one of these two.

code/ch04/dlist.hpp

```
void push_front(T value) { insert(begin(), std::move(value)); }
void push_back(T value) { insert(end(), std::move(value)); }
void pop_front() { erase(begin()); }
void pop_back() { erase(Iterator(sentinel_->prev)); }
T& front() { return sentinel_->next->value; }
T& back() { return sentinel_->prev->value; }
```

Since position is represented by an iterator, the list hands out iterators and takes them back as arguments. The iterator for a doubly linked list can step backward as well as forward.

code/ch04/dlist.hpp

```
class Iterator {
public:
    T& operator*() const { return node_->value; }
    Iterator& operator++() { node_ = node_->next; return *this; }
    Iterator& operator--() { node_ = node_->prev; return *this; }
    bool operator==(const Iterator& other) const = default;

private:
    friend class DList;
```

```
    explicit Iterator(Node* node) : node_(node) {}
    Node* node_;
};

Iterator begin() { return Iterator(sentinel_->next); }
Iterator end() { return Iterator(sentinel_); }
```

One more piece of C++ appears in this class.

code/ch04/dlist.hpp

```
// An empty list is a sentinel whose prev and next point to itself.
DList() : sentinel_(new Node{T{}, nullptr, nullptr}) {
    sentinel_->prev = sentinel_;
    sentinel_->next = sentinel_;
}

~DList() {
    while (!empty()) pop_front();
    delete sentinel_;
}

DList(const DList&) = delete;              // this class cannot be copied
DList& operator=(const DList&) = delete;
```

The two lines ending in `= delete` declare that a `DList` cannot be copied: any attempt is a compile-time error. This is a respectable third option under the rule of five. A class that owns a resource must either copy it correctly or refuse to be copied. What it must not do is let the compiler's member-by-member default through.

> **IN THE WILD**
>
> The standard library has both kinds of list: `std::forward_list` (singly linked) and `std::list` (doubly linked, implemented with a sentinel much as here). `std::list` offers one operation that no array-based container can match: `splice`, which moves a node, or the entire contents of a list, into another list in constant time, without copying or even touching the elements. It also guarantees that iterators and references to an element stay valid until that element is erased, no matter what else is inserted or removed. Contrast that with the invalidation rules for vectors in Section 3.5. Those two properties, and not raw speed, are the reasons to choose `std::list` when you do. Chapter 10 builds a cache that depends on both.

4.4 Stacks and queues

So far we have described containers by how they are built. It is often more useful to describe them by what they *do*. An **abstract data type**, or ADT, is a specification of a set of operations and their behavior, with nothing said about how the data is stored [37]. An ADT answers the book's second question, *what does it promise?*, and deliberately declines to answer the first. The same ADT can be implemented by several data structures with different costs, and code written against the ADT does not care which.

The two simplest ADTs restrict a sequence so that elements can be added and removed only at its ends.

Stacks

A **stack** allows insertion and removal at one end only, called the top. The last element pushed is the first one popped: *last in, first out*, or LIFO.

Operation	Meaning
`push(x)`	add `x` to the top
`pop()`	remove the top element
`top()`	look at the top element
`empty()`	is the stack empty?

Which data structure should implement it? A vector's cheap end is its back, so let the back be the top.

code/ch04/stack_queue.hpp

```
// Last in, first out. All the work is done by the vector underneath.
template <typename T>
class Stack {
public:
    void push(T value) { items_.push_back(std::move(value)); }
    void pop() {
        assert(!items_.empty());
        items_.pop_back();
    }
    T& top() {
        assert(!items_.empty());
        return items_.back();
    }
    bool empty() const { return items_.empty(); }
    std::size_t size() const { return items_.size(); }

private:
    std::vector<T> items_;
};
```

The class is called an *adapter*: it adds nothing but a narrower interface. Every operation is $\Theta(1)$ (amortized, for `push`). Notice also that it declares no destructor and no copy operations: its only member is a `vector`, which manages itself. That is the rule of zero from Section 3.4 at work. (A singly linked list would serve too, with the head as the top, but it would pay for a heap allocation on every push.) The standard library's version is `std::stack`.

You have already met the most important stack in computing. The call stack of Section 1.5 is one: calling a function pushes a frame, returning pops it, and the function that returns is always the most recently called. Stacks turn up wherever things nest: function calls, parentheses, HTML tags, undo histories.

Here is a small, typical application. A compiler, or your editor, needs to check that brackets are balanced. Every closing bracket must match the *most recent* unclosed opening bracket. “Most recent” is the signature of a stack.

code/ch04/brackets.cpp

```
// Are the brackets in s properly matched and nested?
bool balanced(const std::string& s) {
    std::stack<char> open;   // the brackets still waiting to be closed
    for (char c : s) {
        if (c == '(' || c == '[' || c == '{') {
            open.push(c);
        } else if (c == ')' || c == ']' || c == '}') {
            if (open.empty()) return false;          // nothing to close
            char o = open.top();
            open.pop();
            bool match = (o == '(' && c == ')') ||
                         (o == '[' && c == ']') ||
                         (o == '{' && c == '}');
            if (!match) return false;                // wrong kind
        }
    }
    return open.empty();                             // anything left unclosed?
}
```

CHECK YOURSELF 4.3

Trace `balanced` on the string `"([)]"`. At which character does it return, and why?

Queues

A **queue** adds at one end, the back, and removes from the other, the front: *first in, first out*, or FIFO. It is a line at a counter. Queues model anything that must be handled in order of arrival: print jobs, network packets, tasks waiting for a processor, and, as Chapter 9 will show, the frontier of a breadth-first search.

Operation	Meaning
`push(x)`	add `x` at the back
`pop()`	remove the front element
`front()`	look at the front element
`empty()`	is the queue empty?

A vector is awkward here. Adding at the back is cheap, but removing from the front shifts everything: $\Theta(n)$ per pop. A singly linked list with an extra pointer to its last node handles both ends in constant time, and that is a perfectly good implementation. But there is a way to get constant-time operations at both ends *and* keep the elements in a contiguous block.

The idea: when the front element is removed, do not shift anything. Leave a hole, and remember where the front now is. The occupied part of the block drifts rightward as the

queue is used. When it reaches the end of the block, let it wrap around to the beginning, into the space that earlier removals freed. The block is treated as a ring, and the structure is called a **circular buffer** or **ring buffer**.

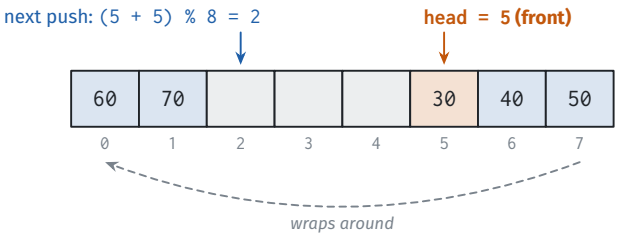


Figure 4.4 A ring buffer of capacity eight holding five elements that have wrapped around the end of the block. head is the index of the front element; the back is `(head + size) % capacity`. In the web edition, push and pop to watch the occupied region travel around the ring. [Interactive in the web edition.]

code/ch04/stack_queue.hpp

```
// First in, first out, in a circular buffer.
template <typename T>
class RingQueue {
public:
    void push(T value) {
        if (size_ == buf_.size()) grow();
        buf_[(head_ + size_) % buf_.size()] = std::move(value);
        ++size_;
    }
    void pop() {
        assert(size_ > 0);
        head_ = (head_ + 1) % buf_.size();   // the front moves; nothing shifts
        --size_;
    }
    T& front() {
        assert(size_ > 0);
        return buf_[head_];
    }
    bool empty() const { return size_ == 0; }
    std::size_t size() const { return size_; }

private:
    // Move to a buffer twice as big, unwrapping so the front is at index 0.
    void grow() {
        std::vector<T> bigger(buf_.empty() ? 4 : 2 * buf_.size());
        for (std::size_t i = 0; i < size_; ++i) {
            bigger[i] = std::move(buf_[(head_ + i) % buf_.size()]);
        }
        buf_ = std::move(bigger);
        head_ = 0;
    }
```

```
    std::vector<T> buf_;      // the slots; buf_.size() is the capacity
    std::size_t head_ = 0;    // index of the front element
    std::size_t size_ = 0;    // number of elements in the queue
};
```

The representation is a block of slots, the index `head_` of the front element, and the count `size_`. The element i places behind the front lives at index `(head_ + i) % capacity`; the remainder operator `%` is what bends the line into a ring. Pushing writes at offset `size_`. Popping advances `head_` by one, wrapping if necessary.

When the buffer is full, `grow` allocates one twice as large and copies the elements into it *in queue order*, starting from the front, so that the queue is "unwrapped" and `head_` returns to zero. The amortized analysis is identical to the vector's.

> **PITFALL**
>
> The classic ring-buffer bug is confusing *full* with *empty*. If you represent the queue by two indices, `head` and `tail`, then `head == tail` when the queue is empty, and also when it is completely full. Implementations that use two indices must either leave one slot permanently unused or keep a separate flag. Storing `head_` and `size_`, as here, avoids the ambiguity entirely.

The standard library's queue is `std::queue`, an adapter like `std::stack`. By default both adapt `std::deque`, a "double-ended queue" that supports constant-time insertion and removal at both ends *and* constant-time indexing. A typical deque is implemented as an array of pointers to fixed-size blocks: a hybrid of the contiguous and linked approaches.

4.5 Contiguous versus linked, on real hardware

Here is the comparison as a traditional textbook would give it.

Operation	Vector	Linked list
access the i-th element	$\Theta(1)$	$\Theta(i)$
insert or remove at the back	$\Theta(1)$ amortized	$\Theta(1)$ (doubly linked)
insert or remove at the front	$\Theta(n)$	$\Theta(1)$
insert or remove at a known position	$\Theta(n)$	$\Theta(1)$
search for a value	$\Theta(n)$	$\Theta(n)$

Read naively, the table says: if your program mostly indexes, use a vector; if it mostly inserts and removes in the middle, use a list. That advice was given to generations of students. Let us test it.

Experiment 1: just walking

Both structures can visit all n elements in $\Theta(n)$ time. The program `traverse.cpp` adds up n integers stored three ways: in a vector; in a linked list whose nodes happen to sit in

memory in list order (the best case for a list, as when it was built all at once and never modified); and in a linked list whose nodes are scattered through memory in random order (what a list tends to look like after a long life of insertions and removals). The numbers are nanoseconds *per element*.

n	vector	list, nodes in order	list, nodes scattered
1,000	0.19	0.99	0.99
10,000	0.19	0.99	2.8
100,000	0.19	1.0	7.4
1,000,000	0.20	1.3	89
10,000,000	0.35	2.3	110

Same data, same $\Theta(n)$. At ten million elements the scattered list takes more than **three hundred times** as long as the vector to do the same additions.

> **CHECK YOURSELF 4.4**
>
> Using what you learned in Section 2.6, explain (a) why the scattered list is no slower than the in-order list at $n = 1{,}000$, and (b) why it becomes catastrophically slower as n grows.

Three things work against the linked list, and all of them come from where it lives.

No locality. Each node may be on a different cache line from the last. A vector of `int`s packs sixteen elements into each cache line; a scattered list gets one node per line fetched, at best.

No prefetching, and no parallelism. The address of the next node is not known until the current node has been loaded, because the address is stored *in* the current node. The processor cannot fetch ahead, and it cannot overlap the waiting. This is called **pointer chasing**, and it forces the memory accesses to happen strictly one after another. With a vector, the addresses of the next hundred elements are known in advance, and modern processors exploit that aggressively.

Overhead. A node holding a four-byte `int` also holds an eight-byte pointer, is padded to sixteen bytes, and usually carries some bookkeeping from the memory allocator. The list uses four or more times the memory of the vector to hold the same data, which means fewer elements fit in the cache to begin with.

Figure 4.5 is a small simulation of the effect.

Experiment 2: inserting in the middle

Traversal is the vector's home ground. Surely the list wins when the workload is insertion in the middle, the very thing it was designed for?

This experiment follows one that Bjarne Stroustrup has used in talks for years [54]. Take n random integers and insert them one at a time into a sequence that is kept sorted. For each, find the position where it belongs by scanning from the front, then insert it there.

code/ch04/bench/sorted_insert.cpp

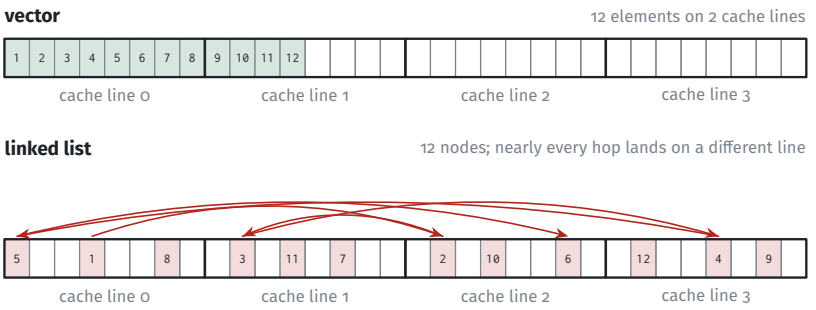


Figure 4.5 Memory drawn as a grid of cache lines. A vector's elements fill consecutive cells, so each line that is fetched serves many accesses. A linked list's nodes are spread over many lines. In the web edition, run both traversals through a small simulated cache and compare the miss counts. [Interactive in the web edition.]

```
template <typename Container>
void insert_all_sorted(Container& c, const std::vector<int>& values) {
    for (int x : values) {
        auto pos = c.begin();
        while (pos != c.end() && *pos < x) ++pos;   // linear scan
        c.insert(pos, x);   // vector: shift the tail. list: relink.
    }
}
```

For the vector, each insertion shifts on average half the elements. For the list, each insertion relinks a couple of pointers.

> **CHECK YOURSELF 4.5**
>
> Predict: which container finishes sooner, and by how much? Does your answer depend on n?

n	`std::vector`	`std::list`	list ÷ vector
10,000	0.007 s	0.072 s	10
20,000	0.025 s	0.35 s	14
40,000	0.10 s	1.9 s	19
80,000	0.40 s	12 s	31
160,000	1.6 s	68 s	44

The vector wins at every size, and its lead *grows* with n.

The reason is in the part of the task that the Big-O table's "known position" quietly assumed away. Before you can insert at a position you must *find* the position, and finding it means walking the sequence. That walk is experiment 1 again, and the list loses it badly. The vector then has to shift its elements, but shifting a contiguous block is among the

fastest things a computer can do: it is a straight-line copy through memory, with perfect locality. Both containers do $\Theta(n)$ work per insertion. The vector's $\Theta(n)$ has a far smaller constant.

(The right way to do this particular task is with neither. Binary search finds the position in a vector in $O(\log n)$, and the structures of Chapter 6 and Chapter 7 make the insertion logarithmic as well.)

So when is a linked list the right choice?

Rarely, but not never. A linked list earns its place when one of its particular promises is what you need.

- **Stable addresses.** Nodes never move. If other parts of the program hold pointers or iterators to elements and those must remain valid while the sequence changes, a vector will not do.
- **Constant-time splicing and removal, given the node.** If you *already hold* a pointer to the node, with no search required, a doubly linked list removes or relocates it in constant time. This is how a least-recently-used cache works (Chapter 10), and how operating system kernels keep track of processes and memory pages, with the links embedded directly in the objects.
- **Elements that are large or expensive to move**, where shifting them would cost more than chasing pointers.
- **A worst-case guarantee per operation**, where the occasional $\Theta(n)$ reallocation of a vector is unacceptable.

And the most important reason to have learned them has nothing to do with lists. Trees, graphs, and hash-table chains are all nodes joined by pointers, and everything you practiced this week (allocating nodes, rewiring links in the right order, cleaning up without leaks) carries over to them directly.

For everyday sequences, the guidance from the people who design and implement C++ is blunt: **use `std::vector` unless you have a specific, measured reason not to** [57].

> **ASK AN LLM**
>
> The claim that linked lists are the right choice for insertion-heavy workloads is one of the most frequently repeated statements in programming education, so it is heavily represented in the text that language models learned from. Ask one:
>
> *When should I choose std::list over std::vector in C++? Be specific.*
>
> Then grade the answer against the measurements in this section. Does it mention cache behavior? Does it distinguish "insertion at a position you already hold" from "insertion at a position you must first find"? Does it give any numbers, and if so, where could they have come from? A good answer exists and a model may well produce it. Your job is to be able to tell, and you can, because you have run the experiment.

THE FOUR QUESTIONS **Linked lists**

Where does it live? One heap allocation per element, anywhere in memory, joined by pointers. The list object holds a head pointer (or a sentinel). Order is explicit in the links, not implicit in the layout.

What does it promise? Following `next` from the head visits every element once and ends (at null, or back at the sentinel). In a doubly linked list, `p->next->prev == p` for every node. Nodes never move, so pointers and iterators to an element stay valid until it is removed.

What does it cost? Insert or remove at the front, or at a node you already hold: $\Theta(1)$. Reaching the i-th element, or finding a value: $\Theta(n)$, with a large constant because of cache misses and pointer chasing. Memory: one or two extra pointers per element, plus allocator overhead.

How do you know? An invariant check that walks the list and counts; differential tests against `std::list` and `std::deque`; and AddressSanitizer, which is at its most valuable on code that allocates and frees many small nodes.

THE FOUR QUESTIONS **Stack and queue**

Where does it live? These are abstract data types; they live wherever the implementation puts them. A stack is naturally a vector with the top at the back. A queue is naturally a ring buffer: a contiguous block used circularly.

What does it promise? Stack: last in, first out. Queue: first in, first out. Nothing else is accessible.

What does it cost? Every operation is $\Theta(1)$, amortized where the underlying block may grow.

How do you know? Differential tests against `std::stack` and `std::queue`; for the ring buffer, tests that force the contents to wrap around the end and to grow while wrapped.

Exercises

1. Draw the list after each statement, including where `head_` points and every `next` pointer.

```
List<int> a;
a.push_front(3);
a.push_front(1);
a.push_front(4);
a.remove(1);
a.reverse();
```

2. Each of these has a bug. Identify it and name its consequence (leak, use after free, lost nodes, or infinite loop).

```
// (a) push_front
Node* fresh = new Node{value, nullptr};
head_ = fresh;
fresh->next = head_;
```

```
// (b) clear
for (Node* p = head_; p != nullptr; p = p->next) delete p;

// (c) remove the node after p
delete p->next;
p->next = p->next->next;
```

3. Add `void push_back(T value)` to `List<T>`. First write it without changing the class's data members; what is its cost? Then add a `tail_` pointer and make it $\Theta(1)$. Which existing member functions must now also maintain `tail_`? (List them all before you look at the code; it is easy to miss one.)
4. Write a member function `T& nth(std::size_t n)` for `List<T>` that returns the element at position `n`. Then explain why this loop is $\Theta(n^2)$, and rewrite it to be $\Theta(n)$.

```
for (std::size_t i = 0; i < list.size(); ++i) total += list.nth(i);
```

5. Rewrite `List<T>::remove` using the pointer-to-pointer technique of `insert_sorted`, so that it has no special case for the first node.
6. Write a function that determines whether a singly linked list (given only a pointer to its first node) contains a cycle, using $O(1)$ extra memory. (Hint: use two pointers that advance at different speeds. This is known as Floyd's "tortoise and hare" algorithm.)
7. Write a function that merges two *sorted* singly linked lists into one sorted list by relinking the existing nodes, allocating nothing. What is its running time?
8. In `DList`, the sentinel node contains a `T value` that is never used. What does this require of the type `T`? Suggest a way to restructure `Node` so that the sentinel does not need to hold a `T`.
9. Implement `void splice(Iterator pos, DList& other)` for `DList<T>`: it moves *all* of `other`'s nodes into this list, just before `pos`, in $\Theta(1)$ time, leaving `other` empty. Check your work with `check_invariant` on both lists.
10. Implement a queue using two stacks: `push` pushes onto the first stack; `pop` pops from the second, and when the second is empty it first moves everything from the first stack onto it. Show that although a single `pop` can take $\Theta(n)$ time, the amortized cost of every operation is $\Theta(1)$. (Use the coin argument: how many times is each element moved in its lifetime?)
11. Extend `balanced` so that it ignores brackets inside double-quoted strings, so that `f("(")` is balanced. Then extend it to report the *position* of the first error.
12. Trace the `RingQueue` through: push 1 through 4 (the first push allocates a buffer of capacity 4); pop twice; push 5 and 6; push 7. For each step give `head_`, `size_`, and the contents of every slot. At which push does `grow` run, and what do the slots look like immediately afterward?
13. Run `traverse.cpp` and `sorted_insert.cpp` on your own machine. Then modify `sorted_insert.cpp` so that each element is a struct containing an `int` key and a `char payload[N]`, and repeat the experiment for `N` $= 64$, 256, and 1024. As the elements

get bigger, shifting them costs more. Is there a size at which the list catches up? Report your numbers.

14. **With a model, as critic.** Write `DList<T>::reverse()`, which reverses the list in place, and test it. Then ask a model to "state the loop invariant of this function, and identify any input for which it fails to restore `p->next->prev == p` for every node." Check its claimed counterexamples by running them.

15. **Lab.** Implement a text editor's *line buffer* in two ways: as a `std::vector<std::string>` and as a `std::list<std::string>`. Support: insert a line at the cursor, delete the line at the cursor, move the cursor up and down, and jump to line k. Write a driver that replays a file of editing commands against either implementation. Construct two command files, one on which the vector is clearly faster and one on which the list is clearly faster, and explain each result in terms of this chapter. (The second file is harder to construct than you might expect. Why?)

Chapter notes

Linked lists date from the mid-1950s; Knuth [33] gives the history along with a thorough treatment of singly, doubly, and circularly linked variants. The idea of specifying a data type by its operations, independent of representation, was formulated by Liskov and Zilles [37], and is one of the organizing ideas of software engineering.

The argument that contiguous storage should be the default on modern hardware has been made forcefully by Stroustrup, whose sorted-insertion experiment [54] is reproduced in this chapter. The underlying facts about caches and memory are laid out by Drepper [21].

Answers to check-yourself questions

4.1 If elements are not going to move, they cannot rely on position in memory to say what order they are in. Each element must instead carry that information itself: the address of the element that follows it. You give up contiguity, and with it the ability to compute where the i-th element is. The only way to reach it is to start at the front and follow i links.

4.2 Pushing each element onto the front as you walk the original would produce a copy in *reverse* order. You could reverse it afterward, or keep a tail pointer as the code does.

4.3 It pushes `(`, then pushes `[`. At `)` it pops the top, which is `[`. That does not match `)`, so it returns `false` at the third character. The count of each kind of bracket is right, but the nesting is wrong, and a stack is exactly what detects nesting.

4.4

(a) A thousand 16-byte nodes occupy 16 kilobytes, which fits entirely in the processor's fastest cache. Once loaded, every node is a cache hit regardless of the order in which they are visited. (b) Ten million nodes occupy 160 megabytes, far more than any cache. In list order, consecutive nodes are adjacent in memory, so cache lines and prefetching still help. In scattered order, each `next` pointer leads to an unpredictable address that is almost never in the cache: a full trip to main memory, about a hundred nanoseconds, for every single element.

4.5 See the table. If you predicted that the list wins for large n, you are in the good company of nearly everyone who has seen only the Big-O table.

CHAPTER 5

Hashing

WEEK 5 OF 10

THIS WEEK

By the end of this week you should be able to:

- explain how a hash table turns a key into an array index, and why collisions are unavoidable;
- say what makes a hash function good, and recognize a bad one;
- implement a hash table with separate chaining and one with linear probing, including growth and deletion;
- define the load factor and use it to state the expected cost of each operation, and the worst case;
- use `std::unordered_map` and `std::unordered_set` fluently, with your own key types;
- explain what a Bloom filter is for, and what it gives up to achieve it.

You have a million names and need to answer, over and over, the question "is this name on the list?" So far the best tool we have is a sorted vector with binary search: about twenty comparisons per question, and painfully slow insertion.

CHECK YOURSELF 5.1

Could a lookup take one or two steps, *no matter how many names there are*? Not twenty: one or two. What would have to be true for that to be possible? Think about the one operation from Chapter 1 that takes constant time regardless of size.

The structure that results, the **hash table**, is the workhorse of modern software. It is behind Python's `dict` and `set`, JavaScript's objects, Java's `HashMap`, the symbol tables in compilers, database indexes, caches, and routers. After the array it is probably the most-used data structure there is. Many textbooks place it after several weeks of trees; this one puts it first, because it is more important in practice and needs nothing you do not already have: an array, a little arithmetic, and the linked chains or the modular wraparound of last week.

5.1 From keys to indices

Two abstract data types run through the rest of the book.

A **set** stores a collection of distinct *keys* and answers membership questions: insert a key, remove a key, is this key present?

A **map** (also called a dictionary or associative array) stores *key–value pairs* and looks up the value belonging to a key: a word and its definition, a student ID and a transcript, a variable name and its type. A set is a map with nothing in the value slot, so we will mostly talk about maps.

If the keys happen to be small whole numbers, there is a wonderful implementation: use the key as an array index. To count how many times each byte value occurs in a file, make an array of 256 counters and, for each byte `b`, increment `count[b]`. Insert, look up, and remove are single array accesses. This is called **direct addressing**, and when it applies, nothing beats it.

It rarely applies. Keys are usually strings, or large numbers, or structs, and the set of *possible* keys is astronomically larger than the set of keys actually in use. There are thousands of times more possible twenty-letter names than there are stars in the observable universe. We cannot have an array slot for each.

Hashing rescues the idea in two steps.

1. A **hash function** converts a key of any type into a large integer, the key's *hash value*. For a 64-bit hash that is a number between 0 and about 1.8×10^{19}.
2. The hash value is reduced to an index into an array of m slots, called **buckets**, most simply by taking the remainder: `index = hash % m`.

To store a pair, compute its key's bucket and put it there. To look up a key, compute its bucket and look there. Neither step involves searching, and neither depends on how many keys the table holds.

What makes a hash function good

A hash function must be **deterministic**: the same key must always produce the same hash value, or we would never find anything again. It follows that keys which compare equal must hash equal. It should be **fast**, since it runs on every operation. And it should **spread the keys out**: the hash values of the keys in use should look as though they were chosen at random, with every bucket equally likely, no matter what patterns the keys themselves contain.

The last requirement is the hard one, and getting it wrong is easy. Here is a hash function for strings that many people invent on their first try.

code/ch05/hash.hpp

```
// A poor hash: add up the character codes. Anagrams collide, and the
// results for short strings are all crowded into a small range.
inline std::uint64_t sum_hash(const std::string& s) {
    std::uint64_t h = 0;
    for (unsigned char c : s) h += c;
    return h;
}
```

And here is one that is used in practice, known as FNV-1a after its inventors Fowler, Noll, and Vo.

code/ch05/hash.hpp

```
// FNV-1a: fold each byte into the running value, then multiply by a
// large prime, which spreads that byte's influence into the higher bits.
inline std::uint64_t fnv1a(const std::string& s) {
    std::uint64_t h = 14695981039346656037ull;   // "offset basis"
    for (unsigned char c : s) {
        h ^= c;
        h *= 1099511628211ull;                   // the FNV prime
    }
    return h;
}
```

They look similar: both walk the string and fold each character into a running value. The difference is that FNV-1a multiplies by a large prime after each character, which spreads every character's influence upward across the higher bits of the result, and makes the order of the characters matter.

To see what the difference means, hash the 10,000 strings `user0` through `user9999` into a table of 16,384 buckets with each function.

Hash function	Buckets used	Longest chain of colliding keys	Average comparisons to find a key
`sum_hash`	91	615	203
`fnv1a`	8,072	4	1.2

With `sum_hash`, ten thousand keys land in just 91 of the sixteen thousand buckets. The sum of the character codes of a short string is a small number, and similar strings have similar sums; `user19` and `user91` have the *same* sum. The table has degenerated into a few long lists. With FNV-1a the keys spread over about half the buckets, which is what randomly thrown keys would do, and almost every lookup finds its key at once.

For integer keys the temptation is to use the integer itself as its hash. Sometimes that is fine. But if the keys follow a pattern, say they are all multiples of 1024, and the table size is a power of two, they will all land in a handful of buckets. A good table therefore scrambles integer hashes first.

code/ch05/hash.hpp

```
// Scrambles the bits of an integer, so that keys which differ only in
// their high bits, or follow a regular pattern, still spread out.
// (This is the finalizer of the SplitMix64 generator.)
inline std::uint64_t mix(std::uint64_t x) {
    x ^= x >> 30;
    x *= 0xbf58476d1ce4e5b9ull;
    x ^= x >> 27;
    x *= 0x94d049bb133111ebull;
    x ^= x >> 31;
    return x;
}
```

> **PITFALL**
>
> Do not invent hash functions. Designing one that spreads real-world keys well is a specialty, and plausible-looking homemade functions like `sum_hash` fail in ways that are invisible until the table is under load. Use the standard library's `std::hash`, or a published function with a name. And if you ask a model for one, **check its constants against an authoritative source**: the two long numbers in `fnv1a` above are part of the definition, and a model reproducing them from memory can get a digit wrong while the surrounding code looks impeccable. A hash function with a wrong constant still compiles and still returns numbers. It just quietly does its job worse.

Collisions are certain

However good the hash function, two different keys will sometimes land in the same bucket. This is a **collision**. If there are more possible keys than buckets, some bucket must receive more than one; that much is the pigeonhole principle. What is surprising is how *soon* collisions begin.

> **CHECK YOURSELF 5.2**
>
> A table has 365 buckets and a perfect hash function that sends each key to a uniformly random bucket. How many keys can you insert before a collision becomes more likely than not? Guess before you calculate.

So a hash table cannot be designed to avoid collisions. It must be designed to *handle* them, and the two major families of hash table differ in how. Both rest on the same quantity. If a table holds n entries in m buckets, its **load factor** is

$$\alpha = \frac{n}{m},$$

the average number of entries per bucket. Keeping α small keeps collisions rare, and costs memory. Everything about hash-table performance is a function of α.

5.2 Separate chaining

The most direct way to handle collisions is to let each bucket hold more than one entry. In **separate chaining**, each bucket is a small collection, traditionally a linked list and hence a "chain," of all the entries whose keys hash there.

code/ch05/chained_map.hpp

```
struct Entry {
    K key;
    V value;
};
// One chain per bucket. Each chain is a small vector of entries.
std::vector<std::vector<Entry>> buckets_;
std::size_t size_ = 0;   // total number of entries
Hash hash_;
```

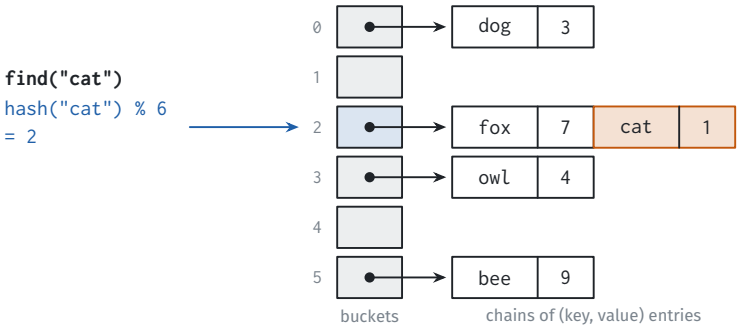


Figure 5.1 Separate chaining. The array holds one chain per bucket. A key's hash selects a bucket; the key is then searched for, or added, within that bucket's short chain. Here `fox` and `cat` have collided in bucket 2.

Our chains are small `std::vectors` instead of linked lists, for the reasons the last chapter gave: a few entries stored contiguously are cheaper to scan than a few nodes scattered in the heap. Since every member manages itself, the class follows the rule of zero.

Finding a key's bucket is one line.

code/ch05/chained_map.hpp

```
std::size_t bucket_of(const K& key) const {
    return hash_(key) % buckets_.size();
}
```

The class takes its hash function as a template parameter, `Hash`, defaulting to the standard library's `std::hash<K>`. A hash function in C++ is an object that can be called like a function; `hash_(key)` calls it.

Lookup scans the one chain where the key must be, if it is anywhere.

code/ch05/chained_map.hpp

```
// Returns a pointer to the value stored under key, or nullptr.
V* find(const K& key) {
    for (Entry& e : buckets_[bucket_of(key)]) {
        if (e.key == key) return &e.value;
    }
    return nullptr;
}
```

`find` returns a *pointer* to the value: the address of the value if the key is present, `nullptr` if it is not. This is a handy use for pointers, as a "reference that might not exist," and it lets the caller both test for presence and reach the value with one lookup.

code/ch05/map_test.cpp

```
ChainedMap<std::string, int> ages;
ages.put("Ada", 36);
ages.put("Grace", 85);
```

```
ages.put("Ada", 37);                  // replaces the earlier value

if (int* a = ages.find("Ada")) {      // found: a points at the value
    *a += 1;
}
assert(ages.find("Alan") == nullptr); // not found
```

Insertion looks for the key first, since a map holds at most one value per key.

code/ch05/chained_map.hpp

```
// Associates value with key, replacing any previous value.
void put(const K& key, V value) {
    if (V* existing = find(key)) {
        *existing = std::move(value);
        return;
    }
    if (size_ + 1 > buckets_.size()) rehash(2 * buckets_.size());
    buckets_[bucket_of(key)].push_back(Entry{key, std::move(value)});
    ++size_;
}
```

Removal finds the entry in its chain and deletes it. Order within a chain means nothing, so instead of shifting the chain's later entries down, we overwrite the doomed entry with the last one and shrink the chain by one: a constant-time removal from the middle of a vector, and a trick worth remembering.

code/ch05/chained_map.hpp

```
bool erase(const K& key) {
    std::vector<Entry>& chain = buckets_[bucket_of(key)];
    for (std::size_t i = 0; i < chain.size(); ++i) {
        if (chain[i].key == key) {
            chain[i] = std::move(chain.back());   // order within a chain
            chain.pop_back();                     // does not matter
            --size_;
            return true;
        }
    }
    return false;
}
```

Growing

If the number of buckets were fixed, the chains would lengthen as entries were added, and the table would slow to a crawl. So, like a vector, a hash table grows. When an insertion would push the load factor above a threshold (here 1.0), `put` calls `rehash` to move everything into a table with twice as many buckets.

code/ch05/chained_map.hpp

```
// Move every entry into a table with new_count buckets.
void rehash(std::size_t new_count) {
    std::vector<std::vector<Entry>> old = std::move(buckets_);
```

```
    buckets_.assign(new_count, {});
    for (auto& chain : old) {
        for (Entry& e : chain) {
            buckets_[bucket_of(e.key)].push_back(std::move(e));
        }
    }
}
```

Notice that entries cannot simply be copied to the same positions in the bigger table. A key's bucket is `hash % m`, and m has changed, so every entry must be assigned to its bucket afresh. Rehashing costs $\Theta(n)$. But because the table doubles, the argument from Section 3.2 applies without change: the cost averages out to a constant per insertion.

What it costs

Every operation computes a hash, which takes time proportional to the size of the key and is independent of n, and then scans one chain. So everything depends on the length of the chains.

If the hash function spreads keys uniformly, the average chain has α entries. An unsuccessful search scans a whole chain, about α comparisons. A successful one finds its key after passing, on average, half of the *other* entries in its chain: about $1 + \alpha/2$ comparisons. With α held below a constant by rehashing, every operation takes **expected constant time**: $\Theta(1)$, independent of n.

The word "expected" is doing real work, and so is the "if." In the *worst* case, every key lands in one bucket, the table is a single list, and every operation is $\Theta(n)$. The table in the previous section showed a hash function getting a fair way toward that worst case with entirely innocent keys.

Operation	Expected	Worst case
find, insert, erase	$\Theta(1)$	$\Theta(n)$
insert that triggers a rehash	$\Theta(n)$, but amortized $\Theta(1)$	$\Theta(n)$
visit every entry	$\Theta(n+m)$	$\Theta(n+m)$

This is a different kind of guarantee from any we have seen. A vector's amortized bound was unconditional. A hash table's constant time is a statement about averages, resting on the assumption that the hash function scatters the keys you actually have. Most of the time the assumption is sound and hash tables are spectacularly fast. Section 5.4 looks at what happens when someone sets out to violate it on purpose.

5.3 Open addressing

Separate chaining stores entries outside the bucket array, in chains that each need their own allocation. The alternative is to store every entry *in the array itself*, at most one per slot. When a key's slot is already taken, we look for another slot according to a fixed rule. This is **open addressing**, and the sequence of slots examined is called the **probe sequence**.

The simplest rule is **linear probing**: if the slot is taken, try the next one, and the next, wrapping around at the end of the array, until an empty slot turns up.

code/ch05/probing_map.hpp

```
enum class State : unsigned char { Empty, Full, Deleted };

struct Slot {
    K key{};
    V value{};
    State state = State::Empty;
};

std::vector<Slot> slots_;   // capacity is always a power of two
std::size_t size_ = 0;      // slots that are Full
std::size_t used_ = 0;      // slots that are Full or Deleted
Hash hash_;
```

Each slot records whether it is empty, full, or (a third state we will need shortly) deleted. The slot where a key would live if nothing were in its way is its *home*.

code/ch05/probing_map.hpp

```
// The slot where key "wants" to live. The capacity is a power of two,
// so (capacity - 1) is a mask of low bits; mix() makes sure those low
// bits depend on the whole hash value.
std::size_t home(const K& key) const {
    return mix(hash_(key)) & (slots_.size() - 1);
}
```

Keeping the capacity a power of two lets us replace the `%` operator, which is a slow division, with a bitwise AND against `capacity - 1`, which keeps just the low bits. The same trick wraps the index around the end of the array. The catch is that only the low bits of the hash are used, so they had better be well scrambled; that is why `home` passes the hash through `mix` first.

To insert a key, start at its home and walk forward to the first slot that is not full. To find a key, start at its home and walk forward, comparing keys, until you find it or reach an **empty** slot.

code/ch05/probing_map.hpp

```
V* find(const K& key) {
    std::size_t i = home(key);
    while (slots_[i].state != State::Empty) {      // an Empty slot ends
        if (slots_[i].state == State::Full && slots_[i].key == key) {
            return &slots_[i].value;
        }
        i = (i + 1) & (slots_.size() - 1);          // next slot, wrapping
    }
    return nullptr;
}
```

The stopping rule is the heart of the method. If the key were in the table, it would have been placed in the first free slot at or after its home. So if the search reaches a slot that is empty, the key cannot be further on. Figure 5.2 lets you try both kinds of table.

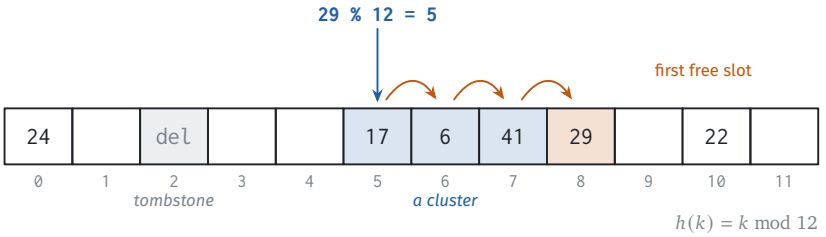


Figure 5.2 Linear probing. The key 29 hashes to slot 5, which is occupied, as are 6 and 7; it is stored in slot 8, the first free slot after its home. A later search for 29 retraces the same path. In the web edition, insert, find, and erase keys in either a chained or a probing table, and watch the load factor and the probe counts. [Interactive in the web edition.]

Deletion and tombstones

Removing an entry from a probing table is subtle, and it is a favorite source of bugs.

> **CHECK YOURSELF 5.3**
>
> Keys A, B, and C all have home slot 3, and were inserted in that order, so they occupy slots 3, 4, and 5. Now B is erased by marking slot 4 *empty*. What happens on a subsequent search for C?

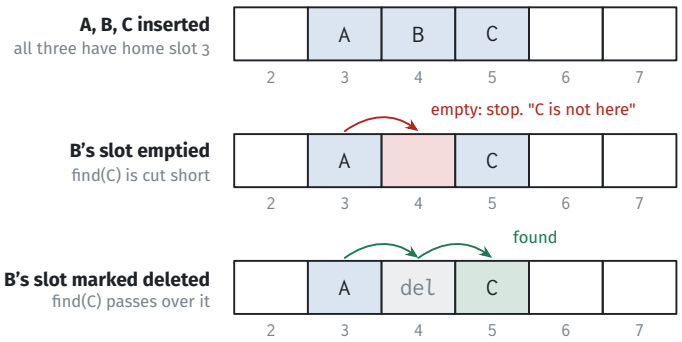


Figure 5.3 Why erasing needs tombstones. A, B, and C share home slot 3. If erasing B simply empties slot 4 (middle), a search for C stops there and wrongly reports C missing. Marking the slot "deleted" instead (bottom) keeps the probe path intact: searches pass over it, and a later insertion may reuse it.

The fix is the third state. Erasing marks the slot **deleted**, leaving what is called a **tombstone**. A search treats a tombstone as occupied and keeps going. An insertion may reuse it.

code/ch05/probing_map.hpp

```
bool erase(const K& key) {
    std::size_t i = home(key);
    while (slots_[i].state != State::Empty) {
        if (slots_[i].state == State::Full && slots_[i].key == key) {
            slots_[i].state = State::Deleted;   // leave a tombstone
            --size_;
            return true;
        }
        i = (i + 1) & (slots_.size() - 1);
    }
    return false;
}
```

Tombstones keep the table correct, at a price: they lengthen probe sequences just as live entries do. A table that has seen many insertions and deletions can be clogged with them even if it holds few live entries. So the load factor that matters counts tombstones as well, and when it gets too high the table is rebuilt, which discards them.

code/ch05/probing_map.hpp

```
void put(const K& key, V value) {
    if (V* existing = find(key)) {
        *existing = std::move(value);
        return;
    }
    if (2 * (used_ + 1) > slots_.size()) rebuild();   // keep it half empty
    std::size_t i = home(key);
    while (slots_[i].state == State::Full) {
        i = (i + 1) & (slots_.size() - 1);
    }
    if (slots_[i].state == State::Empty) ++used_;      // else: reuse a
    ↪ tombstone
    slots_[i].key = key;
    slots_[i].value = std::move(value);
    slots_[i].state = State::Full;
    ++size_;
}
```

code/ch05/probing_map.hpp

```
// Re-insert every live entry into a fresh table. This drops all
// tombstones, and doubles the capacity if the live entries need it.
void rebuild() {
    std::size_t new_capacity = slots_.size();
    if (4 * (size_ + 1) > new_capacity) new_capacity *= 2;
    std::vector<Slot> old = std::move(slots_);
    slots_.assign(new_capacity, Slot{});
    size_ = 0;
```

```
    used_ = 0;
    for (Slot& s : old) {
        if (s.state == State::Full) put(s.key, std::move(s.value));
    }
}
```

BY HAND FIRST

Take a table of eight slots and the hash function $h(k) = k \bmod 8$. By hand, with linear probing: insert 3, 11, 19, 4, 27; erase 11; search for 19; insert 35; search for 12. Draw the table after every step, marking tombstones, and count the probes for each operation. Then check yourself with the interactive figure in the web edition.

Tracing a probing table is precise, mechanical work in which one slip makes everything after it wrong. It is exactly the kind of step-by-step state tracking at which language models are *least* dependable: ask one to carry out a sequence like this and it will often be right and sometimes confidently lose track of a tombstone. You need to be the one who can do it reliably.

Clustering, and what it costs

Linear probing has a characteristic weakness. Occupied slots tend to form contiguous runs, called **clusters**, and clusters feed on themselves: a key that hashes anywhere into a cluster is added to its end, making it longer, and so a still bigger target. As the table fills, the clusters merge and the probe sequences lengthen rapidly.

Knuth analyzed this in the early 1960s; he has called it the first nontrivial algorithm he ever analyzed satisfactorily, and credits it with setting the direction of his life's work. For a table with load factor α, the expected number of slots examined is approximately

$$\frac{1}{2}\left(1 + \frac{1}{1-\alpha}\right) \text{ for a successful search, and } \frac{1}{2}\left(1 + \frac{1}{(1-\alpha)^2}\right) \text{ for an unsuccessful one}$$

[34].

Load factor α	Probes, key present	Probes, key absent
0.25	1.2	1.4
0.50	1.5	2.5
0.75	2.5	8.5
0.90	5.5	50.5
0.95	10.5	200.5

Up to about half full, linear probing is excellent. Beyond three-quarters it deteriorates quickly, and near full it is awful. That is why our table rebuilds itself whenever it would become more than half full. (With chaining, by contrast, α can exceed 1 without disaster.) Other probing rules, such as *quadratic probing* and *double hashing*, spread the probes out to reduce clustering, at some cost in locality.

Which is faster?

On paper, the two designs are both expected constant time. On hardware they are not equal, for the reason that has now come up in three chapters running. A probing table is one contiguous array, and a probe sequence is a short walk through adjacent slots: usually a single cache line. A chained table follows a pointer from the bucket array to a separately allocated chain.

Here are measured times for a successful lookup of a random integer key, in nanoseconds, in five structures: a sorted vector with binary search; `std::map`, a balanced tree we will study in Chapter 7; `std::unordered_map`, the standard library's chained hash table; and the two tables from this chapter.

n	sorted vector	`std::map`	`unordered_map`	`ChainedMap`	`ProbingMap`
1,000	37	35	7.3	6.4	4.5
10,000	47	63	10	6.0	3.3
100,000	64	155	11	9.2	5.1
1,000,000	100	558	31	21	11
10,000,000	183	1,025	39	24	14

All three hash tables beat both logarithmic structures at every size: by a factor of three to six against the sorted vector, and by a factor that grows from about five to more than twenty-five against the tree. And among the hash tables, the probing table is the fastest at every size, roughly one and a half times as fast as our chained table and, from ten thousand keys up, two to three times as fast as `std::unordered_map`. (Even a hash table slows down as it grows, you will notice. The number of steps is constant, but one cache miss per lookup is unavoidable once the table is larger than the cache, since a good hash function sends each lookup to an unpredictable place. That is the price of scattering.)

> **IN THE WILD**
>
> The fastest general-purpose hash tables in use today are open-addressing designs. Google's "Swiss table," published in its Abseil library as `flat_hash_map` [36], keeps one byte of metadata per slot in a separate dense array and uses the processor's vector instructions to examine sixteen slots at a time; Rust's standard `HashMap` is a port of the same design, and Python's `dict` is also open addressing. C++'s `std::unordered_map`, by contrast, is in effect required to use chaining by guarantees in its interface (for instance, that a reference to an entry stays valid when the table grows), which is why performance-sensitive C++ code so often replaces it with a third-party table.

5.4 Using hash tables well

In your own programs you will use the standard library's hash containers far more often than you write one: `std::unordered_map<K, V>` and `std::unordered_set<K>`. Here is the canonical first program, counting how often each word occurs in a text.

code/ch05/wordcount.cpp

```
std::unordered_map<std::string, int> count;
std::string word;
while (std::cin >> word) {
    ++count[word];   // inserts word with count 0 if it is new
}
```

The expression `count[word]` looks up `word` and returns a reference to its value. If the word is not yet in the map, `operator[]` *inserts it* with a default value (zero, for an `int`) and returns a reference to that. So `++count[word]` does the right thing for both new and repeated words.

> **PITFALL**
>
> That convenience has a sharp edge: `map[key]` on a missing key *adds the key*. Writing `if (count["zebra"] > 0)` to test whether a word occurs will silently insert `"zebra"` with a count of zero. To look without inserting, use `count.contains(key)` or `count.find(key)`.

To report the most frequent words we need them in order of frequency, and here we meet the main limitation of hashing.

code/ch05/wordcount.cpp

```
// A hash table has no useful order, so copy the entries into a
// vector and sort that: most frequent first, ties alphabetically.
std::vector<std::pair<std::string, int>> entries(count.begin(), count.end());
std::sort(entries.begin(), entries.end(),
          [](const auto& a, const auto& b) {
              if (a.second != b.second) return a.second > b.second;
              return a.first < b.first;
          });
for (std::size_t i = 0; i < entries.size() && i < 5; ++i) {
    std::cout << entries[i].second << ' ' << entries[i].first << '\n';
}
```

A hash table keeps its entries in no meaningful order; the whole point of a good hash function is to destroy order. Iterating over an `unordered_map` visits the entries in an arbitrary sequence that can change whenever the table grows. If you need the smallest key, or all keys in a range, or the entries in sorted order, a hash table cannot help, and you must sort a copy, as here, or use a structure that maintains order. Those are the subject of the next two chapters.

Your own keys

The standard library knows how to hash numbers, strings, and pointers. To use your own type as a key you must supply two things: an equality test, and a hash function consistent with it, so that equal keys always get equal hashes.

code/ch05/map_test.cpp

```
struct Point {
    int x;
    int y;
```

```
    bool operator==(const Point& other) const = default;
};

// A hash for Point: combine the two coordinates, then scramble.
struct PointHash {
    std::size_t operator()(const Point& p) const {
        std::uint64_t packed = (static_cast<std::uint64_t>(static_
        ↪  cast<std::uint32_t>(p.x)) << 32) |
                               static_cast<std::uint32_t>(p.y);
        return static_cast<std::size_t>(mix(packed));
    }
};
```

code/ch05/map_test.cpp

```
std::unordered_set<Point, PointHash> visited;
visited.insert(Point{2, 7});
visited.insert(Point{2, 7});          // already present: no effect
assert(visited.size() == 1);
assert(visited.contains(Point{2, 7}));
assert(!visited.contains(Point{7, 2}));
```

`operator==` with `= default` asks the compiler for member-by-member comparison. `PointHash` is a small class with a function-call operator, which is what the container's `Hash` parameter expects. It packs the two coordinates into one 64-bit number and scrambles it.

Two cautions. First, the obvious ways to combine fields are bad. Hashing a point as `x + y` or as `x ^ y` (exclusive or) makes `(2, 7)` collide with `(7, 2)`, and the second sends every point on the diagonal to zero. Second, **never modify a key while it is in a hash table**. Its position was computed from its old value. Change it and the entry is stranded in the wrong bucket, where no lookup will ever find it again.

> **ASK AN LLM**
>
> Models are good at spotting weak hash functions, because the weaknesses are structural. As a critic:
>
> > *Here is a hash function I wrote for a struct. Without rewriting it, name specific pairs of distinct keys that collide, and describe any* patterns *of keys that would pile up in few buckets.*
>
> Verify what it says by actually hashing the pairs it names. Then fix the function yourself, preferably by combining standard-library hashes with a mixing step, not by inventing new arithmetic.

When the keys are chosen by an enemy

The expected-constant-time guarantee assumes that the keys are not correlated with the hash function. Suppose instead that someone who knows your hash function deliberately sends you thousands of keys that all hash to the same bucket. Every insertion then scans one ever-longer chain, and inserting n keys takes $\Theta(n^2)$ time.

This is a real attack. In 2003 Crosby and Wallach showed that many widely used programs could be made to stall by feeding them such keys [18], and in 2011 the same

idea was used to show that a single small, carefully constructed web request could occupy a server's processor for minutes. Web frameworks, it turned out, obligingly put every parameter of a request into a hash table. The attack is called **hash flooding**.

The standard defense is to make the hash function unpredictable: mix a random secret, chosen when the program starts, into every hash, using a function designed so that an outsider cannot work out which keys collide. SipHash [3] was created for this purpose and became the default string hash in Python, Ruby, and Rust, among others. One visible consequence: where a container's iteration order follows its hash values, as with Rust's `HashMap` or a Python `set` of strings, that order can differ from one run of a program to the next.

The lesson is broader than hash tables. An *average-case* guarantee is only as good as the assumption behind it, and on the open internet, inputs are not random. They are chosen by whoever most wants your program to fail.

5.5 Bloom filters

We end with a structure that shows how far the idea of hashing can be pushed if you are willing to relax what "correct" means.

Suppose you have a hundred million keys, say every URL known to be malicious, and you want to test membership very fast and in very little memory. A hash set of a hundred million strings would occupy gigabytes. A **Bloom filter** [10] can do the job in about 120 megabytes, a bit over one byte per key, *without storing the keys at all*. The price is that it sometimes gives a wrong answer, in one direction only.

- If the filter says a key is **absent**, the key is definitely absent.
- If the filter says a key is **present**, the key is *probably* present. Occasionally it is not: a **false positive**.

The structure is an array of m bits, all initially zero, and k different hash functions. To **add** a key, compute its k hash values and set those k bits to one. To **test** a key, compute the same k positions and look: if any of those bits is zero, the key was certainly never added, because adding it would have set that bit. If all k are one, the key was probably added, or else other keys happened to set all k of its bits between them.

code/ch05/bloom.hpp

```
class BloomFilter {
public:
    // bits: the size of the bit array. hashes: how many bits each key sets.
    BloomFilter(std::size_t bits, int hashes) : bits_(bits, false), k_(hashes)
↪ {}

    void add(const std::string& key) {
        for (int i = 0; i < k_; ++i) bits_[position(key, i)] = true;
    }

    // false: the key was definitely never added.
    // true:  the key was probably added (or its bits were set by others).
    bool might_contain(const std::string& key) const {
```

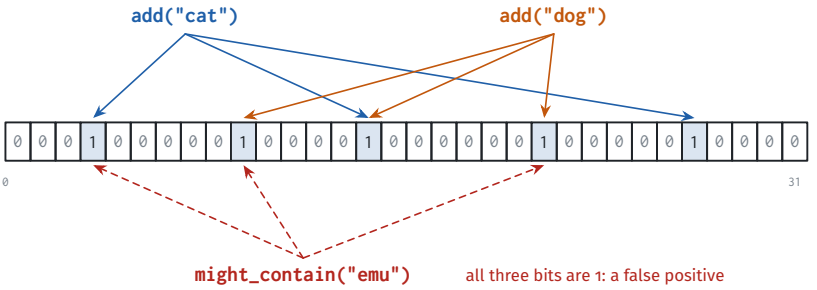


Figure 5.4 A Bloom filter with 32 bits and three hash functions, after adding two keys. Testing a key checks its three bits. A key whose three bits were all set by *other* keys is reported present although it was never added: a false positive. In the web edition, add keys and test others to see the false-positive rate rise as the filter fills. [Interactive in the web edition.]

```
        for (int i = 0; i < k_; ++i) {
            if (!bits_[position(key, i)]) return false;
        }
        return true;
    }

private:
    // The i-th hash of key. Two real hash values are combined to simulate
    // as many hash functions as we need:  h1 + i * h2.
    std::size_t position(const std::string& key, int i) const {
        std::uint64_t h1 = fnv1a(key);
        std::uint64_t h2 = mix(h1) | 1;   // odd, so it is never zero
        return (h1 + static_cast<std::uint64_t>(i) * h2) % bits_.size();
    }

    std::vector<bool> bits_;
    int k_;
};
```

The code simulates k independent hash functions by combining two real hash values, a standard technique. Note what is *missing*: there is no way to list the keys, no way to count them, and no way to remove one, since clearing a key's bits might clear bits that other keys depend on.

How wrong is it?

After n keys have been added to a filter of m bits with k hash functions, the probability that a particular bit is still zero is about $e^{-kn/m}$. A false positive needs all k of a key's bits to be one, so the false-positive rate is approximately

$$\left(1 - e^{-kn/m}\right)^k.$$

For a given number of bits per key, m/n, the rate is lowest when $k \approx (m/n) \ln 2$, about 0.7 times the bits per key. Here is that formula against a measurement, for a hundred thousand keys tested against a million non-members.

Bits per key	k	Measured false positives	Predicted
4	3	14.6%	14.7%
8	6	2.13%	2.16%
10	7	0.81%	0.82%
16	11	0.042%	0.046%

Ten bits per key gives an error rate under one percent, and each further bit and a half or so halves it again. The key itself might be a hundred bytes long; the filter does not care.

IN THE WILD

Bloom filters appear wherever a cheap "definitely not" can save an expensive operation. The storage engines behind many large databases keep their data in a series of sorted files on disk, and keep a Bloom filter for each file in memory. Before reading a file to look for a key, the engine asks the filter, and most of the time the filter says "not in this one," saving a disk read (Chapter 10 returns to this). Web browsers have used them to check URLs against lists of dangerous sites without downloading the lists, and content delivery networks use them to avoid caching objects that are requested only once.

The Bloom filter is the first structure in this book whose answer to *what does it promise?* is deliberately weaker than "the right answer." That is a legitimate thing to promise, provided the weakness is stated exactly and the one-sidedness is used properly: the filter's "no" is trusted absolutely, and its "yes" is treated as "go and check."

ASK AN LLM

Tracing hash tables by hand is good practice, and a model can generate exercises, though it should not be trusted to grade them. Try:

> *Give me a hash table exercise: a table size, the hash function h(k) = k mod m, and a sequence of about ten inserts, erases, and lookups using linear probing with tombstones. Do not show the answer.*

Work it on paper, then check your final table against the interactive figure in this chapter, not against the model. If you do ask the model for its answer and it disagrees with yours, trace the disputed step together; you will sometimes find that the mistake is the model's.

THE FOUR QUESTIONS **Hash tables**

Where does it live? An array of buckets, indexed by a hash of the key. With chaining, each bucket leads to a separate small chain. With open addressing, the entries are in the array itself, and a key lives at or shortly after its home slot.

What does it promise? A key is always in the bucket (or on the probe path) that its hash selects.

Equal keys have equal hashes. No ordering of any kind. In a probing table, no empty slot lies between a key's home and its actual position, which is why erasure leaves tombstones.

What does it cost? Find, insert, erase: expected $\Theta(1)$ when the hash function spreads keys well and the load factor is bounded; $\Theta(n)$ in the worst case. Growth is amortized. Memory: a constant factor more than the entries need. Probing tables are markedly faster in practice because they stay in one block.

How do you know? Differential testing against `std::unordered_map`, with both a small key range (to force repeats, replacements, and erasures of present keys) and a large one (to force growth); an invariant check that every entry is where its hash says; and a look at the chain-length distribution, which reveals a bad hash function at once.

THE FOUR QUESTIONS **Bloom filter**

Where does it live? An array of bits. Each key is represented only by the k bit positions its hashes select. The keys themselves are not stored.

What does it promise? No false negatives, ever. False positives at a predictable rate determined by the bits per key. No deletion, no enumeration.

What does it cost? $\Theta(k)$ per operation, independent of the number of keys. About ten bits per key for a one-percent error rate.

How do you know? Test that every added key is reported present. Measure the false-positive rate on keys known to be absent, and compare it with the formula.

Exercises

1. A table has 10 buckets and uses $h(k) = k \bmod 10$. Insert the keys 12, 44, 13, 88, 23, 94, 11, 39, 5, in that order, (a) using separate chaining, and (b) using linear probing. Draw both tables. For each, what is the largest number of key comparisons any successful search would need?
2. Continuing part (b): erase 23 and then 94, using tombstones. Now search for 5, and count the slots examined. Then insert 33. Which slot does it go in?
3. Explain what is wrong with each of these as a hash function for strings: (a) the length of the string; (b) the character code of the first character; (c) a random number; (d) the address of the string object in memory.
4. Suppose the keys of a `ChainedMap<int, int>` are all multiples of 1024, and the table uses `std::hash<int>`, which on many systems returns the integer unchanged. Our table always has a power-of-two number of buckets. Predict the chain lengths after inserting the first 1,000 multiples of 1024. Verify with `longest_chain()`. Then fix the problem in two different ways: by changing `bucket_of`, and by changing how the number of buckets is chosen. (Many standard library implementations keep the bucket count prime. Why does that help?)
5. In `ProbingMap`, `put` begins by calling `find`, and only then looks for a free slot. Explain why it would be wrong to insert a new key into the first tombstone encountered without first completing the search for the key.

6. Show that if a probing table never rebuilt itself, a long enough sequence of alternating insertions and erasures, of *distinct* keys, would eventually make every unsuccessful search examine every slot, even though the table never holds more than one live entry.
7. Using the formulas in Section 5.3, compute the expected probes for successful and unsuccessful searches at load factors 0.6, 0.8, and 0.99. A colleague proposes saving memory by letting a probing table fill to 95% before growing. Write two sentences in reply.
8. Write a function that determines whether two strings are anagrams of each other in expected $\Theta(n)$ time, using a hash map from characters to counts. Then write a function that, given a vector of words, groups them into sets of mutual anagrams. What should the *key* of your map be?
9. Given a vector of integers and a target sum, determine in expected $\Theta(n)$ time whether any two elements add up to the target. (The brute-force method of Chapter 2 is quadratic.)
10. Implement a `HashSet<K>` class with `insert`, `contains`, `erase`, and `size`, using either collision strategy. You may *not* simply wrap a map: store keys only. Test it differentially against `std::unordered_set`.
11. A Bloom filter has $m = 80{,}000$ bits and will hold $n = 10{,}000$ keys. What k minimizes the false-positive rate, and what is the rate? If you could afford only $k = 2$ (hashing is expensive for your keys), what would the rate be?
12. A *counting* Bloom filter replaces each bit with a small counter: adding a key increments its k counters, and removing a key decrements them. Explain why this permits deletion. What new way can it now fail? What does it cost in space?
13. **With a model, as critic.** Write a hash function for a struct with three `int` fields. Before testing it, ask a model to predict which patterns of keys will collide. Then *measure*: hash every triple with fields in the range 0 to 49 (125,000 keys) into 262,144 buckets and report the longest chain and the number of empty buckets, alongside the same figures for `mix` applied to a packed combination of the fields. Were the model's predictions right?
14. **Lab.** Build a spell checker. Load a word list (your system probably has one at `/usr/share/dict/words`) into (a) your own probing hash set, (b) a `std::unordered_set`, (c) a sorted `std::vector` searched with binary search, and (d) a Bloom filter sized for a 1% false-positive rate. For each, measure the time to build it, the time to check every word of a large text, and the memory used. Present the results as a table and state which you would ship in a spell checker for a phone, and why. For the Bloom filter, also report how many misspelled words in your test text it wrongly accepted.

Chapter notes

Hashing was invented at IBM in the early 1950s; Knuth [34] gives the history and the definitive analysis of both chaining and linear probing, including the formulas quoted in this chapter. Cormen et al. [16] present the analysis under the "simple uniform hashing" assumption used here, and go on to *universal hashing*, due to Carter and Wegman [12],

in which the hash function is chosen at random from a family so that good expected performance holds for every set of keys, not merely for typical ones. That idea is the theoretical ancestor of the keyed hash functions now used to resist hash-flooding attacks [3, 18].

The Bloom filter is from a 1970 paper [10] that is five pages long and still worth reading. For the engineering of a modern open-addressing table, Kulukundis's talk on the design of the Swiss table [36] is an excellent hour.

Answers to check-yourself questions

5.1 Indexing an array takes constant time. So if we could compute, from the name itself, the array index where that name is stored, we would not have to search at all: compute the index, look there. The whole of this chapter is about making that idea work.

5.2 Twenty-three. This is the famous *birthday paradox*: in a group of 23 people, the chance that two share a birthday is just over one half. In general, collisions become likely once the number of keys reaches about the *square root* of the number of buckets. A table with a million buckets will probably see its first collision at around twelve hundred keys, when it is barely a tenth of one percent full.

5.3 The search starts at slot 3 (A, not a match), moves to slot 4, finds it empty, and stops, reporting that C is not in the table. But C is sitting in slot 5. Emptying slot 4 broke the path that leads to it. Figure 5.3 shows the situation.

CHAPTER 6

Recursion and Trees

WEEK 6 OF 10

THIS WEEK

By the end of this week you should be able to:

- write and reason about recursive functions, identifying the base case and the smaller subproblem, and estimate their cost;
- use the vocabulary of trees precisely: root, leaf, depth, height, subtree;
- explain what `std::unique_ptr` is and use it to express ownership of tree nodes;
- implement a binary search tree with search, insertion, removal, and the standard traversals;
- state the binary-search-tree invariant *correctly*, and write a function that checks it;
- explain why the cost of every operation depends on the tree's height, and how badly that can go.

Take stock of where we are. A sorted vector can be searched in $\Theta(\log n)$ time, but inserting into it costs $\Theta(n)$. A linked list can be modified in $\Theta(1)$ time at a known position, but searching it costs $\Theta(n)$. A hash table does both in expected constant time, but in exchange it throws away all information about *order*: it cannot tell you the smallest key, the next key after a given one, or all the keys between two bounds.

CHECK YOURSELF 6.1

Binary search is fast because each comparison discards half of the remaining elements. A linked structure is flexible because elements never have to move. Can you imagine a *linked* structure in which each comparison discards half of the remaining nodes? Where would the search have to start, and how many `next` pointers would each node need?

Trees are the third great family of data structures, after arrays and hash tables, and they are what you reach for when you need both fast search and order. They are also the natural home of **recursion**, a way of thinking about problems that you may have met briefly before and are about to need constantly. So this chapter begins there.

6.1 Recursion

A function is **recursive** if it calls itself. That sounds circular, but a well-formed recursive function is no more circular than a staircase. It solves a problem by reducing it to a *smaller instance of the same problem*, and at the bottom there is an instance small enough to answer directly.

code/ch06/recursion.cpp

```
std::uint64_t factorial(unsigned n) {
    if (n <= 1) return 1;              // base case: nothing left to multiply
    return n * factorial(n - 1);       // recursive case: a smaller problem
}
```

Every recursive function has the same two parts.

- A **base case**: an input so small that the answer is immediate. Here, the factorial of 0 or 1 is 1.
- A **recursive case**: the function calls itself on a *smaller* input and uses the result. Here, $n! = n \cdot (n-1)!$.

Two things must hold for the recursion to be correct. Every chain of recursive calls must eventually reach a base case; that is why the recursive call must be on a smaller problem. And the function must give the right answer *assuming that its recursive calls give the right answers*. Checking the second condition takes a small act of faith, and it is the habit to cultivate: when you write or read `n * factorial(n - 1)`, do not mentally descend into the inner call. Assume that it returns $(n-1)!$, as advertised, and ask only whether, given that, this call returns $n!$.

You already know the machinery that makes recursion work. It is the call stack of Section 1.5. Each call gets its own stack frame with its own copy of `n`. `factorial(4)` calls `factorial(3)`, which calls `factorial(2)`, which calls `factorial(1)`; four frames are stacked up, each waiting for the one above it. Then the base case returns 1, and the frames come off one at a time, each multiplying on its way out.

That picture also tells you what recursion costs. The *time* is the total number of calls times the work done in each. The *space* is the maximum depth of the stack, since each pending call occupies a frame. `factorial(n)` makes n calls and nests them n deep: $\Theta(n)$ time and $\Theta(n)$ space. A loop would use $\Theta(1)$ space, and for a function this simple a loop is the better choice. Recursion shows its worth on problems that split into *more than one* subproblem.

code/ch06/recursion.cpp

```
// Sum of v[lo..hi), by splitting the range in half.
long long sum(const std::vector<int>& v, std::size_t lo, std::size_t hi) {
    if (hi - lo == 0) return 0;                // empty range
    if (hi - lo == 1) return v[lo];            // one element
    std::size_t mid = lo + (hi - lo) / 2;
    return sum(v, lo, mid) + sum(v, mid, hi);  // two half-size problems
}
```

This function sums a range by splitting it in half, summing each half recursively, and adding the results. It is not a sensible way to add up a vector, but its shape, called **divide and conquer**, is the shape of the most important algorithms in the next few chapters. Notice the half-open ranges: the two halves `[lo, mid)` and `[mid, hi)` fit together with no overlap and no gap.

CHECK YOURSELF 6.2

For a vector of 8 elements, how many calls to `sum` are made in all, counting the first? How deep does the stack get?

Recursion can also make a problem dramatically cheaper.

code/ch06/recursion.cpp

```
// x to the power n, using about log2(n) multiplications.
double power(double x, unsigned n) {
    if (n == 0) return 1.0;
    double half = power(x, n / 2);
    if (n % 2 == 0) return half * half;
    return half * half * x;
}
```

Computing x^n by repeated multiplication takes $n - 1$ multiplications. This version computes $x^{n/2}$ once and squares it. Each call halves n, so there are about $\log_2 n$ calls: x^{1000} in ten steps or so.

And it can make a problem catastrophically expensive.

code/ch06/recursion.cpp

```
std::uint64_t calls = 0;

std::uint64_t fib(unsigned n) {
    ++calls;
    if (n < 2) return n;
    return fib(n - 1) + fib(n - 2);   // two overlapping subproblems
}
```

```
fib(10) = 55 took 177 calls
fib(20) = 6765 took 21891 calls
fib(30) = 832040 took 2692537 calls
```

The Fibonacci function calls itself twice, and the two calls do *overlapping* work: `fib(n - 1)` computes `fib(n - 2)` all over again, and so on down. The number of calls grows exponentially. The lesson is that recursion is safe when the subproblems are *disjoint*, as the two halves of a range are. When they overlap, a plain recursive solution recomputes the same things many times. (Remembering the answers to subproblems, so that each is computed once, turns out to be a powerful general technique, called dynamic programming, that you will study in an algorithms course.)

The costs of the common recursive shapes are worth knowing by sight. Write $T(n)$ for the time on an input of size n.

Shape of the recursion	Example	Time
one call on a problem smaller by one	`factorial`	$\Theta(n)$
one call on a problem half the size	`power`, binary search	$\Theta(\log n)$

Shape of the recursion	Example	Time
two calls on halves, constant extra work	`sum`	$\Theta(n)$
two calls on halves, linear extra work	mergesort (Chapter 8)	$\Theta(n \log n)$
two calls on problems smaller by one or two	`fib`	exponential

BY HAND FIRST

Trace `sum(v, 0, 5)` on a five-element vector by drawing its **call tree**: one box per call, labeled with its `lo` and `hi`, with lines down to the calls it makes, and the returned value written beside each box as it finishes. Then do the same for `power(2.0, 13)`.

Being able to trace a recursion by hand is how you debug one, and being able to *stop* tracing and trust the recursive call is how you write one. Both come only from practice, and neither comes from watching a model do it. A model will draw you a call tree instantly and it will usually be right; you will have learned nothing about recursion from it, and recursion is the one idea in this chapter that everything else depends on.

ASK AN LLM

Once you can trace small recursions yourself, a model is a good source of fresh ones.

> *Give me a short recursive C++ function I have probably not seen before, with a specific call to it. Ask me (1) what it returns and (2) how many calls are made in total. Don't reveal the answers until I reply. If I am wrong, don't just correct me: show me the call tree.*

Compile and run the function with a call counter, as `fib` does above, to settle any disagreement. On questions of *counting* calls, a model's arithmetic is not to be taken on trust.

6.2 Trees

A **tree** is a collection of nodes connected so that there is exactly one path between any two of them. One node is singled out as the **root**. Computer scientists draw trees upside down, with the root at the top.

The vocabulary is borrowed from family trees and from botany, with no great consistency.

- Each node except the root has one **parent**, the node directly above it. The nodes directly below a node are its **children**.
- A node with no children is a **leaf**.
- The **depth** of a node is the number of links on the path from the root to it. The root has depth 0.
- The **height** of a tree is the number of nodes on the longest path from the root down to a leaf. An empty tree has height 0, and a tree with one node has height 1. (Some books count links instead of nodes, giving a number one smaller. Check the convention whenever you read about trees.)

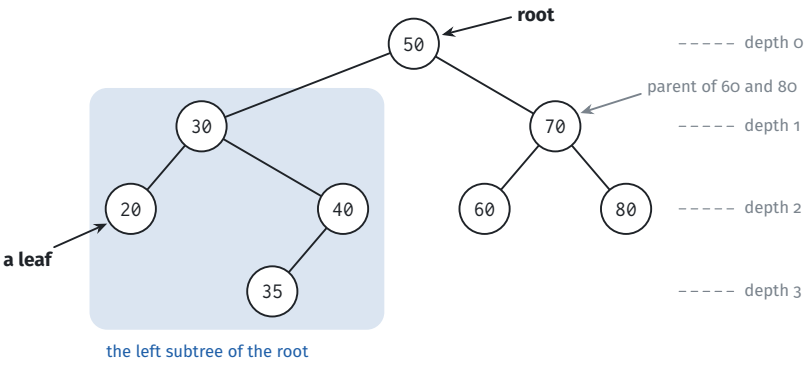


Figure 6.1 The anatomy of a binary tree. Depth is counted from the root, which is at depth 0. The height of this tree is 4: the number of nodes on the longest path from the root to a leaf. The shaded region is the left subtree of the root, which is itself a tree.

- A node together with everything below it is a **subtree**.

In a **binary tree**, each node has at most two children, and they are distinguished: a *left* child and a *right* child.

The last item on the vocabulary list is the important one. It leads to a definition of binary trees that is more useful than any picture:

> *A binary tree is either **empty**, or it is a **node** with a left subtree and a right subtree, each of which is a binary tree.*

The definition is recursive: it defines trees in terms of smaller trees, with the empty tree as its base case. This is why trees and recursion go together. Almost any function on a tree can be written by following the definition: handle the empty tree; otherwise handle the node, and let recursive calls handle the two subtrees.

Nodes that own their children

Here is the representation.

code/ch06/bst.hpp

```
struct Node {
    K key;
    std::unique_ptr<Node> left;    // owns the whole left subtree
    std::unique_ptr<Node> right;   // owns the whole right subtree

    explicit Node(K k) : key(std::move(k)) {}
};

using Link = std::unique_ptr<Node>;   // an owning pointer to a subtree

Link root_;                // null when the tree is empty
std::size_t size_ = 0;
```

In Chapter 4 the nodes of a list were joined by raw pointers, and we wrote the destructor and copy operations by hand. Here the child pointers are `std::unique_ptr<Node>`. This is

the standard library's **owning pointer**, and it is how modern C++ expresses the statement "this pointer is responsible for deleting what it points to."

A `unique_ptr` behaves like a pointer, with three differences.

- When a `unique_ptr` is destroyed, it deletes the object it points to. No `delete` is ever written by hand.
- A `unique_ptr` **cannot be copied**. If it could, two pointers would own the same object and delete it twice, which is the shallow-copy bug of Section 3.3. The compiler rejects the attempt.
- A `unique_ptr` **can be moved**, with `std::move`. Ownership passes to the destination, and the source becomes null.

You create the owned object with `std::make_unique<Node>(...)`, which allocates it and returns the `unique_ptr`. When you need to look at the object without taking ownership, `.get()` returns an ordinary raw pointer. This book uses raw pointers in exactly that role: as *observers* that walk a structure owned by someone else.

Now consider what happens when a `Bst` is destroyed. Its member `root_` is destroyed, which deletes the root node. Destroying the root node destroys *its* members `left` and `right`, which delete the two children, and so on down. The entire tree is freed, correctly, and we did not write a line of code to do it. Nor can a `Bst` be accidentally copied, since its `unique_ptr` member cannot be. Ownership has been stated in the *types*, and the compiler does the rest.

Moving needs one small piece of help. The compiler's default move would move `root_`, leaving the source's root null, but would merely *copy* the plain integer `size_`, leaving the source claiming to hold nodes it no longer has. So the class defines its two move operations by hand, to set the source's size to zero. Keep an eye out for this whenever a class stores a count beside the thing it counts.

> **ASIDE**
>
> The automatic destruction is recursive, one stack frame per level of the tree, and so is the `insert` function later in this chapter. For the tall, thin trees we will meet at the end of the chapter, a tree with a hundred thousand nodes can be a hundred thousand levels deep, and inserting into it or destroying it can overflow the stack. This is also why the linked list of Chapter 4 used raw pointers and a loop: a list is the tallest, thinnest tree there is. Production code that might build degenerate trees destroys them iteratively. Balanced trees, the subject of the next chapter, never get deep enough for this to matter.

Recursion on trees

Here are two functions written directly from the recursive definition.

code/ch06/bst.hpp

```
// Number of nodes on the longest path from n down to a leaf.
static int height(const Node* n) {
    if (n == nullptr) return 0;
    int l = height(n->left.get());
    int r = height(n->right.get());
```

```
    return 1 + (l > r ? l : r);
}

static std::size_t count(const Node* n) {
    if (n == nullptr) return 0;
    return 1 + count(n->left.get()) + count(n->right.get());
}
```

Read `height` as a sentence: the height of an empty tree is zero; otherwise it is one, for this node, plus the height of the taller subtree. Try to write it with a loop and you will appreciate what the recursion is doing for you.

How tall must a binary tree be to hold n nodes? At depth 0 there is room for one node; at depth 1, two; at depth 2, four. A tree of height h holds at most $1+2+4+\cdots+2^{h-1} = 2^h - 1$ nodes. Turned around: a tree with n nodes has height **at least** $\log_2(n+1)$. It has height **at most** n, when every node has a single child and the "tree" is really a linked list. A million nodes can form a tree of height 20, or a tree of height 1,000,000. The difference between those two will occupy us for the rest of this chapter and all of the next.

Traversals

To **traverse** a tree is to visit every node once. For a list there was one natural order. For a tree there are several, and the recursive ones differ only in *when* a node is visited relative to its subtrees.

code/ch06/bst.hpp

```
template <typename Visit>
static void in_order(const Node* n, Visit& visit) {
    if (n == nullptr) return;
    in_order(n->left.get(), visit);    // everything smaller
    visit(n->key);                     // then this node
    in_order(n->right.get(), visit);   // then everything larger
}

template <typename Visit>
static void pre_order(const Node* n, Visit& visit) {
    if (n == nullptr) return;
    visit(n->key);                     // this node first
    pre_order(n->left.get(), visit);
    pre_order(n->right.get(), visit);
}
```

- **Pre-order**: the node, then its left subtree, then its right. Parents come before children. Used for copying a tree, or printing an outline.
- **In-order**: the left subtree, then the node, then the right subtree.
- **Post-order**: both subtrees, then the node. Children come before parents. Used when a node's result depends on its children's, as in `height`, and for deleting a tree.

Each visits n nodes and does constant work at each, so each is $\Theta(n)$.

The fourth traversal is not recursive. **Level-order** visits the root, then all nodes at depth 1, then all at depth 2, and so on. It needs to remember, while working across one level, the

children that are waiting on the next, in the order they were discovered. First discovered, first visited: that is a queue.

code/ch06/bst.hpp

```
// Visits keys level by level, top to bottom, left to right.
template <typename Visit>
void level_order(Visit visit) const {
    std::queue<const Node*> waiting;
    if (root_) waiting.push(root_.get());
    while (!waiting.empty()) {
        const Node* n = waiting.front();
        waiting.pop();
        visit(n->key);
        if (n->left) waiting.push(n->left.get());
        if (n->right) waiting.push(n->right.get());
    }
}
```

Compare this with the recursive traversals. They use a stack (the call stack) and go *deep* first. Level-order uses a queue and goes *wide* first. The same contrast, between depth-first and breadth-first exploration, will reappear for graphs in Chapter 9.

6.3 Binary search trees

A binary tree is just a shape. To make it useful for searching we impose an invariant on where the keys go.

> ***The binary search tree invariant.*** *For every node, all keys in its left subtree are less than the node's key, and all keys in its right subtree are greater.*

A binary tree that satisfies this is a **binary search tree**, or BST. Read the invariant carefully: it says *all keys in the subtree*, not merely "the left child." We will see shortly what goes wrong if you remember it the weaker way.

code/ch06/bst_test.cpp

```
Bst<int> t;
for (int k : {50, 30, 70, 20, 40, 60, 80, 35}) t.insert(k);
```

Searching

To search for a key, start at the root. If the key is smaller than the node's key, it can only be in the left subtree, by the invariant. If it is larger, it can only be in the right. Move to that child and repeat. If you arrive at an empty subtree, the key is not in the tree.

code/ch06/bst.hpp

```
bool contains(const K& key) const {
    const Node* n = root_.get();          // a non-owning "observer" pointer
    while (n != nullptr) {
        if (key < n->key) n = n->left.get();
        else if (n->key < key) n = n->right.get();
```

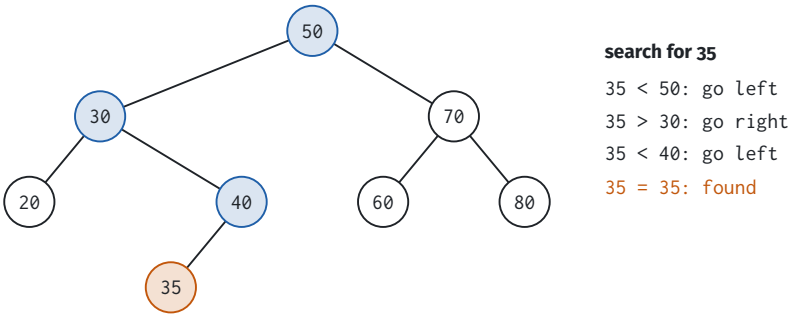


Figure 6.2 The binary search tree produced by inserting 50, 30, 70, 20, 40, 60, 80, 35 in that order. The highlighted path is a search for 35: less than 50, go left; greater than 30, go right; less than 40, go left; found. In the web edition you can insert and remove keys, run the traversals, and try inserting keys in sorted order. [Interactive in the web edition.]

```
        else return true;                  // neither is smaller: equal
    }
    return false;
}
```

This is binary search, performed by following pointers instead of computing midpoints. Each comparison discards an entire subtree. The code uses only < to compare keys: if neither key is less than the other, they are taken to be equal. That is the convention of the C++ standard library, and it means a key type needs only a less-than operator to be stored in a tree.

Because a search only walks downward, a loop is simpler than recursion here, and `n` is a raw observer pointer: we are looking, not owning.

Inserting

A new key always goes in as a new leaf, at the spot where a search for it falls off the tree. The search finds an empty subtree, a null link, where the key would have been. That link is where the new node belongs.

code/ch06/bst.hpp

```
// Insert key into the subtree that `link` owns. `link` is passed by
// reference: it may be root_, or some node's left or right member.
static bool insert(Link& link, K key) {
    if (!link) {                                  // an empty spot: put it here
        link = std::make_unique<Node>(std::move(key));
        return true;
    }
    if (key < link->key) return insert(link->left, std::move(key));
    if (link->key < key) return insert(link->right, std::move(key));
    return false;                                  // already present
}
```

The parameter is the interesting part. `Link& link` is a *reference to the owning pointer* that leads to the current subtree. At the top it refers to `root_`. One level down it refers to some node's `left` or `right` member. When the recursion reaches a null link, assigning to `link` attaches the new node at exactly the right place, whether that place is the root of an empty tree or deep inside a full one, and no special cases are needed. This is the pointer-to-pointer idea of Section 4.2, in the more comfortable form of a reference.

Inserting a key that is already present does nothing and returns `false`: this tree represents a *set*, with no duplicates.

What order buys you

Consider an in-order traversal of a binary search tree. It visits everything in the left subtree, all of which is smaller than the root; then the root; then everything in the right subtree, all of which is larger. Apply the same reasoning within each subtree, and you find that

an in-order traversal of a binary search tree visits the keys in increasing order.

The tree holds its keys sorted, implicitly, at all times, and delivers them in order in $\Theta(n)$ time. This is the first of several things that a hash table cannot do. Here are two more.

code/ch06/bst.hpp

```
// Smallest key in the tree, or nullptr if the tree is empty.
const K* min() const {
    const Node* n = root_.get();
    if (n == nullptr) return nullptr;
    while (n->left) n = n->left.get();    // keep going left
    return &n->key;
}

// Smallest key that is >= key, or nullptr if there is none.
const K* lower_bound(const K& key) const {
    const K* best = nullptr;
    const Node* n = root_.get();
    while (n != nullptr) {
        if (n->key < key) {
            n = n->right.get();           // too small: answer is to the right
        } else {
            best = &n->key;               // a candidate; look for a smaller one
            n = n->left.get();
        }
    }
    return best;
}
```

The smallest key is found by going left until you cannot. `lower_bound` finds the smallest key that is greater than or equal to a given one, the same question that `lower_bound_index` answered for a sorted vector in Section 2.4. It walks down the tree, remembering the best candidate seen so far. With it you can find the key nearest to a value, or the first entry on or after a given date. Add the ability to step from one key to the next, and you can list every key in a range.

All of these, like `contains` and `insert`, walk one path from the root downward and do constant work at each node.

6.4 Removal

Removing a key is the hardest of the basic operations, because removing a node from the middle of a tree leaves a hole that must be repaired without violating the invariant. There are three cases, in increasing order of difficulty.

The node is a leaf. Just remove it. Nothing else is affected.

The node has one child. Remove it and attach its only child in its place. Everything in that child's subtree was already on the correct side of the removed node's parent, so the invariant survives.

The node has two children. We cannot simply remove it, since there are two subtrees to reattach and only one place to put them. Instead we keep the node and replace its *key* with a different key that can legitimately sit in that position: one that is greater than everything in the left subtree and less than everything else in the right subtree. That key is the smallest key in the right subtree, the node's **in-order successor**. The successor is found by taking one step right and then going left as far as possible. Since it is as far left as possible, it has no left child, so removing it from its old position is one of the two easy cases.

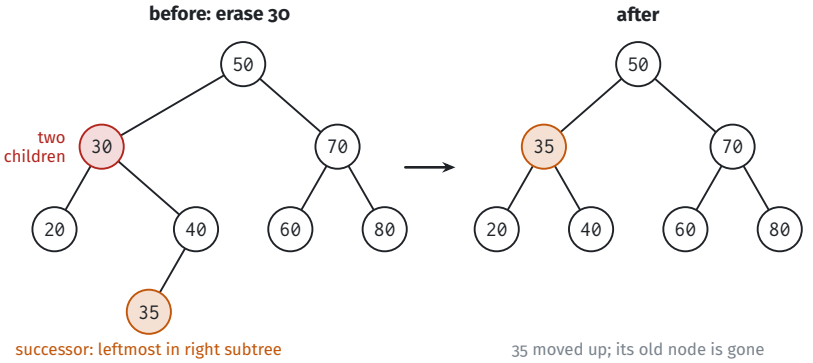


Figure 6.3 Removing 30, a node with two children. Its in-order successor is 35, the leftmost node in its right subtree. The successor's key is moved up to replace 30, and the successor's old node, which has at most a right child, is unlinked.

code/ch06/bst.hpp

```
static bool erase(Link& link, const K& key) {
    if (!link) return false;                                  // not found
    if (key < link->key) return erase(link->left, key);
    if (link->key < key) return erase(link->right, key);

    // link owns the node to remove.
    if (!link->left) {
        link = std::move(link->right);      // no left child: promote the right
    } else if (!link->right) {
        link = std::move(link->left);       // no right child: promote the left
    } else {
        // Two children: find the successor, the leftmost node of the
        // right subtree. Move its key up here, then unlink it.
```

```
        Link* succ = &link->right;
        while ((*succ)->left) succ = &(*succ)->left;
        link->key = std::move((*succ)->key);
        *succ = std::move((*succ)->right);
    }
    return true;
}
```

The function first finds the node, recursing down with a reference to the owning link, just as `insert` did. When it finds the key, `link` is the pointer that owns the doomed node.

In the easy cases, a single move assignment does everything. `link = std::move(link->right)` transfers ownership of the right child into `link`. The node that `link` previously owned is deleted by the `unique_ptr` as part of the assignment. No `delete`, no temporary, and nothing can leak. (If the node was a leaf, the right child is null, and `link` simply becomes null. The leaf case needs no code of its own.)

In the hard case, `succ` walks down to the link that owns the successor. It is a raw pointer to a `Link`: we are again tracking "the pointer that will need to change." The successor's key is moved up, and the successor's node is replaced by its right child, exactly as in the easy case.

> **CHECK YOURSELF 6.3**
>
> In the two-child case, the code uses the smallest key of the right subtree. What other key in the tree could be moved into the node's position instead, without breaking the invariant?

6.5 The shape problem

Every operation in this chapter, searching, inserting, removing, finding a minimum or a lower bound, starts at the root and walks down one path. Each takes time proportional to the length of that path, which is at most the height of the tree, h.

Operation	Cost
`contains`, `insert`, `erase`, `min`, `lower_bound`	$\Theta(h)$
any traversal	$\Theta(n)$

So the cost of a binary search tree is $\Theta(h)$, and everything depends on what h is. We know that h lies somewhere between about $\log_2 n$ and n. Which is it?

> **CHECK YOURSELF 6.4**
>
> Starting from an empty tree, insert the keys 1, 2, 3, 4, 5, 6, 7 in that order. Draw the result. What is its height? How many comparisons does it take to find 7?

The shape of a binary search tree is determined by the order in which keys are inserted. Figure 6.4 shows the same seven keys in the best shape and the worst.

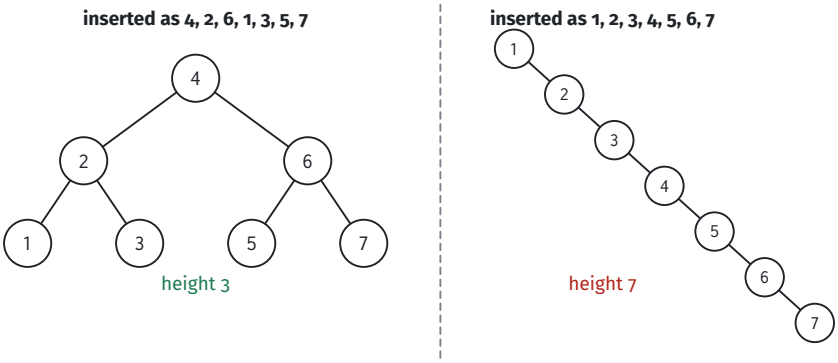


Figure 6.4 The same seven keys in two binary search trees. Both satisfy the invariant. Inserted in the order 4, 2, 6, 1, 3, 5, 7, they form a tree of height 3. Inserted in increasing order, they form a chain of height 7, and every operation is a linear scan.

Sorted input is the worst case, and sorted input is extremely common. Data arrives in order all the time: records with increasing ID numbers, timestamps, the contents of a file that someone already sorted. A data structure that collapses on its most likely input is in trouble.

What about input in no particular order? Here are measurements for trees built by inserting the numbers 0 through $n - 1$ in random order.

n	$\log_2 n$	height	average depth of a node
1,000	10.0	26	11.2
10,000	13.3	28	15.3
100,000	16.6	42	21.4
1,000,000	19.9	53	24.8

Random insertion order produces trees that are not perfectly balanced but are *good enough*. The average node is at a depth of roughly $1.39 \log_2 n$, less a small constant, so a typical search costs only about forty percent more comparisons than the ideal [34]. The height, too, grows in proportion to $\log n$. The same code built from sorted keys:

```
sorted: n = 1000   height = 1000   avg depth = 499.5
sorted: n = 10000   height = 10000   avg depth = 4999.5
```

We are in the same position as with hash tables. The structure is excellent on average and terrible in the worst case, and the worst case is not exotic. For hash tables the remedy was to make the hash function unpredictable. For trees there is a better one: change the algorithms so that the tree *cannot* become tall, whatever order the keys arrive in. That is the subject of Chapter 7.

Checking the invariant

How do you know a tree satisfies the binary-search-tree invariant? The obvious check goes like this.

code/ch06/isbst.cpp

```
// WRONG: checks each node only against its own two children.
bool looks_ordered(const Node* n) {
    if (n == nullptr) return true;
    if (n->left && !(n->left->key < n->key)) return false;
    if (n->right && !(n->key < n->right->key)) return false;
    return looks_ordered(n->left) && looks_ordered(n->right);
}
```

It verifies that every node's left child is smaller than it and its right child larger. It is also wrong, and it is one of the best-known wrong answers in the subject.

> **CHECK YOURSELF 6.5**
>
> Find a binary tree with five nodes that `looks_ordered` accepts but that is *not* a binary search tree. (Hint: put a key in the left subtree of the root that is larger than the root.)

The invariant is about whole subtrees. A key must be on the correct side of *every one of its ancestors*, not only its parent. The correct check carries that information down the tree as a pair of bounds.

code/ch06/bst.hpp

```
// Is every key in n's subtree strictly between *lo and *hi?
// (A null bound means "no limit on that side".)
static bool ordered(const Node* n, const K* lo, const K* hi) {
    if (n == nullptr) return true;
    if (lo != nullptr && !(*lo < n->key)) return false;
    if (hi != nullptr && !(n->key < *hi)) return false;
    return ordered(n->left.get(), lo, &n->key) &&     // left: below this key
           ordered(n->right.get(), &n->key, hi);      // right: above it
}
```

Going left tightens the upper bound to the current key; going right tightens the lower bound. Every node is checked against limits inherited from all the ancestors above it.

Our `check_invariant` uses this function, and the test for this chapter runs it throughout a differential test against `std::set`, the standard library's ordered set, comparing `contains`, `lower_bound`, sizes, and the complete in-order sequence of keys.

> **ASK AN LLM**
>
> The parent-and-children check is so common in tutorials and forum answers that it makes a good probe of any source's reliability, human or machine. Try both directions.
>
> First, hand a model the wrong function, without comment:

> *Is this function a correct check of the binary search tree property? If not, give the smallest counterexample tree.*
>
> Then, in a fresh conversation, ask it to *write* such a check from scratch, and read what it produces. Does it carry bounds down, or compare each node only with its children?
>
> Find the counterexample yourself before you ask. The point of the exercise is not that the model gets it wrong (a good one will probably get it right) but that you now know a precise, checkable fact that a plausible-looking answer can violate, and you know how to test for it.

IN THE WILD

The search trees in this chapter are a small corner of the tree kingdom. A file system is a tree of directories. A web page is a tree of nested elements, and so is any JSON or XML document. A compiler turns your program into a tree, in which `a + b * c` is an addition node whose right child is a multiplication, and then traverses that tree, post-order, to generate code. In all of these, the recursive definition and the recursive traversals carry over directly, and `std::unique_ptr` children are the standard way to represent them in modern C++.

THE FOUR QUESTIONS **Binary search tree**

Where does it live? One heap node per key, each owning its left and right subtrees through `unique_ptr`s. The tree object owns the root. Nodes are scattered in memory, as with a linked list.

What does it promise? For every node, all keys in its left subtree are smaller and all keys in its right subtree are larger. Consequently an in-order traversal yields the keys in sorted order. Nothing is promised about the tree's shape.

What does it cost? Search, insert, remove, minimum, lower bound: $\Theta(h)$, where the height h is between $\log_2 n$ and n. About $1.39 \log_2 n$ on average for keys inserted in random order; n for keys inserted in sorted order. Traversal: $\Theta(n)$.

How do you know? A recursive invariant check that passes bounds down from every ancestor; differential testing against `std::set`, including the in-order key sequence; tests that deliberately insert sorted keys.

Exercises

1. Write a recursive function `int count_digits(unsigned n)` that returns the number of decimal digits in `n`. Identify the base case and the smaller subproblem. How many calls does it make for an input with d digits?
2. Write a recursive function `bool is_palindrome(const std::string& s, std::size_t lo, std::size_t hi)` that reports whether the characters of `s` in the range `[lo, hi)` read the same in both directions.
3. Write binary search recursively: `std::size_t search(const std::vector<int>& v, int target, std::size_t lo, std::size_t hi)`. What is its stack depth? Is there any advantage over the loop of Chapter 2?

4. Draw the call tree for `fib(5)`. How many calls are there? How many of them are calls to `fib(1)`? Then write a version of `fib` that runs in $\Theta(n)$ time using a loop.
5. Draw the binary search tree that results from inserting these keys, in order, into an empty tree: 40, 20, 60, 10, 30, 50, 70, 25, 35, 65. Give its height, and list the keys in pre-order, in-order, post-order, and level-order.
6. Starting from the tree in the previous exercise, draw the tree after each of these removals, performed one after another: 10, 60, 20, 40.
7. Give an insertion order for the keys 1 through 15 that produces a tree of height 4. How many different insertion orders of the keys 1, 2, 3 produce a tree of height 2? Of height 3?
8. Add these member functions to `Bst<K>`: (a) `const K* max() const`; (b) `std::size_t count_leaves() const`; (c) `const K* floor(const K& key) const`, the largest key less than or equal to `key`; (d) `std::size_t count_in_range(const K& lo, const K& hi) const`, which should not visit subtrees that cannot contain keys in the range.
9. A pre-order listing of a binary search tree is enough to reconstruct it exactly. Explain why, and reconstruct the tree with pre-order 50, 30, 20, 40, 35, 70, 60, 80. Is an in-order listing enough? Is a post-order listing?
10. Write a member function `Bst<K> clone() const` that returns a deep copy of the tree with exactly the same shape. Which traversal order does your helper function use, and why?
11. Write a non-recursive in-order traversal using an explicit `std::stack<const Node*>`. Compare its structure with `level_order`. What changes when the queue becomes a stack?
12. `Bst::erase` always uses the successor in the two-child case. Suppose you insert n random keys and then repeatedly remove a random key and insert a new random key, many times. Predict what happens to the tree's balance, then measure the average depth after n^2 such pairs for $n = 1000$. (The answer surprised the researchers who first ran this experiment.)
13. **With a model, as critic.** Write the three-case removal for a BST that uses *raw* pointers (`Node* left; Node* right;`) and an explicit `delete`. Run it under AddressSanitizer on your own tests. Then ask a model: "Without rewriting it, tell me whether any path through this function leaks a node, deletes a node twice, or leaves a dangling pointer." Compare its findings with the sanitizer's. Then write two sentences on what `unique_ptr` saved you from.
14. **Lab.** Turn `Bst<K>` into `BstMap<K, V>`, a map from keys to values, with `put`, `find` (returning a pointer to the value or null), `erase`, and an in-order `for_each`. Use it to rewrite the word-frequency program of Chapter 5 so that it prints *every* word and its count in alphabetical order. Then feed it two inputs, a novel from Project Gutenberg and the first 10,000 lines of the system dictionary file (which is sorted), and report the height of the tree and the running time for each. Explain the difference. (Why only the first 10,000 lines? Try the whole file, under the sanitizers, and explain what happens.)

Chapter notes

Binary search trees were discovered independently by several people around 1960, and the removal algorithm given here is due to Hibbard; Knuth [34] gives the history and the analysis of random trees, including the $1.39 \log_2 n$ average depth and the unexpected long-run effect of removals that Exercise 12 explores. Sedgewick and Wayne [48] give an especially clear treatment of ordered operations such as floor, ceiling, and range counting.

On recursion as a way of thinking, and on trees as recursive data, the classic source remains *Structure and Interpretation of Computer Programs* [1]. For `std::unique_ptr` and the idea of expressing ownership in types, see Meyers [41] and the C++ Core Guidelines [57].

Answers to check-yourself questions

6.1 The search would have to start at the *middle* element, since that is what binary search compares against first. From there it needs to go to the middle of the left half or the middle of the right half. So each node needs two pointers: one to the "middle" of everything smaller, and one to the "middle" of everything larger. That structure is a binary search tree.

6.2 The calls form a tree: one call on 8 elements, two on 4, four on 2, eight on 1. That is 1+2+4+8 = 15 calls. The stack depth is only 4, the number of levels, because the two recursive calls happen one after the other, and the first has finished and released its frames before the second begins. In general: $2n - 1$ calls, depth $\log_2 n + 1$.

6.3 The largest key in the *left* subtree, the in-order **predecessor**: one step left, then right as far as possible. It is greater than everything else in the left subtree and less than everything in the right. Either choice is correct.

6.4 Each new key is larger than every key already present, so each becomes the right child of the previous one. The result is a chain leaning to the right: a linked list in all but name, of height 7. Finding 7 takes seven comparisons. Insert the same keys in the order 4, 2, 6, 1, 3, 5, 7, and you get a perfectly balanced tree of height 3.

6.5 Make 50 the root with children 30 and 70, and give 30 the children 20 and 60. Every parent–child pair is in order: 60 is greater than its parent 30. But 60 is in the left subtree of 50, where every key must be less than 50. A search for 60 would go right at the root and never find it.

CHAPTER 7

Balance

WEEK 7 OF 10

THIS WEEK

By the end of this week you should be able to:

- perform left and right rotations on a binary search tree, by hand and in code, and explain why they preserve the search-tree invariant;
- state the AVL balance condition, explain why it forces the height to be logarithmic, and rebalance a tree after an insertion or removal;
- describe how red-black trees and B-trees relate to AVL trees, and why B-trees suit modern memory hierarchies;
- use `std::map` and `std::set` for ordered queries, and choose sensibly among a hash table, a balanced tree, and a sorted vector.

The binary search tree of the last chapter does everything we asked: search, insertion, removal, ordered traversal, range queries. Its one flaw is that all its costs are proportional to its height, and its height depends on the order in which the keys happened to arrive. Feed it sorted data and it becomes a linked list.

CHECK YOURSELF 7.1

Here is a binary search tree built by inserting 1, 2, 3 in order: a chain leaning right, with 1 at the root. Using the same three nodes, draw a different binary search tree of height 2. Which node is the root? Which pointers changed? And could you make the change *without examining or moving any other part of a larger tree* that this chain might be a piece of?

This chapter is about keeping trees short. The plan is to add a second invariant to the search-tree invariant, one that limits the height, and to repair it after each modification with a small amount of local surgery. A tree that does this is called **self-balancing**. There are many varieties. We will build one completely, the AVL tree, and then survey its relatives, including the one inside `std::map` and the one inside nearly every database.

7.1 Rotations

A **rotation** rearranges a parent and one of its children so that the child becomes the parent, without disturbing the left-to-right order of the keys.

Look at Figure 7.1. On the left, y is the root of some subtree and x is its left child. A is everything less than x; B is everything between x and y; C is everything greater than y. A **right rotation** at y makes x the root, with y as its right child. Subtree A stays under x

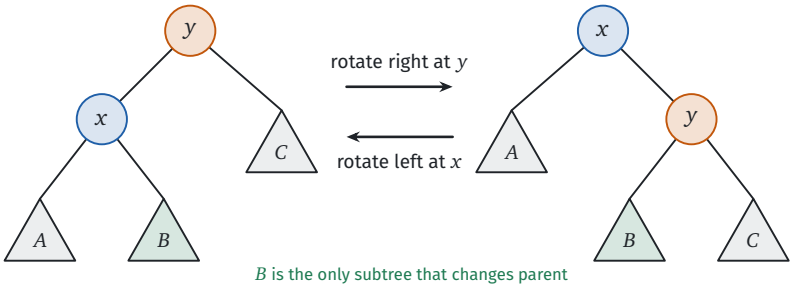


Figure 7.1 A right rotation at y lifts its left child x into its place; a left rotation at x undoes it. The triangles are whole subtrees, which move with their roots. The in-order sequence, $A\,x\,B\,y\,C$, is the same on both sides.

and subtree C stays under y. Only B has to move: it was x's right subtree and becomes y's left subtree, which is the one place a group of keys between x and y can go.

Two facts make rotations the perfect repair tool.

A rotation preserves the search-tree invariant. Read the keys off in order on either side of the figure and you get the same sequence: A, x, B, y, C.

A rotation changes heights. Everything in A moves up one level, and everything in C moves down one. If the left side was too tall, a right rotation shortens it at the expense of the right.

And it is cheap: three links change, regardless of the size of the tree. Here it is in code, for a tree whose nodes are owned by `unique_ptr`s, as in the last chapter.

code/ch07/avl.hpp

```
// Rotate right around the node that `link` owns: its left child rises
// to take its place, and it becomes that child's right child.
static void rotate_right(Link& link) {
    Link pivot = std::move(link->left);       // the child that will rise
    link->left = std::move(pivot->right);     // its inner subtree changes sides
    update(*link);
    pivot->right = std::move(link);           // the old root goes down
    link = std::move(pivot);                  // the pivot takes its place
    update(*link);
    ++rotations;
}

static void rotate_left(Link& link) {         // the mirror image
    Link pivot = std::move(link->right);
    link->right = std::move(pivot->left);
    update(*link);
    pivot->left = std::move(link);
    link = std::move(pivot);
    update(*link);
    ++rotations;
}
```

As in `insert` and `erase` for the plain search tree, `link` is a reference to the owning pointer that leads to the subtree: the root pointer, or a `left` or `right` member of the node above. The four moves are a short dance of ownership, and it is worth following them with a finger on the figure. First the pivot, x, is detached from y, so that for a moment nobody but the local variable owns it. Then B is moved from x to y. Then y itself is moved to become x's right child. Finally x is installed in the link from above. At each step every node has exactly one owner. The compiler helps here: a `unique_ptr` cannot be copied, so the code cannot accidentally leave two pointers owning the same node. (It cannot stop you from forgetting to re-attach a subtree, which would silently delete it; the tests are for that.) (The calls to `update` are bookkeeping that the next section explains.)

A **left rotation** is the mirror image, and each undoes the other.

7.2 AVL trees

Rotations are the mechanism. Now we need a policy: a precise rule for when a tree is "balanced enough" and where to rotate when it is not. The oldest such rule, and still one of the best, was published in 1962 by Georgy Adelson-Velsky and Evgenii Landis [2], after whom the **AVL tree** is named.

The AVL invariant. *For every node, the heights of its left and right subtrees differ by at most one.*

An AVL tree is a binary search tree that also satisfies this. Note that the condition is imposed at *every* node, not just the root, and that it does not demand perfection. A node's two subtrees may differ in height by one, and those small allowances can accumulate, so an AVL tree can be somewhat lopsided. What it cannot be is badly lopsided.

To maintain the invariant cheaply, each node stores the height of its own subtree.

code/ch07/avl.hpp

```
struct Node {
    K key;
    int height = 1;                // height of the subtree rooted here
    std::unique_ptr<Node> left;
    std::unique_ptr<Node> right;

    explicit Node(K k) : key(std::move(k)) {}
};
using Link = std::unique_ptr<Node>;

static int height(const Link& link) { return link ? link->height : 0; }

static void update(Node& n) {
    n.height = 1 + std::max(height(n.left), height(n.right));
}

// Positive: left subtree is taller. Negative: right subtree is taller.
static int balance(const Node& n) { return height(n.left) - height(n.right); }
```

The difference between the two subtree heights is called the node's **balance factor**. The AVL invariant says that every balance factor is -1, 0, or $+1$.

Why the height is logarithmic

How tall can an AVL tree with n nodes be? It is easier to turn the question around: what is the *smallest* number of nodes an AVL tree of height h can have? Call it $N(h)$.

A tree of height 1 is a single node: $N(1) = 1$. A tree of height 2 needs a root and at least one child: $N(2) = 2$. For a taller tree, the root has one subtree of height $h - 1$ (or the tree would not have height h). To use as few nodes as possible, make the other subtree as short as the invariant allows, which is $h - 2$, and make both subtrees as sparse as possible in turn. So

$$N(h) = 1 + N(h-1) + N(h-2).$$

h	1	2	3	4	5	6	7	8	9	10	20
$N(h)$	1	2	4	7	12	20	33	54	88	143	17,710

Each entry is the sum of the two before it, plus one. That is nearly the rule for the Fibonacci numbers, and like them, $N(h)$ grows exponentially: each step multiplies it by about 1.618, the golden ratio. An AVL tree of height h therefore has *at least* an exponential number of nodes, which, turned back around, says that a tree with n nodes has height at most logarithmic in n. Working out the constants gives

$$h < 1.45 \log_2(n + 2).$$

An AVL tree is never more than about 45 percent taller than a perfectly balanced tree with the same number of nodes [34]. A million keys: height at most 28. A billion: at most 42. And this is a guarantee, for every sequence of insertions and removals, with no assumption about the input.

Restoring balance after an insertion

Inserting into an AVL tree starts exactly like inserting into any binary search tree: walk down, and attach a new leaf. The new leaf may have made some subtrees one taller. The only nodes whose subtrees changed are the new leaf's ancestors, the nodes on the path back to the root. So we retrace that path upward, and at each node we update its stored height and check its balance factor. If it has become +2 or −2, we fix it on the spot.

Suppose a node z has become too tall on the left, with balance factor +2. Let y be its left child. There are two cases, depending on which of y's subtrees is the tall one, and Figure 7.2 shows them.

The outside case. The extra height is in y's *left* subtree: the path from z goes left, then left. A single right rotation at z lifts y and its tall left subtree up a level, and lowers z. The subtree is balanced.

The inside case. The extra height is in y's *right* subtree: the path goes left, then right, in a zig-zag. A right rotation at z alone does not help; it would move the tall inner subtree across to hang under z, leaving things just as lopsided in the other direction. Instead we first rotate *left* at y. That straightens the zig-zag into the outside case. Then a right rotation at z finishes, as before. Two rotations, often called a **double rotation**.

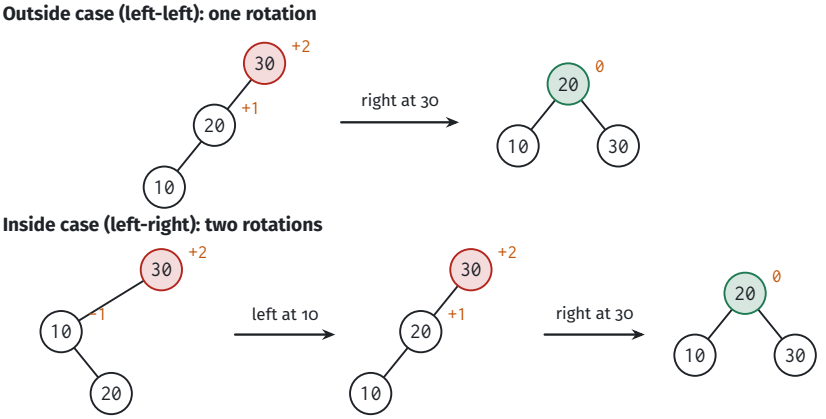


Figure 7.2 The two ways a node can become left-heavy, and their repairs. Top: the extra height is on the *outside* (the left child's left subtree); one right rotation fixes it. Bottom: the extra height is on the *inside* (the left child's right subtree); a left rotation at the child first converts it to the outside case, and then a right rotation finishes the job.

A node that is too tall on the *right* is handled by the mirror images. That makes four cases in all, traditionally named left-left, left-right, right-right, and right-left after the direction of the two steps from the unbalanced node toward the extra height. All four are handled by one short function.

code/ch07/avl.hpp

```
// Restore the AVL invariant at this node, assuming both of its
// subtrees already satisfy it and their heights differ by at most 2.
static void rebalance(Link& link) {
    update(*link);
    int b = balance(*link);
    if (b > 1) {                                    // too tall on the left
        if (balance(*link->left) < 0) {
           rotate_left(link->left);              // left-right: straighten first
        }
        rotate_right(link);
    } else if (b < -1) {                            // too tall on the right
        if (balance(*link->right) > 0) {
            rotate_right(link->right);              // right-left: straighten
↪   first
        }
        rotate_left(link);
    }
}
```

The function reads off the cases directly. If the node is left-heavy by two, look at its left child. If that child leans *right* (the inside case), rotate the child left first. Then rotate the node right.

Insertion calls `rebalance` on each node along the path, on the way back up from the recursion.

code/ch07/avl.hpp

```
static bool insert(Link& link, K key) {
    if (!link) {
        link = std::make_unique<Node>(std::move(key));
        return true;
    }
    bool added;
    if (key < link->key) added = insert(link->left, std::move(key));
    else if (link->key < key) added = insert(link->right, std::move(key));
    else return false;

    if (added) rebalance(link);   // on the way back up, fix this node
    return added;
}
```

Compare this with the plain `Bst::insert` of Section 6.3. The descent is identical. The only addition is the call to `rebalance` after the recursive call returns, which is to say, as the recursion unwinds through the ancestors of the new node from the bottom up. This is a place where recursion earns its keep: "do something at every node on the path back to the root" is exactly what the unwinding call stack does, for free.

CHECK YOURSELF 7.2

Insert the keys 10, 20, 30, 40, 50, 25, in that order, into an empty AVL tree. After which insertions is a rotation needed, and which case is each?

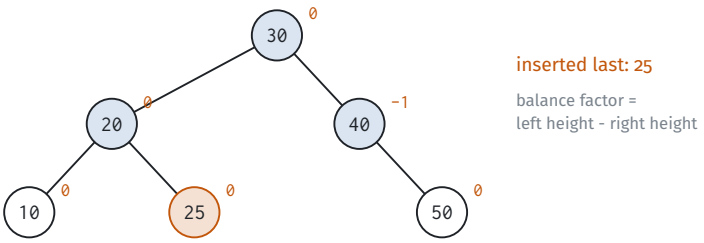


Figure 7.3 An AVL tree after inserting 10, 20, 30, 40, 50, and then 25. Each node shows its balance factor. The last insertion made the root's balance factor −2 with the extra height on the inside, and was repaired by a double rotation. In the web edition, insert and remove keys and watch each rotation; try inserting sorted keys, the input that ruined the plain search tree. [Interactive in the web edition.]

BY HAND FIRST

Insert the keys 1 through 10, in order, into an empty AVL tree, on paper. Draw the tree after every insertion, write the balance factor beside every node, and name the case for every rotation. Then remove 1, 2, and 3. Use the interactive figure to check each step *after* you have

> drawn it.
>
> Rotations are hard to get right from a verbal description and easy once your hands know them. The four cases will blur together until you have drawn each a few times; after that they are obvious, and you will be able to reconstruct the code from the pictures instead of memorizing it.

Removal

Removal follows the same pattern. Remove the node as in a plain search tree (the three cases of Section 6.4), then walk back up, rebalancing every node on the path.

code/ch07/avl.hpp

```
// Detach and return the smallest node of the subtree `link` owns,
// rebalancing every node on the path back up.
static Link extract_min(Link& link) {
    if (!link->left) {
        Link smallest = std::move(link);
        link = std::move(smallest->right);
        return smallest;
    }
    Link smallest = extract_min(link->left);
    rebalance(link);
    return smallest;
}

static bool erase(Link& link, const K& key) {
    if (!link) return false;
    bool removed;
    if (key < link->key) {
        removed = erase(link->left, key);
    } else if (link->key < key) {
        removed = erase(link->right, key);
    } else {
        removed = true;
        if (!link->left) {
            link = std::move(link->right);
        } else if (!link->right) {
            link = std::move(link->left);
        } else {
            Link successor = extract_min(link->right);
            link->key = std::move(successor->key);
        }
    }
    if (removed && link) rebalance(link);
    return removed;
}
```

There is one difference from insertion that the code hides. After an insertion, at most one node needs rotating: the repair restores the subtree to its original height, so nothing above it is affected. After a removal, fixing one node can leave its subtree one shorter than before, which may unbalance its parent, and so on up; removal can require a rotation at

every level. The code does not care. It calls `rebalance` at every node on the path regardless, and each call does constant work.

In the two-child case the successor is detached by `extract_min`, which rebalances along *its* path as well, since removing the successor may have unbalanced the nodes above it.

What it costs, and how you know

Every operation walks one root-to-leaf path and back, doing constant work per node, and the height is at most $1.45 \log_2 n$.

Operation	Plain BST	AVL tree
search, insert, remove	$\Theta(h)$; $\Theta(n)$ worst case	$\Theta(\log n)$ worst case
minimum, lower bound	$\Theta(h)$	$\Theta(\log n)$
in-order traversal	$\Theta(n)$	$\Theta(n)$

Here is what the height looks like in practice, for keys inserted in sorted order (the input that drove the plain tree to height n) and in random order, along with the average number of rotations per insertion.

n	$\log_2 n$	sorted: height	sorted: rotations per insert	random: height	random: rotations per insert
1,000	10.0	10	0.99	12	0.70
10,000	13.3	14	1.00	16	0.67
100,000	16.6	17	1.00	20	0.70
1,000,000	19.9	20	1.00	24	0.70

On sorted input the AVL tree is essentially perfectly balanced. On random input it is within a few levels of perfect. Compare the random-order height of 24 for a million keys with the 53 that the plain tree managed. And the balancing is cheap: on average less than one rotation per insertion.

Now the fourth question, which for balanced trees has a twist. The test for this chapter runs a differential test against `std::set`, as usual. But suppose we made a mistake in `rebalance`, or forgot to call it at all. The tree would still be a valid binary search tree. Every search would still return the right answer. **A differential test, which checks only answers, would pass.** The tree would simply be slow on certain inputs, and nothing would say so.

A balanced tree makes a promise about *cost*, and a promise about cost is invisible to a test of *results*. To check it we must look inside.

code/ch07/avl.hpp

```
// Returns the height of n's subtree, or -1 if any invariant fails:
// search-tree order, stored heights, or the balance condition.
static int check(const Node* n, const K* lo, const K* hi, std::size_t& count) {
    if (n == nullptr) return 0;
    if (lo && !(*lo < n->key)) return -1;
```

```
    if (hi && !(n->key < *hi)) return -1;
    int l = check(n->left.get(), lo, &n->key, count);
    int r = check(n->right.get(), &n->key, hi, count);
    if (l < 0 || r < 0) return -1;
    if (l - r > 1 || r - l > 1) return -1;        // out of balance
    if (n->height != 1 + std::max(l, r)) return -1;   // stale height
    ++count;
    return n->height;
}
```

This one recursive function verifies three things at every node: that the keys are in search-tree order (using the bounds technique of Section 6.5); that the stored height is correct; and that the two subtree heights differ by at most one. The test calls it after *every* operation. It also builds a tree from 100,000 sorted keys and asserts that its height is 17.

PITFALL

Whenever a data structure's value lies in a performance guarantee, test the property that delivers the guarantee, not just the outputs. For a balanced tree that means checking balance. For a hash table it means looking at chain lengths, as we did in Chapter 5. This matters doubly for code you did not write. A balanced tree with broken rebalancing, whether it came from a colleague, a library, or a model, passes every functional test you can devise, and then falls over in production on the first large sorted input.

ASK AN LLM

Models are good at producing test inputs with a specified property, and the AVL cases make a nice exercise in checking them.

> *Give me a sequence of exactly seven distinct integer keys which, inserted in order into an empty AVL tree, causes exactly one double rotation and no single rotations. Don't explain where the rotation happens.*

Insert the sequence in the interactive figure, or in your own implementation with the rotation counter, and see whether it does what was claimed. Requests with words like "exactly one" are precisely where a model's answer needs checking: the sequence it gives will look plausible whether or not it is right.

7.3 The family of balanced trees

AVL trees are one answer to the shape problem. There are several others, all offering the same $O(\log n)$ operations through different invariants. You should know that they exist and what distinguishes them. You will rarely need to implement one.

Red-black trees color each node red or black and constrain the coloring: a red node has no red child, and every path from the root down to an empty subtree passes through the same number of black nodes [26]. Together those rules guarantee that no path is more than twice as long as any other, so the height is at most $2\log_2(n+1)$. That is a looser bound than the AVL tree's, so searches can be slightly slower. In exchange, updates

do less restructuring: an insertion needs at most two rotations and a removal at most three, with the rest of the repair done by recoloring. The red-black tree is the structure inside `std::map` and `std::set` in all the major C++ standard libraries, and inside Java's `TreeMap`. Its insertion algorithm is manageable; its removal algorithm is famously intricate.

Treaps and **skip lists** get balance from randomness instead of from rules. A treap gives every node a random priority and keeps the tree in heap order (Chapter 8) by priority, which makes it shaped like a search tree built in random order, whatever the actual order of insertion [49]. A skip list is not a tree at all but a stack of linked lists, each skipping over more elements than the one below, with each element's height chosen by coin flips [47]. Both are much simpler to implement than red-black trees and are logarithmic *in expectation*.

Splay trees keep no balance information whatever. Each time a key is accessed, it is rotated all the way to the root. Any single operation may be slow, but every sequence of operations is fast on average, an amortized $O(\log n)$ bound, and recently used keys are found especially quickly [52].

All of these are binary, and they all share a weakness that has nothing to do with their height.

7.4 B-trees

A balanced binary tree with a million keys is about twenty levels deep. A search visits twenty nodes. Each node is a separate heap allocation, somewhere in memory. You know from Section 4.5 what that means: twenty pointers chased, and for a large tree, close to twenty cache misses. The comparisons are few. The memory accesses are what hurt.

The step count cannot be improved: $\log_2 n$ comparisons are necessary to find one key among n by comparing. But nothing says each comparison has to be a separate trip to memory.

> **CHECK YOURSELF 7.3**
>
> Suppose each node of a search tree held not one key but fifteen, in sorted order, with sixteen children: one for each gap between keys, plus the two ends. How many levels would a tree of a million keys need? How would you search within a node?

That is the **B-tree** [5]. It is a search tree with fat nodes.

- Every node holds up to some maximum number of keys, kept in sorted order in an array inside the node.
- A node with k keys that is not a leaf has $k + 1$ children. The keys act as dividers: the first child holds everything less than the first key, the second child everything between the first and second keys, and so on.
- **All leaves are at the same depth.** A B-tree is always perfectly balanced in height.

code/ch07/btree.hpp

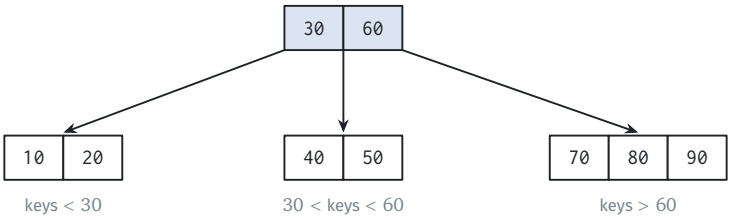


Figure 7.4 A B-tree in which each node holds at most three keys. The keys in a node divide the key space; each child holds the keys that fall in one gap. All leaves are at the same depth. In the web edition, insert keys and watch full nodes split and the tree grow at the root. [Interactive in the web edition.]

```
struct Node {
    std::size_t n = 0;                                  // keys in use
    std::array<K, MAX + 1> keys;                       // sorted; 1 spare slot
    std::array<std::unique_ptr<Node>, MAX + 2> kids;    // n + 1 children, or
    ↪ none

    bool leaf() const { return !kids[0]; }
};
```

Searching descends from the root. At each node, search within the node's array for the key; if it is there, done; if not, the search has found the gap where the key would be, and descends into the corresponding child.

code/ch07/btree.hpp

```
bool contains(const K& key) const {
    const Node* node = root_.get();
    while (node != nullptr) {
        std::size_t i = position(*node, key);            // search within the
        ↪ node
        if (i < node->n && !(key < node->keys[i])) return true;
        node = node->kids[i].get();                     // descend between keys
    }
    return false;
}
```

The same total number of comparisons is performed as in a binary tree, about $\log_2 n$. The difference is that they are grouped: four or five comparisons within one node, all in the same small block of memory, before moving to the next node. A B-tree is a tree of sorted arrays, and it gets the best of both structures: a tree's cheap updates, and an array's locality.

Insertion: growing from the top

How does a B-tree stay perfectly balanced? The answer is one of the most elegant ideas in data structures, and it reverses your expectation of how trees grow.

New keys are always inserted into a leaf, in sorted position within the leaf's array. If the leaf has room, that is all. If the leaf is now overfull, it **splits**: its lower half stays, its upper half moves to a new sibling node, and its *middle key moves up* into the parent, to serve as the divider between the two halves.

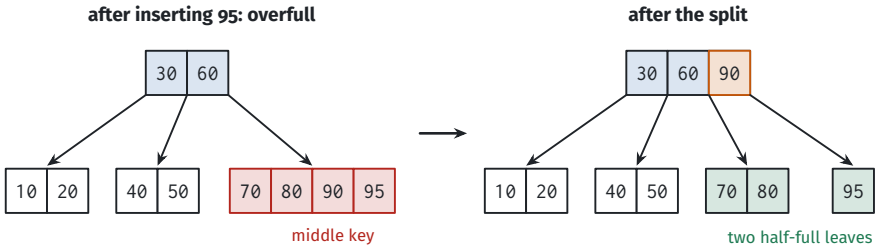


Figure 7.5 Splitting an overfull leaf. Inserting 95 gives the rightmost leaf four keys, one more than allowed. The node splits around its middle key, 90, which moves up into the parent as the divider between the two new leaves.

The parent has gained a key. If *it* is now overfull, it splits in the same way, pushing a key up to its own parent. The splitting can propagate all the way to the root. When the root itself splits, a new root is created above it, holding the single key that was pushed up.

code/ch07/btree.hpp

```
// Child i of parent is overfull. Split it in two around its middle
// key, and move that middle key up into parent.
static void split_child(Node& parent, std::size_t i) {
    Node& left = *parent.kids[i];
    auto right = std::make_unique<Node>();
    std::size_t mid = left.n / 2;

    for (std::size_t j = mid + 1; j < left.n; ++j) {          // upper keys
        right->keys[right->n++] = std::move(left.keys[j]);
    }
    if (!left.leaf()) {
        for (std::size_t j = mid + 1; j <= left.n; ++j) {     // and their
        ↪ children
            right->kids[j - mid - 1] = std::move(left.kids[j]);
        }
    }
    for (std::size_t j = parent.n; j > i; --j) {              // make room above
        parent.keys[j] = std::move(parent.keys[j - 1]);
        parent.kids[j + 1] = std::move(parent.kids[j]);
    }
    parent.keys[i] = std::move(left.keys[mid]);              // middle key rises
    parent.kids[i + 1] = std::move(right);
    ++parent.n;
    left.n = mid;
}
```

code/ch07/btree.hpp

```
bool insert(K key) {
    if (!root_) root_ = std::make_unique<Node>();
    bool added = insert(*root_, std::move(key));
    if (root_->n > MAX) {                  // the root itself overflowed:
        auto top = std::make_unique<Node>();   // grow a new root above it
        top->kids[0] = std::move(root_);
        split_child(*top, 0);
        root_ = std::move(top);            // the tree is one level taller
    }
    if (added) ++size_;
    return added;
}
```

That last step is the only way a B-tree ever gets taller, and it adds one level to *every* path at once. A binary search tree grows downward at its leaves, unevenly, which is where its balance problem comes from. A B-tree grows upward at its root, so all leaves stay at the same depth automatically. No rotations, no balance factors, no colors.

(The B-tree in this chapter's code supports insertion and search. Removal uses the reverse operations, borrowing a key from a sibling or merging two nodes, and is left for the exercises to describe.)

Measurements

Here are times in nanoseconds per operation for random integer keys, for the standard library's red-black tree (`std::set`, abbreviated `set` in the table), this chapter's AVL tree, and this chapter's B-tree with up to 31 keys per node. A sorted vector with binary search is included for lookups, as the contiguous baseline.

n	insert: `set`	insert: AVL	insert: B-tree	find: vector	find: `set`	find: AVL	find: B-tree
10,000	59	92	60	47	60	54	58
100,000	91	156	82	64	147	121	89
1,000,000	232	386	121	103	473	329	138
10,000,000	849	1,023	250	190	1,050	821	305

While everything fits in the cache, the structures are within a factor of two of one another. Once the data outgrows it, all of them slow down, but the two binary trees slow by a factor of eleven to eighteen and the B-tree by about five: at ten million keys it finds a key more than three times as fast as `std::set`, and inserts more than three times as fast too. The AVL tree, being more tightly balanced than the red-black tree, is a little faster to search and a little slower to update, as the theory predicts. The B-tree is within a factor of two of the sorted vector for lookups, while supporting fast insertion, which the vector does not.

IN THE WILD

B-trees were invented at Boeing in 1970 for data on disk, where the gap between "in the same block" and "in a different block" is not a factor of a hundred, as with the cache, but a factor of a hundred thousand or more [5, 15]. With nodes the size of a disk page, each holding hundreds of keys, a B-tree four levels deep can index billions of records, and any one of them can be found with three or four disk reads. Essentially every relational database and most file systems are built on B-trees or their close variant the B+ tree, which Chapter 10 describes.

The same idea is now used in memory, for the cache-related reasons measured above. Rust's standard ordered map, `BTreeMap`, is a B-tree; so is Google's `btree_map` in the Abseil library for C++. When a modern system needs an ordered map and cares about speed, it is increasingly likely to be a B-tree and not a binary tree.

7.5 Ordered maps in practice

In your own programs you will use the standard library's balanced trees: `std::map<K, V>` and `std::set<K>`. Their interface is nearly the same as that of `unordered_map` and `unordered_set`, with the addition of everything that depends on order.

code/ch07/avl_test.cpp

```
std::map<std::string, int> born{{"Hopper", 1906}, {"Lovelace", 1815},
                                {"Turing", 1912}, {"Knuth", 1938}};

// Iteration is in key order, always.
std::string names;
for (const auto& [name, year] : born) names += name + ' ';
assert(names == "Hopper Knuth Lovelace Turing ");

// Smallest and largest keys.
assert(born.begin()->first == "Hopper");
assert(born.rbegin()->first == "Turing");

// First key that is not less than "L".
auto it = born.lower_bound("L");
assert(it->first == "Lovelace");

// Every key in the half-open range ["H", "L").
int count = 0;
for (auto p = born.lower_bound("H"); p != born.lower_bound("L"); ++p) ++count;
assert(count == 2);   // Hopper, Knuth
```

The loop `for (const auto& [name, year] : born)` visits the entries in increasing order of key; the square brackets unpack each key–value pair into two named variables. `begin()` is the smallest entry. `lower_bound(k)` returns an iterator to the first entry whose key is not less than `k`, and from an iterator you can step forward or backward through the keys in order. A range query is a walk from one `lower_bound` to another.

By default the keys are ordered with `<`. To order them differently, or to use a key type that has no `<`, you supply a comparison function.

PITFALL

A comparison given to `std::map`, `std::set`, or `std::sort` must behave like "less than": it must return false when comparing a value with itself, and if it says `a` is before `b` it must not also say `b` is before `a`. A comparison written with `<=` breaks the first rule. The container uses "neither is less than the other" as its test for equality, as our own `contains` does, so with `<=` no key is ever equal to itself, and the tree's behavior is undefined. Always use a strict comparison.

Which associative container should you use? Here are the three candidates side by side.

	Hash table	Balanced tree	Sorted vector
Standard type	`unordered_map`	`map`	`vector` + `lower_bound`
Find	$\Theta(1)$ expected	$\Theta(\log n)$	$\Theta(\log n)$
Insert, remove	$\Theta(1)$ expected	$\Theta(\log n)$	$\Theta(n)$
Worst case	$\Theta(n)$	$\Theta(\log n)$	as above
Smallest, largest, next, range	no	yes	yes
Iteration order	arbitrary	sorted	sorted
Needs from the key type	hash and ==	<	<
Memory per entry	moderate	high (three pointers)	minimal
Cache behavior	fair	poor	excellent

A reasonable policy follows from the table.

- If you only need to look things up by key, use a **hash table**. It is the fastest.
- If you need the keys in order, or the nearest key, or a range, or a guaranteed worst case, use a **balanced tree**.
- If the data is built once and then only queried, sort a **vector** and use binary search. It is smaller and faster than either. (C++23 packages this arrangement as `std::flat_map`.)

THE FOUR QUESTIONS **AVL tree**

Where does it live? As a binary search tree: one heap node per key, each owning its children. Each node also stores the height of its subtree.

What does it promise? The search-tree invariant, and in addition: at every node, the two subtrees differ in height by at most one. Hence the height is less than $1.45 \log_2(n+2)$.

What does it cost? Search, insert, remove, minimum, lower bound: $\Theta(\log n)$ in the worst case, with no assumptions about the input. Less than one rotation per insertion on average. Each step is a pointer chase, so for large trees the constant is dominated by cache misses.

How do you know? Differential testing checks the answers but *cannot* detect broken balancing. Check the balance condition and the stored heights at every node after every operation, and assert the height of a tree built from sorted keys.

THE FOUR QUESTIONS **B-tree**

Where does it live? Nodes that each hold many keys in a sorted, contiguous array, plus one more child pointer than keys. Few, large nodes in place of many small ones.

What does it promise? Keys in each node are sorted and divide the key ranges of the children. No node holds more than the maximum number of keys. All leaves are at the same depth.

What does it cost? Search and insert: $\Theta(\log n)$ comparisons, but only about $\log_B n$ nodes visited, where B is the number of children per node. Far fewer cache misses, or disk reads, than a binary tree.

How do you know? Differential testing against `std::set` with a tiny node size, so that splits happen constantly; an invariant check for sortedness, node capacity, and equal leaf depth.

Exercises

1. Draw the result of a left rotation at the root of the tree with pre-order 20, 10, 40, 30, 50. Then draw the result of a right rotation at the root of *that* tree. Confirm that the in-order sequence is unchanged throughout.
2. For each of the following, draw the tree and say whether it is a valid AVL tree. If not, name the lowest node at which the invariant fails. (Each is given in pre-order.) (a) 30, 20, 10. (b) 30, 10, 20, 40, 50. (c) 50, 30, 20, 10, 40, 70.
3. Insert the keys 50, 40, 30, 60, 70, 65, 45, 47 into an empty AVL tree. Draw the tree after each insertion, and name each rotation case.
4. Using the recurrence for $N(h)$, extend the table to $h = 15$. What is the tallest AVL tree that could hold 1,000 keys? Construct an AVL tree of height 4 with exactly seven nodes.
5. In `rebalance`, the test for the inside case is `balance(*link->left) < 0`. After an *insertion*, the left child's balance factor at this point is never zero. After a *removal*, it can be. Draw a tree in which removing one key leaves a node with balance factor +2 whose left child has balance factor 0, and confirm that the code handles it correctly with a single rotation.
6. Explain why, after an insertion into an AVL tree, at most one call to `rebalance` on the path to the root actually performs a rotation. (Consider the height of the rotated subtree before the insertion and after the rotation.) Then give an example in which a single removal causes rotations at two different levels.
7. Perform mutation testing on `avl.hpp`. For each change, record whether the differential test alone catches it and whether `check_invariant` catches it. (a) Delete the body of `rebalance` except for `update(*link)`. (b) In `rotate_right`, omit the first `update`. (c) Swap the two rotation calls in the left-heavy branch. (d) In `erase`, omit the final call to `rebalance`. What does this tell you about what each kind of test is able to see?
8. Add `const K* lower_bound(const K& key) const` to `AvlTree`. Then add a member `std::size_t rank(const K& key) const` that returns the number of keys less than `key` in $O(\log n)$ time. (Hint: store in each node the size of its subtree, and maintain it wherever the height is maintained.)

9. Insert the keys 10, 20, 30, 40, 50, 60, 70, 80, 90 in order into a B-tree whose nodes hold at most three keys, using the split rule of this chapter. Draw the tree after each split. What is the height at the end?
10. A B-tree's nodes hold at most 255 keys, and every node except the root is at least half full. What is the largest possible height of such a tree holding one billion keys? What is the smallest?
11. Describe, with pictures, how to remove a key from a B-tree *leaf* in each of these situations: (a) the leaf still has enough keys afterward; (b) the leaf is now too empty, but a neighboring sibling has a key to spare; (c) the leaf is too empty and so is its sibling. (You are designing the algorithm; check your design against a textbook afterward.)
12. A program records sensor readings as `(timestamp, value)` pairs and must answer: "what was the most recent reading at or before time t?" Readings arrive in increasing order of timestamp. Which of the three containers in the table of Section 7.5 would you use, and how would you answer the query? Would your answer change if readings could arrive out of order?
13. **With a model, as critic.** Without looking at `avl.hpp`, write `rotate_left` and `rotate_right` for a tree with *raw* child pointers and a `parent` pointer in each node, as many textbook implementations have. Count how many pointers each rotation must update. Then ask a model to "list every pointer that a left rotation must update in a tree with parent pointers, including the cases where a subtree is empty or the rotated node is the root," and compare its list with your code. Whose list was complete?
14. **Lab.** Build `AvlMap<K, V>`, an ordered map, with `put`, `find`, `erase`, `lower_bound`, and in-order iteration by a `for_each` function. Use it to implement an *interval scheduler*: store appointments keyed by start time, and support "add an appointment unless it overlaps an existing one" in $O(\log n)$ time. (Which existing appointments could a new one possibly overlap? You need to look at only two.) Test the map differentially against `std::map`, with the invariant check running after every operation.

Chapter notes

The AVL tree is from a four-page paper of 1962 [2], the first self-balancing search tree. Red-black trees originate with Bayer's "symmetric binary B-trees" of 1972 and were given their colors, and their modern formulation, by Guibas and Sedgewick [26]; they are best understood as binary encodings of B-trees with at most three keys per node, a correspondence that Sedgewick and Wayne [48] develop at length. Cormen et al. [16] give the full red-black algorithms, including removal.

The B-tree is due to Bayer and McCreight [5]; Comer's survey [15], titled "The Ubiquitous B-Tree" when the structure was less than a decade old, remains a fine introduction. On the randomized alternatives, see Pugh on skip lists [47] and Seidel and Aragon on treaps [49]; on splay trees, Sleator and Tarjan [52]. Knuth [34] covers balanced trees in depth and derives the AVL height bound quoted here.

Answers to check-yourself questions

7.1 Make 2 the root, with 1 as its left child and 3 as its right. Two pointers changed: 1 no longer points to 2, and 2 now points to 1 (and whatever pointed to 1 from above must now point to 2). The change is entirely local. That local rearrangement is called a rotation, and it is the tool that every balanced binary tree is built from.

7.2 Inserting 30 unbalances 10 (right-right): rotate left at 10, giving 20 at the root. Inserting 50 unbalances 30 (right-right): rotate left at 30, so 40 has children 30 and 50. Inserting 25 unbalances the root 20, whose right subtree (rooted at 40) is now two taller than its left, and the extra height is on the *inside*, under 40's left child 30. That is the right-left case: rotate right at 40, then left at 20. The final tree has 30 at the root, with children 20 (over 10 and 25) and 40 (over 50). Figure 7.3 shows the last step.

7.3 Each level multiplies the number of reachable keys by sixteen, so the height is about $\log_{16} n$: five levels for a million keys, in place of twenty. Within a node, the keys are a small sorted array, so use binary search (or, for an array this small, even a linear scan). Those comparisons all touch one contiguous block, one or two cache lines, and so cost very little.

CHAPTER 8

Heaps and Sorting

WEEK 8 OF 10

THIS WEEK

By the end of this week you should be able to:

- state the two invariants of a binary heap, and explain how they let a tree be stored in an array with no pointers;
- implement push, pop, and linear-time heap construction, and use `std::priority_queue`;
- explain and implement heapsort, mergesort, and quicksort, and give the time, space, and stability of each;
- explain why no comparison-based sort can beat $n \log n$, and how counting sort gets around the limit;
- use the standard library's sorting facilities well, and say what `std::sort` actually does.

An emergency room does not treat patients in the order they arrive. It treats the most urgent first. An operating system runs the highest-priority task that is ready. A simulation processes whichever pending event has the earliest timestamp. In each case items arrive in any order, and at any moment we want to remove the *smallest* one (most urgent, earliest), with arrivals and removals interleaved.

CHECK YOURSELF 8.1

Consider implementing this with structures we already have. For each, give the cost of adding an item and the cost of removing the smallest: (a) an unsorted vector; (b) a vector kept sorted; (c) a balanced search tree. Is there anything unsatisfying about the best of them?

That last observation is the key to this chapter. Fully sorted order is a strong promise, and expensive to maintain. If all we need is fast access to the minimum, a much weaker promise will do, and a weaker promise is cheaper to keep. The structure that makes exactly the right promise is the **heap**. It will turn out to be a tree that lives in an array, which gives it the best of both worlds, and it leads directly into the oldest problem in the field: sorting.

(The word "heap" here has nothing to do with the heap of Chapter 1, the region of memory that `new` allocates from. It is an unfortunate coincidence of names.)

8.1 Priority queues and the binary heap

The abstract data type is called a **priority queue**.

Operation	Meaning
`push(x)`	add an element
`top()`	look at the smallest element
`pop()`	remove the smallest element

A **binary heap** implements it with a binary tree that obeys two invariants.

Shape. *The tree is* complete*: every level is full except possibly the last, and the last level is filled from the left with no gaps.*

Order. *Every node's key is less than or equal to the keys of its children.*

The order invariant is the weak promise. It says nothing about left versus right; siblings are in no particular order, and a key on the left side of the tree can be larger than a key on the right. But it is enough to guarantee one thing: following it up from any node, keys only get smaller, so **the smallest key is at the root**. A heap ordered this way is called a *min-heap*. Reverse the comparison and the largest key is at the root: a *max-heap*.

The shape invariant is what makes the structure remarkable. Because a complete tree has no gaps, its nodes can be numbered level by level, left to right, starting from 0 at the root, and stored in an array in that order. And then the tree structure does not need to be stored at all, because it can be *computed*. For the node at index i:

$$\text{left child: } 2i+1 \qquad \text{right child: } 2i+2 \qquad \text{parent: } \lfloor (i-1)/2 \rfloor$$

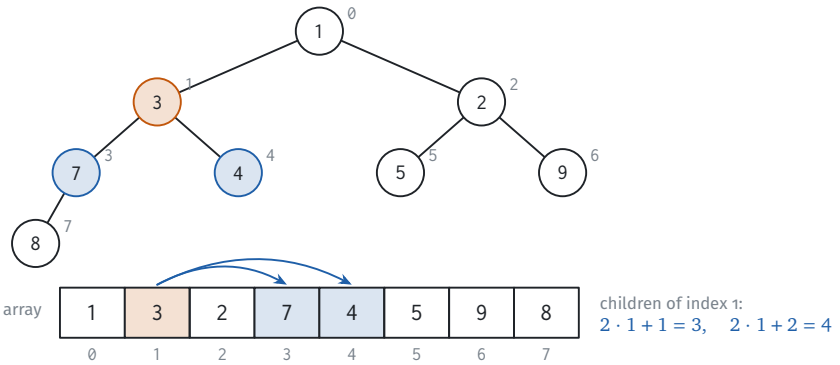


Figure 8.1 A binary min-heap, shown as a tree and as the array that actually stores it. The children of the element at index i are at indices $2i+1$ and $2i+2$. No pointers are stored. In the web edition, push and pop elements and watch them sift up and down in both views. [Interactive in the web edition.]

Look at what this does to the first of our four questions. A binary search tree lives in scattered nodes joined by pointers. A heap is a tree that lives in *one contiguous block*, with no pointers and no per-node allocation, and whose links are a multiplication and an addition. It has the logarithmic height of a balanced tree and the memory layout of a vector. The whole representation is a `std::vector`.

Push

To add an element, there is only one place it can go without breaking the shape invariant: the next free position at the end of the array, which is the next gap on the bottom level. That may break the order invariant, if the new element is smaller than its parent. So swap it with its parent. Now it may be smaller than its *new* parent; swap again. The element rises until it meets a parent that is no larger, or reaches the root. This is called **sifting up**.

Pop

The smallest element is at the root, and removing it leaves a hole at the top. Shape again dictates the repair: the only node that can be removed without leaving a gap is the *last* one. So move the last element into the root and shrink the array by one. The shape is now correct, and the order is probably wrong at the root. Compare the root with its children and swap it with the *smaller* child; repeat from there. The element sinks until both its children are no smaller, or it reaches the bottom. This is **sifting down**.

code/ch08/heap.hpp

```
const T& top() const {          // the smallest element
    assert(!a_.empty());
    return a_[0];
}

void push(T value) {
    a_.push_back(std::move(value));   // add at the bottom...
    sift_up(a_.size() - 1);           // ...and let it rise
}

void pop() {                    // remove the smallest element
    assert(!a_.empty());
    std::swap(a_[0], a_.back());      // last element takes the root's place
    a_.pop_back();
    if (!a_.empty()) sift_down(0);    // ...and sinks to where it belongs
}
```

code/ch08/heap.hpp

```
void sift_up(std::size_t i) {
    while (i > 0) {
        std::size_t parent = (i - 1) / 2;
        if (!(a_[i] < a_[parent])) break;     // parent is no larger: done
        std::swap(a_[i], a_[parent]);
        i = parent;
    }
}

void sift_down(std::size_t i) {
    const std::size_t n = a_.size();
    while (true) {
        std::size_t left = 2 * i + 1;
        if (left >= n) break;                 // no children: done
        std::size_t smaller = left;
```

```
        std::size_t right = left + 1;
        if (right < n && a_[right] < a_[left]) smaller = right;
        if (!(a_[smaller] < a_[i])) break;    // both children are no smaller
        std::swap(a_[i], a_[smaller]);
        i = smaller;
    }
}
```

CHECK YOURSELF 8.2

When sifting down, why must the element be swapped with the *smaller* of its two children, and not just any child that is smaller than it?

A complete tree with n nodes has height $\lfloor \log_2 n \rfloor + 1$, and each of these operations moves an element along a single root-to-leaf path. So push and pop are $\Theta(\log n)$ in the worst case, and top is $\Theta(1)$. There is no shape problem to worry about and no rebalancing: the shape invariant makes the tree perfectly balanced by construction.

Building a heap in linear time

Given n elements all at once, we could build a heap by pushing them one at a time, for a total of $\Theta(n \log n)$. There is a better way, found by Robert Floyd [22]. Put the elements in the array in any order. Then, working backward from the last node that has children to the root, sift each one down.

code/ch08/heap.hpp

```
// Build a heap from an arbitrary vector in linear time: sift down
// every node that has children, from the last such node back to the root.
explicit MinHeap(std::vector<T> items) : a_(std::move(items)) {
    for (std::size_t i = a_.size() / 2; i-- > 0;) sift_down(i);
}
```

When a node is sifted down, both of its subtrees are already heaps, since they were handled earlier, so sifting down makes the whole subtree rooted there into a heap. The surprise is the cost. Half the nodes are leaves and are not touched at all. A quarter are one level above the leaves and can sink at most one level. An eighth can sink at most two. The total number of moves is at most

$$\frac{n}{4} \cdot 1 + \frac{n}{8} \cdot 2 + \frac{n}{16} \cdot 3 + \cdots < n.$$

Building a heap takes $\Theta(n)$ time. The trick is that the expensive nodes, the ones near the top with a long way to sink, are few, and the numerous nodes near the bottom are cheap.

In the standard library

The standard priority queue is `std::priority_queue`, an adapter over a vector, like `std::stack`.

code/ch08/sort_test.cpp

```cpp
std::priority_queue<int> largest_first;          // a max-heap by default
for (int x : {3, 9, 4}) largest_first.push(x);
assert(largest_first.top() == 9);

// To get the smallest first, supply std::greater as the comparison.
std::priority_queue<int, std::vector<int>, std::greater<int>> smallest_first;
for (int x : {3, 9, 4}) smallest_first.push(x);
assert(smallest_first.top() == 3);
```

PITFALL

`std::priority_queue` is a **max**-heap by default: `top()` is the *largest* element. This is the opposite of the convention in most textbooks and of what many algorithms want. To get a min-heap you must name all three template parameters, as above. Getting this backward is a rite of passage, and the bug is quiet: the program runs, and processes things in the worst possible order.

A heap is not a general-purpose container, and it pays to be clear about what it has given up. There is no efficient way to search it for a particular element, and no way to visit its elements in sorted order short of popping them all. It does one thing.

Operation	Sorted vector	Balanced tree	Binary heap
find the minimum	$\Theta(1)$	$\Theta(\log n)$	$\Theta(1)$
remove the minimum	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$
insert	$\Theta(n)$	$\Theta(\log n)$	$\Theta(\log n)$
build from n elements	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$
search for an arbitrary key	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(n)$
extra memory per element	none	two or three pointers	none

IN THE WILD

Priority queues are everywhere that "next" means "most important" and not "oldest." Operating systems use them to pick the next timer to fire. Network routers use them to schedule packets. Discrete-event simulators, from chip design to epidemiology, are a loop around a priority queue of future events. In the next chapter a priority queue will drive the most famous shortest-path algorithm.

And there is one in the loop that generates text from a language model. At each step the model assigns a score to every token in its vocabulary, perhaps a hundred thousand of them, and a common sampling strategy considers only the k highest-scoring ones. Finding the top k of a hundred thousand numbers does not require sorting them all: a heap, or the selection algorithm at the end of this chapter, does it in a fraction of the time, and it happens for every token of every response.

8.2 Sorting with a heap, and by merging

Sorting, putting a sequence into order, is the most studied problem in computer science. It matters in itself, and it matters even more as a preliminary: binary search requires sorted input, finding duplicates is trivial once equal items are adjacent, and many problems that look hard become easy once the data is in order. We met insertion sort in Chapter 2 and found it to be $\Theta(n^2)$ in the worst case. We can do much better, and a heap gives the first way.

Heapsort

Popping every element from a min-heap delivers them in increasing order, so n pushes followed by n pops is a sort, in $\Theta(n \log n)$ time. J. W. J. Williams, who invented the heap in 1964, invented it for this purpose [61]. With one twist the sort can be done *in place*, within the array being sorted, using no extra memory at all.

Build a **max**-heap out of the whole array. Its largest element is at index 0. Swap it with the last element of the array: the largest element is now in its final position. Regard the heap as one element shorter, and sift the new root down to restore it. Repeat. The heap shrinks from the back as the sorted region grows behind it.

code/ch08/sorts.hpp

```
// Sift a[i] down within the max-heap a[0..n).
template <typename T>
void sift_down_max(std::vector<T>& a, std::size_t i, std::size_t n) {
    while (true) {
        std::size_t left = 2 * i + 1;
        if (left >= n) break;
        std::size_t larger = left;
        if (left + 1 < n && a[left] < a[left + 1]) larger = left + 1;
        if (!(a[i] < a[larger])) break;
        std::swap(a[i], a[larger]);
        i = larger;
    }
}

template <typename T>
void heap_sort(std::vector<T>& a) {
    const std::size_t n = a.size();
    for (std::size_t i = n / 2; i-- > 0;) sift_down_max(a, i, n);   // build a
    ↪ max-heap
    for (std::size_t end = n; end > 1; --end) {
        std::swap(a[0], a[end - 1]);        // largest goes to its final place
        sift_down_max(a, 0, end - 1);       // restore the heap on what is left
    }
}
```

Heapsort takes $\Theta(n \log n)$ time in the worst case and $\Theta(1)$ extra space, a combination no other algorithm in this chapter achieves. Its weakness is one you can now predict from the index arithmetic: sifting down hops from index i to $2i + 1$ to $4i + 3$, further apart at

every step, all over the array. It has poor locality, and on large inputs it is the slowest of the fast sorts in practice.

Mergesort

The second fast sort is the divide-and-conquer shape from Section 6.1, applied in earnest. It rests on one observation: two sequences that are *already sorted* can be combined into one sorted sequence quickly.

> **CHECK YOURSELF 8.3**
>
> You have two stacks of exam papers, each already in alphabetical order, and you want one alphabetized stack. Describe your procedure. If the stacks hold n papers altogether, how many times do you compare two names?

That procedure is called a **merge**, and it takes linear time. Figure 8.2 shows one in progress.

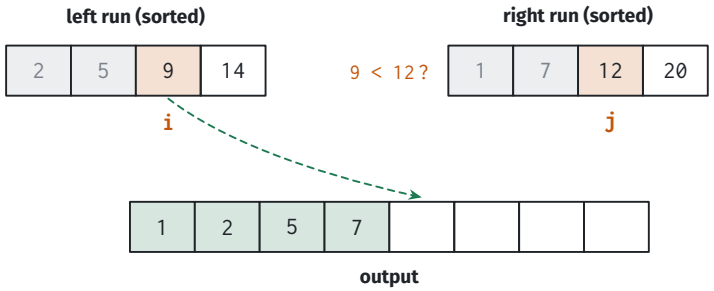


Figure 8.2 Merging two sorted runs. At each step the smaller of the two front elements is moved to the output. Here 9 and 12 are being compared, and 9 will be taken next. Every element is moved exactly once.

Mergesort is then three lines of idea. To sort a sequence: sort its left half; sort its right half; merge the two. And a sequence of zero or one elements is already sorted, which is the base case.

code/ch08/sorts.hpp

```
// Sort a[lo..hi), using scratch[lo..hi) as working space.
template <typename T>
void merge_sort(std::vector<T>& a, std::vector<T>& scratch,
                std::size_t lo, std::size_t hi) {
    if (hi - lo < 2) return;                // 0 or 1 elements: already sorted
    std::size_t mid = lo + (hi - lo) / 2;
    merge_sort(a, scratch, lo, mid);        // sort the left half
    merge_sort(a, scratch, mid, hi);        // sort the right half

    // Merge the two sorted halves into scratch.
    std::size_t i = lo, j = mid, k = lo;
    while (i < mid && j < hi) {
```

```
        if (a[j] < a[i]) scratch[k++] = std::move(a[j++]);
        else scratch[k++] = std::move(a[i++]);   // on a tie, take from the left
    }
    while (i < mid) scratch[k++] = std::move(a[i++]);
    while (j < hi) scratch[k++] = std::move(a[j++]);
    for (k = lo; k < hi; ++k) a[k] = std::move(scratch[k]);
}

template <typename T>
void merge_sort(std::vector<T>& a) {
    std::vector<T> scratch(a.size());
    merge_sort(a, scratch, 0, a.size());
}
```

The two recursive calls sort the halves `[lo, mid)` and `[mid, hi)`. The loop merges them into a scratch array, and the result is copied back. Trusting the recursion is essential here: do not try to follow the calls all the way down. Assume the two halves come back sorted, and check that the merge then produces a sorted whole.

How long does it take? Let $T(n)$ be the time to sort n elements. The function does two recursive calls on half-size inputs and then a linear amount of merging:

$$T(n) = 2\,T(n/2) + \Theta(n).$$

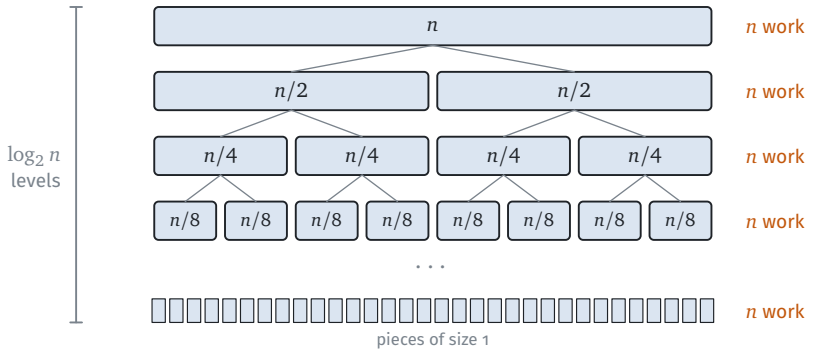


Figure 8.3 The recursion tree of mergesort on n elements. Each level of the tree partitions all n elements into pieces, and the merging at that level touches each element once: n work per level. Halving reaches pieces of size 1 after $\log_2 n$ levels. Total: $n \log_2 n$.

The picture in Figure 8.3 solves it. At the top level there is one merge of n elements. At the next, two merges of $n/2$: again n in total. Then four merges of $n/4$. Every level does n work, and there are $\log_2 n$ levels. Mergesort takes $\Theta(n \log n)$ time, and since it does the same splitting regardless of the data, that is its best, worst, and average case alike.

Mergesort has two further properties worth knowing.

It needs $\Theta(n)$ extra space for the scratch array. That is its main cost.

It is **stable**: elements that compare equal come out in the same relative order they went in. The code ensures this by taking from the *left* run when the two front elements are equal. Stability matters whenever records are sorted by one field and already have a

meaningful order in another. Sort a roster by name, then stably by graduation year, and each year's students are still in alphabetical order. An unstable sort would scramble them. Heapsort is not stable.

8.3 Quicksort

The third fast sort also divides and conquers, but it does the hard work *before* the recursive calls instead of after. Tony Hoare invented it in 1959, as a visiting student in Moscow, and it remains the basis of most library sorts [28].

Pick one element of the sequence and call it the **pivot**. Rearrange the sequence so that everything smaller than the pivot comes before everything larger, with the pivot's equals in between. This is called **partitioning**. Now the pivot is in its final sorted position, and no element will ever need to cross from one side to the other. Sort the two sides, recursively and independently. There is nothing to merge.

Partitioning can be done in place in a single pass. The version here divides the array into three regions as it goes: less than the pivot, equal to it, and greater.

code/ch08/sorts.hpp

```
// Sort a[lo..hi).
template <typename T>
void quick_sort(std::vector<T>& a, std::size_t lo, std::size_t hi,
↪  std::mt19937& rng) {
    if (hi - lo < 2) return;
    T pivot = a[lo + rng() % (hi - lo)];     // a randomly chosen element

    // Partition into three regions. Invariant:
    //   a[lo..lt) < pivot,   a[lt..i) == pivot,   a[gt..hi) > pivot,
    //   and a[i..gt) has not been examined yet.
    std::size_t lt = lo, i = lo, gt = hi;
    while (i < gt) {
        if (a[i] < pivot) std::swap(a[lt++], a[i++]);
        else if (pivot < a[i]) std::swap(a[i], a[--gt]);
        else ++i;
    }
    quick_sort(a, lo, lt, rng);             // everything smaller than the pivot
    quick_sort(a, gt, hi, rng);               // everything larger
}

template <typename T>
void quick_sort(std::vector<T>& a) {
    std::mt19937 rng(std::random_device{}());
    quick_sort(a, 0, a.size(), rng);
}
```

The loop is a fine example of programming by invariant, in the manner of binary search. The comment states what is true of the four regions, and Figure 8.4 draws it. Each iteration looks at the first unexamined element, `a[i]`. If it is smaller than the pivot, it is swapped to the end of the "less" region, and both boundaries advance. If it is larger, it is swapped with the last unexamined element, which extends the "greater" region leftward; `i` does

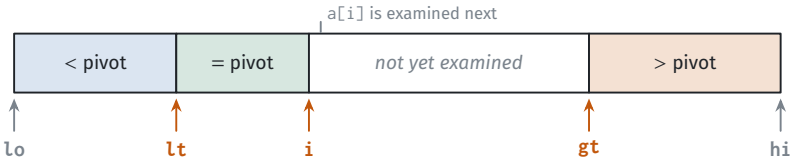


Figure 8.4 The invariant of the three-way partition, part of the way through. Everything left of `lt` is less than the pivot; everything from `lt` up to `i` equals it; everything from `gt` onward is greater. The elements from `i` up to `gt` have not yet been examined. Each step examines `a[i]` and shrinks the unexamined region by one.

not advance, because the element that was swapped in has not been examined yet. If it is equal, it is already in the right place. The unexamined region shrinks by one each time, and when it is empty the partition is complete.

How fast is it?

Partitioning n elements takes $\Theta(n)$ time. What happens next depends entirely on how evenly the pivot splits the array.

If the pivot always lands near the middle, the two sides are each about half the size, and the recurrence is the same as mergesort's: $\Theta(n \log n)$.

If the pivot is always the smallest or largest element, one side is empty and the other has $n - 1$ elements. The recursion goes n levels deep, with partitions of size n, $n - 1$, $n - 2$, and so on. That is our familiar sum: $\Theta(n^2)$.

So quicksort's worst case is as bad as insertion sort's. How likely is it? That depends on how the pivot is chosen, and the most natural choice is a trap. Here are comparison counts for a textbook quicksort that always uses the *first* element as the pivot.

n	random input	already-sorted input	$n \log_2 n$
1,000	10,684	499,500	9,966
10,000	157,017	49,995,000	132,877
100,000	1,965,347	4,999,950,000	1,660,964

On random input it is splendid: only a little more than $n \log_2 n$ comparisons. On input that is already sorted, the first element is always the smallest, every partition is as lopsided as it can be, and the count is exactly $n(n-1)/2$. Five *billion* comparisons to "sort" a hundred thousand elements that were in order to begin with. We have seen this pattern twice before, with binary search trees and with hash tables: fine on random data, disastrous on a pattern, and the pattern (sorted input) is one that real data exhibits all the time.

The remedy is the one used for hashing. Take the choice away from the input. The `quick_sort` above picks its pivot **at random**. Now no particular input is bad. The already-sorted array is no worse than any other, because the pivots are not chosen by position. For *every* input, a quicksort with random pivots is expected to examine about $1.39\, n \log_2 n - 2.85\, n$ elements in its partitioning passes, which is what the random-input column of the table

shows, and the probability of running significantly slower than that is vanishingly small. (Our three-way version tests some elements twice, once for "less" and once for "greater", so it performs about half again as many < comparisons.) The worst case still exists, but it has been moved from "a bad input," which an unlucky or malicious user can supply, to "a bad run of random numbers," which nobody controls.

Quicksort sorts in place (using only the stack space for the recursion, $O(\log n)$ deep in expectation), and it has excellent locality: partitioning is a pair of sequential sweeps through the array. That is why, despite its worst case, it is usually the fastest of the three in practice. It is not stable.

Figure 8.5 animates all of these algorithms on the same data.

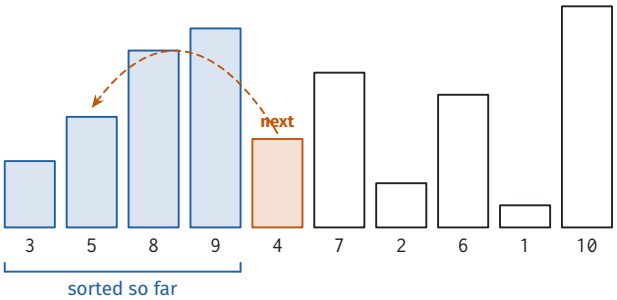


Figure 8.5 Insertion sort part of the way through an array, drawn as bars. The shaded bars at the left are already in order; the next bar will be slid into place among them. In the web edition, choose insertion sort, heapsort, mergesort, or quicksort, and an input that is random, sorted, or reversed, and compare the counts of comparisons and moves. [Interactive in the web edition.]

BY HAND FIRST

With pencil and paper: (1) Starting from the array `{9, 4, 7, 1, 8, 2}`, build a min-heap by Floyd's method, drawing the tree after each sift-down. Then pop twice. (2) Partition `{5, 3, 8, 5, 1, 9, 5, 2}` around the pivot 5 using the three-way scheme, writing out the array and the positions of `lt`, `i`, and `gt` after every step. (3) Merge `{2, 5, 9}` with `{1, 5, 7, 12}`, and say which 5 comes out first.

These are the three inner loops of the chapter. Each has an invariant, and you understand the algorithm when you can maintain the invariant by hand. Index arithmetic in a heap and boundary movement in a partition are also places where a model asked to "show the steps" will sometimes drift, producing a trace that looks right and has one swap wrong in the middle.

8.4 How fast can sorting be?

Three different algorithms, three different ideas, and all of them arrive at $n \log n$. That is suspicious. Is it a coincidence, or a barrier?

It is a barrier, and the argument that establishes it is one of the prettiest in the subject, because it applies to *every possible* sorting algorithm of a certain kind, including ones not yet invented.

Consider any algorithm that learns about its input only by **comparing** pairs of elements. All the sorts so far are like this. Each comparison has two outcomes, so the algorithm's whole execution on a given input can be described by the sequence of yes-or-no answers it received. After k comparisons there are at most 2^k different sequences of answers.

Now, n distinct elements can arrive in $n!$ different orders, and a correct algorithm must do something different for each one, since each needs a different rearrangement. If two input orders produced the *same* sequence of answers, the algorithm could not tell them apart and would rearrange them identically, getting at least one wrong. So the number of distinguishable answer sequences must be at least the number of possible inputs:

$$2^k \geq n!, \qquad \text{that is,} \qquad k \geq \log_2(n!).$$

And $\log_2(n!)$ is approximately $n \log_2 n - 1.44\, n$. Therefore:

Any sorting algorithm based on comparisons must make at least about $n \log_2 n$ comparisons in the worst case.

Mergesort and heapsort are *optimal*, up to a constant factor. No cleverness will find a comparison sort that runs in linear time, because the limit comes from information: there are $n!$ possibilities to distinguish, and each comparison yields one bit.

Sorting without comparing

The theorem has a loophole, stated right in its hypothesis: *based on comparisons*. An algorithm that examines the keys themselves can do better, when the keys have exploitable structure.

Suppose the keys are integers known to lie between 0 and $k - 1$. Make an array of k counters; pass over the input once, tallying each value; then write out each value as many times as it was seen.

code/ch08/sorts.hpp

```
// Sort integers known to lie in the range [0, k), without comparing them.
inline void counting_sort(std::vector<int>& a, int k) {
    std::vector<std::size_t> count(static_cast<std::size_t>(k), 0);
    for (int x : a) ++count[static_cast<std::size_t>(x)];     // tally each
    ↪ value
    std::size_t out = 0;
    for (int v = 0; v < k; ++v) {                             // write them back
    ↪ in order
        for (std::size_t c = 0; c < count[static_cast<std::size_t>(v)]; ++c)
        ↪ a[out++] = v;
    }
}
```

This is **counting sort**. It never compares two elements. It uses each key as an array index, the direct-addressing idea from Section 5.1, and it runs in $\Theta(n + k)$ time. When k is not much larger than n, that is linear. **Radix sort** extends the idea to larger keys by sorting on one digit at a time, with a stable counting sort for each digit.

The stopwatch

Here are the times, in seconds, to sort n random integers between 0 and 999,999 with each algorithm in this chapter, alongside the standard library's `std::sort`.

n	insertion	heapsort	mergesort	quicksort	`std::sort`	counting
1,000	0.0001	0.0001	0.0001	0.0001	0.0000	0.0037
10,000	0.0071	0.0006	0.0006	0.0005	0.0003	0.0026
100,000	0.73	0.0061	0.0052	0.0060	0.0039	0.0020
1,000,000		0.076	0.062	0.066	0.043	0.0070
10,000,000		1.13	0.72	0.61	0.50	0.026

Several lessons from earlier chapters are visible at once.

Insertion sort is quadratic: each tenfold increase in n multiplies its time by a hundred. At ten million elements it would need about two hours. The three $n \log n$ sorts finish in around a second.

Among those three, the ranking at large n follows locality, not operation counts. Heapsort, which jumps around the array, is nearly twice as slow as quicksort, which sweeps through it.

Counting sort is twenty times faster than `std::sort` at ten million elements, because it is linear and the others are not. But look at the top of its column: for a thousand elements it is the *slowest*, because it must still initialize and scan a million counters. Its cost is $\Theta(n+k)$, and when k dwarfs n, the k is what you pay for.

And the standard library's sort is faster than all of our comparison sorts, which is the subject of the next section.

8.5 Sorting in practice

You should understand the algorithms above, and you should essentially never use your own implementations of them. Every language's standard library has a sort that is the product of decades of engineering, and it will be faster and more robust than anything written in an afternoon.

What these library sorts have in common is that they are **hybrids**, combining several of the algorithms in this chapter so that each covers for the others' weaknesses.

- `std::sort` in the common C++ libraries is **introsort** [42]. It begins as quicksort, with pivots chosen carefully. If the recursion gets suspiciously deep, a sign that the pivots are splitting badly, it switches to heapsort, whose worst case is guaranteed. And when a piece is small, a few dozen elements or fewer, it finishes with insertion sort, which has the lowest overhead on tiny inputs. The result has quicksort's speed and heapsort's guarantee.
- `std::stable_sort` is a mergesort, with the same small-piece optimization.
- Python's `sorted` and Java's sort for objects use **Timsort** [45], a mergesort that first looks for stretches of the input that are already in order and merges those. On real-world data, which is often partly sorted, it does far fewer comparisons than $n \log n$.

Insertion sort, the "slow" algorithm, is inside every one of them. This is the practical meaning of the warning in Section 2.2 that Big-O hides constants: for twenty elements, the quadratic algorithm with the tiny constant wins.

Using the library

`std::sort` takes a pair of iterators and, optionally, a comparison that says when one element should come before another.

code/ch08/sort_test.cpp

```
struct Student {
    std::string name;
    int year;
    double gpa;
};
```

code/ch08/sort_test.cpp

```
std::vector<Student> roster{
    {"Okafor", 2027, 3.6}, {"Lindqvist", 2026, 3.9}, {"Abara", 2027, 3.9},
    {"Tran", 2026, 3.2},   {"Moreau", 2028, 3.6},
};

// Sort with a comparison: "a comes before b when..." Highest GPA first.
std::sort(roster.begin(), roster.end(),
          [](const Student& a, const Student& b) { return a.gpa > b.gpa; });
assert(roster[0].gpa == 3.9 && roster[4].name == "Tran");

// C++20 ranges: sort the whole container by one member.
std::ranges::sort(roster, {}, &Student::name);
assert(roster[0].name == "Abara" && roster[4].name == "Tran");

// A stable sort keeps equal elements in their existing order, so the
// roster is now grouped by year and still alphabetical within each year.
std::ranges::stable_sort(roster, {}, &Student::year);
assert(roster[0].name == "Lindqvist" && roster[1].name == "Tran");
assert(roster[2].name == "Abara" && roster[3].name == "Okafor");
```

The comparison in the first call is a *lambda*, an unnamed function written in place. The square brackets introduce it; then come its parameters and its body.

The second call uses the C++20 **ranges** library: `std::ranges::sort` takes the container itself instead of two iterators, and its third argument is a *projection*, here a pointer to the member that each element should be sorted by. That is often clearer than writing a comparison. (The empty braces in the middle say "use the default comparison, <.")

The third call shows what stability is for. The roster was just sorted by name; a stable sort by year leaves each year's students in name order.

When you do not need the whole sequence sorted, there are cheaper tools.

code/ch08/sort_test.cpp

```
std::vector<int> times{52, 17, 88, 31, 45, 9, 73, 26};
```

```
// The three smallest, in order, without sorting the rest.
std::partial_sort(times.begin(), times.begin() + 3, times.end());
assert(times[0] == 9 && times[1] == 17 && times[2] == 26);

// The median: put the element that belongs at the middle position
// there, with smaller elements before it and larger ones after.
std::nth_element(times.begin(), times.begin() + 4, times.end());
assert(times[4] == 45);
```

`std::partial_sort` uses a heap to deliver the smallest few elements in order. `std::nth_element` uses the partitioning step of quicksort, repeatedly, on only the side that contains the position of interest; it finds the median, or any other rank, in expected *linear* time, without sorting.

PITFALL

The comparison passed to any of these must be a strict "comes before" relation, as discussed for `std::map` in Section 7.5. If it uses <=, or is otherwise inconsistent (for instance, by comparing floating-point values where some are "not a number"), the behavior of `std::sort` is undefined, and in practice it can read outside the array. A sort that crashes mysteriously is very often a sort with a bad comparison.

ASK AN LLM

Questions about what a standard library "really does" are a good test of a model's precision, because the true answer has layers. Ask:

> *What algorithm does std::sort use in C++? Is that required by the standard, or is it an implementation choice? What does the standard actually guarantee?*

A precise answer separates three things: what the C++ standard *requires* (a bound on the number of comparisons, and nothing about which algorithm), what common implementations *do* (introsort or a relative), and what is *not* promised (stability). Check the first against cppreference. An answer that says flatly "std::sort is quicksort" is the kind that is nearly right and would lose marks on an exam.

ASK AN LLM

For practice with the mechanics, use the examiner pattern, and let your own tested code be the referee.

> *Give me an array of eight small integers and ask me to carry out ONE of: a single three-way partition around a stated pivot; one bottom-up heap construction; or one merge of its two sorted halves. Don't show the result until I answer.*

Run the same step in your implementation, printing the array, to see who is right.

The algorithms compared

Algorithm	Worst case	Typical	Extra space	Stable?	Notes
Insertion sort	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(1)$	yes	$\Theta(n)$ on nearly sorted input; best for tiny n
Heapsort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(1)$	no	guaranteed and in place; poor locality
Mergesort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$	yes	predictable; the basis of stable library sorts
Quicksort	$\Theta(n^2)$	$\Theta(n \log n)$	$\Theta(\log n)$	no	fastest in practice; choose pivots at random or carefully
Counting sort	$\Theta(n + k)$	$\Theta(n + k)$	$\Theta(k)$	can be	small integer keys only; not comparison-based

THE FOUR QUESTIONS **Binary heap**

Where does it live? In a single array, with no pointers. The element at index i has its children at $2i + 1$ and $2i + 2$ and its parent at $\lfloor (i - 1)/2 \rfloor$. It is a complete binary tree that exists only as index arithmetic.

What does it promise? Shape: the tree is complete, so the array has no gaps. Order: no element is smaller than its parent, so the minimum is at index 0. Nothing is promised about the order of siblings, and there is no efficient search.

What does it cost? `top`: $\Theta(1)$. `push` and `pop`: $\Theta(\log n)$ worst case. Building from n elements: $\Theta(n)$. No per-element memory overhead.

How do you know? Check that every element is no smaller than its parent, after every operation; test differentially against `std::priority_queue`; pop everything and confirm that the output is sorted.

Exercises

1. Which of these arrays are min-heaps? For those that are not, name an element that violates the order invariant. (a) `{1, 3, 2, 7, 4, 5}` (b) `{2, 5, 3, 4, 9, 8}` (c) `{1, 2, 3, 4, 5, 6, 7}` (d) `{4, 6, 5, 9, 8, 3}`.
2. Starting from an empty min-heap, push 8, 3, 10, 1, 6, 14, 4 in that order. Draw the tree and the array after each push. Then pop three times, again drawing each state.

3. In a min-heap of n distinct elements stored in an array: (a) at which indices can the *largest* element be? (b) at which indices can the second-smallest be? (c) at which indices can the third-smallest be?
4. Add a member `void replace_top(T value)` to `MinHeap`: it removes the smallest element and inserts `value`, using a single sift-down. Then use it to write a function that returns the k largest elements of a vector of n elements in $O(n \log k)$ time and $O(k)$ extra space. (Which kind of heap do you need, of what size?)
5. Building a heap by n successive `push` operations is $\Theta(n \log n)$ in the worst case. Describe an input order that actually takes that long. Then explain, in terms of *which* nodes travel far, why Floyd's bottom-up method is faster.
6. Trace mergesort on `{38, 27, 43, 3, 9, 82, 10}` by drawing the recursion tree: show each subarray as it is split on the way down, and each merged result on the way up.
7. Mergesort as written allocates one scratch vector for the whole sort. A simpler version would allocate a new vector inside every call to hold the merged result. How much total memory would that version allocate over the whole sort? Would its Big-O running time change? Why might its measured running time change a lot?
8. Trace the three-way partition on `{6, 2, 9, 6, 1, 8, 6, 3}` with pivot 6, showing the array and the three boundaries after each step.
9. Our `quick_sort` recurses on both sides. In the worst case the recursion can be n levels deep. Rewrite it so that it recurses only on the *smaller* side and loops on the larger. What is the maximum recursion depth now, and why?
10. Explain why a quicksort with a two-way partition ("less than pivot" on one side, "greater than or equal" on the other) takes quadratic time on an array in which all the elements are equal, whatever pivot it chooses. Why does the three-way partition not have this problem?
11. The lower bound says a comparison sort of 5 elements needs at least $\lceil \log_2 5! \rceil = 7$ comparisons in the worst case. How many does mergesort use, in the worst case, on 5 elements? (Count carefully.)
12. You are given a vector of a million people, each with a `birthday` field giving a day of the year from 1 to 366. Describe the fastest way to sort them by birthday. Should your sort be stable? Implement it.
13. Given a vector of integers, find the pair of elements whose values are closest together. Do it in $O(n \log n)$ time. Then say why hashing, the tool that solved the "find a duplicate" problem in linear time, does not obviously help here.
14. **With a model, as critic.** Write a comparison function for sorting `Student` records by GPA, highest first, breaking ties by name alphabetically. Then ask a model: "Is this a valid strict weak ordering for std::sort? If not, give two specific students that expose the problem." Next, deliberately break it by changing a < to <=, and ask again. Does the model catch it? Does AddressSanitizer, on a large enough roster with many ties?
15. **Lab.** Write a program that finds the k most frequent words in a large text file, for k given on the command line, using a hash map for the counts and a heap of size k for the

selection. Then implement a second version that sorts all the distinct words by count. Time both on a large text (several megabytes from Project Gutenberg, concatenated if need be) for $k = 10$, 1,000, and 100,000, and explain what you see. Your heap must be your own; your hash map may be the library's.

Chapter notes

Knuth's *Sorting and Searching* [34] is the encyclopedic treatment of everything in this chapter, including the history: mergesort goes back to von Neumann in 1945, quicksort to Hoare [28], and the heap and heapsort to Williams [61], with the linear-time construction added by Floyd [22] within the year. The decision-tree lower bound is presented carefully by Cormen et al. [16], who also analyze randomized quicksort.

The engineering of real sorts is its own literature. The three-way partition used here is Dijkstra's solution to what he called the Dutch national flag problem [20]. Bentley and McIlroy's "Engineering a Sort Function" [7] showed how much three-way partitioning matters in a production quicksort, and is a model of careful experimental work. Musser's introsort [42] is the basis of `std::sort`; Peters's description of Timsort [45] is unusually readable for a document that lives in a source tree. Sedgewick and Wayne [48] give the most thorough textbook treatment of quicksort variants.

Answers to check-yourself questions

8.1

(a) Add: $\Theta(1)$, at the end. Remove smallest: $\Theta(n)$, since it must be searched for. (b) Add: $\Theta(n)$, to shift elements; remove smallest: $\Theta(1)$ if the vector is sorted in decreasing order so that the smallest is at the back. (c) Both $\Theta(\log n)$. The tree is the only one with no linear-time operation, but it is doing far more than was asked. It keeps *every* element in order, in a pointer-heavy structure, when all we ever want is the smallest one.

8.2 After the swap, the child that moved up becomes the parent of the other child. If the larger child were moved up, it would sit above its smaller sibling, violating the order invariant at that very node. Moving the smaller child up guarantees that the new parent is no larger than both of the nodes below it.

8.3 Look at the top paper of each stack. Whichever name comes first goes onto the output pile. Repeat until one stack runs out, then add what remains of the other. Each comparison places one paper, so there are at most $n - 1$ comparisons. You never look anywhere but the tops of the two stacks.

CHAPTER 9

Graphs

WEEK 9 OF 10

THIS WEEK

By the end of this week you should be able to:

- model a problem as a graph, saying what the vertices and edges are;
- represent a graph with adjacency lists or an adjacency matrix, and choose between them;
- implement breadth-first and depth-first search, and say what each is good for;
- find a topological order of a directed acyclic graph;
- implement Dijkstra's algorithm with a priority queue, and explain why it requires non-negative weights;
- implement union-find with union by size and path compression.

In a linked list, every node points to at most one other. In a tree, a node may point to several, but no node is pointed to by more than one, and there is a root from which everything hangs. Take away those restrictions, and let any node be connected to any others, and you have a **graph**: the most general linked structure, and the one that describes the world best.

A road map is a graph: towns, and the roads between them. So is a social network (people, and who knows whom), the web (pages and links), an airline's route map, the wiring of a circuit, the prerequisite structure of a curriculum, the dependency structure of a large program, the atoms and bonds of a molecule. Whenever the interesting thing about some objects is not the objects but the *relationships* between pairs of them, the right picture is a graph.

CHECK YOURSELF 9.1

Six people are at a party. Some pairs are friends. You want to know the smallest number of introductions needed to get from Ana to Farid, where each introduction is made by a mutual friend. You start by listing Ana's friends, then *their* friends, and so on. As you work outward, you need to keep track of people you have discovered but whose friends you have not yet listed. In what order should you process them: most recently discovered first, or earliest discovered first? Which data structure is that?

This chapter is different in character from the ones before it. A graph is less a single data structure than a *setting*, in which the structures you already have (vectors, queues, stacks, priority queues, hash tables) are put to work together. Most of what follows is algorithms. Pay attention to which earlier structure sits at the heart of each one. It is rarely an accident.

9.1 Graphs and how to store them

A graph consists of a set of **vertices** (also called nodes) and a set of **edges**, each of which connects two vertices.

- In an **undirected** graph, an edge is a two-way connection: if u is joined to v, then v is joined to u. Friendship is like this.
- In a **directed** graph, each edge has a direction, from one vertex to another. "Page u links to page v" and "course u is a prerequisite of course v" are like this.
- In a **weighted** graph, each edge carries a number: a distance, a cost, a time.

Two vertices joined by an edge are **adjacent**, or **neighbors**. A **path** is a sequence of vertices in which each is adjacent to the next. A **cycle** is a path that returns to where it began. An undirected graph is **connected** if there is a path between every pair of vertices; if not, it falls into several **connected components**.

It is conventional to write V for the number of vertices and E for the number of edges, and to state costs in terms of both. A graph with V vertices can have at most about V^2 edges (every pair). Most graphs that arise in practice have far fewer: a road junction joins three or four roads however large the country, and a person has a few hundred friends however large the network. Such graphs, where E is much nearer to V than to V^2, are called **sparse**.

How should a graph be stored? There are two standard answers, and Figure 9.1 shows both for one small graph.

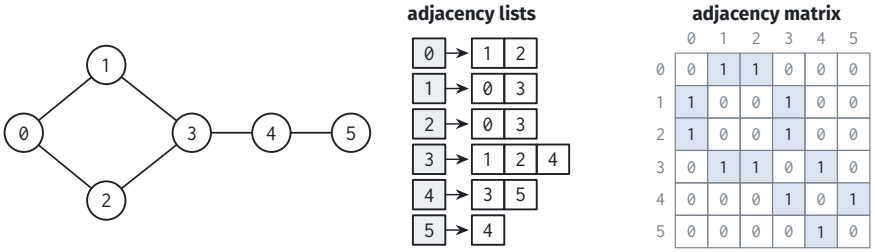


Figure 9.1 An undirected graph with six vertices and six edges, and its two standard representations. The adjacency lists give, for each vertex, the vertices it is joined to. The adjacency matrix has a 1 in row u, column v exactly when there is an edge between u and v.

An **adjacency matrix** is a $V \times V$ table in which the entry in row u and column v says whether there is an edge from u to v (or gives its weight). Testing for a particular edge is a single array access. But the matrix occupies $\Theta(V^2)$ space no matter how few edges there are, and finding all the neighbors of a vertex means scanning a whole row. For a sparse graph with a million vertices, the matrix would have a trillion entries, almost all of them zero.

An **adjacency list** representation stores, for each vertex, a list of its neighbors. The space is $\Theta(V + E)$: proportional to the actual size of the graph. Finding the neighbors of a vertex, which turns out to be the operation that graph algorithms perform constantly, takes time proportional to how many there are.

	Adjacency matrix	Adjacency lists
Space	$\Theta(V^2)$	$\Theta(V+E)$
Is there an edge from u to v?	$\Theta(1)$	$\Theta(\text{number of neighbors of } u)$
Visit all neighbors of u	$\Theta(V)$	$\Theta(\text{number of neighbors of } u)$
Visit every edge	$\Theta(V^2)$	$\Theta(V+E)$

For sparse graphs, which is to say nearly always, adjacency lists are the right choice, and they are what we will use. The representation is a vector of vectors.

code/ch09/graph.hpp

```
struct Edge {
    int to;          // the vertex this edge leads to
    double weight;   // its cost (1 if the graph is unweighted)
};

class Graph {
public:
    explicit Graph(int vertices) : adj_(static_cast<std::size_t>(vertices)) {}

    int size() const { return static_cast<int>(adj_.size()); }

    // A directed edge from u to v.
    void add_edge(int u, int v, double weight = 1.0) {
        adj_[static_cast<std::size_t>(u)].push_back(Edge{v, weight});
    }

    // An undirected edge is a pair of directed ones.
    void add_undirected(int u, int v, double weight = 1.0) {
        add_edge(u, v, weight);
        add_edge(v, u, weight);
    }

    // The edges leaving u.
    const std::vector<Edge>& neighbors(int u) const {
        return adj_[static_cast<std::size_t>(u)];
    }

private:
    std::vector<std::vector<Edge>> adj_;   // adj_[u] lists u's outgoing edges
};
```

Vertices are identified by the integers 0 through $V-1$, so that a vertex can index directly into a vector. This is simple and fast, and it is how nearly all graph code is written. When the vertices of your problem are really names, or coordinates, or web addresses, use a hash map to translate each one to an integer once, at the door.

code/ch09/graph_test.cpp

```
// Six vertices; the undirected edges of the chapter's running example.
Graph g(6);
g.add_undirected(0, 1);
```

```
g.add_undirected(0, 2);
g.add_undirected(1, 3);
g.add_undirected(2, 3);
g.add_undirected(3, 4);
g.add_undirected(4, 5);
```

IN THE WILD

At very large scale even a vector of vectors is too scattered, with one heap allocation per vertex. Graph-processing systems and scientific libraries pack the whole thing into two flat arrays, a format called *compressed sparse row*: one long array holding every vertex's neighbor list end to end, and a second array giving the position where each vertex's list starts. It is the adjacency-list idea with the contiguity lesson of Section 4.5 applied.

9.2 Breadth-first search

The most basic question about a graph is: starting here, what can I reach, and how far away is it? **Breadth-first search**, or BFS, answers it by exploring outward from a source vertex in waves: first the source, then all its neighbors, then all of *their* neighbors that have not been seen, and so on. Figure 9.2 shows the waves on our example graph.

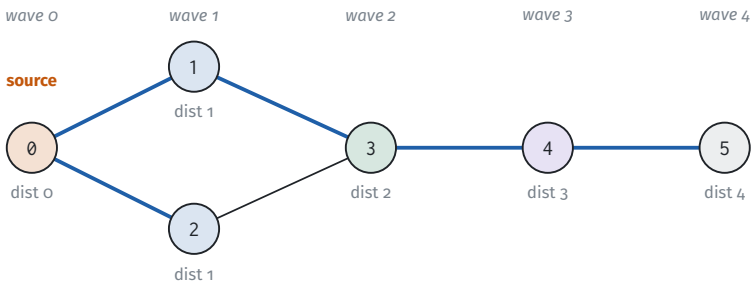


Figure 9.2 Breadth-first search from vertex 0. The number beside each vertex is its distance from the source, in edges. The search discovers vertices in order of distance: everything at distance 1, then everything at distance 2, and so on. The heavy edges are the ones along which each vertex was first discovered; they form a tree of shortest paths.

Two pieces of bookkeeping are needed. First, a record of which vertices have already been discovered, so that the search does not go around a cycle forever. Second, the collection of vertices that have been discovered but not yet processed, called the **frontier**. As the opening question established, the frontier must be a queue.

code/ch09/algorithms.hpp

```
struct BfsResult {
    std::vector<int> dist;     // edges on a shortest path from the source; -1
    ↪  if unreachable
    std::vector<int> parent;   // previous vertex on that path; -1 for the
    ↪  source
```

```
};

inline BfsResult bfs(const Graph& g, int source) {
    BfsResult r{std::vector<int>(idx(g.size()), -1),
↪   std::vector<int>(idx(g.size()), -1)};
    std::queue<int> frontier;
    r.dist[idx(source)] = 0;
    frontier.push(source);

    while (!frontier.empty()) {
        int u = frontier.front();
        frontier.pop();
        for (const Edge& e : g.neighbors(u)) {
            if (r.dist[idx(e.to)] == -1) {           // first time we have seen
             ↪  e.to
                r.dist[idx(e.to)] = r.dist[idx(u)] + 1;
                r.parent[idx(e.to)] = u;
                frontier.push(e.to);
            }
        }
    }
    return r;
}
```

The `dist` vector does double duty. A value of -1 means "not yet discovered"; any other value is the vertex's distance from the source. The algorithm takes a vertex `u` from the front of the queue and examines each of its neighbors. Any neighbor seen for the first time is one edge further from the source than `u` is; it is given its distance and added to the back of the queue.

Why the distances are right. The queue always holds vertices in order of distance: first some at distance d, then some at distance $d + 1$, and never anything else. So vertices are taken from it in order of distance, and a vertex is always discovered from a neighbor that is as close to the source as any of its neighbors. BFS computes the *shortest* path, in number of edges, from the source to every reachable vertex.

What it costs. Each vertex enters the queue at most once, because it is marked when first discovered. When a vertex leaves the queue, its adjacency list is scanned once. So over the whole run, every vertex is handled once and every edge is examined once (twice, in an undirected graph). The running time is $\Theta(V + E)$: linear in the size of the graph. You cannot do better and still look at everything.

Recovering the path

Knowing that Farid is three introductions away is less useful than knowing who should make them. The `parent` vector records, for each vertex, the neighbor from which it was discovered. Following parents from any vertex leads back, step by step, to the source, along a shortest path in reverse.

code/ch09/algorithms.hpp

```
// Follow parent links back from target, then reverse.
inline std::vector<int> path_to(const std::vector<int>& parent, int source,
↪  int target) {
    std::vector<int> path;
    for (int v = target; v != -1; v = parent[idx(v)]) path.push_back(v);
    std::reverse(path.begin(), path.end());
    if (path.front() != source) path.clear();   // target was not reachable
    return path;
}
```

The parent links form a tree, rooted at the source, containing one shortest path to every reachable vertex: the heavy edges in Figure 9.2. Storing one parent per vertex is a compact way to store all those paths at once.

Graphs that are never built

Sometimes the graph is too large to construct, or there is simply no need. A maze drawn on a grid is a graph, with a vertex for each open cell and an edge between adjacent open cells, but the neighbors of a cell can be computed from its coordinates whenever they are wanted.

code/ch09/graph_test.cpp

```
// Fewest steps from (r0, c0) to (r1, c1) in a grid where '#' is a wall,
// moving up, down, left or right. Returns -1 if there is no route.
// The graph is never built: a cell's neighbours are computed on demand.
int maze_steps(const std::vector<std::string>& grid, int r0, int c0, int r1,
↪  int c1) {
    const int rows = static_cast<int>(grid.size());
    const int cols = static_cast<int>(grid[0].size());
    std::vector<std::vector<int>> dist(grid.size(),
    ↪  std::vector<int>(grid[0].size(), -1));
    std::queue<std::pair<int, int>> frontier;
    dist[idx(r0)][idx(c0)] = 0;
    frontier.push({r0, c0});

    const int dr[] = {-1, 1, 0, 0};
    const int dc[] = {0, 0, -1, 1};
    while (!frontier.empty()) {
        auto [r, c] = frontier.front();
        frontier.pop();
        if (r == r1 && c == c1) return dist[idx(r)][idx(c)];
        for (int k = 0; k < 4; ++k) {
            int nr = r + dr[k], nc = c + dc[k];
            if (nr < 0 || nr >= rows || nc < 0 || nc >= cols) continue;   // off
            ↪  the grid
            if (grid[idx(nr)][idx(nc)] == '#') continue;                  // a
            ↪  wall
            if (dist[idx(nr)][idx(nc)] != -1) continue;                   //
            ↪  seen already
            dist[idx(nr)][idx(nc)] = dist[idx(r)][idx(c)] + 1;
            frontier.push({nr, nc});
        }
```

```
    }
    return -1;
}
```

This is the same BFS, with "for each neighbor" replaced by "for each of the four directions that stays on the grid and is not a wall." A graph like this is called **implicit**. The technique applies to any puzzle or game: a vertex is a configuration (the arrangement of a sliding puzzle, the state of a Rubik's cube), an edge is a legal move, and BFS finds the solution with the fewest moves. The graph of all configurations may be astronomically large. The search builds only the part it visits.

9.3 Depth-first search

Breadth-first search is cautious: it explores everything nearby before venturing further. **Depth-first search**, or DFS, is the opposite. It follows a path as far as it can go, and only when it reaches a dead end does it back up to the most recent vertex with an unexplored neighbor and try another way. It is how you would explore a maze on foot, with chalk to mark where you had been.

"Back up to the most recent" is the behavior of a stack, and the simplest way to get one is to use the call stack, by writing the search recursively.

code/ch09/algorithms.hpp

```
// Visit everything reachable from u that has not been visited, depth first.
inline void dfs(const Graph& g, int u, std::vector<bool>& visited,
↪   std::vector<int>& order) {
    visited[idx(u)] = true;
    order.push_back(u);
    for (const Edge& e : g.neighbors(u)) {
        if (!visited[idx(e.to)]) dfs(g, e.to, visited, order);
    }
}
```

Mark the vertex; then, for each neighbor not yet visited, search from *there* before looking at the next neighbor. When the recursive call returns, everything reachable through that neighbor has been visited.

Like BFS, DFS visits each vertex once and examines each edge once: $\Theta(V + E)$. It does not find shortest paths. What it offers instead is that it finishes exploring everything below a vertex before it leaves that vertex, and several problems are naturally solved by that structure.

> **CHECK YOURSELF 9.2**
>
> Compare `bfs` and `dfs` with the `level_order` and `pre_order` tree traversals of Section 6.2. What corresponds to what? What did the tree traversals not need that the graph searches do?

Connected components

A search from one vertex of an undirected graph visits exactly the vertices in that vertex's connected component. To find all the components, start a search from each vertex that no earlier search has reached.

code/ch09/algorithms.hpp

```
// Number of connected components of an undirected graph.
inline int count_components(const Graph& g) {
    std::vector<bool> visited(idx(g.size()), false);
    std::vector<int> order;
    int components = 0;
    for (int v = 0; v < g.size(); ++v) {
        if (!visited[idx(v)]) {      // v is in a component we have not explored
            ++components;
            dfs(g, v, visited, order);
        }
    }
    return components;
}
```

The outer loop looks as though it might make this quadratic, but it does not: the visited marks persist across the searches, so each vertex is still visited once and each edge examined once in total.

Ordering tasks: topological sort

Suppose the vertices are tasks and a directed edge from u to v means "u must be done before v." Courses and their prerequisites are the classic example. So are the steps of a recipe, the source files of a program that must be compiled before the files that include them, and the cells of a spreadsheet that must be recalculated before the cells that refer to them.

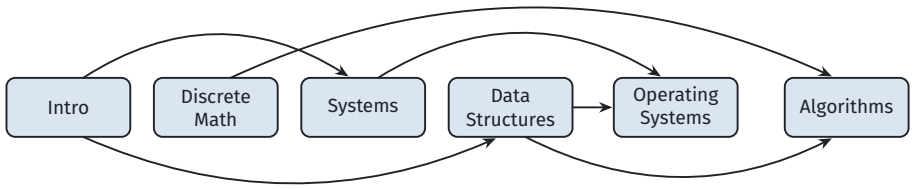


Figure 9.3 A prerequisite graph and one topological order of it. Every edge points from left to right, so taking the courses in this order never violates a prerequisite. There are usually several valid orders.

A **topological order** of a directed graph is a listing of its vertices in which every edge points from an earlier vertex to a later one. Such an order exists exactly when the graph has no directed cycle. (If u must precede v and v must precede u, nothing can be done.) A directed graph with no cycles is called a **directed acyclic graph**, or DAG, and DAGs are everywhere that dependencies are.

One algorithm follows the way you would actually plan your courses. Find a course with no prerequisites, and take it. That frees up any course for which it was the last unmet prerequisite. Repeat.

code/ch09/algorithms.hpp

```
// An ordering of a directed graph's vertices in which every edge points
// forward. Returns an empty vector if the graph has a cycle (no such order).
inline std::vector<int> topological_order(const Graph& g) {
    std::vector<int> indegree(idx(g.size()), 0);
    for (int u = 0; u < g.size(); ++u)
        for (const Edge& e : g.neighbors(u)) ++indegree[idx(e.to)];

    std::queue<int> ready;                       // vertices with no unmet
    ↪ prerequisites
    for (int v = 0; v < g.size(); ++v)
        if (indegree[idx(v)] == 0) ready.push(v);

    std::vector<int> order;
    while (!ready.empty()) {
        int u = ready.front();
        ready.pop();
        order.push_back(u);
        for (const Edge& e : g.neighbors(u)) {   // u is done: release its
        ↪ successors
            if (--indegree[idx(e.to)] == 0) ready.push(e.to);
        }
    }
    if (order.size() != idx(g.size())) order.clear();   // something never
    ↪ became ready
    return order;
}
```

The algorithm first counts, for each vertex, how many edges point into it: its **in-degree**. The vertices with in-degree zero are ready. Each time a vertex is output, the in-degrees of its successors are decremented, and any that reach zero become ready. If the graph has a cycle, the vertices on the cycle never become ready, the queue runs dry early, and the function reports failure by returning an empty order. So this algorithm also *detects* cycles. Its cost is again $\Theta(V + E)$.

code/ch09/graph_test.cpp

```
// Edges point from a prerequisite to the course that requires it.
enum { Intro, Systems, DataStructures, DiscreteMath, OperatingSystems,
↪ Algorithms };
Graph prereq(6);
prereq.add_edge(Intro, Systems);
prereq.add_edge(Intro, DataStructures);
prereq.add_edge(Systems, OperatingSystems);
prereq.add_edge(DataStructures, OperatingSystems);
prereq.add_edge(DataStructures, Algorithms);
prereq.add_edge(DiscreteMath, Algorithms);
std::vector<int> plan = topological_order(prereq);
```

> **IN THE WILD**
>
> Topological ordering runs constantly in the software you use. Build tools such as `make` and package managers use it to decide what to compile or install first. Spreadsheets use it to recalculate cells. And it is at the center of how neural networks are trained: the arithmetic that a model performs is recorded as a DAG of operations, and the training algorithm, backpropagation, computes its results by walking that graph in *reverse* topological order, so that each operation's contribution is known before it is needed by the operations that fed into it.

9.4 Shortest paths with weights

Breadth-first search finds the path with the fewest edges. On a road map that is the route with the fewest road segments, which is not what anyone wants. When edges have weights, the length of a path is the *sum* of its weights, and the shortest path may well use more edges.

The classic algorithm for this was devised by Edsger Dijkstra in 1956, by his own account in about twenty minutes at a café table [19]. It works whenever **no edge has a negative weight**.

The idea is to generalize BFS's waves. BFS processes vertices in order of their distance from the source, and that worked because the next-closest vertex was always at the front of a queue. With weights, the vertices still need to be processed in order of distance, but the next-closest one is no longer simply the oldest. We need a structure that hands over the *closest* unprocessed vertex each time: a priority queue.

Keep, for every vertex, the length of the shortest path to it found *so far*, initially infinity. Repeatedly take the unprocessed vertex with the smallest such distance. That distance is now final; we say the vertex is **settled**. Then look at each edge leaving it: if going through this vertex and along that edge gives a shorter route to the neighbor than any found before, record the improvement. This update is called **relaxing** the edge.

code/ch09/algorithms.hpp

```
const double INF = std::numeric_limits<double>::infinity();

// Shortest-path distances from source. All edge weights must be >= 0.
inline std::vector<double> dijkstra(const Graph& g, int source) {
    std::vector<double> dist(idx(g.size()), INF);
    using Item = std::pair<double, int>;                      // (distance,
    ↪ vertex)
    std::priority_queue<Item, std::vector<Item>, std::greater<Item>> pq;   //
    ↪ min-heap

    dist[idx(source)] = 0.0;
    pq.push({0.0, source});

    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();
```

```
        if (d > dist[idx(u)]) continue;          // a stale entry: u was already
         ↪  settled
        for (const Edge& e : g.neighbors(u)) {
            double candidate = d + e.weight;
            if (candidate < dist[idx(e.to)]) {   // found a shorter way to e.to
                dist[idx(e.to)] = candidate;
                pq.push({candidate, e.to});
            }
        }
    }
    return dist;
}
```

The priority queue holds pairs of (distance, vertex), ordered by distance, smallest first. (Recall from Section 8.1 that `std::priority_queue` needs `std::greater` to behave as a min-heap.)

One practical detail deserves a comment. When a shorter route to a vertex is found, the ideal would be to lower that vertex's priority where it sits in the heap. The standard priority queue has no such operation. So the code simply pushes a *second* entry for the vertex, with the better distance, and leaves the old one in place. The better entry will come out first. When the outdated one eventually surfaces, it is recognized, because its distance is greater than the one now on record, and skipped. That is the meaning of the line marked "a stale entry." This trick is known as *lazy deletion*, and it is the usual way to write Dijkstra's algorithm in practice.

code/ch09/graph_test.cpp

```
Graph roads(5);
roads.add_undirected(0, 1, 4);
roads.add_undirected(0, 2, 1);
roads.add_undirected(2, 1, 2);
roads.add_undirected(1, 3, 5);
roads.add_undirected(2, 3, 8);
roads.add_undirected(3, 4, 3);
std::vector<double> d = dijkstra(roads, 0);
```

Here is the run on that graph, step by step.

Step	Popped	Action	Distances to 0, 1, 2, 3, 4
		start	0, ∞, ∞, ∞, ∞
1	(0, vertex 0)	relax 0→1 to 4; relax 0→2 to 1	0, 4, 1, ∞, ∞
2	(1, vertex 2)	relax 2→1 to 3; relax 2→3 to 9	0, 3, 1, 9, ∞
3	(3, vertex 1)	relax 1→3 to 8	0, 3, 1, 8, ∞
4	(4, vertex 1)	stale: skip	
5	(8, vertex 3)	relax 3→4 to 11	0, 3, 1, 8, 11
6	(9, vertex 3)	stale: skip	
7	(11, vertex 4)	nothing to improve	0, 3, 1, 8, 11

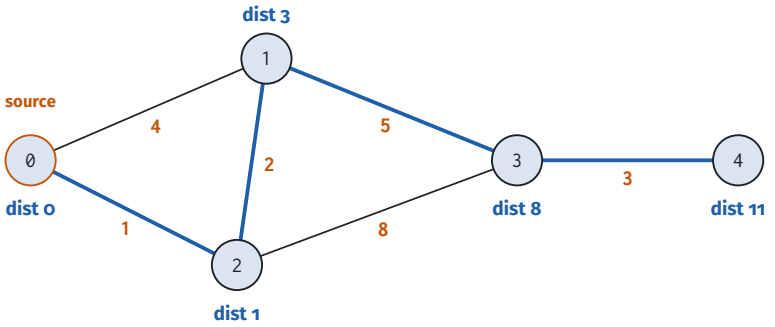


Figure 9.4 Dijkstra's algorithm on a small road network, complete. Each vertex shows its final distance from vertex 0, and the heavy edges form the tree of shortest paths. Note that the best route from 0 to 1 goes through 2, with total weight 3, and not along the direct edge of weight 4. In the web edition, step through breadth-first search, depth-first search, and Dijkstra's algorithm on this and other graphs, and watch the queue, stack, or priority queue as it changes. [Interactive in the web edition.]

Why it works, and when it does not

The crucial claim is that when a vertex is taken from the priority queue (and is not stale), its recorded distance is truly the shortest. Why can no shorter route turn up later?

Suppose vertex u is popped with distance d. Any other route to u would have to leave the set of already-settled vertices at some point, passing through some vertex x that is still in the queue. But u was the *closest* thing in the queue, so the distance to x is at least d. The rest of the route, from x on to u, can only add to that. So no route through x is shorter than d.

Look at the last step of that argument: "can only add." It assumes that edge weights are not negative. With a negative edge, a route that wanders further away could come back shorter, and a vertex could be settled too soon.

> **CHECK YOURSELF 9.3**
>
> Consider a directed graph with edges A→B of weight 1, A→C of weight 3, and C→B of weight −5. What distance does Dijkstra's algorithm, as written, settle for B on first popping it? What is the true shortest distance?

What it costs

Every edge can cause at most one push onto the priority queue, so the queue never holds more than E entries, and each push or pop costs $O(\log E)$, which is $O(\log V)$. The total is

$$O\big((V + E)\log V\big).$$

Before heaps were in common use, the algorithm was implemented by scanning an array of all the vertices to find the closest unsettled one, which takes $\Theta(V^2)$ time in total. On sparse graphs the difference is dramatic.

Vertices (5 edges each)	Array scan	Priority queue
2,000	0.005 s	0.0002 s
8,000	0.086 s	0.0011 s
32,000	2.2 s	0.0058 s
128,000		0.027 s

Quadrupling the graph multiplies the array-scan time by about twenty and the heap time by about five. The same algorithm, with a better data structure inside it, goes from unusable to instantaneous. That is the reason this is a course in data structures.

BY HAND FIRST

On paper, run Dijkstra's algorithm on a weighted graph of six or seven vertices that you draw yourself, from a source of your choosing. Keep a table like the one above: what was popped, what was relaxed, the distance array, and the contents of the priority queue after each step, including stale entries. Then run BFS on the same graph, ignoring weights, and compare the two shortest-path trees.

Then check your work by running the code, not by asking a model to "trace Dijkstra on this graph." A language model produces a trace by predicting what such a trace looks like, and over a dozen steps of carefully interdependent state it is quite capable of relaxing an edge that does not exist or losing an entry from the queue, while the prose around the error remains perfectly confident. For anything beyond a toy, the reliable way to get an algorithm's output from a model is to have it write the program and then run the program yourself.

9.5 Union-find

The last structure of the chapter solves a narrower problem than the searches do, and solves it almost unbelievably well.

Suppose a graph is being built up one edge at a time, and at any moment you may be asked: are these two vertices connected yet? This is the **dynamic connectivity** problem. Running a new search for every question would take linear time per question. But we do not need paths. We need only to know which vertices are in the same component, and to be able to merge two components when an edge joins them.

The abstract data type is called **disjoint sets**, or **union-find** after its two operations.

Operation	Meaning
`find(x)`	return a representative element of the set containing `x`
`unite(a, b)`	merge the sets containing `a` and `b`

Two elements are in the same set exactly when `find` returns the same representative for both.

The representation is a forest: each set is a tree, in which every element points to a **parent**, and the root of the tree, which is its own parent, is the set's representative. These

trees are unusual in two ways. The pointers go *up*, from child to parent, since the only thing we ever do is climb to the root. And the trees do not need nodes at all. Since elements are numbered 0 through $n-1$, the entire forest is one array: `parent[x]` is the parent of `x`.

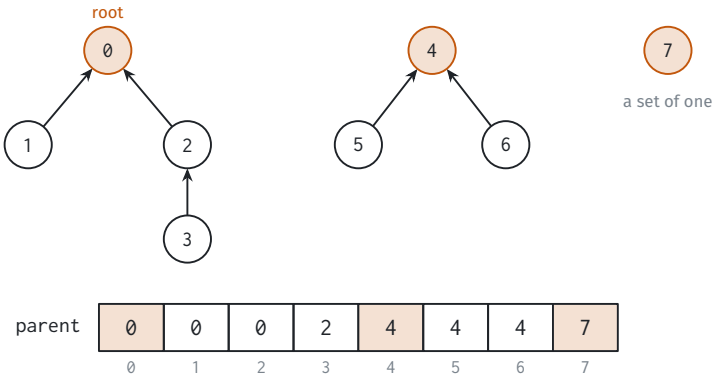


Figure 9.5 A union-find structure over eight elements, shown as a forest and as the `parent` array that actually stores it. There are three sets, with representatives 0, 4, and 7. In the web edition, unite sets and run `find`, and watch path compression flatten the trees. [Interactive in the web edition.]

`find` climbs parent links to the root. `unite` finds the two roots and makes one the parent of the other. That is the whole idea, and in its naive form it has the same flaw as the binary search tree: nothing stops the trees from growing tall and thin, in which case `find` takes linear time. Two small refinements fix it.

Union by size. When merging two trees, always attach the *smaller* one beneath the root of the larger. An element's depth increases only when its tree is absorbed into one at least as big, so each time an element gets one level deeper, the size of its tree has at least doubled. That can happen at most $\log_2 n$ times. The trees have logarithmic height.

Path compression. After `find` has climbed from `x` to the root, it goes back over the path and points every element on it *directly* at the root. The next `find` for any of them takes one step. The tree gets flatter every time it is used.

code/ch09/union_find.hpp

```
class UnionFind {
public:
    // n elements, 0 .. n-1, each initially in a set by itself.
    explicit UnionFind(int n) : parent_(static_cast<std::size_t>(n)),
                                size_(static_cast<std::size_t>(n), 1), sets_(n) {
        std::iota(parent_.begin(), parent_.end(), 0);   // parent_[i] = i
    }

    // The representative ("root") of the set containing x.
    int find(int x) {
        int root = x;
        while (parent(root) != root) root = parent(root);   // climb to the root
        while (parent(x) != root) {                          // then point every
        ↪  node
            int next = parent(x);                          // on the path straight
            parent(x) = root;                                  // at it
```

```
            x = next;
        }
        return root;
    }

    bool same(int a, int b) { return find(a) == find(b); }

    // Merge the sets containing a and b. Returns false if already together.
    bool unite(int a, int b) {
        int ra = find(a), rb = find(b);
        if (ra == rb) return false;
        if (size(ra) < size(rb)) std::swap(ra, rb);   // ra is the larger tree
        parent(rb) = ra;                              // hang the smaller under it
        size(ra) += size(rb);
        --sets_;
        return true;
    }

    int sets() const { return sets_; }

private:
    int& parent(int x) { return parent_[static_cast<std::size_t>(x)]; }
    int& size(int x) { return size_[static_cast<std::size_t>(x)]; }

    std::vector<int> parent_;   // parent_[x] == x means x is a root
    std::vector<int> size_;     // for a root: how many elements in its tree
    int sets_;
};
```

With both refinements, the cost of a sequence of operations was determined by Robert Tarjan in 1975, in one of the most celebrated analyses in the field [58]. The amortized cost per operation is $O(\alpha(n))$, where α is the *inverse Ackermann function*, a function that grows so slowly that $\alpha(n) \le 4$ for any n up to far beyond the number of atoms in the universe. It is not constant. But no computation that could ever be carried out will see it exceed four.

Here is the difference the two refinements make. The test unites n elements into one long chain, the worst case for the naive version, and then calls `find` on each.

n	naive	union by size + path compression
10,000	0.078 s	0.00006 s
20,000	0.28 s	0.00011 s
40,000	1.1 s	0.00021 s
80,000	4.4 s	0.00046 s

The naive version is quadratic. The refined one is, to the eye, linear: about ten lines of code separate a structure that chokes on eighty thousand elements from one that handles twenty million operations on ten million elements at around 36 nanoseconds apiece.

An application: the cheapest network

Suppose the vertices are buildings, and each weighted edge is the cost of laying cable between two of them. We want to connect every building, directly or indirectly, as cheaply as possible. The solution is a set of edges that connects all the vertices with no cycles (a cycle would mean a redundant cable) and with the smallest possible total weight: a **minimum spanning tree**.

Kruskal's algorithm [35] finds one with a strategy of pure greed. Consider the edges from cheapest to most expensive. Take each edge unless its two endpoints are already connected by edges taken earlier, in which case it would close a cycle, so skip it.

code/ch09/graph_test.cpp

```
struct WeightedEdge {
    int u, v;
    double weight;
};

// Total weight of a minimum spanning tree of a connected undirected graph.
double mst_weight(int vertices, std::vector<WeightedEdge> edges) {
    std::ranges::sort(edges, {}, &WeightedEdge::weight);   // cheapest first
    UnionFind uf(vertices);
    double total = 0.0;
    for (const WeightedEdge& e : edges) {
        if (uf.unite(e.u, e.v)) total += e.weight;   // joins two components:
        ↪  keep it
    }                                                // else it would close a cycle
    return total;
}
```

"Are these two vertices already connected?" is exactly the question union-find answers, and "now they are" is `unite`. With it, the algorithm is a sort followed by a single pass: $O(E \log E)$ for the sort, and effectively linear for everything else. It is not obvious that so greedy a strategy gives the best possible answer. It does, and the proof belongs to an algorithms course.

ASK AN LLM

With graphs, the algorithms are the easy part. The hard part, and the part that makes someone good at this, is *modeling*: seeing that a problem which never mentions graphs is one. Use a model to practice.

> *Give me a short word problem, with no mention of graphs or of any algorithm by name, that is best solved by exactly one of: breadth-first search, topological sort, Dijkstra's algorithm, or union-find. I will tell you what the vertices are, what the edges are, whether they are directed or weighted, and which algorithm applies. Then critique my modeling, especially if there is a better choice of vertices.*

Do ten of these. Pay particular attention to problems where a vertex is not a physical thing but a *state*: a configuration of a puzzle, a position plus the amount of fuel remaining, a set of tasks already completed.

Here are the chapter's algorithms side by side.

Algorithm	Structure at its heart	Cost	Answers
Breadth-first search	queue	$\Theta(V + E)$	fewest-edge paths; what is reachable
Depth-first search	stack (recursion)	$\Theta(V + E)$	reachability; components; cycles
Topological sort	queue, in-degree counts	$\Theta(V + E)$	a valid order for dependencies
Dijkstra's algorithm	priority queue	$O((V + E) \log V)$	least-weight paths, weights ≥ 0
Union-find	forest in an array	nearly $O(1)$ per operation	are two things connected yet?
Kruskal's algorithm	sort, then union-find	$O(E \log E)$	minimum spanning tree

THE FOUR QUESTIONS **Graph (adjacency lists)**

Where does it live? A vector with one entry per vertex, each entry a vector of that vertex's outgoing edges. Vertices are integers, so they index directly. Space is proportional to vertices plus edges.

What does it promise? `neighbors(u)` lists exactly the edges leaving `u`. An undirected edge is stored as two directed ones, and the two must be kept consistent.

What does it cost? Visiting a vertex's neighbors takes time proportional to their number. Any algorithm that looks at the whole graph, such as BFS or DFS, takes $\Theta(V + E)$.

How do you know? Test each algorithm against a slower, simpler, independent one on many small random graphs: Dijkstra against Bellman–Ford, BFS against Dijkstra with unit weights, Kruskal against a different spanning-tree algorithm. Include disconnected graphs, self-loops, and repeated edges.

THE FOUR QUESTIONS **Union-find**

Where does it live? Two arrays of integers. `parent[x]` is the element above `x` in its tree; a root is its own parent. A forest with no nodes and no pointers.

What does it promise? Two elements are in the same set exactly when climbing from each reaches the same root. With union by size, a tree of height h has at least 2^{h-1} elements.

What does it cost? Amortized $O(\alpha(n))$ per operation, where $\alpha(n) \leq 4$ in practice. It cannot split a set or list a set's members efficiently.

How do you know? Compare its answers with connectivity computed by a graph search over the same edges; check that the number of sets equals the number of connected components.

Exercises

1. Draw the directed graph with vertices 0 through 5 and edges 0→1, 0→3, 1→2, 1→3, 3→4, 4→1, 5→4. Give its adjacency lists and its adjacency matrix. Does it contain a cycle? Which vertices are reachable from 5? From 2?
2. For each scenario, say what the vertices and edges are, whether the graph is directed and whether it is weighted, and whether you would expect it to be sparse or dense: (a) flights between airports; (b) which pairs of students have a class together; (c) the "follows" relation on a social network; (d) the squares of a chessboard and a knight's moves; (e) the tiles of a sliding puzzle.
3. Run breadth-first search by hand on the graph of Figure 9.1 starting from vertex 5. Give the order in which vertices are removed from the queue, and the final `dist` and `parent` arrays. Then give the order in which `dfs` visits vertices starting from 5.
4. A graph has V vertices and is stored as an adjacency matrix. What is the running time of BFS? Explain why it differs from the $\Theta(V + E)$ of the adjacency-list version.
5. Rewrite `dfs` without recursion, using an explicit `std::stack<int>`. Does your version visit vertices in the same order as the recursive one? If not, what would you change to make it do so? Why might an iterative version be necessary for a very large graph?
6. An undirected graph is **bipartite** if its vertices can be colored with two colors so that every edge joins vertices of different colors. Write a function that determines whether a graph is bipartite, using BFS. (What must be true of the distances of two adjacent vertices?)
7. Give all the valid topological orders of the course graph in Section 9.3. Then add the smallest set of edges you can that leaves exactly one valid order.
8. Run Dijkstra's algorithm by hand on the `roads` graph from the source vertex 4, with a table like the one in Section 9.4. Which entries in the priority queue turn out to be stale?
9. Modify `dijkstra` to return a `parent` vector as `bfs` does, so that the shortest paths themselves can be recovered. At exactly which line should a vertex's parent be recorded?
10. Suppose every edge of a weighted graph has its weight increased by the same constant c. Does the shortest path between two given vertices stay the same? Give a proof or a small counterexample. (This is why "just add a constant to make all the weights positive" does not rescue Dijkstra's algorithm from negative edges.)
11. Start with elements 0 through 7 in separate sets, and perform `unite(0,1)`, `unite(2,3)`, `unite(4,5)`, `unite(6,7)`, `unite(0,2)`, `unite(4,6)`, `unite(0,4)`, using union by size, with ties broken as in the code. Draw the forest after each step. Then perform `find(7)` with path compression and draw the result.
12. Prove that with union by size (and no path compression), a tree of height h contains at least 2^{h-1} elements. Conclude that `find` takes $O(\log n)$ time.
13. An image is a grid of pixels, each black or white. Two black pixels belong to the same *blob* if they are adjacent horizontally or vertically. Write a function that counts the blobs, once using a graph search and once using union-find. Which would you use

if pixels were being turned black one at a time and you needed the count after each change?

14. **With a model, as critic.** A classic puzzle: you have a 3-liter jug and a 5-liter jug, and you must measure exactly 4 liters. Model it as a graph yourself, on paper: what is a vertex, what is an edge, and what does a shortest path mean? Then ask a model to critique your modeling without solving the puzzle. Finally, write a BFS over your implicit graph that prints the shortest sequence of moves.
15. **Lab.** *Word ladders.* Given a dictionary and two words of the same length, find a shortest sequence of dictionary words leading from one to the other in which each word differs from the previous one in exactly one letter (`cold`, `cord`, `card`, `ward`, `warm`). The graph is implicit: a word's neighbors are the dictionary words one letter away. Use a hash set for the dictionary and BFS for the search. Consider two ways to generate a word's neighbors, by comparing it with every dictionary word, or by trying every single-letter change and looking each up in the set, and measure both. Report the ladder from `stone` to `money`, or the fact that none exists in your dictionary, and the number of words your search discovered along the way.

Chapter notes

Graph theory is usually dated to Euler's 1736 solution of the Königsberg bridges problem. The algorithms in this chapter are from the first two decades of computer science: Dijkstra's shortest-path algorithm [19], Kahn's topological sort [30], Kruskal's spanning-tree algorithm [35], and the systematic use of depth-first search by Hopcroft and Tarjan [29], which earned them a Turing Award. The forest representation of disjoint sets is from Galler and Fischer [25]; the two refinements followed within a few years, and the remarkable analysis of the combination is Tarjan's [58].

Sedgewick and Wayne [48] devote a quarter of their book to graphs, with many applications and an outstanding treatment of union-find as an opening case study. Cormen et al. [16] give the correctness proofs omitted here, including those for Dijkstra's and Kruskal's algorithms, and the algorithms for graphs with negative weights.

Answers to check-yourself questions

9.1 Earliest discovered first. You want to finish with everyone one introduction away before moving on to anyone two away, or you might reach Farid by a roundabout route and report too large a number. First discovered, first processed is a queue. The procedure is called breadth-first search, and it is the first algorithm of this chapter.

9.2 Level-order is BFS: a queue of discovered-but-unprocessed nodes. Pre-order is DFS: recursion, going deep first. A tree is a graph with no cycles and exactly one path to each node, so a tree traversal can never arrive at a node twice. A graph search can, which is why it needs the visited marks. A tree traversal is a graph search that gets to skip the bookkeeping.

9.3 Dijkstra's algorithm pops B first, with distance 1, since it is the closest. The true shortest path is A→C→B, with length $3 + (-5) = -2$. With this lazy-deletion implementation the later relaxation would push a new entry for B and the final number would happen to come out right in this tiny case, but anything computed *from* B in the meantime would have used the wrong value, and in

general the algorithm's guarantee is gone. Graphs with negative edges need a different algorithm (Bellman–Ford), which you will meet in an algorithms course.

CHAPTER 10

Data Structures in the Wild

WEEK 10 OF 10

THIS WEEK

By the end of this week you should be able to:

- choose a data structure for a problem by asking what operations it needs and what each candidate promises and costs;
- combine structures, so that each covers for another's weakness, as in an LRU cache;
- implement a trie, and explain when it beats a hash table or a search tree;
- recognize the structures of this course inside real systems: a tokenizer, a database, a version-control system, a vector search engine;
- say what you would check before trusting a data structure you did not write.

Nine weeks ago this book claimed that a data structure is a decision about where things live, together with a set of promises about how you can get at them. You now have a dozen such decisions in hand. This last chapter is about using them: choosing among them, combining them, and recognizing them in the software you depend on every day, including the software that may have helped you study for this course.

CHECK YOURSELF 10.1

A music service keeps, for each user, the 100 most recently played songs. Playing a song that is already on the list moves it to the top. Playing a new one adds it at the top and, if the list is full, drops the song at the bottom. Both must be fast. Which *single* structure from this book supports "is this song on the list, and where?" and "move this entry to the top" and "drop the bottom entry," all in constant time?

10.1 Choosing

When you face a problem that needs a container, resist the urge to pick the one you used last. Ask, in order:

1. **What operations do I need?** Write them down. Lookup by key? By position? Insertion where: at the end, anywhere, in sorted order? Removal? The smallest element? Everything in a range? Iteration in some order?
2. **Which of them are frequent?** A structure that makes the common operation fast and the rare one slow is usually the right trade.
3. **How big will it get?** For twenty elements, nothing matters; use a vector. For twenty million, everything matters.

4. **What guarantees do I need?** Is an average-case bound acceptable, or must every operation be fast? Could an adversary choose the input? Must pointers to elements stay valid while the container changes?

Then consult the table. It summarizes the whole book.

If you need…	Reach for	Costs to remember
a sequence; access by position; append	`std::vector`	$\Theta(1)$ index and amortized append; $\Theta(n)$ insert in the middle
last in, first out	`std::stack`, or a vector	$\Theta(1)$
first in, first out	`std::queue`, or a ring buffer	$\Theta(1)$
both ends, plus indexing	`std::deque`	$\Theta(1)$ at either end
lookup by key, order irrelevant	`std::unordered_map` / `_set`	$\Theta(1)$ expected; $\Theta(n)$ worst
lookup by key, *and* order, ranges, nearest key	`std::map` / `std::set`	$\Theta(\log n)$ worst case
build once, then look up many times	sorted `std::vector` + binary search	$\Theta(\log n)$; smallest and most cache-friendly
repeatedly take the smallest (or largest)	`std::priority_queue`	$\Theta(\log n)$ push and pop
membership only, in very little memory, errors tolerable	Bloom filter	$\Theta(k)$; false positives
"are these two connected yet?"	union-find	nearly $\Theta(1)$
relationships between pairs of things	adjacency lists	$\Theta(V + E)$ to traverse
splice or remove in $O(1)$ given the node; stable addresses	`std::list`	$\Theta(n)$ to find anything
keys that are strings, with prefix queries	trie	proportional to key length

Three pieces of advice go with the table.

Start with a vector. It is the right answer more often than everything else combined, and when it is the wrong answer, it is usually an adequate one until you have measurements.

Then a hash table. Most problems that a vector does not solve are lookup problems, and for those the hash table is the default. Move to a tree when you find you need order.

Measure before you get clever. You have seen, repeatedly, that the structure with the better Big-O is not always the faster one, and that the difference between two structures with the same Big-O can be a factor of ten. Intuition about performance is unreliable, yours and everyone else's. The stopwatch is not.

ASK AN LLM

A model is a useful partner for design decisions if you ask it to argue and not to pronounce.

> *I need a data structure for the following workload: [describe the operations, how often each happens, and the expected size]. Propose two different designs. For each, give the cost of every operation, the memory overhead, and the strongest argument* against *it. Do not recommend*

> *one.*
>
> The instruction not to recommend matters. Asked for "the best data structure," a model tends to give a single confident answer, and you learn nothing about the alternatives. Asked for the trade-offs, it gives you what you need to decide for yourself, and then you measure.

10.2 Combining structures: an LRU cache

A **cache** keeps the results of expensive operations (a web page fetched over the network, a block read from disk, a database query) so that a repeated request can be answered from memory. Memory is finite, so a cache has a capacity, and when it is full something must be thrown out to make room. A good and very common policy is to evict the entry that was **least recently used**, on the theory that what has not been needed for the longest time is least likely to be needed soon. This is an **LRU cache**, and it is the music-service problem from the start of the chapter.

It needs two things done in constant time.

- *Find an entry by key.* That is a job for a hash table.
- *Keep the entries in order of last use*, moving any entry to the front when it is used and removing the one at the back when space is needed. That is a job for a doubly linked list, provided we can get to an entry's node without searching.

So use both, and connect them. The list holds the entries, in order of use. The hash table maps each key not to the value but to the entry's *position in the list*.

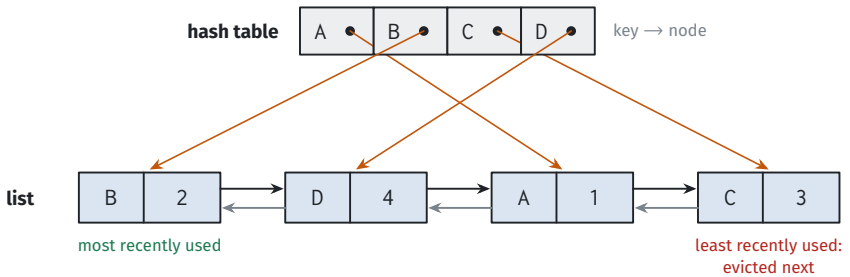


Figure 10.1 An LRU cache of capacity four. The list holds the entries, most recently used first. The hash table maps each key to that key's node in the list, so an entry can be found in constant time and then moved to the front, or removed, in constant time.

code/ch10/lru.hpp

```
template <typename K, typename V>
class LruCache {
    using Entry = std::pair<K, V>;
    using Position = typename std::list<Entry>::iterator;

    std::size_t capacity_;
    std::list<Entry> order_;                // most recently used at the front
```

```
    std::unordered_map<K, Position> where_;  // key -> its node in order_

public:
    explicit LruCache(std::size_t capacity) : capacity_(capacity) {
        assert(capacity > 0);
    }

    // A member-by-member copy would leave the copy's where_ holding
    // iterators into the ORIGINAL's order_. Forbid copying; moving is
    // fine, because a moved list keeps its nodes and so its iterators.
    LruCache(const LruCache&) = delete;
    LruCache& operator=(const LruCache&) = delete;
    LruCache(LruCache&&) = default;
    LruCache& operator=(LruCache&&) = default;

    std::size_t size() const { return order_.size(); }

    // Returns a pointer to key's value, or nullptr if it is not cached.
    // A successful lookup counts as a use.
    V* get(const K& key) {
        auto found = where_.find(key);
        if (found == where_.end()) return nullptr;
        // Move the entry's node to the front of the list. splice relinks
        // the node in O(1); nothing is copied, and the iterator stored
        // in where_ remains valid.
        order_.splice(order_.begin(), order_, found->second);
        return &found->second->second;
    }

    void put(const K& key, V value) {
        if (V* existing = get(key)) {        // already cached: update it
            *existing = std::move(value);
            return;
        }
       if (order_.size() == capacity_) {    // full: evict the entry at the back
            where_.erase(order_.back().first);
            order_.pop_back();
        }
        order_.emplace_front(key, std::move(value));
        where_[key] = order_.begin();
    }
};
```

code/ch10/wild_test.cpp

```
LruCache<std::string, int> cache(2);
cache.put("a", 1);
cache.put("b", 2);
cache.get("a");            // "a" is now the most recently used
cache.put("c", 3);         // full: evicts "b", the least recently used
assert(cache.get("b") == nullptr);
assert(*cache.get("a") == 1 && *cache.get("c") == 3);
```

Every operation is a hash lookup followed by a constant amount of pointer manipulation: $\Theta(1)$ expected.

This is, at last, a job that a linked list does better than anything else, and it is worth being precise about why. In Section 4.5 the list lost to the vector because every operation began with a search, and searching a list is slow. Here the hash table does the searching. The list is asked to do only what it is good at: relink a node, given the node. And the design depends on a promise that only a linked structure makes: a list iterator stays valid when *other* nodes are inserted, removed, or moved. The hash table can hold on to those iterators indefinitely. Positions in a vector would be invalidated by the very first eviction.

The class is built entirely from standard parts and needs no destructor: its members, a `std::list` and a `std::unordered_map`, clean up after themselves. Copying is another matter, and it is a useful warning about the rule of zero. A member-by-member copy would duplicate the list and duplicate the map, but the copied map would hold iterators that still point into the *original's* list. Each member knows how to copy itself; neither knows about the other. So the class declares its copy operations deleted, which makes copying a compile-time error instead of a use-after-free waiting to happen. Whenever one member of a class refers into another, the compiler-generated copy is wrong.

The general lesson is that real problems are often solved by a small *system* of data structures, each compensating for another's weakness: a hash table for finding, with a list for order; a heap for priority, with a hash table for locating entries in it; a sorted array for searching, with a small buffer in front for cheap insertions. When no single structure fits your operations, ask whether two would.

> **IN THE WILD**
>
> LRU caches are everywhere there is something slow to avoid: in operating systems (which pages of memory to keep), in databases (which disk blocks), in web browsers and content delivery networks (which responses), and in processors themselves. The hash-map-plus-linked-list design shown here is the standard one; Python's `functools.lru_cache` and Java's `LinkedHashMap` are both built this way.

10.3 Tries

Every structure so far has treated a key as an indivisible thing, to be hashed or compared as a whole. When keys are *strings*, there is another possibility: use the characters of the key, one at a time, to steer through a tree.

A **trie** (the name comes from re*trie*val; most people say "try") is a tree in which each edge is labeled with a character, and each node represents the string spelled out by the path from the root to it [23]. To look up a word, start at the root and follow the edge for its first letter, then the edge for its second, and so on. The word is in the set if the path exists and the node where it ends is marked as the end of a word.

code/ch10/trie.hpp

```
struct Node {
    bool is_word = false;                          // does a word end here?
    std::array<std::unique_ptr<Node>, 26> next;    // one child per letter
```

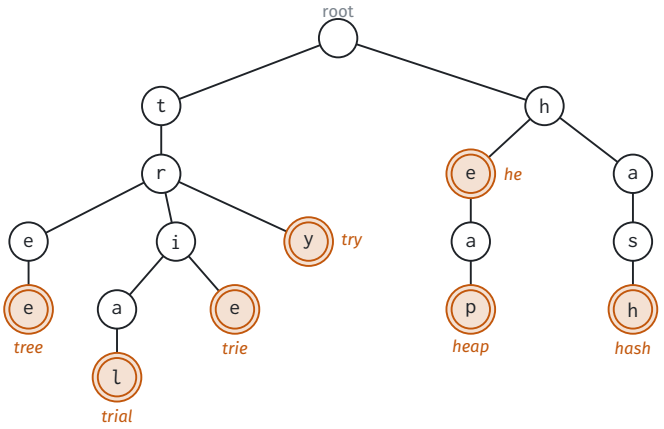


Figure 10.2 A trie holding seven words. Each path from the root spells a prefix; double-ringed nodes mark the ends of words. Words with a common prefix share the nodes for it, so “tree”, “trie”, “trial”, and “try” share the path t–r. In the web edition, insert words and type a prefix to see its completions. [Interactive in the web edition.]

```
};

Node root_;
std::size_t words_ = 0;

static std::size_t slot(char c) {
    assert(c >= 'a' && c <= 'z');
    return static_cast<std::size_t>(c - 'a');
}
```

Each node has an array of 26 child pointers, one per letter, most of them null. A key’s characters are used directly as indices, the direct-addressing idea once more.

code/ch10/trie.hpp

```
// Follow the path spelled by s. Returns the node reached, or nullptr
// if the path leaves the trie.
const Node* walk(const std::string& s) const {
    const Node* n = &root_;
    for (char c : s) {
        n = n->next[slot(c)].get();
        if (n == nullptr) return nullptr;
    }
    return n;
}
```

code/ch10/trie.hpp

```
void insert(const std::string& word) {
    Node* n = &root_;
    for (char c : word) {
        std::unique_ptr<Node>& child = n->next[slot(c)];
```

```
        if (!child) child = std::make_unique<Node>();   // extend the path
        n = child.get();
    }
    if (!n->is_word) {
        n->is_word = true;
        ++words_;
    }
}

bool contains(const std::string& word) const {
    const Node* n = walk(word);
    return n != nullptr && n->is_word;
}

bool has_prefix(const std::string& prefix) const {
    return walk(prefix) != nullptr;
}

// Up to `limit` words that begin with prefix, in alphabetical order.
std::vector<std::string> complete(const std::string& prefix, std::size_t
↪  limit) const {
    std::vector<std::string> out;
    if (const Node* n = walk(prefix)) {
        std::string path = prefix;
        collect(n, path, limit, out);
    }
    return out;
}
```

code/ch10/wild_test.cpp

```
Trie t;
for (const char* w : {"tree", "trie", "try", "trial", "heap", "hash", "he"}) {
    t.insert(w);
}
assert(t.contains("trie") && !t.contains("tri"));   // "tri" is only a prefix
assert(t.has_prefix("tri") && !t.has_prefix("tra"));
assert((t.complete("tr", 10) ==
        std::vector<std::string>{"tree", "trial", "trie", "try"}));
assert((t.complete("he", 10) == std::vector<std::string>{"he", "heap"}));
```

Two properties set tries apart.

The cost depends on the length of the key, not on the number of keys. Looking up a word of length L takes L steps whether the trie holds ten words or ten million. A hash table also takes time proportional to L, to compute the hash; a balanced tree takes $O(L \log n)$, since each of its $\log n$ comparisons may examine up to L characters.

Prefixes are first-class. All the words that begin with a given prefix live in the subtree below that prefix's node. To list them, walk to the node and traverse its subtree. That is `complete` above, and it is how autocompletion works. A hash table cannot do this at all, since hashing scatters words that share a prefix.

The price is memory. Our node holds 26 pointers, 208 bytes, and a trie can have a node for every character of every key. Practical tries economize in various ways: by storing

children in a small sorted array or a hash map, or by collapsing chains of single-child nodes into one node labeled with a whole substring, which gives a *radix tree*.

> **IN THE WILD**
>
> Network routers find the destination of every packet with a trie over the bits of its address, looking for the longest matching prefix. The Linux kernel uses radix trees to index cached file contents. And tries are central to how text is fed to language models, which is the first of our case studies.

10.4 Four case studies

The rest of this chapter looks inside four real systems. Nothing in them is beyond you now. Each is assembled from structures you have built, and in each the interesting part is how the pieces are chosen and combined.

How a language model reads: tokenizers

A language model does not see characters, and it does not see words. It sees **tokens**: integers that index into a fixed vocabulary of tens of thousands to a few hundred thousand text fragments. Before any text reaches the model, a *tokenizer* chops it into fragments from the vocabulary. Common words are single tokens; rarer ones are split into pieces, so that `unbelievably` might become `un`, `believ`, `ably`.

Where does the vocabulary come from? Most current models use a method called **byte-pair encoding**, which began life as a data-compression algorithm [24] and was adapted for language models in 2016 [50]. It is learned from a large body of text by a strikingly simple loop.

1. Start with every word split into single characters. The vocabulary is the set of characters.
2. Count how often each *pair of adjacent tokens* occurs in the whole corpus.
3. Take the most frequent pair, and merge it into a single new token everywhere it occurs. Add that token to the vocabulary.
4. Repeat from step 2, tens of thousands of times.

code/ch10/bpe.cpp

```
using Word = std::vector<std::string>;   // a word as a sequence of tokens

// Count every adjacent pair of tokens across the corpus.
std::map<std::pair<std::string, std::string>, int> count_pairs(
    const std::vector<std::pair<Word, int>>& corpus) {
    std::map<std::pair<std::string, std::string>, int> counts;
    for (const auto& [word, freq] : corpus) {
        for (std::size_t i = 0; i + 1 < word.size(); ++i) {
            counts[{word[i], word[i + 1]}] += freq;
        }
    }
    return counts;
}
```

```
// Replace every occurrence of the pair (a, b) by the single token a+b.
void merge_pair(std::vector<std::pair<Word, int>>& corpus, const std::string&
↪  a,
                const std::string& b) {
    for (auto& [word, freq] : corpus) {
        Word merged;
        for (std::size_t i = 0; i < word.size(); ++i) {
            if (i + 1 < word.size() && word[i] == a && word[i + 1] == b) {
                merged.push_back(a + b);
                ++i;   // skip the second half of the pair
            } else {
                merged.push_back(word[i]);
            }
        }
        word = std::move(merged);
    }
}
```

Here is the program run on a toy corpus of four words: “low” (occurring 5 times), “lower” (2), “newest” (6), and “widest” (3), with _ marking the end of each word.

```
merge 1: e + s -> es   (seen 9 times)
merge 2: es + t -> est   (seen 9 times)
merge 3: est + _ -> est_   (seen 9 times)
merge 4: l + o -> lo   (seen 7 times)
merge 5: lo + w -> low   (seen 7 times)
merge 6: e + w -> ew   (seen 6 times)
merge 7: ew + est_ -> ewest_   (seen 6 times)
merge 8: n + ewest_ -> newest_   (seen 6 times)

the corpus, tokenized:
low _   x5
low e r _   x2
newest_   x6
w i d est_   x3
```

Without being told anything about English, the procedure has discovered that `est` at the end of a word is a unit, and that `low` is one too. Run for fifty thousand merges on billions of words, it produces a vocabulary of common words, word stems, prefixes, and suffixes.

Now look at it as a data structures problem. The pair counts live in a map from pairs to integers: a hash table, in a real implementation. Finding the most frequent pair, over and over, is the job of a priority queue. Our toy version recounts everything after each merge, which is hopelessly slow at scale; production tokenizer trainers update the counts incrementally and keep the pairs in a heap. When a trained byte-pair tokenizer is *used*, it replays its merges on the new text in the order they were learned, which again means looking up pairs in a hash table by their rank. Other families of tokenizer work differently, repeatedly taking the longest vocabulary entry that matches the text at the current position; that is prefix matching, and it is commonly done with a trie. A hash table, a heap, and a trie: three chapters of this book, running every time you send a message to a model.

ASK AN LLM

Here is an experiment in the limits of introspection. Ask a model:

> *How exactly would your tokenizer split the word "antidisestablishmentarianism"? List the tokens.*

Then check the answer with an actual tokenizer or token-counting tool; several model providers publish one. The model's answer is often wrong. A model never sees its own tokenization. It receives a sequence of token numbers and has no direct access to the fragments of text they stand for, so its description of how a word is split is a guess from general knowledge, not an observation. It is a neat illustration of a general rule: a model's account of its own inner workings is a generated text like any other, and deserves the same checking.

How a database finds a row: B+ trees and LSM trees

A database table may hold billions of rows on disk, and a query such as "find the customer with this ID" must not read them all. The table has an **index**: a data structure, also on disk, that maps keys to the locations of rows. For half a century the standard index has been a variant of the B-tree of Section 7.4.

In a **B+ tree**, the interior nodes hold only keys and child pointers, and all the actual entries live in the leaves. That makes the interior nodes as small as possible, so that each holds as many keys as possible and the tree is as shallow as possible. A node is one disk page; with several hundred keys per node, four levels reach billions of rows. The leaves are also linked to their neighbors in a chain, like a linked list, so that a range query, "every order placed in March," finds the first matching leaf by descending the tree and then simply walks along the leaves. The upper levels of the tree are small enough to stay in memory, in an LRU cache of pages. A lookup therefore typically costs a single disk read.

B-trees are superb for reading. Writing is harder on them: inserting rows in random key order modifies pages all over the disk. Many modern storage engines, built for workloads with heavy writing, use a different design called a **log-structured merge tree**, or LSM tree [44]. Its parts will all be familiar.

- New entries go into a sorted structure *in memory*, usually a skip list or a balanced tree. Writing is fast because the only disk activity is appending a record to the end of a log file, kept so that nothing is lost in a crash.
- When the in-memory structure grows large, it is written to disk in one sequential pass as an immutable **sorted file**.
- To look up a key, check the in-memory structure, then the sorted files from newest to oldest, using binary search within each. To avoid reading files that cannot contain the key, each file has a **Bloom filter** (Section 5.5) in memory. Most files are ruled out without a disk read.
- In the background, sorted files are combined into larger sorted files, discarding overwritten entries, by exactly the **merge** step of mergesort (Section 8.2).

A balanced tree, sorted arrays, binary search, Bloom filters, and merging: an LSM tree is practically a review session for this course, and it is what sits beneath many of the largest data stores in the world [32].

How Git remembers: hashing and a graph

Git, the version-control system, stores the entire history of a project, every version of every file, and does so compactly, with a design of real elegance built from two ideas in this book [13].

The first is hashing, used in a new way. Git stores everything as *objects*, and the name of an object is **a hash of its contents**. The contents of a file are stored as an object called a *blob*, under a name computed by a cryptographic hash function from the bytes of the file. This is called **content addressing**, and it has an immediate consequence: two files with identical contents have the same name, and so are stored only once, no matter how many directories or versions they appear in.

A directory is stored as a *tree* object: a list of names, each with the hash of the blob or subtree it refers to. A *commit* object holds the hash of the tree for the project's top-level directory at that moment, plus the hash of the previous commit, its parent.

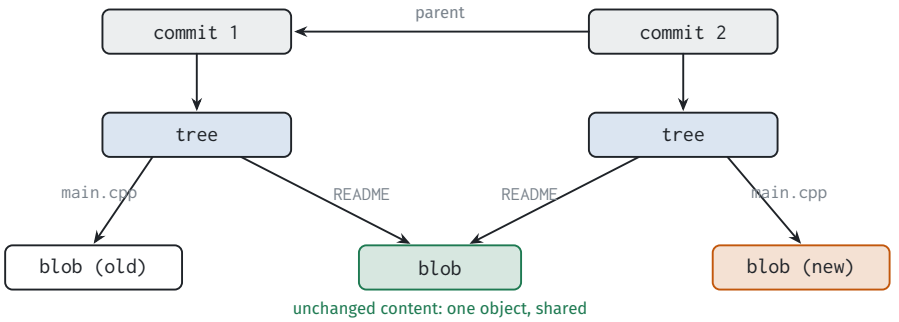


Figure 10.3 Two commits in a Git repository. Each commit points to a tree (a directory), which points to blobs (file contents) by their hashes. The second commit changed only `main.cpp`; its tree points to a new blob for that file and to the *same* blob as before for `README`, which is stored once.

Because every object refers to others by the hash of their contents, a change to one file changes that file's hash, which changes the contents, and so the hash, of the directory that lists it, and so on up to the root. Two versions of a project that differ in one file share every object except those on the path from that file to the top. And two directory trees can be compared quickly: if their hashes are equal, they are identical throughout, and there is no need to look inside. A tree whose nodes are identified by hashes of their contents in this way is called a **Merkle tree** [40].

The second idea is the graph. Each commit points to its parent, and a merge commit has two parents. The history of a project is therefore a directed acyclic graph of commits. A branch is nothing more than a named pointer to one commit. Finding the common ancestor of two branches, listing the history of a file, deciding what must be sent to another machine: all of these are graph traversals of the kind in Chapter 9, over a DAG whose edges are hashes.

How a model looks things up: vector search

Language models are often connected to a store of documents, so that a question can be answered from relevant text and not from the model's memory alone. Finding the

relevant text is a search problem of a new kind. Each passage is converted, by a model, into an **embedding**: a list of perhaps a thousand numbers, a point in a space of a thousand dimensions, arranged so that passages with similar meanings are nearby points. A question is converted the same way, and the task is then: among millions or billions of stored points, find the ones **nearest** to the query point.

None of our structures does this. A hash table finds exact matches; "nearby" is meaningless to it. A search tree needs a one-dimensional order, and a thousand-dimensional space has none that helps. The straightforward method is to compute the distance from the query to every stored point: a linear scan. With the embeddings stored contiguously, that is a cache-friendly sweep and is perfectly good for a few hundred thousand points. For a billion it is far too slow.

The most widely used solution takes two ideas from this book and a third from Section 5.5.

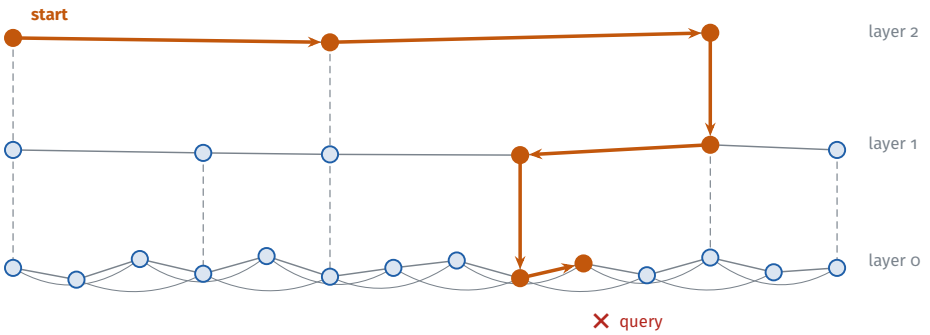


Figure 10.4 The idea behind a layered proximity graph. Every point appears in the bottom layer, linked to some of its near neighbors. A random subset also appears in each layer above, with longer links. A search starts at the top, hops greedily toward the query, drops down a layer whenever it can get no closer, and finishes with a careful search of the dense bottom layer.

The first idea is a **graph**. Make every stored point a vertex, and connect each to a handful of its nearest neighbors. To search, start at some vertex and repeatedly move to whichever neighbor is closest to the query, until no neighbor is closer: a greedy walk through the graph, examining only a few vertices at each step.

The second is the layering of a **skip list** (Section 7.3). A walk through a graph of only short links would take many steps. So a random subset of the points is also placed in a sparser graph "above," with longer links, and a subset of those in a still sparser graph above that. The search begins in the top layer, takes a few long strides toward the right region, and descends layer by layer to the dense bottom layer for the final approach. The lower layers are explored with a best-first search driven by a **priority queue** of candidates. The structure is called a *hierarchical navigable small-world* graph, or HNSW [38].

The third idea is the Bloom filter's: **relax the promise**. An HNSW search does not guarantee to find the true nearest neighbors. It finds points that are *very probably* among the nearest, and it examines a tiny fraction of the data to do so. As with the Bloom filter, the trade is stated explicitly (a measured *recall*, such as 95 percent, at a given speed), and for this application an approximate answer is entirely acceptable.

A graph, a skip list's layers, a priority queue, and an approximate promise. The structure was published in 2016. People are still inventing data structures, and they are inventing them out of the parts in this book.

10.5 What we left out, and what to keep

Ten weeks is not long, and this book made choices. Here, briefly, is some of what lies beyond it, so that you will recognize the names.

Persistent data structures never modify anything; every "update" produces a new version that shares most of its memory with the old one, as Git's trees do. They are the foundation of functional programming and make it trivial to keep old versions around [43].

Concurrent data structures can be used safely by many threads at once. Some use locks; the most intricate, called *lock-free*, use none. This is where linked structures regain ground against arrays, and it is a deep subject.

Probabilistic sketches follow the Bloom filter in trading exactness for space. One counts the number of distinct items in a stream of billions using a few kilobytes; another estimates how often each item occurred.

Spatial structures, such as k-d trees and R-trees, index points and shapes in two or three dimensions, for maps and graphics. **String structures**, such as suffix arrays, index every position of a long text at once, for search and for genomics.

And beyond data structures lies the study of **algorithms** proper: systematic techniques for designing them (greedy methods, dynamic programming), proofs of their correctness, and the theory of which problems can be solved efficiently at all. That is your next course.

The four questions, once more

The particular structures in this book will be joined by others over your career, and some will fade. What is meant to last is the habit of asking four questions of every one.

Where does it live? Contiguous or linked, one block or many, who owns it. You have seen this question decide performance again and again, in ways that no operation count predicted: the vector against the list, probing against chaining, the B-tree against the binary tree, the heap that needed no pointers.

What does it promise? The invariant. Sorted order, heap order, balance, "every key is where its hash says." An invariant is what turns bytes into a data structure, and stating it precisely is most of the work of implementing it. You have also seen structures gain a great deal by promising *less*: a heap that does not sort, a Bloom filter that may err, a search that is only probably right.

What does it cost? Worst case, average case, amortized. In steps, and then on a real machine. And always: on *which* inputs? The plain search tree, the naive quicksort, and the unprotected hash table all taught the same lesson, that the dangerous input is not random but patterned, or hostile.

How do you know? An invariant you can execute. A differential test against something trusted, on thousands of small random cases. A sanitizer under all of it. A deliberate worst-case input. A stopwatch and a doubling experiment. A mutation to check that the tests can fail.

That last question is the one this book has pressed hardest, for a reason given in its first pages. You are entering a profession in which code is abundant. A model will write you a red-black tree, a lock-free queue, or an LSM tree on request, in seconds, and most of the time it will be right. What is scarce, and what you are now equipped to supply, is the judgment to choose the right structure, the precision to say what it must guarantee, and the skill to find out whether the thing in front of you actually does.

BY HAND FIRST

Here is a checklist to keep. Before you rely on a data structure that you did not write yourself, whether it came from a model, a colleague, or the internet:

1. **State its invariant** in a sentence. If you cannot, you do not yet know what it is supposed to do.
2. **Get a checker.** Write a function that verifies the invariant, including any property (balance, load factor) that a performance guarantee rests on, or have one written and read it carefully.
3. **Test it differentially** against a standard container on random operations, with small sizes and a small key range, checking the invariant as you go.
4. **Run it under the sanitizers.**
5. **Feed it the awkward inputs:** empty, one element, all equal, already sorted, reverse sorted, and whatever its particular weakness would be.
6. **Measure its growth** with a doubling experiment, and compare with what was promised.
7. **Read it.** You can, now.

None of the seven steps requires you to have written the code. All of them require you to understand it. That is what this course was for.

Exercises

1. For each workload, choose a structure (or combination) and justify the choice by the costs of the operations involved. (a) A spell checker's dictionary: loaded once, then queried millions of times. (b) The "undo" history of a text editor. (c) A leaderboard showing the top ten scores among a million players, with scores changing constantly. (d) A phone's contact list, searched as the user types a name. (e) Detecting whether any of ten million uploaded files is a duplicate of another. (f) A print server's job list, where urgent jobs may jump ahead.
2. A colleague stores a set of 50 integers in a `std::set` "because lookups are logarithmic." Suggest a simpler structure and predict how the two compare in speed and memory. Then measure.
3. Trace an `LruCache` of capacity 3 through: put A, put B, put C, get A, put D, get B, put E, get A. After each operation give the order of the list, and say which gets fail.
4. Add to `LruCache` a member `bool erase(const K& key)`, and a member that returns the key that would be evicted next without evicting it. Both should be $\Theta(1)$.

5. An **LFU** cache evicts the *least frequently* used entry instead. Explain why keeping the entries in a priority queue ordered by use count makes `get` cost $\Theta(\log n)$, and why a `std::priority_queue` is not sufficient as it stands. (What operation is missing?)
6. Draw the trie containing `car`, `card`, `care`, `cat`, `do`, `dog`, and `dot`. How many nodes does it have? How many would a trie containing only `dog` have?
7. Add `bool erase(const std::string& word)` to `Trie`. A simple version just clears the `is_word` flag. A better version also deletes any nodes that are no longer on the path to any word. Implement the better version, recursively.
8. Our trie node uses 208 bytes for its children. Rewrite `Node` to use a `std::map<char, std::unique_ptr<Node>>` instead. What are the costs of each operation now? Build both versions from a large word list and compare memory use (count nodes and multiply) and lookup time.
9. Run the byte-pair-encoding program by hand on the corpus `aaab` (3 times), `aab` (2 times), and `ab` (4 times), for three merges.
10. Explain how the idea of content addressing lets Git detect that a file has been corrupted on disk. Why must the hash function be *cryptographic* for this to be trustworthy against a deliberate attacker, when the hash functions of Chapter 5 were not?
11. An LSM tree answers a lookup by checking several sorted files, newest first. Suppose there are 20 files and each has a Bloom filter with a 1% false-positive rate. For a key that is in none of the files, what is the expected number of files actually read? What would it be with no Bloom filters?
12. A nearest-neighbor search over one million 1,000-dimensional embeddings, stored as `floats`, is done by linear scan. How much memory do the embeddings occupy? If computing one distance takes 250 nanoseconds, how long does one query take? At what number of queries per second does this stop being acceptable, and what does the approximate method give up to do better?
13. **With a model, as ghostwriter, at last.** Choose a structure this book did not implement: a skip list, a treap, or a red-black tree. Have a model write it, as a set with `insert`, `contains`, and `erase`. Then apply the seven-step checklist from the end of this chapter, in full, and write a one-page report: what you checked, what you found, and whether you would ship it.
14. **Lab.** *A tiny search engine.* Given a directory of text files, build an **inverted index**: a map from each word to the sorted list of documents that contain it. Support queries for a single word, for `A AND B` (intersect two sorted lists, with a merge), and for words beginning with a given prefix (a trie over the vocabulary, or a sorted vector with binary search). Put an LRU cache in front of the query function. Report the structures you chose and why, the time to build the index, and the query time with and without the cache.

Chapter notes

The trie was named by Fredkin [23]; the structure itself had been described a year earlier by René de la Briandais. Byte-pair encoding was introduced as a compression method by

Gage [24] and brought to neural language models by Sennrich, Haddow, and Birch [50], whose paper uses the same four-word example as this chapter. The model architecture that those tokens are fed to is described in the paper that introduced it [60].

For storage engines, B-trees, and LSM trees, Kleppmann's *Designing Data-Intensive Applications* [32] is the best single source, and a pleasure to read; the LSM tree's original paper is O'Neil et al. [44]. The internals of Git are explained in the last chapter of *Pro Git* [13], which is free online; hash trees are from Merkle [40]. The HNSW graph is from Malkov and Yashunin [38]. Okasaki's *Purely Functional Data Structures* [43] is the classic on persistent structures.

Answers to check-yourself questions

10.1 None of them. A hash table finds the song in constant time but has no notion of order, so it cannot say which entry is the oldest. A linked list can move an entry to the front and drop the last one in constant time, *given a pointer to the entry*, but finding the entry takes a linear scan. A vector is linear for the move. Each structure is good at half of the job. The solution, in Section 10.2, is to use two structures at once and have each do the half it is good at.

APPENDIX A

The Toolchain

This appendix is a reference for the mechanics of working in C++: getting a compiler, building programs, and reading what the tools tell you. Skim it in the first week and come back when something breaks.

A.1 Getting a compiler

You need a compiler that supports C++20. Either of the two major open-source compilers will do: GCC (whose C++ compiler is the command `g++`), version 11 or later, or Clang (`clang++`), version 14 or later. The book's code was tested with GCC 13.

Linux. Install your distribution's compiler package. On Debian or Ubuntu: `sudo apt install g++ make`.

macOS. Run `xcode-select --install` in a terminal. This installs Apple's version of Clang, which also answers to the name `g++`. It handles everything in this book, with one caveat about leak detection noted below.

Windows. The smoothest route is the Windows Subsystem for Linux (WSL), which gives you a real Linux environment; then follow the Linux instructions.

Lab machines. Your instructor will tell you what is installed and how to reach it.

Check your installation:

```
$ g++ --version
g++ (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
```

A.2 Building programs

The one command you need:

```
$ g++ -std=c++20 -Wall -Wextra -g -fsanitize=address,undefined prog.cpp -o prog
$ ./prog
```

Flag	What it does	Why
`-std=c++20`	selects the language version	older defaults reject some of the book's code
`-Wall -Wextra`	enables most warnings	warnings are free bug reports; fix them all

Flag	What it does	Why
`-g`	records line numbers in the program	so that sanitizers and debuggers can name source lines
`-fsanitize=address,undefined`	adds run-time error checking	catches memory errors and undefined behavior as they happen
`-o prog`	names the output	otherwise it is called `a.out`

For *timing* a program, use a different set of flags: optimization on, sanitizers off.

```
$ g++ -std=c++20 -O2 prog.cpp -o prog
```

Never time a program built with sanitizers, and never trust the correctness of a program you have only run without them.

Programs in several files. List all the `.cpp` files on the command line. Header files (`.hpp`) are not listed; they are pulled in by `#include`. To tell the compiler where to look for headers, use `-I`:

```
$ g++ -std=c++20 -Wall -Wextra -g -fsanitize=address,undefined -I. main.cpp
↪  util.cpp -o main
```

The book's code. The `code/` directory has a `Makefile` that builds everything.

Command	Effect
`make test`	compiles every program with warnings as errors and both sanitizers, then runs each one
`make bench`	compiles the timing programs with `-O2`
`make bugs`	compiles the deliberately broken programs, so that you can watch them fail
`make clean`	removes everything that was built

Typing long commands repeatedly is tedious and error-prone. For your own assignments, a three-line `Makefile` is enough to start with (the last line, the command, must begin with a tab character, not spaces):

```
CXXFLAGS = -std=c++20 -Wall -Wextra -g -fsanitize=address,undefined
prog: prog.cpp
    g++ $(CXXFLAGS) prog.cpp -o prog
```

A.3 Reading compiler errors

C++ compilers are thorough and not concise. Three habits make their output manageable.

Fix the first error first. One mistake often produces a cascade of consequent errors. Scroll to the top, fix the first, and recompile. Many of the rest will vanish.

Find the line that is yours. An error involving a template can run to dozens of lines, most of them naming files deep inside the standard library. Look for the line that mentions *your* file. That is where the mistake is.

Read the "note:" lines. They often say what the compiler expected.

Message contains	Usually means
`expected ';' before ...`	a missing semicolon, on the line *before* the one named
`'x' was not declared in this scope`	a misspelling, a missing `#include`, or a missing `std::`
`no matching function for call to ...`	wrong number or types of arguments; the notes list the candidates
`invalid conversion from 'const char*' to 'int'`	a value of the wrong type
`passing 'const X' as 'this' argument discards qualifiers`	calling a non-`const` member function on a `const` object; the function probably should be marked `const`
`use of deleted function`	an attempt to copy something that cannot be copied, such as a `std::unique_ptr`; use `std::move`, or pass by reference
`undefined reference to ...`	this is from the *linker*: a function was declared but its definition was never compiled in; a `.cpp` file is missing from the command line
`comparison of integer expressions of different signedness`	(warning) comparing an `int` with a `std::size_t`; make the loop variable a `std::size_t`
`unused variable` / `unused parameter`	(warning) often a sign of a typo or unfinished logic
`control reaches end of non-void function`	(warning) some path through the function forgets to `return` a value; that path is undefined behavior

A.4 Reading sanitizer reports

A report from AddressSanitizer has the same parts every time. Here is one from Chapter 1, abbreviated.

```
==46472==ERROR: AddressSanitizer: heap-buffer-overflow on address
↪  0x50200000001c
READ of size 4 at 0x50200000001c thread T0
    #0 0x62a34ecde731 in main ch01/bugs/oob.cpp:8
    #1 ...
0x50200000001c is located 0 bytes after 12-byte region
                                    [0x502000000010,0x50200000001c)
allocated by thread T0 here:
    #0 ... in operator new(unsigned long)
    ...
```

```
    #7 0x62a34ecde5b3 in main ch01/bugs/oob.cpp:5
```

1. **The first line names the kind of error.** See the table below.
2. **The second says what the program was doing**: reading or writing, and how many bytes.
3. **The first stack trace says where.** Each line is a function call, innermost first. Read down until you reach a line in *your* code: here, `oob.cpp`, line 8.
4. **The next line describes the memory involved**: here, just past the end of a 12-byte block.
5. **Further stack traces say where that memory was allocated**, and for a use-after-free, where it was freed.

Report	Meaning	Typical cause
`heap-buffer-overflow`	access outside a heap block	index out of range; off-by-one in a loop
`stack-buffer-overflow`	access outside a local array	the same, on a local array
`heap-use-after-free`	access to memory already deleted	dangling pointer, reference, or iterator; an invalidated vector iterator
`attempting double-free`	the same block deleted twice	a missing copy constructor (shallow copy); deleting through two pointers
`stack-use-after-scope`, `stack-use-after-return`	access to a local variable after it is gone	keeping a pointer to a variable from an inner block; returning a pointer to a local (as Clang reports it)
`SEGV on unknown address 0x000000000000`	dereferenced a null pointer	forgetting to check for `nullptr`; following `next` off the end of a list; with GCC, using a returned reference to a local (GCC turns the address into null)
`detected memory leaks`	(at exit) blocks never deleted	a missing `delete`, or a destructor that does not free everything
`alloc-dealloc-mismatch`	`new[]` paired with `delete`, or the reverse	use `delete[]` for arrays

UndefinedBehaviorSanitizer prints one line per problem, beginning with the file and line and the words `runtime error`: `signed integer overflow`, `index out of bounds`, `load of null pointer`, `division by zero`.

> **ASIDE**
>
> Leak detection is on by default on Linux. With Apple's compiler on macOS it is not available; the other AddressSanitizer checks work normally. If you develop on a Mac, run your tests on a Linux machine as well before you hand them in.

A.5 When the program is simply wrong

The sanitizers catch errors of *memory*. Errors of *logic*, where the program runs cleanly and gives the wrong answer, need other methods.

Make it small. Find the smallest input on which the program misbehaves. For data structures the bug almost always appears with three or four elements. A differential test with small random inputs (Section 3.6) will often find that input for you; print the sequence of operations that led to the failure.

Assert what you believe. Put `assert` statements wherever you can state something that must be true: a precondition at the top of a function, the class invariant at the end of every member function that modifies the object. The first assertion to fail is close to the bug.

Print. A few well-placed output statements, showing the state of the structure after each operation, are often the fastest route to understanding. Write a `print` or `dump` function for each structure you build. Send debugging output to `std::cerr`, which is not buffered, so that it appears even if the program crashes on the next line.

Use a debugger. A debugger (`gdb` on Linux, `lldb` on macOS) runs your program under your control: you can stop at any line, examine any variable, and step forward one statement at a time. Half a dozen commands cover most needs.

gdb command	Effect
`break file.cpp:42`	stop when execution reaches that line
`run`	start the program
`next` / `step`	execute one line, stepping over / into function calls
`print expr`	show the value of an expression
`backtrace`	show the chain of function calls that led here
`continue`	resume until the next breakpoint

Explain it. Describe, line by line and out loud, what the code is supposed to do: to a classmate, to a rubber duck, or in writing. The act of explaining forces you to look at what the code says and not at what you meant. A language model can play the listener, and is good at it, particularly for compiler errors and sanitizer reports. Ask it what the message means, and do the fixing yourself.

APPENDIX B

A Little Mathematics

The analysis in this book uses a small amount of mathematics, over and over. This appendix collects it in one place.

B.1 Exponents and logarithms

The **logarithm** undoes exponentiation: $\log_b x$ is the power to which b must be raised to give x.

$$\log_b x = y \quad \text{means exactly} \quad b^y = x.$$

In this book, log with no base means base 2. Three ways to think of $\log_2 n$ are worth having.

- It is the number of times n can be **halved** before reaching 1.
- It is the number of times 1 must be **doubled** to reach n.
- It is, to within one, the number of **bits** needed to write n in binary.

Since doubling grows very quickly, the logarithm grows very slowly.

n	1	2	8	16	1,024	about 10^6	about 10^9	about 10^{18}
$\log_2 n$	0	1	3	4	10	20	30	60

Memorize the middle of that table: $2^{10} = 1024$ is about a thousand, so 2^{20} is about a million and 2^{30} about a billion.

The rules for logarithms mirror the rules for exponents.

Exponents	Logarithms
$b^x \cdot b^y = b^{x+y}$	$\log(xy) = \log x + \log y$
$b^x / b^y = b^{x-y}$	$\log(x/y) = \log x - \log y$
$(b^x)^y = b^{xy}$	$\log(x^y) = y \log x$
$b^0 = 1$	$\log 1 = 0$

Logarithms to different bases differ only by a constant factor:

$$\log_a x = \frac{\log_b x}{\log_b a}.$$

That is why the base is omitted inside Big-O: $O(\log_2 n)$, $O(\log_{10} n)$, and $O(\ln n)$ are all the same class. The *natural* logarithm, ln, has base $e \approx 2.718$, and $\ln x \approx 0.693 \log_2 x$.

B.2 Floors and ceilings

The **floor** $\lfloor x \rfloor$ is the largest integer not greater than x; the **ceiling** $\lceil x \rceil$ is the smallest integer not less than it. So $\lfloor 3.7 \rfloor = 3$ and $\lceil 3.2 \rceil = 4$. In C++, dividing one non-negative integer by another gives the floor of the quotient: `7 / 2` is 3. The number of elements in the left half of a range of n elements split at `mid = n / 2` is $\lfloor n/2 \rfloor$, and the right half has $\lceil n/2 \rceil$.

B.3 Sums

Three sums account for nearly every analysis in the book.

The arithmetic sum. Adding the first n whole numbers:

$$1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}.$$

To see why, write the sum forwards and backwards, one above the other. Each of the n columns adds to $n + 1$, and the two rows together are twice the sum. This sum is $\Theta(n^2)$. It appears whenever a loop does 1 unit of work, then 2, then 3: nested loops over pairs, insertion sort, quicksort's worst case, growing a vector by a fixed amount.

The geometric sum. Adding successive powers of two:

$$1 + 2 + 4 + \cdots + 2^k = 2^{k+1} - 1.$$

Each term is one more than the sum of all the terms before it, so the total is just under twice the last term. This sum is dominated by its *largest* term. It appears whenever work doubles at each step: the copies made by a doubling vector, the nodes in a complete binary tree.

Run the other way, with terms that *halve*, the sum stays bounded however long it goes on:

$$1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \cdots < 2, \qquad \text{and so} \qquad n + \frac{n}{2} + \frac{n}{4} + \cdots < 2n.$$

The harmonic sum. Adding the reciprocals:

$$1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} \approx \ln n.$$

It grows logarithmically. It appears in the analysis of quicksort and of random binary search trees.

B.4 Induction

Mathematical induction proves that a statement holds for every whole number n from some starting point on. It has two steps.

1. **Base case.** Show that the statement holds for the smallest n.
2. **Inductive step.** Show that *if* the statement holds for some n, *then* it holds for $n + 1$.

Together these give the statement for all n: the base case gives it for 1; the step then gives it for 2; then for 3; and so on, like a row of dominoes.

Example. Claim: $1 + 2 + \cdots + n = n(n + 1)/2$ for every $n \geq 1$.

Base case. For $n = 1$ the left side is 1 and the right side is $1 \cdot 2/2 = 1$.

Inductive step. Suppose the claim holds for n. Then

$$1 + 2 + \cdots + n + (n + 1) = \frac{n(n + 1)}{2} + (n + 1) = \frac{(n + 1)(n + 2)}{2},$$

which is the claim for $n + 1$.

Induction is the same idea as two things you do constantly in this course. A **recursive function** is correct for the same reason an inductive proof is: the base case is handled directly, and the recursive case is correct *assuming* the recursive calls are. And a **loop invariant** is an induction over the iterations of a loop: it holds before the first iteration, and if it holds before any iteration it holds after. When you check an invariant, you are doing an inductive proof.

A variant called *strong induction* assumes the statement for *all* smaller values, not just the previous one. It is what justifies recursion on halves, as in mergesort.

B.5 Factorials

The **factorial** $n! = 1 \cdot 2 \cdot 3 \cdots n$ is the number of different orders in which n distinct things can be arranged. It grows faster than any exponential. Its logarithm, though, is tame:

$$\log_2(n!) = \log_2 1 + \log_2 2 + \cdots + \log_2 n \approx n \log_2 n - 1.44\, n,$$

which is $\Theta(n \log n)$. This is the origin of the lower bound for sorting in Section 8.4.

B.6 A little probability

The analysis of hash tables, quicksort, and Bloom filters uses a few facts about chance.

The **expected value** of a random quantity is its average over all the ways the randomness could turn out, each weighted by its probability. If a fair die is rolled, the expected value is $(1 + 2 + 3 + 4 + 5 + 6)/6 = 3.5$.

Expectations add. The expected value of a sum is the sum of the expected values, *always*, whether or not the quantities are independent. This one fact does a remarkable amount of work. For example: throw n keys into m buckets uniformly at random. How many keys are expected to land in bucket 0? Each key lands there with probability $1/m$,

contributing an expected $1/m$ to the count. Add up over n keys: n/m. That is the load factor, with no further calculation needed.

Waiting for a success. If each attempt succeeds independently with probability p, the expected number of attempts up to and including the first success is $1/p$. If a fraction α of a hash table's slots are full, and probes landed on slots at random, each probe would find an empty slot with probability $1 - \alpha$, so about $1/(1 - \alpha)$ probes would be needed. (Linear probing does worse than this, because its probes are not independent; see Section 5.3.)

Collisions come early. Among k keys thrown into m buckets there are about $k^2/2$ pairs, and each pair collides with probability $1/m$. So the expected number of colliding pairs is about $k^2/2m$, which reaches 1 when k is about $\sqrt{2m}$. A collision is likely long before the table is full. This is the birthday paradox of Section 5.1.

APPENDIX C

A Field Guide to the Standard Containers

This appendix is a quick reference to the containers of the C++ standard library, each of which you have now built or studied. For full details of any of them, consult cppreference.com [17].

C.1 The containers

Container	Built as	Chapter	Strengths	Weaknesses
`vector<T>`	contiguous block that doubles	3	indexing; append; iteration; cache behavior	insert or erase in the middle
`string`	a vector of characters	1, 3	as `vector`	as `vector`
`array<T, N>`	fixed-size block, no heap	1	no allocation at all	size fixed at compile time
`deque<T>`	array of fixed-size blocks	4	push and pop at both ends; indexing	slower iteration than `vector`
`list<T>`	doubly linked list	4	$O(1)$ splice and erase given an iterator; iterators stay valid	no indexing; slow to traverse
`forward_list<T>`	singly linked list	4	as `list`, smaller	forward traversal only
`stack<T>`, `queue<T>`	adapters, over deque by default	4	restricted, intention-revealing interface	nothing else
`priority_queue<T>`	binary heap in a `vector`	8	$O(1)$ largest; $O(\log n)$ push and pop	no search; no iteration
`unordered_map<K,V>`, `unordered_set<K>`	hash table, chained	5	$O(1)$ expected lookup, insert, erase	no order; $O(n)$ worst case

Container	Built as	Chapter	Strengths	Weaknesses
`map<K,V>`, `set<K>`	red-black tree	6, 7	ordered; range queries; $O(\log n)$ worst case	slower than hashing; pointer-heavy
`multimap`, `multiset`	red-black tree	7	as above, duplicate keys allowed	as above

C.2 What each operation costs

In the table, n is the number of elements; "am." means amortized and "exp." means expected.

	`vector`	`deque`	`list`	`unordered_map`	`map`	`priority_queue`
access by position	1	1	n			
find by key or value	n	n	n	1 exp.	$\log n$	
insert or erase at the back	1 am.	1 am.	1			
insert or erase at the front	n	1 am.	1			
insert or erase in the middle	n	n	1, given an iterator			
insert or erase by key				1 exp.	$\log n$	
smallest or largest	n	n	n	n	1	1 (one end only)
push; pop the top						$\log n$

C.3 When iterators and references stop being valid

An iterator, pointer, or reference to an element refers to a place in memory. Operations that move elements make it dangle (Section 3.5). These rules are worth knowing by heart.

Container	Operation	What is invalidated
`vector`, `string`	`push_back`, `insert`, `reserve` that causes reallocation	everything
	`insert` without reallocation	the insertion point and everything after it
	`erase`	the erased position and everything after it
`deque`	insertion or erasure in the middle	everything
	`push_back`, `push_front`	all iterators (but not references to elements)
`list`, `forward_list`	insert	nothing
	erase	only the erased element
`map`, `set`	insert	nothing
	erase	only the erased element
`unordered_map`, `unordered_set`	insert that causes rehashing	all iterators (but not references to elements)
	erase	only the erased element

The pattern: contiguous containers invalidate freely, because their elements move; node-based containers almost never do, because their elements stay put.

C.4 Idioms

These fragments are from `code/appc/stl_idioms.cpp`, which is compiled and tested with the rest of the book's code.

Removing elements from a vector.

code/appc/stl_idioms.cpp

```
std::vector<int> v{4, 8, 15, 16, 23, 42};

// Remove every element matching a condition (C++20).
std::erase_if(v, [](int x) { return x % 2 == 1; });   // {4, 8, 16, 42}

// Remove one element by position, keeping order: O(n).
v.erase(v.begin() + 1);                               // {4, 16, 42}

// Remove one element by position when order does not matter: O(1).
v[0] = v.back();
v.pop_back();                                         // {42, 16}
```

Never erase from a container inside a range-`for` loop over it. Use `std::erase_if`, or loop with an explicit iterator and assign it the return value of `erase`.

Working with maps.

code/appc/stl_idioms.cpp

```
std::unordered_map<std::string, int> stock{{"apple", 3}, {"pear", 0}};

// Look up without inserting.
if (auto it = stock.find("apple"); it != stock.end()) {
    it->second -= 1;              // it->first is the key, it->second the value
}
bool has_fig = stock.contains("fig");            // false, and "fig" is not
↪  added

// Insert only if absent; otherwise leave the existing value alone.
stock.try_emplace("pear", 99);                   // no effect: "pear" exists
stock.try_emplace("plum", 7);                    // inserted

// Visit every entry. Structured bindings name the two halves.
int total = 0;
for (const auto& [name, count] : stock) total += count;
```

Remember that `m[key]` inserts when the key is missing. Use `find` or `contains` to look without changing the map.

Algorithms instead of loops. The header `<algorithm>` holds dozens of functions that work on any iterator range. They are tested, fast, and say what they mean.

code/appc/stl_idioms.cpp

```
std::vector<int> a{5, 3, 9, 1, 7};

int sum = std::accumulate(a.begin(), a.end(), 0);                  // 25
int biggest = *std::max_element(a.begin(), a.end());               // 9
bool any_neg = std::any_of(a.begin(), a.end(), [](int x) { return x < 0; });
auto n_big = std::count_if(a.begin(), a.end(), [](int x) { return x > 4; });

std::sort(a.begin(), a.end());                                     // {1, 3, 5,
↪  7, 9}
bool has7 = std::binary_search(a.begin(), a.end(), 7);             // needs
↪  sorted input
int next = *std::lower_bound(a.begin(), a.end(), 6);               // 7: first
↪  >= 6

std::reverse(a.begin(), a.end());                                  // {9, 7, 5,
↪  3, 1}
```

Notice that `biggest` holds the *value* of the largest element. An iterator obtained before the call to `sort` would still be valid afterward, since sorting does not reallocate, but it would refer to the same *position*, which now holds a different element. Iterators name places, not values.

General habits.

- Pass containers to functions by `const` reference, or by reference if the function is to modify them. Return them by value; the move is cheap.
- Call `reserve` on a vector or hash table when you know roughly how many elements are coming.

- Prefer `v.empty()` to `v.size() == 0`, and never write `v.size() - 1` without being sure `v` is not empty.
- Use `std::size_t` for indices and sizes.
- Let `auto` name iterator types for you: `auto it = m.find(key);`.

Bibliography

[1] Harold Abelson, Gerald Jay Sussman, and Julie Sussman. *Structure and Interpretation of Computer Programs*. MIT Press, 2 edition, 1996.

[2] G. M. Adelson-Velsky and E. M. Landis. An algorithm for the organization of information. *Soviet Mathematics Doklady*, 3:1259–1263, 1962. English translation of the Russian original in Doklady Akademii Nauk SSSR 146:263–266.

[3] Jean-Philippe Aumasson and Daniel J. Bernstein. SipHash: A fast short-input PRF. In *Progress in Cryptology – INDOCRYPT 2012*, volume 7668 of *Lecture Notes in Computer Science*, pages 489–508. Springer, 2012.

[4] Hamsa Bastani, Osbert Bastani, Alp Sungu, Haosen Ge, Özge Kabakcı, and Rei Mariman. Generative AI without guardrails can harm learning: Evidence from high school mathematics. *Proceedings of the National Academy of Sciences*, 122(26): e2422633122, 2025.

[5] Rudolf Bayer and Edward M. McCreight. Organization and maintenance of large ordered indexes. *Acta Informatica*, 1(3):173–189, 1972.

[6] Jon Bentley. *Programming Pearls*. Addison-Wesley, 2 edition, 2000.

[7] Jon L. Bentley and M. Douglas McIlroy. Engineering a sort function. *Software: Practice and Experience*, 23(11):1249–1265, 1993.

[8] Elizabeth L. Bjork and Robert A. Bjork. Making things hard on yourself, but in a good way: Creating desirable difficulties to enhance learning. In Morton A. Gernsbacher, Richard W. Pew, Leaetta M. Hough, and James R. Pomerantz, editors, *Psychology and the Real World: Essays Illustrating Fundamental Contributions to Society*, pages 56–64. Worth Publishers, New York, 2011.

[9] Joshua Bloch. Extra, extra - read all about it: Nearly all binary searches and mergesorts are broken. Google Research Blog, 2006. URL https://research.google/blog/extra-extra-read-all-about-it-nearly-all-binary-searches-and-mergesorts-are-broken/.

[10] Burton H. Bloom. Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM*, 13(7):422–426, 1970.

[11] Randal E. Bryant and David R. O'Hallaron. *Computer Systems: A Programmer's Perspective*. Pearson, 3 edition, 2016.

[12] J. Lawrence Carter and Mark N. Wegman. Universal classes of hash functions. *Journal of Computer and System Sciences*, 18(2):143–154, 1979.

[13] Scott Chacon and Ben Straub. *Pro Git*. Apress, 2 edition, 2014. URL https://git-scm.com/book.

[14] Koen Claessen and John Hughes. QuickCheck: A lightweight tool for random testing of Haskell programs. In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP '00)*, pages 268–279, 2000.

[15] Douglas Comer. The ubiquitous B-tree. *ACM Computing Surveys*, 11(2):121–137, 1979.

[16] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 4 edition, 2022.

[17] cppreference. cppreference.com: C++ reference, 2026. URL https://en.cppreference.com/. Accessed November 2026.

[18] Scott A. Crosby and Dan S. Wallach. Denial of service via algorithmic complexity attacks. In *Proceedings of the 12th USENIX Security Symposium*, pages 29–44, 2003.

[19] Edsger W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1):269–271, 1959.

[20] Edsger W. Dijkstra. *A Discipline of Programming*. Prentice-Hall, 1976.

[21] Ulrich Drepper. What every programmer should know about memory, 2007. URL https://www.akkadia.org/drepper/cpumemory.pdf.

[22] Robert W. Floyd. Algorithm 245: Treesort 3. *Communications of the ACM*, 7(12):701, 1964.

[23] Edward Fredkin. Trie memory. *Communications of the ACM*, 3(9):490–499, 1960.

[24] Philip Gage. A new algorithm for data compression. *The C Users Journal*, 12(2):23–38, 1994.

[25] Bernard A. Galler and Michael J. Fischer. An improved equivalence algorithm. *Communications of the ACM*, 7(5):301–303, 1964.

[26] Leo J. Guibas and Robert Sedgewick. A dichromatic framework for balanced trees. In *Proceedings of the 19th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 8–21, 1978.

[27] John L. Hennessy and David A. Patterson. *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, 6 edition, 2017.

[28] C. A. R. Hoare. Quicksort. *The Computer Journal*, 5(1):10–16, 1962.

[29] John Hopcroft and Robert Tarjan. Algorithm 447: Efficient algorithms for graph manipulation. *Communications of the ACM*, 16(6):372–378, 1973.

[30] Arthur B. Kahn. Topological sorting of large networks. *Communications of the ACM*, 5(11):558–562, 1962.

[31] Manu Kapur. Productive failure. *Cognition and Instruction*, 26(3):379–424, 2008.

[32] Martin Kleppmann. *Designing Data-Intensive Applications*. O'Reilly Media, 2017.

[33] Donald E. Knuth. *The Art of Computer Programming, Volume 1: Fundamental Algorithms*. Addison-Wesley, 3 edition, 1997.

[34] Donald E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, 2 edition, 1998.

[35] Joseph B. Kruskal. On the shortest spanning subtree of a graph and the traveling salesman problem. *Proceedings of the American Mathematical Society*, 7(1):48–50, 1956.

[36] Matt Kulukundis. Designing a fast, efficient, cache-friendly hash table, step by step. Talk at CppCon 2017, Bellevue, WA, 2017.

[37] Barbara Liskov and Stephen Zilles. Programming with abstract data types. *ACM SIGPLAN Notices*, 9(4):50–59, 1974.

[38] Yu. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020.

[39] William M. McKeeman. Differential testing for software. *Digital Technical Journal*, 10(1):100–107, 1998.

[40] Ralph C. Merkle. A digital signature based on a conventional encryption function. In *Advances in Cryptology – CRYPTO '87*, volume 293 of *Lecture Notes in Computer Science*, pages 369–378. Springer, 1988.

[41] Scott Meyers. *Effective Modern C++*. O'Reilly Media, 2014.

[42] David R. Musser. Introspective sorting and selection algorithms. *Software: Practice and Experience*, 27(8):983–993, 1997.

[43] Chris Okasaki. *Purely Functional Data Structures*. Cambridge University Press, 1998.

[44] Patrick O'Neil, Edward Cheng, Dieter Gawlick, and Elizabeth O'Neil. The log-structured merge-tree (LSM-tree). *Acta Informatica*, 33(4):351–385, 1996.

[45] Tim Peters. listsort.txt. Description of the timsort algorithm, in the file `Objects/listsort.txt` of the CPython source distribution, 2002. URL https://github.com/python/cpython/blob/main/Objects/listsort.txt.

[46] James Prather, Brent N. Reeves, Juho Leinonen, Stephen MacNeil, Arisoa S. Randrianasolo, Brett A. Becker, Bailey Kimmel, Jared Wright, and Ben Briggs. The widening gap: The benefits and harms of generative AI for novice programmers. In *Proceedings of the 2024 ACM Conference on International Computing Education Research (ICER '24)*, pages 469–486. ACM, 2024.

[47] William Pugh. Skip lists: A probabilistic alternative to balanced trees. *Communications of the ACM*, 33(6):668–676, 1990.

[48] Robert Sedgewick and Kevin Wayne. *Algorithms*. Addison-Wesley, 4 edition, 2011.

[49] Raimund Seidel and Cecilia R. Aragon. Randomized search trees. *Algorithmica*, 16 (4–5):464–497, 1996.

[50] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1715–1725, 2016.

[51] Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitry Vyukov. AddressSanitizer: A fast address sanity checker. In *Proceedings of the 2012 USENIX Annual Technical Conference*, pages 309–318, 2012.

[52] Daniel D. Sleator and Robert E. Tarjan. Self-adjusting binary search trees. *Journal of the ACM*, 32(3):652–686, 1985.

[53] Alexander Stepanov and Meng Lee. The standard template library. Technical Report HPL-95-11, Hewlett-Packard Laboratories, 1995.

[54] Bjarne Stroustrup. C++11 style. Keynote address, GoingNative 2012, Redmond, WA, 2012.

[55] Bjarne Stroustrup. *A Tour of C++*. Addison-Wesley, 3 edition, 2022.

[56] Bjarne Stroustrup. *Programming: Principles and Practice Using C++*. Addison-Wesley, 3 edition, 2024.

[57] Bjarne Stroustrup and Herb Sutter. C++ core guidelines, 2026. URL `https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines`. Accessed November 2026.

[58] Robert E. Tarjan. Efficiency of a good but not linear set union algorithm. *Journal of the ACM*, 22(2):215–225, 1975.

[59] Robert E. Tarjan. Amortized computational complexity. *SIAM Journal on Algebraic and Discrete Methods*, 6(2):306–318, 1985.

[60] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, pages 5998–6008, 2017.

[61] J. W. J. Williams. Algorithm 232: Heapsort. *Communications of the ACM*, 7(6): 347–348, 1964.

How This Book Was Made

This book asks, of every data structure, *how do you know?* It would be poor form not to answer the same question about itself.

Who wrote it

The text, the code, the figures, and the interactive widgets were drafted by Claude, a large language model made by Anthropic (the Fable 5.2 model), working in a single extended session in November 2026 at the request of the instructor of CMSC 270 at Lawrence University. The instructor's brief asked for a ten-week data structures text in C++ in two editions, one for the browser and one typeset with LaTeX, and left the content, the order, and the presentation to the model's judgment. The decisions described in the preface, such as putting memory first and hashing before trees, the four recurring questions, and the stance on language models, are the model's own.

That is an unusual way for a textbook to come about, and you are entitled to ask how far to trust the result. The answer has two halves.

What was checked, and how

The programs were compiled and run. Every C++ listing with a file path printed above it is an excerpt, extracted mechanically at build time, from that file in the `code/` directory. Running `make test` there compiles each program with `-std=c++20 -Wall -Wextra -Wpedantic -Wshadow -Wconversion -Werror` and both AddressSanitizer and UndefinedBehaviorSanitizer, and executes it; all of them pass. The timing programs in the `bench/` directories are the exception: they are compiled with optimization and without sanitizers, as timing requires, and were run by hand to produce the tables. A few short fragments in check-yourself questions, warnings, and exercises have no file path above them. Those were typed directly into the text and were not compiled; several are wrong on purpose.

The data structures were tested against independent references. The vector, lists, queue, hash tables, search trees, AVL tree, B-tree, heap, sorts, LRU cache, and trie are each put through thousands of randomized operations and compared with a standard-library container or a deliberately simple second implementation. Where a structure has an invariant that can be checked from inside (the vector, the lists, the hash tables, all the trees, the heap), an executable check runs alongside. The shortest-path, spanning-tree, topological-sort, and union-find code is checked on hundreds of small random graphs against slower independent algorithms. A few functions (depth-first visiting order, path reconstruction, the maze search) are tested only on hand-built examples.

The tests found real mistakes. This is worth recording, because it is the book's argument in miniature. While the book was being written, the tests caught, among other things: an expected value for a maze's shortest path that the author had worked out by hand and gotten wrong (the code was right); an example in Appendix C that held an iterator across a call to `sort` and then expected it to still refer to the largest element; and a demonstration of quicksort's worst case that did not exhibit the worst case at all, because the particular partitioning scheme happened to scramble sorted input. Each of these would have gone to print as a confident, plausible, wrong statement. None of them was caught by rereading.

Every timing was measured. The numbers in the performance tables are the output of the programs in the `bench/` directories, run on one laptop (a 2023 Intel Core i9-13900H with 64 GB of memory, Ubuntu Linux, GCC 13.3 at `-O2`). They are single runs, reported as observed, noise included. Your machine will give different numbers and, with luck, the same patterns. Where a measurement contradicted what the author expected, the measurement stayed and the text was changed.

The figures are generated. Each static figure is described once, in a short Python program, and rendered from that single description both as vector graphics for the web edition and as TikZ for print. The interactive figures compute their states by running the actual algorithm in the browser; they are not hand-drawn animations. (The one exception is the memory explorer in Chapter 1, which replays a scripted trace of four small programs.)

What was not checked

The prose, by a person. The explanations, historical remarks, and descriptions of how real systems work were written from the model's training. After drafting, three separate instances of the model, given the source cold and told only to find errors, reviewed the chapters; they reported about forty problems, among them an exercise that overfilled its own hash table, three passages that overstated what the measurements beside them showed, two real bugs in the code (both in copying or moving objects, which the tests had never exercised), and several claims in an earlier draft of this very colophon that the repository did not support. All were corrected. That is a useful second pass, and it is not the same as review by a human expert, which the book has not had. The preface says that a language model sounds exactly as confident when it is wrong as when it is right. That applies to this book.

Claims of the kind most likely to be wrong are: dates and attributions in the chapter notes; statements about what particular libraries and products do internally, which change over time; and the summaries of research on learning in the preface, which compress nuanced findings into a sentence or two. If something here contradicts your instructor, a primary source, or a measurement you have made yourself, believe them and not the book.

The bibliography. A language model can produce a citation that is perfectly formatted and entirely fictitious. The references here were restricted to well-known works, and each was then checked against publisher records, library catalogs, and primary copies on the open web by a separate automated pass. All sixty were found to exist as described; two needed corrections (a link, and a title), and a handful gained missing page numbers. One

entry was added afterward and has not been through that check: Dijkstra's *A Discipline of Programming*. That is a weaker check than a librarian's. Before you cite any of these works in your own writing, look it up.

Pedagogy. No student has yet learned from this book. Whether the order works, whether the exercises are the right difficulty, and whether a ten-week term can hold all of it are empirical questions, and the people best placed to answer them are the instructor and you.

How to report a problem

Tell your instructor. The source of the book is a directory of Markdown files, C++ programs, and a build script, meant to be edited. A correction to the text is a one-line change; a correction to a program is checked by the same tests as everything else.

Tools

The source is written in Pandoc Markdown. A Python script assigns numbers, pulls the listings out of the code files, and runs Pandoc with a Lua filter to produce HTML for the web edition and LaTeX for print. The print edition is typeset by pdfLaTeX in XCharter, with Fira Sans for headings and Inconsolata for code; the web edition uses the same typefaces, which are distributed under open licenses. The web edition makes no network requests and runs from a local folder.